

**FULLSTACKUTVECKLARE
JAVASCRIPT**

Kursbenämning: Backendutveckling i Node.js, databaser och säkerhet	
Kurspoäng: 60 yhp	Undervisningsspråk: Svenska, delvis engelska
Utbildningsnummer- och omgång: YH02031-2023-03	Kursplan fastställd: 10 december 2025

KURSENS HUVUDSAKLIGA INNEHÅLL

Kursen ger en praktisk och strukturerad introduktion till backendutveckling med Node.js och Express. Studerande lär sig skapa och typa stateless REST API:er, hantera persistent data i relations- och dokumentdatabaser samt implementera och testa CRUD-operationer. Fokus ligger på säker och konsekvent datahantering genom robust inputvalidering, korrekt felhantering och grundläggande skydd mot OWASP-relaterade risker såsom injektioner och osäker autentisering. Kursen introducerar även principen "don't trust the client" och hur rollbaserad åtkomst (RBAC) och miljövariabler används för att skapa säkrare system. Studerande tränar på GDPR-medveten datahantering genom dataminimering och strukturerad loggning. Kursen tränar även förmågan att visualisera och dokumentera dataflöden i fullstackapplikationer, samt genomföra enklare driftsättning till molnmiljö. Kursen avslutas med ett teamprojekt där studerande arbetar med systemarkitektur, dokumentation och delat ansvar i en gemensam kodbas.

KURSMÅL

KUNSKAPER

1. Förklara hur Node.js och Express används för att bygga backendtjänster och hantera kommunikation mellan klient och server via stateless REST API:er.
2. Redogöra för hur data lagras och hämtas i relations- och dokumentdatabaser samt hur CRUD-operationer implementeras och testas.
3. Beskriva hur TypeScript används för struktur och typning i backendkod, inklusive rutter, modeller och API-kontrakt.
4. Redogöra för grundläggande säkerhetsprinciper såsom autentisering, felhantering, OWASP Top 3, principen "don't trust the client", skillnaden mellan verklig säkerhet och "security by obscurity", samt hur lagkrav (inklusive GDPR, dataminimering och säker loggning) påverkar API-design och datahantering.
5. Förklara hur frontend, backend och databas samverkar i ett fullstacksystem, hur rollbaserad åtkomstkontroll (RBAC) fungerar, samt hur dataflöden dokumenteras.
6. Beskriva hur miljövariabler och separerad konfiguration används för att hantera känslig information och miljöspecifika inställningar på ett säkert sätt.

FÄRDIGHETER

7. Skapa och strukturera backendapplikationer i Node.js och Express, inklusive rutter, logik, stateless API-anrop och typade dataflöden.
8. Implementera och testa CRUD-operationer mot en databas samt koppla backendkod till ett datalager på ett säkert och stabilt sätt.
9. Implementera robust inputvalidering i backend genom att validera inkommande data, tillämpa "don't trust the client", och hantera fel konsekvent och säkert.

10. Implementera grundläggande skydd mot OWASP-relaterade risker, exempelvis parametriserade databasanrop, säker hantering av autentisering, enkel rollbaserad åtkomstkontroll (RBAC) och hantering av känslig information via miljövariabler.
11. Utföra grundläggande driftsättning till molnmiljö samt dokumentera systemflöden och API-anrop på ett överskådligt sätt med hänsyn till GDPR och dataminimering.
12. Förstå och visualisera dataflödet mellan klient, server och databas i en fullstackapplikation samt samarbeta i större teamprojekt med tydlig ansvarsfördelning.

FORMER FÖR KOMPETENSKONTROLL

SUMMATIV BEDÖMNING

Summativ bedömning utvärderar studerandes kunskaper och färdigheter retroaktivt för att sammanfatta vad den studerande kan vid examinationstillfället. Den summativa bedömningen täcker samtliga kursmål och sker vid följande avgränsade examinationstillfällen:

- Skriftligt kunskapstest – En (1) individuell prövning av teoretiska kunskaper.
- Skriftlig reflektion – En (1) individuell redogörelse för egna lösningar och tillvägagångssätt.
- Projektinlämning – En (1) gruppinlämning där varje studerandes individuella insats bedöms.

FORMATIV BEDÖMNING

Formativ bedömning utvärderar studerandes kunskaper och färdigheter löpande under kursens interaktiva lärandemoment. Syftet är att effektivisera bedömningen och samtidigt stödja lärandet. Den formativa bedömningen sker i samband med:

- Workshops – Interaktiva undervisningstillfällen där klassen och utbildaren övar tillsammans.
- Checkpoints – Regelbundna avstämningar där studerande presenterar sina projekt och får direkt återkoppling.

Formativ bedömning är ett komplement till summativ bedömning och tillämpas enbart till fördel för studerande. Studerande kan uppnå kursmål redan innan de summativa examinationerna genom formativ bedömning, men ett underkänt resultat kan aldrig enbart baseras på denna bedömningsform.

PRINCIPER FÖR BETYGSSÄTTNING

Betyg sätts i form av icke godkänt (IG), godkänt (G) eller väl godkänt (VG)

IG

Icke godkänt (IG): Samtliga kursmål är inte uppfyllda

G

Godkänt (G): Samtliga kursmål är uppfyllda

VG

Alla lärandemål är uppfyllda på en tillfredsställande nivå. Utöver ovan uppvisar den studerande:

- Hög precision att skapa och strukturera backendapplikationer i Node.js och Express, inklusive rutter, logik, stateless API-anrop och typade dataflöden.
- Stor skicklighet att implementera och testa CRUD-operationer mot en databas samt koppla backendkod till ett datalager på ett säkert och stabilt sätt.
- Hög precision att implementera robust inputvalidering i backend genom att validera inkommande data, tillämpa "don't trust the client", och hantera fel konsekvent och säkert.

KOMPLETTERING

För varje examinationsmoment har den studerande rätt att delta på en ordinarie examination och en (1) omexamination. Utbildare avgör i vilka fall omexamination kan ske i form av komplettering.

Tid och form för omexamination alternativt komplettering fastställs av utbildningsanordnaren och utbildare som sedan förmedlar detta till den studerande.

Fullstackutvecklare Javascript

Kursbenämning: Backendutveckling i Node.js, databaser och säkerhet	
Kurspoäng: 60 yhp	Undervisningsspråk: Svenska, delvis engelska
Utbildarens namn: Robin Andersson	Antal veckor: 12

ÖVERSIKTSSCHEMA

Preliminär grovplanering av kursen. Upplägget kan komma att ändras.

Vecka	Lektion/ handledning	Ämne
Kursvecka 1	Boiler Room	Inget Boiler Room denna vecka - introduktionsvecka.
Kursvecka 1	Workshops	Kursmål: 1 - Introduktion till kursen, kursens upplägg och examinerande moment. - Vad är backend? Klient-server-modellen och hur frontend och backend kommunicerar. - Introduktion till Node.js - runtime, eventloop och non-blocking I/O. - npm och pakethantering - skapa ett Node.js-projekt med package.json. - Introduktion till Express - skapa en enkel server. - Grundläggande routing i Express: GET, POST, PUT, DELETE. - Request- och Response-objekt - förstå req.params, req.query och req.body. - Middleware-konceptet i Express - vad det är och hur det fungerar. - Stateless HTTP och vad det innebär för REST API:er. - Code-along: Bygg en enkel Express-server med grundläggande routes. - Övning: Skapa egna endpoints som hanterar olika HTTP-metoder.
Kursvecka 2	Boiler Room	Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 2	Workshops	Kursmål: 1, 7 - Repetition av Node.js och Express från vecka 1. - RESTful API-design - principer, namnkonventioner och

		resursorienterade URL:er. • HTTP-statuskoder - när och varför man använder 200, 201, 400, 404, 500 etc. • Strukturera ett Express-projekt: routes, controllers och separation of concerns. • Router-moduler i Express - bryta ut routes i separata filer. • Middleware i praktiken: <code>express.json()</code>, <code>cors</code>, <code>morgan</code> för loggning. • Skriv egna middleware-funktioner för validering och loggning. • Använda Postman/Thunder Client för att testa API:er. • Versionshantering av API:er - best practices. • Code-along: Strukturera ett REST API med separata route-filer och middleware. • Övning: Bygg ett komplett REST API med tydlig mappstruktur och middleware-kedja.
Kursvecka 3	Boiler Room	• Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 3	Workshops	Kursmål: 3 • Repetition av TypeScript-grunder från kurs 3 (typer, interfaces, generics). • Konfigurera TypeScript i ett Node.js/Express-projekt (<code>tsconfig.json</code>, <code>ts-node</code>). • Typa Express - Request, Response och NextFunction. • Skapa typade route handlers och controllers. • Definiera interfaces för datamodeller och API-kontrakt. • Typa middleware-funktioner i Express. • Generics i praktiken - typade request bodies och response-objekt. • Utility types (Partial, Pick, Omit) för flexibla datamodeller. • Strikt felhantering med TypeScript - typade Error-objekt. • Code-along: Konvertera ett Express-API från JavaScript till TypeScript. • Övning: Skapa ett fullständigt typat Express-API med interfaces för alla dataflöden.
Kursvecka 4	Boiler Room	• Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 4	Workshops	Kursmål: 2, 6 • Introduktion till databaser - varför persistent data behövs. • Relationsdatabaser vs dokumentdatabaser - skillnader och användningsområden. • Introduktion till MongoDB - dokumentbaserad datalagring. • MongoDB Atlas - skapa ett konto och en molnbaserad databas. • Mongoose som ODM (Object Document Mapper) - varför och

		hur. - Definiera Mongoose schemas och modeller med TypeScript. - Ansluta Express-applikationen till MongoDB med Mongoose. - Grundläggande databasoperationer: skapa och läsa dokument. - Miljövariabler med dotenv - hantera databasanslutningssträngar säkert. - Code-along: Koppla Express till MongoDB och skapa första modellen. - Övning: Designa schemas för en applikation och spara data i databasen.
Kursvecka 5	Boiler Room	Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 5	Workshops	Kursmål: 2, 8 - Fullständiga CRUD-operationer med Mongoose: create, find, findById, updateOne, deleteOne. - Filtrera, sortera och paginera data med Mongoose queries. - Relationer i MongoDB: embedding vs referencing - när ska man använda vad? - Populate i Mongoose - hämta relaterad data. - Indexering i MongoDB - förbättra prestanda på sökningar. - Introduktion till testning av API:er - varför och hur. - Jest och Supertest - enhetstester och integrationstester för Express-routes. - Testa CRUD-operationer mot en testdatabas (MongoDB Memory Server). - Code-along: Implementera fullständig CRUD med relationer och tester. - Övning: Bygg och testa CRUD-endpoints med relationer mellan modeller.
Kursvecka 6	Boiler Room	Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 6	Workshops	Kursmål: 4, 9 - Principen "don't trust the client" - varför all input måste valideras på servern. - Introduktion till Zod - schemabaserad validering med TypeScript-integration. - Validera request body, params och query med Zod-schemas. - Skapa återanvändbara validerings-middleware med Zod. - Mongoose-validering vs applikationsvalidering - olika lager av säkerhet. - Robust felhantering i Express - centraliserad error handler middleware. - Skapa typade felklasser: ValidationError, NotFoundError, UnauthorizedError.

		• Konsekvent felrespons - standardiserat error-format i API:et. • Undvika att exponera känslig information i felmeddelanden. • Code-along: Bygg validering med Zod och centraliserad felhantering. • Övning: Implementera inputvalidering och felhantering i ert API.
Kursvecka 7	Boiler Room	• Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 7	Workshops	Kursmål: 4, 10 • Grundläggande säkerhetsprinciper för webbutveckling. • OWASP Top 3: Broken Access Control, Cryptographic Failures, Injection. • Skillnaden mellan verklig säkerhet och "security by obscurity". • Autentisering: registrering och inloggning med lösenord. • Hasha lösenord säkert med bcrypt - varför klartext aldrig är okej. • JSON Web Tokens (JWT) - hur tokenbaserad autentisering fungerar. • Skapa och verifiera JWT:er i Express. • Auth-middleware - skydda routes med tokenverifiering. • Skydda mot injection-attacker med parametriserade databasanrop i Mongoose. • Code-along: Implementera registrering, inloggning och skyddade routes med JWT. • Övning: Lägg till autentisering i ert API med bcrypt och JWT.
Kursvecka 8	Boiler Room	• Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 8	Workshops	Kursmål: 5, 6, 10 • Skillnaden mellan autentisering och auktorisering. • Rollbaserad åtkomstkontroll (RBAC) - principer och implementering. • Designa roller och behörigheter i ett API (admin, user, etc.). • RBAC-middleware i Express - kontrollera roller per route. • Lagra roller i databasen och koppla till användare. • Miljövariabler och separerad konfiguration - .env-filer i olika miljöer. • Hantera känslig information: JWT-hemligheter, databasnycklar, API-nycklar. • Säker konfigurationshantering - aldrig committa hemligheter till Git. • Hur frontend, backend och databas samverkar i ett fullstacksystem. • Code-along: Implementera RBAC med roller och miljövariabler.

		Övning: Lägg till rollbaserad åtkomst och säker konfiguration i ert API.
Kursvecka 9	Boiler Room	Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 9	Workshops	Kursmål: 4, 5, 11 - GDPR och vad det innebär för API-design och datahantering. - Dataminimering - samla bara in den data som behövs. - Säker loggning - vad får och bör loggas? Undvika persondata i loggar. - Strukturerad loggning med bibliotek (winston / pino). - Mjuk radering (soft delete) och datalivscykel. - Relationsdatabaser i korthet - SQL-grunder och jämförelse med MongoDB. - Visualisera dataflöden: klient → server → databas. - Sekvensdiagram för att dokumentera API-anrop och systemflöden. - Verktyg för dokumentation: Mermaid, draw.io och API-dokumentation med Swagger/OpenAPI. - Code-along: Implementera strukturerad loggning och dokumentera API-flöden. - Övning: Skapa sekvensdiagram och API-dokumentation för ert projekt.
Kursvecka 10	Boiler Room	Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 10	Workshops	Kursmål: 5, 11, 12 - Koppla ihop frontend (React) med backend (Express API). - CORS i praktiken - konfiguration och felsökning. - Proxy-konfiguration i utvecklingsmiljön. - Hantera autentisering i fullstack - JWT i frontend och backend. - Visualisera det kompletta dataflödet i en fullstackapplikation. - Introduktion till driftsättning - vad krävs för att köra i produktion? - Driftsätta backend till molnmiljö (Render / Railway / Vercel). - Miljövariabler i produktionsmiljö. - Grundläggande produktionsförberedelser: HTTPS, CORS-policy, rate limiting. - Code-along: Driftsätt API:et till molnmiljö med MongoDB Atlas. - Övning: Driftsätt ert API och verifiera att det fungerar i produktion.
Kursvecka 11	Boiler Room	Se BoilerRoomProjekt.pdf för instruktioner.
Kursvecka 11	Workshops	Kursmål: 11, 12

		• Dokumentation av API:er - README, endpoints och autentiseringsflöde. • Dokumentera systemarkitektur med diagram och flödesbeskrivningar. • Skriva överlämningsdokumentation för ett teamprojekt. • GDPR-hänsyn i dokumentationen - vilken data lagras och varför. • Samarbete i gemensam kodbas - Git-workflow och ansvarsfördelning. • Kodgranskning (code review) - checklistor och best practices. • Slutförberedelser inför examination. • Projektarbete: Finslipa kod, tester och dokumentation. • Genomgång av examinationskrav. • Repetition och frågestund inför kunskapstestet.
Kursvecka 12	Boiler Room	• Färdigställ projektet och förbered för redovisning.
Kursvecka 12	Workshops	Kursmål: Samtliga • Tisdag: Checkpoint och sista minuten-frågor. • Onsdag: Skriftligt kunskapstest (kursmål 1-6). • Torsdag: Projektinlämning och skriftlig reflektion.

EXAMINERANDE MOMENT

Examinerande moment	Presenteras	Deadline	Kursmål
Skriftligt kunskapstest		Kursvecka 12	Kursmål 1-6
Skriftlig reflektion		Kursvecka 12	Kursmål 1-6
Projektinlämning i grupp		Kursvecka 12	Kursmål 7-12
Kompletteringstillfälle		Senast 3 veckor efter kursslut	De som saknas

KURSMÅL - REFERENS

Kunskaper:

- 1. Förklara hur Node.js och Express används för att bygga backentjänster och hantera kommunikation mellan klient och server via stateless REST API:er.
- 2. Redogöra för hur data lagras och hämtas i relations- och dokumentdatabaser samt hur CRUD-operationer implementeras och testas.
- 3. Beskriva hur TypeScript används för struktur och typning i backendkod, inklusive rutter, modeller och API-kontrakt.

- 4. Redogöra för grundläggande säkerhetsprinciper såsom autentisering, felhantering, OWASP Top 3, principen "don't trust the client", skillnaden mellan verklig säkerhet och "security by obscurity", samt hur lagkrav (inklusive GDPR, dataminimering och säker loggning) påverkar API-design och datahantering.
- 5. Förklara hur frontend, backend och databas samverkar i ett fullstacksystem, hur rollbaserad åtkomstkontroll (RBAC) fungerar, samt hur dataflöden dokumenteras.
- 6. Beskriva hur miljövariabler och separerad konfiguration används för att hantera känslig information och miljöspecifika inställningar på ett säkert sätt.

Färdigheter:

- 7. Skapa och strukturera backendapplikationer i Node.js och Express, inklusive rutter, logik, stateless API-anrop och typade dataflöden.
- 8. Implementera och testa CRUD-operationer mot en databas samt koppla backendkod till ett datalager på ett säkert och stabilt sätt.
- 9. Implementera robust inputvalidering i backend genom att validera inkommande data, tillämpa "don't trust the client", och hantera fel konsekvent och säkert.
- 10. Implementera grundläggande skydd mot OWASP-relaterade risker, exempelvis parametriserade databasanrop, säker hantering av autentisering, enkel rollbaserad åtkomstkontroll (RBAC) och hantering av känslig information via miljövariabler.
- 11. Utföra grundläggande driftsättning till molnmiljö samt dokumentera systemflöden och API-anrop på ett överskådligt sätt med hänsyn till GDPR och dataminimering.
- 12. Förstå och visualisera dataflödet mellan klient, server och databas i en fullstackapplikation samt samarbeta i större teamprojekt med tydlig ansvarsfördelning.

Studiematerial

Introduktion till backend, Node.js och Express

Kursvecka 1

Introduktion

Välkommen till Backendutveckling i Node.js, databaser och säkerhet – kurs 4 i utbildningen Fullstackutvecklare JavaScript. Vi har hittills byggt ett starkt fundament: vi behärskar HTML, CSS och JavaScript-grunderna, kan manipulera DOM och konsumera externa API:er, och vi har lärt oss att bygga moderna webbgränssnitt i React med TypeScript och till och med mobilappar med React Native. Nu tar vi nästa stora steg – vi ska lära oss att bygga de API:er vi hittills bara anropat.

Under föregående kurser har vi alltid befunnit oss på klientsidan: vi har skickat HTTP-anrop till servrar som någon annan byggt och tagit emot svar som vi presenterat för användaren. Vi har läst dokumentation om endpoints, HTTP-metoder och svarsformat – men vad som händer innanför servern har varit en svart låda. Den svarta lådan öppnar vi nu.

Backendutveckling handlar om att bygga den infrastruktur som frontendar och mobilappar kommunicerar med. Det innebär att designa API:er, hantera datalagring, validera indata, skydda känslig information och säkerställa att systemet beter sig förutsägbart och säkert. Det är ett område som kräver ett annat tankesätt än frontend – vi ska inte längre tänka på vad användaren ser, utan på vad servern vet, vad den tillåter och hur den svarar.

Vi använder Node.js och Express som vår plattform. Det är ett naturligt val för oss som redan kan JavaScript: vi kan återanvända vår kunskap om syntaxen, asynkron programmering och TypeScript, men nu tillämpa den på servern istället för i webbläsaren. Kursen bygger vidare direkt på det vi lärt oss – du kommer att känna igen mönster, men se dem från en helt ny vinkel.

Under vecka 1 lägger vi grunden för hela kursen. Vi förstår skillnaden mellan frontend och backend, lär oss vad Node.js är och varför det fungerar som det gör, sätter upp vår första Express-server och skapar våra första API-endpoints. Det kan verka enkelt till en början – och det är det delvis – men det är viktigt att vi förstår varför saker fungerar som de gör, inte bara hur. Det är den förståelsen som gör oss till bra backendutvecklare.

Vad är backend?

Backend är allt som händer på serversidan – den del av systemet som klienten (webbläsaren, mobilappen) kommunicerar med men inte direkt ser. Medan frontend handlar om vad användaren möter och interagerar med, handlar backend om logik, datalagring, säkerhet och kommunikation.

I ett modernt system spelar backend ofta rollen som en central koordinator: den tar emot anrop från frontendar, validerar och bearbetar datan, kommunicerar med databaser och andra tjänster, och returnerar strukturerade svar. Frontendar behöver inte veta hur datan lagras eller var den kommer ifrån – de litar på att backenden sköter det.

Klient-server-modellen

Klient-server-modellen är det grundläggande arkitekturmönstret för webben. En klient (exempelvis vår React-app) initierar kommunikation genom att skicka ett HTTP-anrop till en server. Servern tar emot anropet, utför nödvändig logik och returnerar ett svar. Klienten tar emot svaret och presenterar eller använder informationen.

Det är viktigt att förstå tre saker om den här modellen:

- Klienten initierar alltid – servern väntar passivt på inkommande anrop och kan inte på eget initiativ skicka data till klienten (med undantag för tekniker som WebSockets, men det är utanför den här kursens scope).
- Servern är tillståndslös (stateless) – varje HTTP-anrop behandlas oberoende av tidigare anrop. Servern minns inte vem vi är eller vad vi gjort tidigare. Det är därför vi behöver mekanismer som tokens för att hantera inloggning.
- Svaret är strukturerat – servern returnerar data i ett standardformat, vanligtvis JSON, som klienten kan tolka och använda.

Under kursen 2 (Frontendutveckling) använde vi `fetch()` och `axios` för att göra HTTP-anrop mot externa API:er. Nu ska vi bygga servern som tar emot sådana anrop. Det är en perspektivförskjutning som kräver att vi tänker på hur anropet ser ut från serverns synvinkel – vad klienten skickar med, vad vi kan lita på och vad vi måste verifiera.

HTTP-metoder och deras semantik

HTTP definierar ett antal metoder som anger syftet med ett anrop. Som frontendare har vi läst dokumentation om dessa, men nu när vi designar API:er måste vi förstå deras semantik på djupet:

- GET – Hämta en resurs. Ska aldrig förändra data. Kan sparas i cache. Parametrar skickas i URL:en.
- POST – Skapa en ny resurs eller utföra en åtgärd. Data skickas i request body. Är inte idempotent (flera identiska anrop kan ge olika resultat).
- PUT – Ersätt en hel resurs. Idempotent: samma anrop ger alltid samma resultat.
- PATCH – Uppdatera delar av en resurs. Idempotent i praktiken, men inte per definition.
- DELETE – Ta bort en resurs. Idempotent: att radera något som inte finns är tekniskt sett samma slutresultat.

Att välja rätt HTTP-metod är inte bara en konvention – det signalerar till klienten och till infrastrukturen (proxy-servrar, cache-lager) hur anropet ska hanteras. En GET-endpoint som ändrar data är ett allvarligt design-misstag.

HTTP-statuskoder

HTTP-statuskoder kommunicerar resultatet av ett anrop på ett standardiserat sätt. Som backendutvecklare är det vårt ansvar att returnera korrekta statuskoder – det gör det möjligt för klienten att hantera svar korrekt utan att behöva analysera svarstexten.

De viktigaste kategorierna att känna till:

- 2xx Framgång – 200 OK (vanligaste svaret), 201 Created (ny resurs skapad), 204 No Content (lyckades men inget att returnera).
- 3xx Omdirigering – Används sällan direkt i API:er, men förekommer i autentiseringsflöden.
- 4xx Klientfel – 400 Bad Request (felaktig indata), 401 Unauthorized (ej inloggad), 403 Forbidden (inloggad men saknar behörighet), 404 Not Found (resursen finns inte), 422 Unprocessable Entity (valideringsfel).
- 5xx Serverfel – 500 Internal Server Error (oväntat fel i servern), 503 Service Unavailable (servern är tillfälligt otillgänglig).

En vanlig nybörjarmiss är att returnera 200 OK även när något gick fel, med felmeddelandet inbakat i svarstexten. Det bryter mot HTTP-standarden och gör det svårt för klienter att hantera fel automatiskt. Vi ska alltid returnera rätt statuskod.

Node.js – JavaScript på servern

Node.js är en JavaScript-runtime byggd på V8 – samma JavaScript-motor som används i Chrome. Det gör att vi kan köra JavaScript-kod utanför webbläsaren, direkt i terminalen eller på en server. För oss som kan JavaScript sedan tidigare är det en enorm fördel: vi behöver inte lära oss ett nytt programmeringsspråk för att bygga backend.

Node.js skapades 2009 av Ryan Dahl med ett tydligt mål: bygga en plattform för nätverksorienterade applikationer som kan hantera många simultana anslutningar utan att använda mycket minne. Lösningen var att basera Node.js på en arkitektur som vi känner igen från frontendprogrammering – eventloopen.

Eventloopen och non-blocking I/O

För att förstå Node.js måste vi förstå skillnaden mellan blocking och non-blocking I/O. I ett traditionellt serverprogram (t.ex. ett Java- eller PHP-program med trådbaserad modell) startas en ny tråd för varje inkommande anslutning. Tråden väntar (blockar) tills operationen är klar – exempelvis tills databasen svarat – och kan inte göra något annat under den tiden. Om tusentals anrop kommer in samtidigt krävs tusentals trådar, vilket är resurskrävande.

Node.js löser detta annorlunda. Det kör på en enda tråd, men delegerar tidskrävande operationer (nätverksanrop, fil läsning, databasfrågor) till operativsystemet via libuv. Medan operativsystemet utför arbetet kan Node.js-tråden hantera andra inkommande anrop. När operationen är klar lägger operativsystemet ett event i eventloopen, och Node.js hanterar det när det är klart.

Det här mönstret är exakt samma som vi känner igen från asynkron JavaScript i webbläsaren – Promises, async/await och callbacks. Vi har använt det när vi hämtat data med fetch(). Nu förstår vi varför det ser ut som det gör: det är inte bara en konvention utan en konsekvens av hur den underliggande arkitekturen fungerar.

En viktig konsekvens är att vi aldrig ska blockera eventloopen med CPU-intensiva operationer. Om vi exekverar en lång beräkning synkront (utan await) kan servern inte hantera andra anrop under den tiden. Det är en av de viktigaste principerna i Node.js-programmering.

Node.js vs webbläsarens JavaScript

Trots att både Node.js och webbläsaren kör JavaScript finns viktiga skillnader vi behöver ha koll på:

- Ingen DOM – Node.js har ingen webbläsarmiljö, inga window- eller document-objekt. Vi arbetar inte med HTML-element.
- Moduler – Node.js använder CommonJS-moduler (require/module.exports) historiskt, men stödjer nu även ES-moduler (import/export). Vi kommer att använda ES-moduler med TypeScript i kursen.
- Inbyggda moduler – Node.js har tillgång till inbyggda moduler för filsystemet (fs), nätverket (http/https), sökvägar (path) och mer.
- Global scope – Istället för window heter det globala objektet global i Node.js, men vi ska undvika att använda det direkt.

npm och pakethantering

npm (Node Package Manager) är det inbyggda pakethanteringssystemet för Node.js. Det är det samma npm vi använt när vi installerat paket i React-projekten – npm install, package.json och node_modules är välbekanta begrepp. Nu ska vi förstå dem lite djupare.

En Node.js-applikation startar med att skapa en package.json-fil. Den fungerar som projektets manifest och innehåller information om projektet, dess beroenden och scripts för att köra, bygga och testa applikationen.

För att skapa ett nytt Node.js-projekt kör vi:

```
mkdir mitt-projekt
cd mitt-projekt
npm init -y          # Skapar package.json med standardvärden
```

package.json ser ut ungefär så här:

```
{
  "name": "mitt-projekt",
  "version": "1.0.0",
  "description": "Min första backend-app",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "dependencies": {},
  "devDependencies": {}
}
```

Det finns en viktig distinktion mellan dependencies och devDependencies: dependencies är paket som behövs när applikationen körs i produktion (t.ex. Express), medan devDependencies är paket som bara behövs under utveckling (t.ex. nodemon för automatisk omstart, eller TypeScript-kompilatorn).

nodemon är ett verktyg vi kommer att använda flitigt under kursen. Det övervakar våra filer och startar om servern automatiskt när vi sparar ändringar – precis som Vites hot-reload fungerar för React, fast för vår Node.js-server. Vi installerar det som en devDependency:

```
npm install --save-dev nodemon
```

En sak vi alltid ska göra är att lägga till node_modules i .gitignore. node_modules innehåller alla installerade paket och kan bli väldigt stor – det ska aldrig checkas in i versionshanteringen. Alla som klonar projektet kör npm install för att installera beroendena på nytt.

Express – ett webbramverk för Node.js

Node.js har ett inbyggt http-bibliotek som låter oss skapa en webbserver, men det är relativt lågnivå och kräver mycket kod för enkla saker. Express är ett minimalistiskt webbramverk som bygger ovanpå Node.js http-biblioteket och ger oss ett bekvämare API för att definiera routes, hantera requests och skicka responses.

Express är det dominerande ramverket för Node.js-backends och används av stora företag världen över. Det är lätt, flexibelt och har ett enormt ekosystem av tilläggspplugins (middleware). Många andra ramverk (som NestJS eller Fastify) bygger på liknande principer, så det vi lär oss här ger oss en solid grund att stå på.

Vi installerar Express så här:

```
npm install express
```

Vår första Express-server

Här är den absolut enklaste Express-servern vi kan skriva:

```
const express = require('express');

const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hej från min server!');
});

app.listen(port, () => {
  console.log(`Servern lyssnar på http://localhost:${port}`);
});
```

Låt oss gå igenom varje del:

- `express()` – Skapar en Express-applikation. App-objektet är kärnan i allt vi gör.
- `app.get('/', callback)` – Registrerar en route som svarar på GET-anrop till sökvägen '/'. Callback-funktionen tar emot request-objektet (req) och response-objektet (res).
- `res.send()` – Skickar ett svar till klienten. Express sätter automatiskt rätt Content-Type-header baserat på vad vi skickar.
- `app.listen(port, callback)` – Startar servern och börjar lyssna på det angivna portnumret. Callback-funktionen körs när servern är redo.

Vi kör servern med `node index.js` (eller via `npm start` om vi definierat det scriptet). Sedan kan vi öppna en webbläsare och navigera till `http://localhost:3000` för att se svaret.

Routing i Express

Routing handlar om att definiera hur vår server svarar på olika HTTP-anrop till olika URL-sökvägar. I Express registrerar vi routes med `app.METHOD(path, handler)` där METHOD är den HTTP-metod vi vill lyssna på (get, post, put, patch, delete) och path är URL-mönstret.

Ett mer komplett exempel med flera routes:

```
const express = require('express');
const app = express();

// Middleware för att kunna läsa JSON i request body
app.use(express.json());

// Enkel lista med böcker (ersätter databas tills vidare)
const books = [
  { id: 1, title: 'Clean Code', author: 'Robert C. Martin' },
  { id: 2, title: 'You Don't Know JS', author: 'Kyle Simpson' },
];

// GET /books - hämta alla böcker
app.get('/books', (req, res) => {
  res.json(books);
});

// GET /books/:id - hämta en specifik bok
app.get('/books/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const book = books.find(b => b.id === id);

  if (!book) {
    return res.status(404).json({ message: 'Boken hittades inte' });
  }

  res.json(book);
});

// POST /books - skapa en ny bok
app.post('/books', (req, res) => {
  const { title, author } = req.body;
  const newBook = { id: books.length + 1, title, author };
  books.push(newBook);
  res.status(201).json(newBook);
});

app.listen(3000, () => console.log('Servern körs!'));
```

Notera `res.json()` – det är Express smarta version av `res.send()` för JSON-data. Det sätter automatiskt `Content-Type: application/json` i svaret och serialiserar JavaScript-objektet till JSON-format. Vi kommer att använda det för nästan alla API-svar.

Request och Response-objekten

Request-objektet (req) och response-objektet (res) är de två viktigaste verktygen vi har i en route handler. Express bygger på Node.js inbyggda http-objekt och lägger till en mängd hjälpmetoder och egenskaper.

Request-objektet (req)

req innehåller all information om det inkommande anropet:

- req.params – URL-parametrar definierade med kolon i routemönstret. I /books/:id ger req.params.id oss värdet ur URL:en. Viktigt: värdet är alltid en sträng, så vi måste konvertera med parseInt() eller Number() om vi vill ha ett tal.
- req.query – Query string-parametrar efter frågetecknet i URL:en. En anrop till /books?genre=fiction&sort=title ger req.query.genre === 'fiction' och req.query.sort === 'title'. Används för filtrering, sortering och paginering.
- req.body – Request body, vanligtvis JSON-data som klienten skickar med POST/PUT/PATCH-anrop. Kräver att vi använder express.json()-middleware för att fungera.
- req.headers – HTTP-headrar som klienten skickat med. Vi kommer att använda Authorization-headern när vi implementerar JWT-autentisering senare i kursen.

Här är ett exempel som visar alla tre sätten att skicka data:

```
// Klienten anropar: GET /search/fiction?page=2&limit=10
// Med JSON body: { 'author': 'Tolkien' }

app.get('/search/:genre', (req, res) => {
  console.log(req.params.genre); // 'fiction'
  console.log(req.query.page);   // '2' (sträng!)
  console.log(req.query.limit);  // '10' (sträng!)
  console.log(req.body.author);  // 'Tolkien'
  res.json({ message: 'OK' });
});
```

Response-objektet (res)

res ger oss metoder för att skicka svar tillbaka till klienten:

- res.json(data) – Skickar ett JSON-svar. Sätter Content-Type: application/json automatiskt.
- res.status(kod) – Sätter HTTP-statuskoden. Kan kedjas: res.status(404).json({ message: 'Inte hittad' }).
- res.send(data) – Skickar ett svar. Kan ta emot sträng, Buffer eller objekt.
- res.set(header, value) – Sätter en HTTP-svarshuvud. Exempelvis res.set('X-Custom-Header', 'värde').

En viktig regel: vi kan bara skicka ett svar per request. Om vi anropar res.json() eller res.send() flera gånger i samma handler får vi ett fel. Det är en vanlig bugg för nybörjare, särskilt när man glömmer return efter att man skickat ett felssvar:

```
// FEL - kan skicka två svar!
app.get('/example', (req, res) => {
```

```
if (!req.query.name) {
  res.status(400).json({ message: 'name krävs' });
  // Koden fortsätter köras!
}
res.json({ message: 'OK' }); // Körs även om name saknas!
});

// RÄTT - return efter felsvar
app.get('/example', (req, res) => {
  if (!req.query.name) {
    return res.status(400).json({ message: 'name krävs' });
  }
  res.json({ message: 'OK' });
});
```

Middleware-konceptet

Middleware är ett av de viktigaste koncepten i Express och ett mönster vi kommer att använda genomgående i kursen. En middleware-funktion är en funktion som har tillgång till request-objektet, response-objektet och en next-funktion. Den kan utföra kod, modifiera req och res, avsluta request-response-cykeln eller skicka vidare till nästa funktion i kedjan.

Tänk på det som en pipeline: ett inkommande HTTP-anrop passerar igenom en serie funktioner i ordning. Varje funktion kan välja att hantera anropet (och skicka ett svar) eller skicka det vidare till nästa funktion med next().

Middleware-funktionens signatur

En middleware-funktion har alltid tre parametrar:

```
function minMiddleware(req, res, next) {  
  // Gör något med req eller res  
  console.log('Anrop mottaget:', req.method, req.path);  
  
  // Skicka vidare till nästa middleware eller route handler  
  next();  
}  
  
// Registrera globalt (gäller alla routes)  
app.use(minMiddleware);
```

Inbyggda och populära middleware

Express har ett par inbyggda middleware-funktioner som vi kommer att använda ofta:

- `express.json()` – Tolkar inkommande JSON-data i request body och gör den tillgänglig via `req.body`. Utan denna middleware är `req.body` undefined för POST/PUT-anrop med JSON-data.
- `express.urlencoded({ extended: true })` – Tolkar HTML-formulärdata. Vi behöver den sällan i moderna API:er men det är bra att känna till den.

Utöver de inbyggda finns populära tredjepartsmiddleware. `cors` är ett av de vanligaste – det låter oss konfigurera vilka domäner som får anropa vårt API (Cross-Origin Resource Sharing). Vi installerar det som ett separat paket:

```
npm install cors  
  
const cors = require('cors');  
app.use(cors()); // Tillåt alla domäner (ej för produktion!)
```

Middleware-ordning är viktig

Express kör middleware i den ordning de registreras. Det innebär att vi måste registrera `express.json()` innan de routes som behöver `req.body`, och vi bör registrera loggning och `cors` tidigt i kedjan för att de ska gälla alla requests:

```
const express = require('express');  
const cors = require('cors');
```

```
const app = express();

// 1. Registrera middleware i rätt ordning
app.use(cors());           // CORS-hantering
app.use(express.json());   // Tolka JSON i request body

// 2. Definiera routes EFTER middleware
app.get('/books', (req, res) => {
  // Nu fungerar req.body om klienten skickat JSON
  res.json({ books: [] });
});

// 3. Felhantering-middleware registreras SIST
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: 'Något gick fel' });
});
```

Lägg märke till felhanteringsmiddleware – den har fyra parametrar (err, req, res, next) istället för tre. Express känner igen den som en felhanterare just för att den har fyra parametrar. Den anropas när vi skickar ett fel med next(error) från en annan middleware eller route handler. Vi återkommer till robust felhantering i detalj under vecka 6.

Stateless HTTP och REST API:er

HTTP är ett stateless protokoll – varje anrop är helt oberoende av alla tidigare anrop. Servern har inget minne av att vi skickat ett anrop tidigare. Det här är en fundamental egenskap som påverkar hur vi designar våra API:er.

Tänk på det som att ringa ett kundtjänstnummer varje gång. Varje gång du ringer måste du identifiera dig på nytt och förklara ditt ärende – operatören minns inte ditt förra samtal. Det låter ineffektivt, men det ger enorma fördelar: servrar kan skalas horisontellt (fler servrar kan läggas till utan att dela sessionstillstånd), felsökning blir enklare (vi kan reproducera ett anrop exakt), och systemet är mer robust (om en server kraschar kan en annan ta över utan förlust av tillstånd).

REST – Representational State Transfer

REST är en arkitekturstil för att designa nätverkstjänster. Det är inte ett protokoll eller en standard – det är en uppsättning principer som, om de följs, resulterar i väldesignade och förutsägbara API:er. REST-principerna formulerades av Roy Fielding i hans doktorsavhandling år 2000.

De viktigaste REST-principerna för oss är:

- Resursorientering – Vi tänker på data som resurser med unika identifierare (URL:er). `/books/42` är resursen 'boken med id 42'. URL:er ska vara substantiv, inte verb – `/books`, inte `/getBooks`.
- HTTP-metoderna anger åtgärden – GET för att hämta, POST för att skapa, PUT/PATCH för att uppdatera, DELETE för att ta bort. URL:en identifierar resursen, metoden anger vad vi vill göra med den.
- Stateless – Servern lagrar inget tillstånd per klient. All information som behövs för att behandla ett anrop måste skickas med i anropet.
- Enhetligt gränssnitt – API:et ska vara konsekvent och förutsägbart. Om `/books/1` returnerar en bok, ska `/users/1` returnera en användare på ett liknande sätt.

URL-design för REST API:er

Bra URL-design är en konstform. Här är principerna vi ska följa:

- Använd substantiv i plural: `/books`, `/users`, `/orders` – inte `/book`, `/getUser`, `/createOrder`.
- Använd hierarki för relaterade resurser: `/users/42/orders` ger alla ordrar för användare 42.
- Versionshantera API:et: `/api/v1/books` gör det möjligt att introducera brytande ändringar i v2 utan att bryta befintliga klienter.
- Använd kebab-case för flerordssökvägar: `/book-reviews` inte `/bookReviews` eller `/book_reviews`.

En väldesignad REST API-struktur för en boktjänst kan se ut så här:

GET	<code>/api/v1/books</code>	- Hämta alla böcker
POST	<code>/api/v1/books</code>	- Skapa en ny bok
GET	<code>/api/v1/books/:id</code>	- Hämta en specifik bok
PUT	<code>/api/v1/books/:id</code>	- Ersätt en hel bok
PATCH	<code>/api/v1/books/:id</code>	- Uppdatera delar av en bok
DELETE	<code>/api/v1/books/:id</code>	- Ta bort en bok

GET /api/v1/books/:id/reviews - Hämta recensioner för en bok

Projektstruktur och organisation

En välorganiserad mappstruktur är avgörande för underhållbarhet. Vi börjar enkelt vecka 1, men vi ska direkt tänka på hur projektet kan växa. En mappstruktur vi kommer att bygga på under kursen ser ut så här:

```
mitt-api/  
  src/  
    routes/          # Route-definitioner  
      books.js  
      users.js  
    controllers/     # Route handler-logik (vecka 2+)  
      booksController.js  
    middleware/      # Egna middleware-funktioner  
      auth.js  
    models/          # Datamodeller (vecka 4+)  
      Book.js  
    app.js            # Express-applikation  
    server.js        # Serverstartfil  
  .env               # Miljövariabler (läggs i .gitignore!)  
  .gitignore  
  package.json
```

Anledningen till att vi separerar app.js och server.js är att det gör det enklare att testa API:et. I tester vill vi kunna importera Express-applikationen utan att den startar en server (eftersom testramverket sköter det). Separationen gör det enkelt:

```
// app.js - Express-applikationen  
const express = require('express');  
const booksRouter = require('./routes/books');  
  
const app = express();  
app.use(express.json());  
app.use('/api/v1/books', booksRouter);  
  
module.exports = app; // Exportera, starta inte servern här  
  
// server.js - Startar servern  
const app = require('./app');  
const port = process.env.PORT || 3000;  
  
app.listen(port, () => {  
  console.log(`Servern lyssnar på port ${port}`);  
});
```

Router-moduler

Express Router låter oss bryta ut routes för ett specifikt område i en separat fil. Det håller koden organiserad och gör det enkelt att hitta relevant kod när projektet växer:

```
// routes/books.js  
const express = require('express');
```

```
const router = express.Router();

// Notera: sökvägen här är relativ till där routern monteras
// Om den monteras på /api/v1/books blir GET / -> GET /api/v1/books
router.get('/', (req, res) => {
  res.json({ books: [] });
});

router.get('/:id', (req, res) => {
  res.json({ id: req.params.id });
});

module.exports = router;
```

Verktyg för API-testning

När vi bygger ett API behöver vi ett sätt att testa det manuellt – att skicka HTTP-anrop och se svaren. Webbbläsaren klarar bara av att skicka GET-anrop, så vi behöver ett dedikerat verktyg för POST, PUT och DELETE.

Thunder Client

Thunder Client är ett VS Code-tillägg som låter oss skicka HTTP-anrop direkt från editorn. Det är enkelt att komma igång med, kräver inget separat program och är perfekt för enklare API-testning. Vi rekommenderar det som startpunkt.

Vi installerar det via VS Code Extensions (Ctrl+Shift+X) och söker på 'Thunder Client'. Sedan kan vi:

- Välja HTTP-metod (GET, POST, PUT, DELETE)
- Skriva in URL:en
- Lägga till headers och request body
- Skicka anropet och se svaret direkt i VS Code

Postman

Postman är det mest använda verktyget i industrin för API-testning och -dokumentation och det som jag främst har använt. Det har ett rikt utbud av funktioner: samlingar, miljövariabler, automatiserade tester och teamdelning. Jag rekommenderar att ni skapar ett Postman-konto och lär er grunderna, då det är ett verktyg ni med stor sannolikhet kommer att stöta på i arbetslivet.

Det finns ingen anledning att välja det ena framför det andra i kursen – använd det verktyg som känns bekvämt. Många använder Thunder Client för snabb testning under utveckling och Postman för mer strukturerad testning och dokumentation.

Checkpoint: Förberedelse inför kursvecka 1

Vecka 1 är en specialvecka utan föregående boiler room. Det innebär att checkpoint är vår gemensamma startpunkt för kursen. För att vi ska kunna använda den tiden väl är det viktigt att ni har läst igenom detta studiematerial och förberett er miljö.

Kontrollera att ni har följande klart inför workshopen:

1. Node.js installerat – kontrollera med: `node --version` (minst v18)
2. npm fungerar – kontrollera med: `npm --version`

Ta med er dessa frågor och reflektioner till workshopen: Hur skiljer sig att bygga en backend från att bygga en frontend? Vad tror ni är de största utmaningarna med att designa ett API som andra ska använda? Vilka av era erfarenheter från React-kursen tror ni är mest relevanta för backendutveckling?

Om ni stöter på problem med installationen, notera exakt vilket kommando som misslyckas och vilket felmeddelande ni får. Vi löser det tillsammans under checkpointen.

Sammanfattning

Under kursvecka 1 har vi lagt grunden för hela backendutvecklingskursen. Vi har förstått vad backend är och vilken roll det spelar i ett fullstacksystem: det är den centrala koordinatören som tar emot anrop, bearbetar data, kommunicerar med databaser och returnerar strukturerade svar till klienterna.

Vi har lärt oss om klient-server-modellen och HTTP-protokollets stateless natur. Varje HTTP-anrop är oberoende – servern minns ingenting från tidigare anrop. Det kräver att all nödvändig information skickas med i varje anrop, och det är grunden för hur vi sedan implementerar autentisering med JWT. HTTP-metoderna GET, POST, PUT, PATCH och DELETE har specifik semantik som vi alltid ska respektera, och HTTP-statuskoder kommunicerar resultatet av ett anrop på ett standardiserat sätt.

Node.js ger oss möjligheten att köra JavaScript på servern. Dess eventloopbaserade arkitektur och non-blocking I/O gör det möjligt att hantera många simultana anslutningar effektivt – men det kräver att vi skriver asynkron kod och aldrig blockerar eventloopen med CPU-intensiva operationer. npm och package.json ger oss ekosystemet av paket och ett sätt att hantera projektets beroenden.

Express är vårt webbramverk för Node.js. Det ger oss ett bekvämt API för routing (app.get, app.post, etc.), request-objektet (req) med params, query och body, response-objektet (res) med json() och status(), och middleware-konceptet som låter oss bygga en pipeline av funktioner som bearbetar inkommande anrop.

REST-principerna ger oss ett väldesignat API: resursorienterade URL:er med substantiv i plural, HTTP-metoder som anger åtgärden, stateless kommunikation och ett konsekvent gränssnitt. En bra URL-design och rätt statuskoder är fundamenten för ett API som andra utvecklare kan arbeta med utan frustration.

Nästa vecka bygger vi vidare på detta genom att gå djupare in i REST API-design, lära oss att strukturera projektet med separata route-filer och controllers, och utforska middleware-kedjan i mer detalj. Boiler room-projektet startar också – mer information om det i BoilerRoomProjekt.pdf.

Resurser för fördjupning

Följande resurser rekommenderas för de som vill fördjupa sina kunskaper inom Node.js och Express.

Officiell dokumentation

Node.js officiella dokumentation – fullständig referens för Node.js API:er och inbyggda moduler:

<https://nodejs.org/en/docs>

Express officiella dokumentation – guide och API-referens för Express:

<https://expressjs.com>

MDN Web Docs: HTTP – djupdykning i HTTP-protokollet, metoder och statuskoder:

<https://developer.mozilla.org/en-US/docs/Web/HTTP>

Rekommenderade guider

The Odin Project – "NodeJS"-sektionen innehåller välstrukturerade övningar och förklaringar om Node.js och Express som kompletterar kursmaterialet väl. Projekten i Odin Project är utmärکتa för extra träning.

"Node.js Best Practices" – ett community-drivet GitHub-repository med rekommenderade mönster och anti-patterns för Node.js-utveckling. Relevant nu och ännu mer relevant när vi kommit längre in i kursen:

<https://github.com/goldbergonyi/nodebestpractices>

Videomaterial

"Node.js Crash Course" av Traversy Media på YouTube ger en bra introduktion till Node.js-grunderna. Den täcker mycket av vad vi går igenom i detta studiematerial och är ett bra komplement för de som föredrar att lära sig via video.

"REST API Design Best Practices" av freeCodeCamp på YouTube är en genomgång av REST-principer och hur man designar välstrukturerade API:er. Rekommenderas starkt inför vecka 2 då vi fördjupar oss i API-design.

Studiematerial

REST API-design, routing och middleware

Kursvecka 2

Introduktion

Välkommen till kursvecka 2! Under förra veckan lade vi grunden: vi förstår vad backend är, hur Node.js fungerar med sin eventloop och non-blocking I/O, och vi har skapat vår första Express-server med grundläggande routes. Det är en solid bas att stå på, men en enda fil med all kod i – app.js – räcker inte långt när projektet börjar växa.

Den här veckan tar vi nästa steg och fokuserar på tre saker som hänger tätt ihop: hur vi designar ett välstrukturerat REST API, hur vi organiserar koden i separata filer och moduler, och hur vi använder middleware på ett genomtänkt sätt. Det är den här typen av struktur och disciplin som skiljer ett hobbyexperiment från ett produktionsklart system.

Vi börjar med att fördjupa oss i REST API-design – inte bara principerna vi nämnde förra veckan, utan de praktiska beslut vi behöver fatta varje dag: hur ska URL:erna se ut, vilken HTTP-metod ska vi använda, vilket statuskod är rätt att returnera? Sedan tittar vi på hur vi strukturerar projektet med separata route-filer och controllers. Slutligen utforskar vi middleware-kedjan på djupet och lägger till praktisk loggning med morgan.

Det är också den här veckan som boiler room-projektet startar på allvar. Det projektet kommer att löpa under hela kursen och är den plats där ni tillämpar det ni lär er varje vecka – mer information om det finns i BoilerRoomProjekt.pdf.

Repetition från vecka 1

Innan vi går vidare är det värt att snabbt repetera de viktigaste koncepten från förra veckan, eftersom vi bygger direkt vidare på dem.

Express – grundstrukturen

Vi skapar en Express-applikation med `express()`, registrerar routes med `app.METHOD(path, handler)`, och startar servern med `app.listen()`. Request-objektet ger oss information om inkommande anrop via `req.params`, `req.query` och `req.body`, och vi skickar svar med `res.json()` och `res.status()`.

En minimal men korrekt Express-server ser ut så här:

```
const express = require('express');
const app = express();

app.use(express.json()); // Tolka JSON i request body

app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});

app.listen(3000, () => console.log('Server igång på port 3000'));
```

HTTP-metoder och statuskoder

HTTP-metoderna GET, POST, PUT, PATCH och DELETE har specifik semantik: GET hämtar, POST skapar, PUT ersätter hela resursen, PATCH uppdaterar delar av den, och DELETE tar bort. Statuskoderna kommunicerar resultatet – 2xx för framgång, 4xx för klientfel och 5xx för serverfel.

Vi återkommer till statuskoder mer utförligt senare i det här materialet. Det är ett ämne som förtjänar extra uppmärksamhet eftersom fel statuskod är ett av de vanligaste misstagen i API-design.

REST API-design i praktiken

Vi nämnde REST-principerna kortfattat förra veckan. Nu ska vi gå på djupet och titta på de konkreta beslut vi fattar när vi designar ett API. Bra API-design handlar om att skapa ett gränssnitt som är intuitivt, konsekvent och förutsägbart – ett API som en annan utvecklare kan förstå utan att behöva läsa hela dokumentationen.

Resurser och URL-design

I REST tänker vi på allt som resurser. En resurs är ett substantiv – något vi kan identifiera, hämta, skapa, uppdatera eller ta bort. URL:en är resursens adress, och HTTP-metoden talar om vad vi vill göra med den.

Principerna för bra URL-design:

- Använd substantiv i plural – /users, /orders, /products. Aldrig verb som /getUsers, /createOrder eller /deleteProduct. Verbet är redan HTTP-metoden.
- Använd kebab-case för flerordssökvägar – /blog-posts, /user-profiles, /order-items. Undvik camelCase (/blogPosts) och underscore (/blog_posts) i URL:er.
- Använd hierarki för att uttrycka relationer – /users/42/orders ger alla ordrar för användare 42. /users/42/orders/7 ger specifikt order 7 för den användaren.
- Versionshantera API:et från dag ett – /api/v1/users. Det gör det möjligt att introducera brytande ändringar i /api/v2/ utan att befintliga klienter slutar fungera.
- Håll URL:erna platta – undvik djupa hierarkier som /users/42/orders/7/items/3/details. Om hierarkin känns djup är det ofta ett tecken på att resurserna bör omdesignas.

Här är ett väldesignat API för ett e-handelsystem:

GET	/api/v1/products	- Hämta alla produkter
POST	/api/v1/products	- Skapa en ny produkt
GET	/api/v1/products/:id	- Hämta en specifik produkt
PUT	/api/v1/products/:id	- Ersätt produkten helt
PATCH	/api/v1/products/:id	- Uppdatera delar av produkten
DELETE	/api/v1/products/:id	- Ta bort produkten
GET	/api/v1/users/:id/orders	- Hämta en användares ordrar
POST	/api/v1/users/:id/orders	- Skapa en ny order för användaren
GET	/api/v1/users/:id/orders/:oid	- Hämta en specifik order

Filtrering, sortering och paginering

När vi hämtar listor av resurser behöver vi ofta möjligheten att filtrera, sortera och paginera resultaten. I REST görs det med query string-parametrar – aldrig i URL-sökvägen.

Konventioner vi ska följa:

- Filtrering: /api/v1/products?category=electronics&inStock=true – filtrera på ett eller flera fält.
- Sortering: /api/v1/products?sort=price&order=asc – ange vilket fält och riktning.
- Paginering: /api/v1/products?page=2&limit=20 – ange sida och antal per sida. Returnera alltid information om totalt antal sidor i svaret.

Så här kan en route se ut som hanterar alla tre:

```
app.get('/api/v1/products', (req, res) => {
  const { category, inStock, sort, order, page, limit } = req.query;

  // Konvertera och sätt standardvärden
  const pageNum = parseInt(page) || 1;
  const limitNum = parseInt(limit) || 20;
  const sortOrder = order === 'desc' ? -1 : 1;

  // Tillämpas mot databasen senare i kursen
  // För nu returnerar vi bara ett exempelsvar
  res.json({
    data: [],
    meta: {
      page: pageNum,
      limit: limitNum,
      total: 0,
      totalPages: 0
    }
  });
});
```

Notera svarformatet: vi returnerar data i ett data-fält och metadata i ett meta-fält. Det är ett mönster vi ska använda konsekvent i hela kursen – det gör det enkelt för klienter att hantera svaren och ger oss plats att lägga till ytterligare information utan att bryta det befintliga kontraktet.

HTTP-statuskoder – välja rätt kod

HTTP-statuskoder är ett av de områden där nybörjare och erfarna utvecklare skiljer sig mest åt. Det är lätt att alltid returnera 200 OK och stoppa in felmeddelandet i svarstexten – men det bryter mot HTTP-standarden och gör API:et svårare att använda och felsöka.

En grundprincip: statuskoden ska återspegla resultatet av anropet på protokollnivå, inte på applikationsnivå. Om klienten skickade felaktig data är det ett klientfel (4xx), oavsett vad vi skriver i svarstexten. Om något gick fel på servern är det ett serverfel (5xx).

2xx – Framgång

- 200 OK – Det generiska framgångssvaret. Används för GET, PUT och PATCH när allt gick bra och vi returnerar data.
- 201 Created – En ny resurs skapades (typiskt svar på en POST). Vi bör också inkludera en Location-header som pekar på den nya resursen: Location: /api/v1/users/42.
- 204 No Content – Operationen lyckades men det finns ingenting att returnera. Används ofta för DELETE eller för PATCH-operationer som inte returnerar uppdaterad data.

Exempel – korrekt hantering av POST och DELETE:

```
// POST /api/v1/users - Skapa användare
app.post('/api/v1/users', (req, res) => {
  const newUser = { id: 42, name: req.body.name };
  // Spara till databas...
  res
    .status(201)
    .set('Location', `/api/v1/users/${newUser.id}`)
    .json(newUser);
});

// DELETE /api/v1/users/:id - Ta bort användare
app.delete('/api/v1/users/:id', (req, res) => {
  // Ta bort från databas...
  res.status(204).send(); // Inget att returnera
});
```

4xx – Klientfel

4xx-koder används när klienten gjort något fel – skickat ogiltig data, anropar en resurs den inte har tillgång till, eller försöker nå något som inte finns.

- 400 Bad Request – Anropet är felformat. Klienten skickade data som vi inte kan förstå eller använda, exempelvis ett ogiltigt JSON-format eller ett saknat obligatoriskt fält.
- 401 Unauthorized – Klienten är inte autentiserad. Trots namnet betyder det egentligen 'inte inloggad'. Används när en token saknas eller är ogiltig.
- 403 Forbidden – Klienten är autentiserad men saknar behörighet. En inloggad användare som försöker ändra en annan användares data ska få 403, inte 401.

- 404 Not Found – Resursen finns inte. Används när vi letar efter en specifik resurs (t.ex. /users/9999) och den inte hittas i databasen.
- 409 Conflict – Anropet är i konflikt med nuvarande tillstånd. Vanligt vid POST för att skapa en resurs som redan finns, exempelvis en e-postadress som redan är registrerad.
- 422 Unprocessable Entity – Formatet är korrekt men innehållet kan inte behandlas. Används för valideringsfel när data är semantiskt felaktig, exempelvis ett ogiltigt datum eller ett negativt pris.

5xx – Serverfel

- 500 Internal Server Error – Ett oväntat fel inträffade på servern. Det är vår kod som gick sönder, inte klientens anrop. Vi ska aldrig returnera 500 med ett felmeddelande som avslöjar intern information som stack traces eller databasstrukturer.
- 503 Service Unavailable – Servern kan inte hantera anropet just nu, exempelvis för att en beroende tjänst (databas, extern API) är nere. Kan inkludera en Retry-After-header.

Konsekvent felformat

Oavsett vilken 4xx-kod vi returnerar ska felsvaret alltid ha samma struktur. Det gör det enkelt för klienter att hantera fel konsekvent:

```
// Standardiserat fel svar - alltid samma struktur
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "Ingen användare med id 42 hittades",
    "status": 404
  }
}
```

Notera att vi inkluderar en maskinläsbar kod (USER_NOT_FOUND) utöver det mänskligt läsbara meddelandet. Det gör det möjligt för frontendutvecklare att skriva kod som reagerar på specifika feltyper utan att beroende på att felmeddelandets text aldrig förändras. Vi återkommer till systematisk felhantering i kursvecka 6 – men vi börjar tänka i de banorna redan nu.

Strukturera Express-projektet

En av de viktigaste färdigheterna i backendutveckling är att organisera kod på ett sätt som gör den underhållbar när projektet växer. En enda fil med hundratals rader kod – med routes, logik och dataåtkomst huller om buller – är ett recept på teknisk skuld.

Vi tillämpar principen Separation of Concerns: varje fil och funktion ska ha ett tydligt, avgränsat ansvar. I ett Express-projekt innebär det att vi separerar routing, affärslogik och dataåtkomst i olika lager.

Mapstruktur

Vår rekommenderade mappstruktur för kursen ser ut så här:

```
mitt-api/  
  src/  
    routes/  
      index.js          # Samlad router - monterar alla subrouters  
      users.js          # Routes för /users  
      products.js       # Routes för /products  
    controllers/  
      usersController.js # Logik för användar-endpoints  
      productsController.js # Logik för produkt-endpoints  
    middleware/  
      logger.js         # Egen loggningmiddleware  
      errorHandler.js   # Centraliserad felhantering  
    app.js              # Express-app, middleware och routes  
    server.js           # Startar HTTP-servern  
  .env                  # Miljövariabler (ALDRIG i Git!)  
  .gitignore  
  package.json
```

Routes – var och hur

En route-fil definierar vilka HTTP-anrop vi lyssnar på och kopplar dem till rätt controller-funktion. Route-filen ska inte innehålla affärslogik – den ska bara vara en karta över URL:er och HTTP-metoder.

Så här ser en välstrukturerad route-fil ut:

```
// src/routes/users.js  
const express = require('express');  
const router = express.Router();  
const usersController = require('../controllers/usersController');  
  
// Samlingssökväg: /api/v1/users  
router.get('/', usersController.getAllUsers);  
router.post('/', usersController.createUser);  
  
// Resurssökväg: /api/v1/users/:id  
router.get('/:id', usersController.getUserById);  
router.put('/:id', usersController.updateUser);  
router.delete('/:id', usersController.deleteUser);
```



```
module.exports = router;
```

Notera att vi monterar routern utan prefixet `/api/v1/users` – det gör vi i `app.js`. Det innebär att sökvägen `'/'` i routern faktiskt motsvarar `/api/v1/users`. Det håller route-filerna rena och gör det enkelt att byta prefix om vi vill.

Controllers – logiken

En controller är en samling funktioner som innehåller logiken för en specifik resurs. Varje funktion tar emot `req` och `res` och hanterar ett specifikt anrop. Just nu returnerar de hårdkodad data – när vi lägger till databasen i vecka 4 och 5 är det här vi pratar med den.

```
// src/controllers/usersController.js

// Simulerad databas (ersätts med MongoDB i vecka 4)
let users = [
  { id: 1, name: 'Anna Andersson', email: 'anna@example.com' },
  { id: 2, name: 'Erik Eriksson', email: 'erik@example.com' },
];

// GET /api/v1/users
const getAllUsers = (req, res) => {
  res.json({ data: users, meta: { total: users.length } });
};

// GET /api/v1/users/:id
const getUserById = (req, res) => {
  const id = parseInt(req.params.id);
  const user = users.find(u => u.id === id);

  if (!user) {
    return res.status(404).json({
      error: { code: 'USER_NOT_FOUND', message: `Användare ${id} hittades inte`,
status: 404 }
    });
  }
  res.json({ data: user });
};

// POST /api/v1/users
const createUser = (req, res) => {
  const { name, email } = req.body;

  if (!name || !email) {
    return res.status(400).json({
      error: { code: 'MISSING_FIELDS', message: 'name och email krävs', status:
400 }
    });
  }

  const newUser = { id: users.length + 1, name, email };
  users.push(newUser);
```

```

    res.status(201).set('Location', `/api/v1/users/${newUser.id}`).json({ data:
newUser });
};

// PUT /api/v1/users/:id
const updateUser = (req, res) => {
  const id    = parseInt(req.params.id);
  const index = users.findIndex(u => u.id === id);

  if (index === -1) {
    return res.status(404).json({
      error: { code: 'USER_NOT_FOUND', message: `Användare ${id} hittades inte`,
status: 404 }
    });
  }

  users[index] = { id, ...req.body };
  res.json({ data: users[index] });
};

// DELETE /api/v1/users/:id
const deleteUser = (req, res) => {
  const id    = parseInt(req.params.id);
  const index = users.findIndex(u => u.id === id);

  if (index === -1) {
    return res.status(404).json({
      error: { code: 'USER_NOT_FOUND', message: `Användare ${id} hittades inte`,
status: 404 }
    });
  }

  users.splice(index, 1);
  res.status(204).send();
};

module.exports = { getAllUsers, getUserById, createUser, updateUser,
deleteUser };

```

app.js – allt hänger ihop

app.js är navet i applikationen – det är här vi konfigurerar Express, registrerar middleware och monterar route-modulerna. Filen exporterar app-objektet men startar inte servern (det gör server.js).

```

// src/app.js
const express = require('express');
const cors    = require('cors');

const usersRouter    = require('./routes/users');
const productsRouter = require('./routes/products');

```

```
const app = express();

// === Globala middleware ===
app.use(cors());
app.use(express.json());

// === Routes ===
app.use('/api/v1/users', usersRouter);
app.use('/api/v1/products', productsRouter);

// === 404-hantering för okända routes ===
app.use((req, res) => {
  res.status(404).json({
    error: { code: 'NOT_FOUND', message: `Sökvägen ${req.path} finns inte`,
    status: 404 }
  });
});

module.exports = app;

// src/server.js
const app = require('./app');
const port = process.env.PORT || 3000;

app.listen(port, () => {
  console.log(`Server lyssnar på http://localhost:${port}`);
});
```

Middleware i praktiken

Vi introducerade middleware-konceptet förra veckan. Nu ska vi använda det på riktigt, med verkliga middleware-bibliotek och egna middleware-funktioner. Middleware är kärnan i hur Express fungerar – utan det är Express bara ett tunt lager ovanpå Node.js inbyggda http-modul.

Kom ihåg pipelinemodellen: ett inkommande HTTP-anrop passerar igenom en serie middleware-funktioner i ordning. Varje funktion anropar `next()` för att skicka anropet vidare, eller skickar ett svar för att avsluta kedjan. Ordningen spelar stor roll.

cors – Cross-Origin Resource Sharing

CORS är ett webbläsarsäkerhetsprotokoll som styr vilka domäner som får göra HTTP-anrop till vår server. Som standard blockerar webbläsare alla anrop till en annan domän än den sidan laddades från – det kallas Same-Origin Policy.

När vår React-frontend på `http://localhost:5173` försöker anropa vår Express-backend på `http://localhost:3000` är det tekniskt sett en cross-origin request – de är på olika portar, vilket räknas som olika origins. Utan CORS-konfiguration kommer webbläsaren att blockera anropet.

Vi installerar cors-paketet och konfigurerar det i `app.js`:

```
npm install cors

const cors = require('cors');

// Tillåt alla origins (bra för utveckling, INTE för produktion)
app.use(cors());

// Konfigurerad CORS för produktion
app.use(cors({
  origin: 'https://min-frontend.com', // Tillåt bara vår frontend
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));
```

En viktig distinktion: CORS är ett webbläsarskydd, inte ett serverskydd. Postman och curl skickar anrop utan CORS-kontroller oavsett hur vi konfigurerat servern. CORS skyddar slutanvändare från att skadliga webbsidor i deras webbläsare anropar vår server i smyg.

morgan – HTTP-loggning

morgan är ett populärt loggningsbibliotek för Express som loggar inkommande HTTP-anrop på ett standardiserat sätt. Det är ett av de första paketen vi lägger till i ett nytt projekt – loggning är helt nödvändigt för att förstå vad som händer i en körande applikation.

```
npm install morgan

const morgan = require('morgan');

// 'dev' ger färgkodad kompakt output - bra för utveckling
```

```
app.use(morgan('dev'));

// Output i terminalen ser ut ungefär så här:
// GET /api/v1/users 200 12.345 ms - 124
// POST /api/v1/users 201 8.123 ms - 67
// GET /api/v1/users/9999 404 3.456 ms - 89
```

morgan har flera inbyggda format: dev ger kortfattad färgad output i terminalen, combined ger Apache-kompatibelt format för produktionsloggar, och tiny ger minimal information. Vi rekommenderar dev under kursen.

Vi placerar morgan-anropet tidigt i middleware-kedjan i app.js, EFTER cors() men FÖRE routes – på så sätt loggas alla anrop, inklusive de som resulterar i CORS-fel eller 404.

Egna middleware-funktioner

Utöver tredjepartsbibliotek skriver vi egna middleware-funktioner för specifika behov i vår applikation. Det är ett mönster vi kommer att använda mycket – autentisering, auktorisering, validering och felhantering implementeras alla som middleware.

En middleware-funktion har alltid signaturen (req, res, next):

```
// src/middleware/logger.js
// En enkel loggnings-middleware som loggar varje inkommande anrop

const logger = (req, res, next) => {
  const timestamp = new Date().toISOString();
  const method    = req.method;
  const path      = req.path;

  console.log(`[${timestamp}] ${method} ${path}`);

  // Vi måste anropa next() för att kedjan ska fortsätta!
  // Om vi inte gör det hänger anropet och klienten får aldrig svar.
  next();
};

module.exports = logger;
```

En vanligare och mer avancerad pattern är middleware som validerar headers:

```
// src/middleware/validateContentType.js
// Kontrollerar att POST/PUT/PATCH skickar JSON

const validateContentType = (req, res, next) => {
  const methodsWithBody = ['POST', 'PUT', 'PATCH'];

  if (methodsWithBody.includes(req.method)) {
    const contentType = req.headers['content-type'] || '';

    if (!contentType.includes('application/json')) {
      return res.status(415).json({
        error: {

```

```

        code:    'UNSUPPORTED_MEDIA_TYPE',
        message: 'Content-Type måste vara application/json',
        status:  415
      }
    });
  }
}

next();
};

module.exports = validateContentType;

```

Vi kan applicera middleware på hela applikationen, på en router, eller på enskilda routes:

```

// Globalt - gäller alla routes
app.use(logger);
app.use(validateContentType);

// Per router - gäller alla routes i routern
router.use(validateContentType);

// Per route - gäller bara den specifika routen
router.post('/', validateContentType, usersController.createUser);

```

Felhanteringsmiddleware

Express har ett speciellt format för felhanteringsmiddleware: funktionen ska ha fyra parametrar (err, req, res, next). Express identifierar en middleware som en felhanterare just för att den har fyra parametrar och anropar den automatiskt när next(error) anropas från någon annan middleware eller route handler.

Vi placerar felhanteringsmiddleware **SIST** i app.js, efter alla routes:

```

// src/middleware/errorHandler.js
const errorHandler = (err, req, res, next) => {
  // Logga alltid felet på servern (men aldrig till klienten!)
  console.error('[ERROR]', err.message);
  console.error(err.stack);

  // Skicka ett generellt felmeddelande till klienten
  // Vi avslöjar aldrig stack traces eller intern information
  res.status(500).json({
    error: {
      code:    'INTERNAL_SERVER_ERROR',
      message: 'Ett oväntat fel inträffade',
      status:  500
    }
  });
};

module.exports = errorHandler;

```

I app.js importerar och registrerar vi felhanteraren sist:

```
const errorHandler = require('./middleware/errorHandler');

// ... routes ...

// Felhanteraren MÅSTE vara sist
app.use(errorHandler);

module.exports = app;
```

Vi triggar felhanteraren från en route handler genom att anropa next(error):

```
const getUser = (req, res, next) => {
  try {
    // Kod som potentiellt kastar ett fel
    const user = findUser(req.params.id); // Kan kasta Error
    res.json({ data: user });
  } catch (error) {
    // Skicka felet till felhanteringsmiddleware
    next(error);
  }
};
```

Komplett app.js med all middleware

Nu sätter vi ihop allt. Här är en komplett app.js som visar den rekommenderade ordningen för middleware-registrering – ordningen är inte godtycklig, den spelar stor roll för hur applikationen beter sig:

```
// src/app.js
const express      = require('express');
const cors          = require('cors');
const morgan        = require('morgan');

const usersRouter   = require('./routes/users');
const productsRouter = require('./routes/products');
const logger         = require('./middleware/logger');
const errorHandler  = require('./middleware/errorHandler');

const app = express();

// =====
// 1. Säkerhets- och protokoll-middleware (FÖRST)
// =====
app.use(cors({
  origin: process.env.CORS_ORIGIN || '*',
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
})));

// =====
// 2. Request-parsning
// =====
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

// =====
// 3. Loggning
// =====
app.use(morgan('dev'));
app.use(logger);

// =====
// 4. Routes
// =====
app.use('/api/v1/users', usersRouter);
app.use('/api/v1/products', productsRouter);

// =====
// 5. 404-hantering för okända routes
// =====
app.use((req, res) => {
  res.status(404).json({
    error: {
      code: 'NOT_FOUND',
      message: `Sökvägen ${req.path} finns inte`,
      status: 404
    }
  });
});
```



```
    }  
  });  
});  
  
// =====  
// 6. Felhantering (SIST - alltid)  
// =====  
app.use(errorHandler);  
  
module.exports = app;
```

Ordningen är kritisk och logisk: säkerhetsmiddleware ska köras innan vi tolkar request body (annars kan en angripare skicka en enorm JSON-payload som crashar servern), loggning ska köras innan routes (annars loggas inte anrop som avvisas av tidigare middleware), och felhanteraren ska vara sist (annars fångar den inte fel från routes).

API-versionshantering

Versionshantering av ett API handlar om att hantera förändring utan att bryta befintliga klienter. Utan versionshantering sitter vi fast – vi kan inte ändra svarsformatet, ta bort fält eller ändra beteende utan att riskera att krascha alla appar och webbsidor som anropar vårt API.

URL-baserad versionshantering

Den vanligaste och enklaste strategin är att inkludera versionen i URL:en: `/api/v1/users`. Det är tydligt, enkelt att förstå, och fungerar med alla HTTP-klienter utan specialkonfiguration. Det är den strategi vi använder i kursen.

Så här monterar vi flera versioner parallellt i `app.js`:

```
const v1UsersRouter = require('./routes/v1/users');
const v2UsersRouter = require('./routes/v2/users');

// v1 och v2 körs parallellt
app.use('/api/v1/users', v1UsersRouter);
app.use('/api/v2/users', v2UsersRouter);
```

I praktiken behöver vi inte versionshantera från dag ett i ett nytt projekt. Men vi ska alltid ha `/api/v1/` i URL:erna från start – det kostar ingenting och vi sparar oss enormt mycket arbete den dag vi vill göra en brytande ändring. Att lägga till versionsprefix i efterhand kräver att alla klienter uppdateras.

Vad räknas som en brytande ändring?

En brytande ändring (breaking change) är en förändring som kan orsaka att befintliga klienter slutar fungera. Det är viktigt att vi vet vad som är en brytande ändring och vad som inte är det:

- Brytande ändringar (kräver ny version): ta bort ett fält i svaret, byta namn på ett fält, ändra datatypen på ett fält, ändra beteendet på en befintlig endpoint, ta bort en endpoint.
- Icke-brytande ändringar (kan göras i befintlig version): lägga till ett nytt fält i svaret, lägga till en ny endpoint, lägga till en ny optional query-parameter.

En bra tumregel: om en klient som inte vet om förändringen kan fortsätta fungera utan modifikation, är det en icke-brytande ändring.

Testa API:et med Postman och Thunder Client

Vi behöver ett sätt att manuellt testa API:et under utveckling. Webbbläsarens adressfält klarar bara GET-anrop, så vi behöver ett dedikerat verktyg för att skicka POST, PUT, PATCH och DELETE med rätt headers och request body.

Grundläggande testning

Oavsett om vi använder Postman eller Thunder Client är arbetsflödet detsamma. Här är de viktigaste sakerna att testa för varje endpoint:

1. Happy path – skicka korrekt data och kontrollera att vi får rätt statuskod och svarformat.
2. Saknade fält – utelämna obligatoriska fält och kontrollera att vi får 400 Bad Request.
3. Icke-existerande resurser – hämta/uppdatera/ta bort en resurs med ett id som inte finns och kontrollera att vi får 404.
4. Felaktiga Content-Type – skicka POST utan Content-Type: application/json och se hur servern reagerar.

Postman Collections

En av Postmans starkaste funktioner är Collections – samlingar av sparade anrop som representerar hela API:et. Vi rekommenderar starkt att skapa en Collection för ert projekt tidigt. Det tjänar två syften: ni kan enkelt köra alla testanrop om och om igen under utvecklingen, och det fungerar som levande dokumentation för API:et.

En välstrukturerad Collection för vårt API kan se ut så här:

- Users – GET alla, GET enskild, POST, PUT, DELETE
- Products – GET alla (med filter/sort), GET enskild, POST, PUT, DELETE
- Error cases – 404-anrop, anrop med saknade fält, anrop med felaktig Content-Type

Miljövariabler i Postman

Postman stödjer miljövariabler – variabler som vi kan byta ut beroende på om vi testar mot vår lokala server eller mot en produktionsserver. Det är ett mönster vi ska använda från dag ett.

Skapa en miljö i Postman (Environment) med variablerna:

```
BASE_URL = http://localhost:3000 (för lokal development)
BASE_URL = https://mitt-api.onrender.com (för produktion)
```

Sedan refererar vi till variabeln i URL-fältet:

```
{{BASE_URL}}/api/v1/users
```

Nu kan vi byta mellan lokal och produktionsmiljö med ett klick, utan att behöva manuellt ändra URL:erna i alla sparade anrop.

Checkpoint: Förberedelse inför kursvecka 2

Inför onsdagens workshop ska ni ha läst igenom det här studiematerialet och helst provat att implementera projektstrukturen och middleware-kedjan i ett eget testprojekt. Det är inte ett krav att allt fungerar perfekt – men ju mer vi har kodat på förhand, desto mer tid kan vi lägga på att förstå på djupet under workshopen.

Kontrollera att ni har följande klart inför workshopen:

5. Installerat cors och morgan: `npm install cors morgan`
6. Skapat projektstrukturen med mapparna `src/routes/`, `src/controllers/` och `src/middleware/`
7. Testat minst ett GET- och ett POST-anrop i Postman eller Thunder Client
8. Läst igenom boiler room-instruktionerna i `BoilerRoomProjekt.pdf` och kommit igång med projektet i gruppen

Ta med er dessa frågor till workshopen: Hur avgör man om ett fält ska vara i URL-sökvägen (`req.params`), i query string (`req.query`) eller i request body (`req.body`)? När är det rätt att returnera 204 istället för 200? Hur tänker ni kring ansvarsfördelningen i ert boiler room-projekt?

Sammanfattning

Under kursvecka 2 har vi fördjupat oss i de tre grundpelarna för ett välbyggt REST API: design, struktur och middleware.

När det gäller API-design har vi sett att URL:er ska vara resursorienterade substantiv i plural (/users, /products), att hierarki används för att uttrycka relationer (/users/42/orders), och att filtrering, sortering och paginering hanteras med query string-parametrar. Vi har också fördjupat oss i HTTP-statuskodernas semantik: 201 Created för lyckad POST, 204 No Content när vi inte har något att returnera, 400/422 för klientfel, 401/403 för autentiserings- och behörighetsproblem, och 500 för oväntade serverfel. Ett konsekvent felformat med en maskinläsbar kod-sträng gör API:et mycket enklare att använda.

Projektstrukturen med separata route-filer och controllers är fundamentet för underhållbar kod. Route-filen är en karta – den kopplar URL:er och HTTP-metoder till controller-funktioner utan att innehålla affärslogik. Controllers innehåller logiken och pratar med databasen. app.js monterar allt och konfigurerar middleware-kedjan, medan server.js enbart ansvarar för att starta HTTP-servern. Den separationen gör det enkelt att testa applikationen isolerat.

Middleware-kedjan och dess ordning är kritisk: cors() och säkerhetsmiddleware kommer först, sedan request-parsning med express.json(), sedan loggning med morgan, sedan routes, sedan 404-hantering för okända sökvägar, och slutligen felhanteringsmiddleware med fyra parametrar (err, req, res, next) allra sist. cors skyddar slutanvändare i webbläsaren, morgan ger oss insyn i trafiken, och felhanteraren ser till att oväntade fel inte exponerar intern information till klienten.

Vi har också pratat om versionshantering – varför vi alltid ska inkludera /api/v1/ i URL:erna från start och vad som räknas som en brytande ändring. Och vi har tittat på hur vi organiserar testanrop i Postman Collections med miljövariabler för att enkelt kunna byta mellan lokal utveckling och produktion.

Nästa vecka introducerar vi TypeScript i backendkoden. Vi behåller allt vi lärt oss om Express och projektstruktur, men lägger till statiska typer för att göra koden mer robust och läsbar. Boiler room-projektet fortsätter – grupperna ska börja implementera grundstrukturen för sitt API.

Resurser för fördjupning

Följande resurser rekommenderas för de som vill fördjupa sina kunskaper.

Officiell dokumentation

Express Router – dokumentation för Router-moduler och hur routes organiseras:

<https://expressjs.com/en/guide/routing.html>

Express Middleware – guide till hur middleware fungerar och hur man skriver egna:

<https://expressjs.com/en/guide/writing-middleware.html>

morgan – dokumentation och alla tillgängliga format:

<https://github.com/expressjs/morgan>

cors – dokumentation och konfigurationsalternativ:

<https://github.com/expressjs/cors>

REST API-design

"REST API Design Best Practices" av freeCodeCamp – en grundlig genomgång av de principer vi arbetar med i den här veckan. Täcker URL-design, statuskoder, versionshantering och mer:

<https://www.freecodecamp.org/news/rest-api-best-practices-rest-endpoint-design-examples/>

MDN Web Docs: HTTP response status codes – komplett referens för alla HTTP-statuskoder med förklaringar och användningsexempel:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

Verktyg

Postman Learning Center – officiella guider för att komma igång med Postman, skapa Collections och använda miljövariabler:

<https://learning.postman.com/docs/getting-started/overview/>

Thunder Client – VS Code-tilläggets GitHub-sida med dokumentation och exempel:

<https://github.com/rangav/thunder-client-support>

Studiematerial

TypeScript i backendkod

Kursvecka 3

Introduktion

Under kurs 3 – React, TypeScript och React Native – lärde vi oss grunderna i TypeScript: hur man deklarerar typer, skapar interfaces, arbetar med generics och drar nytta av typkompilatorn för att fånga fel tidigt i utvecklingsprocessen. Vi såg hur TypeScript gör React-komponenter tydligare och mer robusta, och hur typade props och hooks eliminerar en hel kategori av vanliga buggar.

Nu ska vi ta med oss den kunskapen in i backendkoden. TypeScript i en Express-applikation fungerar på samma sätt som i React – men tillämpningarna är annorlunda. Istället för att typa props och komponenttillstånd ska vi typa request-objekt, response-objekt, middleware-funktioner, datamodeller och API-kontrakt. Det är en ny kontext, men samma verktyg.

Varför är TypeScript extra viktigt på backend? På frontend kan en typfel ofta resultera i ett synligt gränssnittsproblem som vi snabbt märker. På backend kan ett typfel i ett API-svar eller en databasoperation leda till subtila buggar som är svåra att spåra, och i värsta fall till säkerhetsproblem. En backend-server hanterar också ofta flera typer av data från olika källor – request bodies, databasobjekt, externa API-svar – som alla behöver hanteras korrekt och konsekvent.

I detta studiematerial går vi igenom hur vi konfigurerar TypeScript i ett Node.js/Express-projekt från grunden, hur vi typar Express-specifika konstruktioner, och hur vi designar typade API-kontrakt som gör koden självdokumenterande och lättare att underhålla. Vi avslutar med ett genomgång av generics och utility types i en backendkontext – verktyg vi redan sett i kurs 3, men som vi nu tillämpar på nya och mer avancerade sätt.

Konfigurera TypeScript i ett Node.js/Express-projekt

Till skillnad från ett React-projekt som skapas med Vite eller create-react-app, och där TypeScript-stödet redan är förkonfigurerat, behöver vi i ett Node.js/Express-projekt sätta upp TypeScript manuellt. Det är en process som är värd att förstå i detalj, dels för att vi gör det ofta, och dels för att konfigurationsalternativen påverkar hur strikt TypeScript är och hur projektet kompileras.

Installation av nödvändiga paket

Vi behöver installera TypeScript-kompilatorn och typdeklarationer för Node.js och Express. Typdeklarationerna (paket med prefixet @types/) är separata paket som berättar för TypeScript hur olika JavaScript-bibliotek ser ut invändigt – de är alltid devDependencies eftersom de bara används under utveckling och kompilering, aldrig i produktion.

```
# TypeScript-kompilatorn och körtidshantering
npm install --save-dev typescript tsx

# Typdeklarationer för Node.js och Express
npm install --save-dev @types/node @types/express

# Express själv (runtime-beroende, inte devDependency)
npm install express
```

Vi använder tsx istället för det klassiska ts-node, eftersom tsx är snabbare och inte kräver separata konfigurationssteg för ES-moduler. tsx kompilerar TypeScript-filer i farten och låter oss köra dem direkt med Node.js, ungefär som nodemon gör för JavaScript-filer. För produktion kompilerar vi däremot alltid till JavaScript med tsc-kompilatorn.

tsconfig.json – TypeScript-konfigurationen

tsconfig.json är hjärtat i vår TypeScript-konfiguration. Den talar om för TypeScript-kompilatorn hur den ska tolka och kompilera koden. Vi skapar den med det inbyggda init-kommandot och anpassar sedan inställningarna:

```
npx tsc --init
```

En välkonfigurerad tsconfig.json för ett Express-projekt ser ut så här:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "lib": ["ES2022"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true
  },
  "include": ["src/**/*"],
```

```
"exclude": ["node_modules", "dist"]
}
```

Låt oss gå igenom de viktigaste inställningarna:

- target: "ES2022" – Vilken version av JavaScript-standard som TypeScript kompilerar till. ES2022 ger oss tillgång till moderna JavaScript-funktioner och fungerar med alla aktuella Node.js-versioner.
- module: "CommonJS" – Vilket modulsystem som används i den kompilerade outputen. CommonJS (require/module.exports) är standard i Node.js-miljön, till skillnad från ESM (import/export) som används i webbläsaren och i React-projekt.
- outDir: "./dist" och rootDir: "./src" – Var källkoden (TypeScript) befinner sig och var den kompilerade outputen (JavaScript) placeras. Vi håller dessa separerade för tydlighet.
- strict: true – Aktiverar alla strikta typkontroller: noImplicitAny, strictNullChecks, strictFunctionTypes med flera. Det är det viktigaste alternativet – utan strict är TypeScript-typningen bara kosmetisk.
- esModuleInterop: true – Gör att vi kan använda import-syntax för CommonJS-moduler som inte har en default export, exempelvis: import express from "express" istället för det klumpigare import * as express from "express".

Scripts i package.json

Vi uppdaterar package.json med scripts för att köra projektet i utvecklingsläge och kompilera för produktion:

```
"scripts": {
  "dev": "tsx watch src/server.ts",
  "build": "tsc",
  "start": "node dist/server.js",
  "typecheck": "tsc --noEmit"
}
```

tsx watch fungerar som nodemon för TypeScript-filer: den övervakar alla .ts-filer och startar om processen automatiskt vid ändringar. Kommandot typecheck kör TypeScript-kompilatorn utan att faktiskt generera filer – det är ett snabbt sätt att kontrollera om koden är typkorrekt utan att behöva kompilera hela projektet.

Projektstruktur med TypeScript

En välorganiserad mappstruktur för ett TypeScript-baserat Express-projekt:

```
mitt-api/
  src/
    routes/
      books.ts
      users.ts
    controllers/
      booksController.ts
    middleware/
      auth.ts
      validate.ts
```

```
models/          # Mongoose-modeller (vecka 4+)
  Book.ts
types/           # Delade typdefinitioner och interfaces
  index.ts
app.ts
server.ts
dist/            # Kompilerad JavaScript (genereras av tsc)
.env
.gitignore
package.json
tsconfig.json
```

Mappen `types/` är ny jämfört med JavaScript-varianten. Här samlar vi interfaces och typdefinitioner som används på flera ställen i koden – exempelvis typer för API-request bodies, response-format och delade datastrukturer. Att ha en central plats för dessa minskar dubblering och gör dem lätta att hitta.

Typa Express – Request, Response och NextFunction

När vi arbetar med Express i TypeScript är det tre grundläggande typer från @types/express-paketet som vi behöver behärska: Request, Response och NextFunction. Dessa generiska typer låter oss specificera exakt vilken typ av data vi förväntar oss i olika delar av ett HTTP-anrop.

Request-typen och dess generiska parametrar

Express Request-typen är generisk och tar fyra valfria typparametrar i ordningen: params, resBody (oanvänd), reqBody och query. Det ser ut lite ovanligt, men vi behöver sällan specificera alla – vi kan hoppa över dem vi inte behöver med unknown.

```
import { Request, Response } from 'express';

// Interface för request body
interface CreateBookBody {
  title: string;
  author: string;
  year: number;
}

// Interface för URL-parametrar
interface BookParams {
  id: string; // URL-parametrar är alltid strängar
}

// Interface för query string
interface BooksQuery {
  genre?: string;
  page?: string; // Query-parametrar är alltid strängar
  limit?: string;
}

// Typade route handlers
const createBook = (
  req: Request<{}, {}, CreateBookBody>,
  res: Response
): void => {
  const { title, author, year } = req.body; // Fullständigt typat!
  res.status(201).json({ title, author, year });
};

const getBook = (
  req: Request<BookParams>,
  res: Response
): void => {
  const id = req.params.id; // TypeScript vet att det är en sträng
  res.json({ id });
};

const getBooks = (
  req: Request<{}, {}, {}, BooksQuery>,
```

```

    res: Response
  ): void => {
    const { genre, page, limit } = req.query; // Alla är string | undefined
    res.json({ genre, page, limit });
  };

```

En viktig detalj att komma ihåg: både URL-parametrar (`req.params`) och query string-parametrar (`req.query`) är alltid strängar i HTTP, oavsett vad vi skriver i URL:en. Om vi förväntar oss ett numeriskt ID måste vi konvertera det explicit med `parseInt()` eller `Number()`. TypeScript hjälper oss att komma ihåg detta – om vi definierar `id` som `number` istället för `string` i `params`-interfacet, klagar TypeScript på att vi använder det utan konvertering.

Response-typen och typade svar

`Response`-typen kan också typsättas med en generisk parameter för response body. Det gör det möjligt för TypeScript att kontrollera att vi faktiskt returnerar den form av data vi lovar:

```

interface Book {
  id: number;
  title: string;
  author: string;
  year: number;
}

interface ApiResponse<T> {
  data: T;
  message?: string;
}

interface ErrorResponse {
  message: string;
  code?: string;
}

// Route handler som returnerar en bok eller ett fel
const getBookById = (
  req: Request<BookParams>,
  res: Response<ApiResponse<Book> | ErrorResponse>
): void => {
  const id = parseInt(req.params.id);

  if (isNaN(id)) {
    // TypeScript vet att ErrorResponse passar här
    res.status(400).json({ message: 'Ogiltigt ID-format' });
    return;
  }

  const book: Book = { id, title: 'Clean Code', author: 'Martin', year: 2008 };
  // TypeScript vet att ApiResponse<Book> passar här
  res.json({ data: book });
};

```

Det är viktigt att notera att typning av Response-body inte tvingar Express att returnera just den datan vid körningstid – det är fortfarande vi som ansvarar för att koden faktiskt gör rätt. TypeScript hjälper oss att fånga upp misstag under utvecklingen, men det ersätter inte att vi skriver korrekt kod.

NextFunction och asynkrona handlers

NextFunction är typen för next-parametern som vi ser i route handlers och middleware. Det är en funktion som antingen tar emot ingenting (för att gå vidare i middleware-kedjan) eller ett Error-objekt (för att trigga felhanteringsmiddleware):

```
import { Request, Response, NextFunction } from 'express';

// Synkron handler med next
const syncHandler = (
  req: Request,
  res: Response,
  next: NextFunction
): void => {
  try {
    res.json({ message: 'OK' });
  } catch (error) {
    next(error); // Skickar felet till felhanteringsmiddleware
  }
};

// Asynkron handler – vi MÅSTE hantera fel med try/catch
const asyncHandler = async (
  req: Request,
  res: Response,
  next: NextFunction
): Promise<void> => {
  try {
    const data = await someAsyncOperation();
    res.json({ data });
  } catch (error) {
    next(error);
  }
};
```

En vanlig fallgrop är att glömma try/catch i asynkrona handlers. Om ett fel kastas i en async-funktion och vi inte fångar det, kraschar Node.js-processen (i äldre versioner) eller så hänger anropet utan svar (i nyare). TypeScript löser inte detta åt oss – det är upp till oss att se till att alla asynkrona handlers har korrekt felhantering. Vi ska i vecka 6 titta på ett wrapparmönster som gör det lättare.

Interfaces för datamodeller och API-kontrakt

Ett av de mest värdefulla användningsområdena för TypeScript i backend är att definiera tydliga kontrakt för hur data ser ut – både data som rör sig inuti systemet och data som exponeras via API:et. Det är ofta värt att skilja på dessa, eftersom de kan ha olika form.

Datamodell vs API-representation

En datamodell representerar hur data ser ut i databasen eller i affärslogiken. En API-representation (ibland kallad DTO, Data Transfer Object) representerar hur data ser ut när det exponeras via API:et. Dessa behöver inte vara identiska – och det är ofta viktigt att de inte är det. Tänk på ett användarobjekt: internt kan det ha ett lösenordshash-fält, men det ska aldrig skickas tillbaka i API-svaret.

```
// src/types/index.ts

// Fullständig datamodell (intern representation)
export interface UserModel {
  id: number;
  email: string;
  passwordHash: string; // Känsligt - ska aldrig exponeras i API
  role: 'user' | 'admin';
  createdAt: Date;
  updatedAt: Date;
}

// API-representation (vad klienten ser)
export interface UserResponse {
  id: number;
  email: string;
  role: 'user' | 'admin';
  createdAt: Date;
}

// Request body för att skapa en ny användare
export interface CreateUserBody {
  email: string;
  password: string; // Klartext - hashas på servern
}

// Request body för inloggning
export interface LoginBody {
  email: string;
  password: string;
}

// Svar vid lyckad inloggning
export interface AuthResponse {
  token: string;
  user: UserResponse;
}
```

Genom att samla dessa typdefinitioner i en separat fil (src/types/index.ts) kan vi importera dem var vi behöver dem i projektet. Controllers, middleware och route-filer delar alla samma typdefinitioner, vilket garanterar konsistens genom hela kodbasen.

Typa controllers separat från routes

I en välstrukturerad Express-applikation separerar vi route-definitionerna (var endpoints finns) från controller-logiken (vad som faktiskt händer). Det gör koden mer testbar och enklare att underhålla. Här ser vi hur detta ser ut med TypeScript:

```
// src/controllers/usersController.ts
import { Request, Response, NextFunction } from 'express';
import { CreateUserBody, UserResponse, LoginBody, AuthResponse } from
'../types';

// Vi exporterar controller-funktionerna för att routers ska kunna använda dem
export const createUser = async (
  req: Request<{}, {}, CreateUserBody>,
  res: Response<UserResponse>,
  next: NextFunction
): Promise<void> => {
  try {
    const { email, password } = req.body;
    // Här skulle vi i verkligheten hasha lösenordet och spara i databasen
    const newUser: UserResponse = {
      id: 1,
      email,
      role: 'user',
      createdAt: new Date()
    };
    res.status(201).json(newUser);
  } catch (error) {
    next(error);
  }
};

// src/routes/users.ts
import { Router } from 'express';
import { createUser } from '../controllers/usersController';

const router = Router();

router.post('/', createUser);

export default router;
```

Notera att vi exporterar controllers som namngivna funktioner och routers som default export. Det är en vanlig konvention i TypeScript-Express-projekt som gör imports tydliga och förutsägbara.

Typa middleware-funktioner

Middleware-funktioner är kanske det område där TypeScript tillför mest värde i en Express-applikation. Utan typning är det lätt att göra misstag med vad som finns på request-objektet – exempelvis om en middleware lägger till användarinformation på req men en senare middleware försöker läsa den utan att TypeScript vet om det.

Utöka Request-typen med egna fält

En vanlig situation är att en autentiseringsmiddleware hämtar användarinformationen och lägger den på request-objektet, så att efterföljande route handlers kan komma åt den utan att behöva autentisera igen. I TypeScript behöver vi utöka Express Request-typ för att detta ska fungera korrekt.

Det görs med en teknik som kallas declaration merging – vi utökar Express egna typdefinitioner med ett eget interface i en .d.ts-fil:

```
// src/types/express.d.ts
import { UserModel } from './index';

// Utökar Express inbyggda Request-typ
declare global {
  namespace Express {
    interface Request {
      user?: Pick<UserModel, 'id' | 'email' | 'role'>;
    }
  }
}
```

Nu kan autentiseringsmiddleware sätta req.user och TypeScript vet att det fältet existerar på alla Request-objekt. Vi märker hur vi använder Pick – en utility type som vi återkommer till i detalj – för att bara inkludera de fält från UserModel som vi faktiskt behöver:

```
// src/middleware/auth.ts
import { Request, Response, NextFunction } from 'express';

export const authenticate = (
  req: Request,
  res: Response,
  next: NextFunction
): void => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    res.status(401).json({ message: 'Autentisering krävs' });
    return;
  }

  const token = authHeader.split(' ')[1];
  // Vecka 7: här verifierar vi JWT. Just nu simulerar vi det.
  req.user = { id: 1, email: 'user@example.com', role: 'user' };
  next();
};
```

```
// src/routes/profile.ts
import { Router, Request, Response } from 'express';
import { authenticate } from '../middleware/auth';

const router = Router();

router.get('/me', authenticate, (req: Request, res: Response) => {
  // TypeScript vet nu att req.user kan existera (men är optional)
  if (!req.user) {
    res.status(401).json({ message: 'Ej autentiserad' });
    return;
  }
  res.json({ user: req.user });
});

export default router;
```

Observera att `req.user` är definierat som optional (med `?`) i vår utökade typ. Det är korrekt: inte alla routes passerar autentiseringsmiddleware, så vi kan inte garantera att `req.user` alltid finns. TypeScript tvingar oss att kontrollera detta innan vi använder det – precis som vi vill.

Valideringsmiddleware med TypeScript

En annan vanlig typ av middleware är valideringsmiddleware – en funktion som kontrollerar att request body har rätt format innan den når route handleren. Med TypeScript kan vi göra detta på ett typsäkert sätt:

```
// src/middleware/validate.ts
import { Request, Response, NextFunction, RequestHandler } from 'express';

// En factory-funktion som skapar valideringsmiddleware
// Vi återkommer till Zod för riktigt bra validering i vecka 6
export const requireFields = (
  fields: string[]
): RequestHandler => {
  return (req: Request, res: Response, next: NextFunction): void => {
    const missingFields = fields.filter(
      (field) => req.body[field] === undefined || req.body[field] === ''
    );

    if (missingFields.length > 0) {
      res.status(400).json({
        message: 'Obligatoriska fält saknas',
        missingFields
      });
      return;
    }

    next();
  };
};
```

```
// Användning i en route  
router.post('/books', requireFields(['title', 'author', 'year']), createBook);
```

Typen `RequestHandler` är en Express-inbyggd typ som representerar en middleware-funktion med signaturen `(req, res, next) => void`. Det är ett bra alternativ när vi inte behöver specificera de generiska typparametrarna – det ger oss typkontroll utan att vi behöver skriva ut hela signaturen varje gång.

Generics i backendkod

Vi använde generics i kurs 3 för att skapa återanvändbara React-komponenter och hooks. I backendkod är de lika användbara, men vi tillämpar dem på andra problem: generiska API-svarformat, återanvändbara hjälpfunktioner och typade databasfrågor.

Generiska API-svarsformat

Ett vanligt mönster i API-design är att ha ett konsekvent format för alla svar, oavsett vilken typ av data de innehåller. Med generics kan vi definiera detta format en gång och återanvända det överallt:

```
// src/types/index.ts

// Generiskt svarformat för lyckade anrop
export interface SuccessResponse<T> {
  success: true;
  data: T;
  message?: string;
}

// Generiskt svarformat för paginerade listor
export interface PaginatedResponse<T> {
  success: true;
  data: T[];
  pagination: {
    page: number;
    limit: number;
    total: number;
    totalPages: number;
  };
}

// Svarformat för fel
export interface ErrorResponse {
  success: false;
  message: string;
  code?: string;
}

// Hjälpfunktioner för att skapa standardiserade svar
export const createSuccess = <T>(data: T, message?: string): SuccessResponse<T>
=> ({
  success: true,
  data,
  ...(message && { message })
});

export const createPaginated = <T>(
  data: T[],
  page: number,
  limit: number,
  total: number
```

```

): PaginatedResponse<T> => ({
  success: true,
  data,
  pagination: { page, limit, total, totalPages: Math.ceil(total / limit) }
});

```

Dessa hjälpfunktioner gör det enkelt att skapa konsistenta svar i controllers:

```

import { createSuccess, createPaginated, SuccessResponse } from '../types';
import { Book } from '../types';

const getBooks = async (
  req: Request<{}, {}, {}, BooksQuery>,
  res: Response<PaginatedResponse<Book> | ErrorResponse>
): Promise<void> => {
  const page = parseInt(req.query.page ?? '1');
  const limit = parseInt(req.query.limit ?? '10');

  // Simulerad databasrespons
  const books: Book[] = [{ id: 1, title: 'Clean Code', author: 'Martin', year:
2008 }];
  const total = 1;

  res.json(createPaginated(books, page, limit, total));
};

```

Generiska hjälpfunktioner

Generics är också användbara för att skapa typade hjälpfunktioner som arbetar med data av okänd typ. Här är ett exempel på en funktion som plockar ut specifika fält från ett objekt – en operation vi ofta behöver för att filtrera bort känsliga fält:

```

// En generisk pick-funktion som plockar ut specifika fält från ett objekt
function pick<T, K extends keyof T>(obj: T, keys: K[]): Pick<T, K> {
  const result = {} as Pick<T, K>;
  keys.forEach((key) => {
    result[key] = obj[key];
  });
  return result;
}

// Användning: säker filtrering av känsliga fält
const user: UserModel = {
  id: 1,
  email: 'user@example.com',
  passwordHash: '$2b$12$...', // Ska aldrig skickas till klienten!
  role: 'user',
  createdAt: new Date(),
  updatedAt: new Date()
};

// TypeScript garanterar att vi bara kan ange fält som finns i UserModel

```

```
const safeUser = pick(user, ['id', 'email', 'role', 'createdAt']);  
// safeUser har nu typen Pick<UserModel, 'id' | 'email' | 'role' | 'createdAt'>  
// passwordHash finns inte - TypeScript klagar om vi försöker accessa det
```

Notera syntaxen `K extends keyof T` – det är en constraint (begränsning) som säger att `K` måste vara en delmängd av nycklar i `T`. Det gör att TypeScript kan verifiera att vi bara ber om fält som faktiskt existerar i objektet. Om vi skriver `pick(user, ['id', 'password'])` – där `password` inte finns i `UserModel` – klagar TypeScript direkt.

Utility types i praktiken

TypeScript har en uppsättning inbyggda utility types som transformerar befintliga typer till nya typer. Vi använde några av dem i kurs 3, men i backendkod är de ännu mer värdefulla. De låter oss undvika att definiera liknande typer om och om igen, och de uttrycker relationen mellan typer på ett tydligt sätt.

Partial – gör alla fält valfria

`Partial<T>` skapar en typ där alla fält i `T` är valfria. Det är perfekt för PATCH-endpoints, där klienten bara skickar de fält som ska uppdateras:

```
interface Book {
  id: number;
  title: string;
  author: string;
  year: number;
}

// För PATCH /books/:id - alla fält är valfria
type UpdateBookBody = Partial<Omit<Book, 'id'>>;
// Ekvivalent med: { title?: string; author?: string; year?: number }

const updateBook = async (
  req: Request<BookParams, {}, UpdateBookBody>,
  res: Response
): Promise<void> => {
  const updates = req.body; // Kan vara ett tomt objekt, och det är OK
  // ... uppdatera i databasen
  res.json({ message: 'Bok uppdaterad' });
};
```

Pick och Omit – välj eller uteslut fält

`Pick<T, K>` och `Omit<T, K>` är varandras motsatser. `Pick` väljer ut specifika fält, `Omit` tar bort specifika fält. Båda är användbara för att skapa specialiserade versioner av en basstyp:

```
interface UserModel {
  id: number;
  email: string;
  passwordHash: string;
  role: 'user' | 'admin';
  createdAt: Date;
  updatedAt: Date;
}

// Pick: välj ut exakt de fält vi vill ha
type UserPublic = Pick<UserModel, 'id' | 'email' | 'role'>;
// { id: number; email: string; role: 'user' | 'admin' }

// Omit: ta bort fält vi inte vill ha
type UserWithoutPassword = Omit<UserModel, 'passwordHash'>;
// { id: number; email: string; role: ...; createdAt: Date; updatedAt: Date }
```

```
// Kombinera: skapa request body-typen för att skapa en användare
type CreateUserBody = Pick<UserModel, 'email'> & { password: string };
// { email: string; password: string }
// Vi tar email från UserModel men password (klartext) istället för passwordHash
```

Required – gör alla fält obligatoriska

Required<T> är motsatsen till Partial – den gör alla fält obligatoriska. Det är användbart när vi vill säkerställa att ett objekt är fullständigt ifyllt, till exempel efter att vi hämtat det från databasen och fyllt i eventuella default-värden:

```
interface BookDraft {
  title?: string;
  author?: string;
  year?: number;
}

// Required gör alla fält obligatoriska
type CompletedBook = Required<BookDraft>;
// { title: string; author: string; year: number }

// En funktion som validerar och kompletterar ett utkast
function validateAndComplete(draft: BookDraft): CompletedBook {
  if (!draft.title || !draft.author || !draft.year) {
    throw new Error('Alla fält krävs');
  }
  // TypeScript förstår att vi kontrollerat att alla fält finns
  return { title: draft.title, author: draft.author, year: draft.year };
}
```

Record – nyckel-värde-mappningar

Record<K, V> är en utility type för att skapa ett objekt-typ med nycklar av typ K och värden av typ V. Det är särskilt användbart för konfigurationsobjekt och mappningar:

```
type UserRole = 'user' | 'admin' | 'moderator';

// Mapper varje roll till en lista av tillåtna åtgärder
const rolePermissions: Record<UserRole, string[]> = {
  user: ['read:own', 'update:own', 'delete:own'],
  moderator: ['read:own', 'read:all', 'update:own', 'delete:all'],
  admin: ['read:own', 'read:all', 'update:own', 'update:all', 'delete:all', 'create:admin']
};

// TypeScript klagar om vi missar en roll eller stavat fel
function hasPermission(role: UserRole, action: string): boolean {
  return rolePermissions[role].includes(action);
}
```


Typade felklasser och felhantering

Felhantering är ett område där TypeScript kan hjälpa oss att vara mer systematiska och explicita. Istället för att kasta generiska Error-objekt kan vi definiera egna felklasser med specifik information – statuskoder, felkoder och strukturerade felmeddelanden.

Egna felklasser

En felklass ärver från den inbyggda Error-klassen och lägger till extra information. Vi skapar en basklass och sedan specialiserade subclasser för vanliga feltyper:

```
// src/errors/AppError.ts

// Basklass för alla applikationsspecifika fel
export class AppError extends Error {
  constructor(
    message: string,
    public readonly statusCode: number,
    public readonly code?: string
  ) {
    super(message);
    this.name = this.constructor.name; // Sätter felnamnet till klassnamnet
    // Behövs för att instanceof ska fungera korrekt i TypeScript
    Object.setPrototypeOf(this, new.target.prototype);
  }
}

// 404 Not Found
export class NotFoundError extends AppError {
  constructor(resource: string) {
    super(`${resource} hittades inte`, 404, 'NOT_FOUND');
  }
}

// 400 Bad Request / valideringsfel
export class ValidationError extends AppError {
  constructor(message: string) {
    super(message, 400, 'VALIDATION_ERROR');
  }
}

// 401 Unauthorized
export class UnauthorizedError extends AppError {
  constructor(message: string = 'Autentisering krävs') {
    super(message, 401, 'UNAUTHORIZED');
  }
}

// 403 Forbidden
export class ForbiddenError extends AppError {
  constructor(message: string = 'Åtkomst nekad') {
    super(message, 403, 'FORBIDDEN');
  }
}
```

```
}
```

Centraliserad felhanteringsmiddleware

Med våra felklasser kan vi nu skapa en central felhanteringsmiddleware som tar hand om alla fel på ett konsekvent sätt. Det är en av de viktigaste principerna i Express-arkitektur – felhantering på ett ställe, inte utspridd i varje route handler:

```
// src/middleware/errorHandler.ts
import { Request, Response, NextFunction } from 'express';
import { AppError } from '../errors/AppError';

// Felhanteringsmiddleware har fyra parametrar
// Express känner igen den just för att den har fyra parametrar
export const errorHandler = (
  err: unknown,
  req: Request,
  res: Response,
  next: NextFunction
): void => {
  // Om det är ett känt applikationsfel
  if (err instanceof AppError) {
    res.status(err.statusCode).json({
      success: false,
      message: err.message,
      code: err.code
    });
    return;
  }

  // Om det är ett okänt fel - logga det men exponera inte detaljer
  console.error('Okänt fel:', err);
  res.status(500).json({
    success: false,
    message: 'Ett internt serverfel uppstod'
  });
};
```

Vi registrerar felhanteringsmiddleware sist i app.ts, efter alla routes:

```
// src/app.ts
import express from 'express';
import { errorHandler } from '../middleware/errorHandler';
import usersRouter from '../routes/users';
import booksRouter from '../routes/books';

const app = express();
app.use(express.json());

app.use('/api/v1/users', usersRouter);
app.use('/api/v1/books', booksRouter);
```

```
// Felhanteringsmiddleware SIST
app.use(errorHandler);

export default app;
```

Nu kan vi kasta felklasser var som helst i koden och veta att de hanteras korrekt:

```
// I en controller
import { NotFoundError, ValidationError } from '../errors/AppError';

const getBook = async (
  req: Request<BookParams>,
  res: Response,
  next: NextFunction
): Promise<void> => {
  try {
    const id = parseInt(req.params.id);

    if (isNaN(id)) {
      throw new ValidationError('ID måste vara ett heltal');
    }

    // Simulerar databasfråga (vecka 4 och framåt)
    const book = null; // Hittades inte i databasen

    if (!book) {
      throw new NotFoundError('Bok');
      // Ger automatiskt: 404, 'Bok hittades inte'
    }

    res.json({ success: true, data: book });
  } catch (error) {
    next(error); // Skickas till errorHandler
  }
};
```

TypeScript och unknown i catch-block

En viktig förändring från JavaScript till TypeScript (med strict-läge) är att felvariabeln i ett catch-block är av typen unknown – inte Error. Det är faktiskt mer korrekt, eftersom vad som helst kan kastas med throw i JavaScript. För att komma åt felmeddelandet behöver vi kontrollera typen:

```
try {
  // Kod som kan kasta fel
} catch (error) {
  // error är 'unknown' – vi kan inte direkt accessa error.message

  if (error instanceof Error) {
    // Nu vet TypeScript att det är ett Error-objekt
    console.error(error.message);
    console.error(error.stack);
  }
}
```

```
    } else {  
      // Någon kastade något som inte är ett Error  
      console.error('Okänt fel:', error);  
    }  
  }  
  
  // Hjälpfunktion för att extrahera felmeddelanden säkert  
  export function getErrorMessage(error: unknown): string {  
    if (error instanceof Error) return error.message;  
    if (typeof error === 'string') return error;  
    return 'Okänt fel uppstod';  
  }  
}
```

Det kan verka som en omständlighet jämfört med JavaScript, men det speglar verkligheten: i JavaScript kan vi kasta bokstavligen vad som helst – strängar, objekt, number – inte bara Error-instanser. TypeScript tvingar oss att hantera det faktum att vi inte alltid vet vad som kastades.

Sätta ihop det hela – ett fullständigt typat mini-API

Vi ska nu titta på hur alla delar vi gått igenom hänger ihop i ett komplett mini-API. Det här är ett förenklade exempel – vi har inga riktiga databaser ännu – men strukturen och typningen speglar hur vi ska arbeta under resten av kursen.

```
// src/types/index.ts
export interface Book {
  id: number;
  title: string;
  author: string;
  year: number;
}

export type CreateBookBody = Omit<Book, 'id'>;
export type UpdateBookBody = Partial<Omit<Book, 'id'>>;
export interface BookParams { id: string; }

export interface SuccessResponse<T> {
  success: true;
  data: T;
}

export interface ErrorResponse {
  success: false;
  message: string;
}

// src/controllers/booksController.ts
import { Request, Response, NextFunction } from 'express';
import { Book, CreateBookBody, UpdateBookBody, BookParams, SuccessResponse }
from '../types';
import { NotFoundError, ValidationError } from '../errors/AppError';

// In-memory lager (ersätts av MongoDB i vecka 4)
let books: Book[] = [
  { id: 1, title: 'Clean Code', author: 'Robert C. Martin', year: 2008 },
  { id: 2, title: 'The Pragmatic Programmer', author: 'Hunt & Thomas', year:
1999 }
];
let nextId = 3;

export const getAllBooks = (req: Request, res: Response): void => {
  res.json({ success: true, data: books });
};

export const getBookById = (
  req: Request<BookParams>,
  res: Response<SuccessResponse<Book> | ErrorResponse>,
  next: NextFunction
): void => {
  const id = parseInt(req.params.id);
  if (isNaN(id)) throw new ValidationError('ID måste vara ett heltal');
```

```

    const book = books.find((b) => b.id === id);
    if (!book) throw new NotFoundError('Bok');

    res.json({ success: true, data: book });
  };

export const createBook = (
  req: Request<{}, {}, CreateBookBody>,
  res: Response<SuccessResponse<Book>>
): void => {
  const { title, author, year } = req.body;
  const newBook: Book = { id: nextId++, title, author, year };
  books.push(newBook);
  res.status(201).json({ success: true, data: newBook });
};

export const updateBook = (
  req: Request<BookParams, {}, UpdateBookBody>,
  res: Response,
  next: NextFunction
): void => {
  const id = parseInt(req.params.id);
  const index = books.findIndex((b) => b.id === id);
  if (index === -1) throw new NotFoundError('Bok');

  books[index] = { ...books[index], ...req.body };
  res.json({ success: true, data: books[index] });
};

export const deleteBook = (
  req: Request<BookParams>,
  res: Response,
  next: NextFunction
): void => {
  const id = parseInt(req.params.id);
  const index = books.findIndex((b) => b.id === id);
  if (index === -1) throw new NotFoundError('Bok');

  books.splice(index, 1);
  res.status(204).send();
};

// src/routes/books.ts
import { Router } from 'express';
import {
  getAllBooks, getBookById, createBook, updateBook, deleteBook
} from '../controllers/booksController';

const router = Router();

router.get('/', getAllBooks);
router.get('/:id', getBookById);

```

```
router.post('/', createBook);  
router.patch('/:id', updateBook);  
router.delete('/:id', deleteBook);  
  
export default router;
```

Det är värt att notera hur rent detta ser ut. Routes-filen innehåller enbart routing-logik – den vet ingenting om hur böckerna hanteras. Controllers-filen innehåller logiken men vet ingenting om URL-strukturen. Types-filen definierar kontrakten som alla delar måste följa. TypeScript binder ihop allt och garanterar att kontrakt respekteras.

Checkpoint: Förberedelse inför kursvecka 3

Under boiler room vecka 3 ska ni ta det REST API ni byggde under vecka 2 och migrera det till TypeScript. Det innebär att ni konfigurerar TypeScript i projektet, lägger till typdeklarationer och sedan successivt typar upp befintlig kod. Börja med att sätta upp tsconfig.json och byta filändelse på filerna, sedan typa request och response-objekt, och avsluta med att definiera interfaces för era datamodeller.

Kontrollera att ni har följande klart inför workshopen:

1. typescript och tsx installerade som devDependencies i ert projekt
2. @types/node och @types/express installerade
3. tsconfig.json skapad och konfigurerad med strict: true
4. npm run dev fungerar och startar er TypeScript-server
5. Ni har börjat skapa interfaces för era datamodeller i en separat types-fil

Ta med er dessa reflektioner till workshopen: Var stötte ni på de mest intressanta TypeScript-felen när ni migrerade koden? Vilka typer av buggar hittade TypeScript som hade varit svåra att hitta annars? Hur väljer ni vad som ska vara en interface vs vad som ska vara ett type alias?

Sammanfattning

Under kursvecka 3 har vi tagit TypeScript-kunskaperna från kurs 3 och tillämpat dem i en backendkontext. Vi har sett att verktygen är desamma, men att tillämpningarna är annorlunda – istället för att typa React-komponenter och hooks typar vi HTTP-handlers, middleware-funktioner och API-kontrakt.

Vi har lärt oss att konfigurera TypeScript i ett Node.js/Express-projekt från grunden: installera nödvändiga paket (`@types/node`, `@types/express`), skapa en korrekt `tsconfig.json` med strict-läge aktiverat, och sätta upp scripts för utveckling och produktion. Vi har sett varför `tsx` är ett bekvämt val för att köra TypeScript-filer direkt under utveckling.

Express Request-typen är generisk och tar fyra parametrar för `params`, `resBody`, `reqBody` och `query`. Vi kan specificera precis de typparametrar vi behöver och hoppa över resten med `{}`. Det ger oss fullt typade request-objekt där TypeScript vet exakt vilka fält som finns och vilken typ de har. Vi har också sett hur vi utökar Express egna Request-typ med egna fält – exempelvis `req.user` i autentiseringsmiddleware – med hjälp av declaration merging i en `.d.ts`-fil.

Interfaces och typer låter oss definiera tydliga kontrakt för hur data ser ut i systemet. Skillnaden mellan en datamodell (intern representation) och en API-representation (vad klienten ser) är viktig – och TypeScript hjälper oss att hålla dem isär. Utility types – `Partial`, `Pick`, `Omit`, `Required` och `Record` – gör det möjligt att definiera varianter av befintliga typer utan att duplicera kod.

Generics i backendkod används för generiska API-svarformat, återanvändbara hjälpfunktioner och typade data-operationer. Constraints (`K extends keyof T`) låter oss skriva generiska funktioner som fortfarande ger oss typkontroll på fältnivå.

Typade felklasser ger oss ett systematiskt sätt att hantera fel i Express-applikationer. En hierarki av felklasser – `AppError`, `NotFoundError`, `ValidationError` och så vidare – tillsammans med en centraliserad felhanteringsmiddleware gör att felhantering sker konsekvent och på ett enda ställe. TypeScript's `unknown`-typ i catch-block är en påminnelse om att vi alltid bör kontrollera typen av felvariabeln innan vi antar att det är ett `Error`-objekt.

Nästa vecka introducerar vi MongoDB och Mongoose – vi lär oss att ansluta vår Express-applikation till en riktig databas och definiera typade schemas och modeller. Det är ett stort steg: vi går från in-memory-data till persistent lagring, och TypeScript hjälper oss att hantera de komplexa typer som uppstår i gränssnittet mellan applikation och databas.

Resurser för fördjupning

Följande resurser rekommenderas för de som vill fördjupa sina kunskaper inom TypeScript och backendkod.

Officiell dokumentation

TypeScript officiella handbok – fullständig referens för TypeScript-språket:

<https://www.typescriptlang.org/docs/handbook/intro.html>

TypeScript officiella referens för utility types:

<https://www.typescriptlang.org/docs/handbook/utility-types.html>

DefinitelyTyped – källkoden för @types-paket, inklusive @types/express:

<https://github.com/DefinitelyTyped/DefinitelyTyped>

Guider och artiklar

"TypeScript Express Tutorial" av Marcin Wanago på wanago.io är en välskriven serie artiklar om hur man använder TypeScript med Express. Artiklarna täcker allt från grundkonfiguration till avancerade mönster och är ett utmärkt komplement till studiematerialet.

"How to extend the Express Request object in TypeScript" – det finns många bra artiklar om declaration merging med Express. Sök på denna fras för att hitta uppdaterade guider som täcker den teknik vi använde för att lägga till req.user.

Videomaterial

"TypeScript Express REST API" av Ben Awad på YouTube ger en praktisk genomgång av hur man bygger ett fullständigt TypeScript/Express-API. Det är ett bra sätt att se hur koncepten vi gått igenom tillämpas i ett verkligt projekt.

"No BS TS" av Jack Herrington på YouTube är en serie korta, praktiska videor om TypeScript-koncept. Avsnitten om generics och utility types är särskilt relevanta för detta studiematerial och rekommenderas varmt.

Studiematerial

MongoDB och Mongoose - grunder

Kursvecka 4

Introduktion

Under de tre första veckorna har vi byggt upp en solid grund i backendutveckling: vi kan skapa Express-servrar med välstrukturerade routes och middleware, vi förstår HTTP och REST API-design, och vi kan nu skriva vår backendkod med TypeScript för ökad typsäkerhet. Men vi har saknat en kritisk komponent – persistent datalagring. Alla data vi hittills arbetat med har förvarats i minnet och försvunnit varje gång servern startats om. Det är dags att ta nästa steg och koppla vår server till en riktig databas.

Den här veckan introducerar vi MongoDB – en dokumentbaserad databas – och Mongoose, som är det mest använda biblioteket för att kommunicera med MongoDB från Node.js. Det är ett stort och viktigt steg: när vi kombinerar Express med MongoDB har vi alla delar på plats för att bygga riktiga fullstackapplikationer som faktiskt kan lagra och hämta data på ett varaktigt sätt.

Innan vi dyker in i MongoDB är det viktigt att förstå varför vi behöver en databas över huvud taget, och varför vi väljer en dokumentdatabas framför en relationsdatabas. Vi ska titta på de grundläggande skillnaderna, förstå när varje typ passar bäst, och sedan lägga all vår energi på att lära oss MongoDB och Mongoose i praktiken.

Vi avslutar veckan med att veta hur man ansluter en Express-applikation till MongoDB Atlas (MongoDBs molntjänst), skapar schemas och modeller med Mongoose och TypeScript, och utför de grundläggande databasoperationerna – att spara och hämta dokument. Nästa vecka bygger vi vidare med fullständiga CRUD-operationer och relationer.

Varför behöver vi persistent datalagring?

En applikation utan persistent datalagring är fundamentalt begränsad. Tänk på vad som händer med vår Express-server just nu: när vi startar den kan vi skapa, uppdatera och ta bort data i en JavaScript-array i minnet. Men så fort servern startas om – antingen för att vi gjort en kodändring, för att det uppstår ett fel, eller för att vi driftsätter en ny version – är all data borta. Alla användare som registrerat sig, alla inlägg som skapats, alla beställningar som gjorts: allt försvinner.

För en verklig applikation är det självklart oacceptabelt. Men persistent datalagring ger oss mer än bara beständighet. Det ger oss möjligheten att hantera data som är mycket större än vad som ryms i minnet, att söka och filtrera data effektivt med index, att köra flera instanser av vår server parallellt (alla kopplade till samma databas), och att överleva serveromstarter och driftsättningar utan dataförlust.

Databaser som lösning

En databas är ett system optimerat för att lagra, organisera och hämta data. Till skillnad från en vanlig fil erbjuder en databas strukturerade sätt att söka och filtrera data, transaktionsstöd (garantier om att flera operationer antingen alla lyckas eller alla misslyckas), index för snabba sökningar, och mekanismer för att hantera flera läsare och skrivare samtidigt utan att data korrumpas.

Det finns många typer av databaser, men i webbutvecklingsvärlden dominerar två kategorier: relationsdatabaser (SQL) och dokumentdatabaser (NoSQL). Vi ska förstå skillnaderna för att kunna välja rätt verktyg för rätt uppgift.

Relationsdatabaser vs dokumentdatabaser

Den stora skilljelinjen inom databaser går mellan relationsdatabaser (SQL-databaser som PostgreSQL, MySQL, SQLite) och dokumentdatabaser (NoSQL-databaser som MongoDB, CouchDB). Att förstå skillnaderna hjälper oss att fatta bra arkitekturella beslut. Vi kommer att gå djupare in på SQL senare i kursen, men det är viktigt att ha en grundläggande förståelse för båda redan nu.

Relationsdatabaser (SQL)

Relationsdatabaser organiserar data i tabeller med rader och kolumner – en struktur som liknar ett kalkylblad. Varje tabell representerar en typ av data (exempelvis användare eller produkter), och varje rad representerar en post. Relationer mellan tabeller skapas med nycklar: en order-tabell kan till exempel innehålla en kolumn `user_id` som refererar till primärnyckeln i user-tabellen.

Det som definierar relationsdatabaser är några centrala egenskaper:

- Strikt schema – Tabellstrukturen definieras i förväg och alla rader måste följa strukturen. Vi kan inte lägga till en kolumn för en enskild rad – strukturen gäller alla rader i tabellen.
- SQL – Structured Query Language används för att kommunicera med databasen. Det är ett kraftfullt och standardiserat frågespråk som används av nästan alla relationsdatabaser.
- ACID-transaktioner – Garantier om Atomicity (allt eller inget), Consistency (data alltid i giltigt tillstånd), Isolation (transaktioner påverkar inte varandra) och Durability (bekräftad data går inte förlorad).
- Normalisering – Principen att undvika duplicering av data genom att dela upp information i separata tabeller och koppla ihop dem med relationer.

Relationsdatabaser passar utmärkt för applikationer med komplex, väldefinierad datastruktur, starka konsistenskrav och komplexa relationer – till exempel banksystem, bokföringssystem och ERP-system. SQL är också ett av de viktigaste verktygen i en utvecklarens verktygslåda och vi kommer att titta på det mer ingående i vecka 9.

Dokumentdatabaser (NoSQL)

Dokumentdatabaser lagrar data i dokument – vanligtvis JSON-liknande strukturer som kallas BSON (Binary JSON) i MongoDB:s fall. Istället för tabeller med rader organiseras data i samlingar (collections) av dokument. Varje dokument är ett fristående objekt som kan ha sin egen unika struktur.

Dokumentdatabaser erbjuder en annan uppsättning egenskaper:

- Flexibelt schema – Dokument i samma samling behöver inte ha samma struktur. Vi kan lägga till nya fält i ett dokument utan att det påverkar andra dokument.
- Inbäddning – Relaterad data kan lagras direkt inuti ett dokument (embedding) istället för att delas upp i separata tabeller. En users-samling kan till exempel innehålla en lista med adresser direkt i varje användardokument.
- Horisontell skalning – Dokumentdatabaser är generellt enklare att skala horisontellt (lägga till fler servrar) eftersom data inte behöver delas upp och sammankopplas via joins.
- Naturlig mappning till JavaScript – JSON-dokument mappas direkt till JavaScript-objekt, vilket gör det intuitivt att arbeta med från Node.js.

Dokumentdatabaser passar bra för applikationer med varierande datastrukturer, snabb iteration (schemat kan ändras utan komplexa migrationer), och innehåll som naturligt representeras som dokument – till exempel blogginlägg, produktkataloger och användarprofiler.

Varför MongoDB i den här kursen?

Vi väljer MongoDB i den här kursen av flera skäl. Dels är det den populäraste dokumentdatabasen och används brett i JavaScript-ekosystemet. Dels är den naturliga mappningen mellan MongoDB-dokument och JavaScript-objekt en stor fördel när vi arbetar med Node.js och Express – vi slipper en konverteringslager. Dels är Mongoose (som vi introducerar snart) ett moget och välldesignat bibliotek som ger oss mycket av SQL-databasers strukturering (schemas och validering) ovanpå MongoDB:s flexibilitet.

Det är dock viktigt att inte fastna i tänket att dokumentdatabaser alltid är bättre. I arbetslivet är det vanligt att använda relationsdatabaser – speciellt i systemkritiska sammanhang. Kursen ger er grunden för MongoDB och introduktion till SQL senare, så att ni kan värdera och välja rätt verktyg i framtiden.

MongoDB – dokumentbaserad datalagring

MongoDB är en öppen källkods-dokumentdatabas som skapades 2009 av företaget MongoDB, Inc. (då kallat 10gen). Sedan dess har det vuxit till att bli en av världens mest använda databaser. Versionen vi fokuserar på är det klassiska MongoDB:t – en kraftfull, flexibel databas med ett välbekant JSON-liknande gränssnitt.

Grundläggande begrepp

Innan vi börjar använda MongoDB är det viktigt att känna till dess grundläggande terminologi och hur den förhåller sig till relationsdatabasers terminologi:

- Databas (Database) – En logisk behållare för relaterade samlingar. I MongoDB kan en databasserver köra flera databaser samtidigt – exempelvis en för varje applikation vi bygger.
- Samling (Collection) – Motsvarar en tabell i SQL. En samling innehåller en grupp av relaterade dokument – exempelvis en users-samling och en products-samling.
- Dokument (Document) – Motsvarar en rad i SQL, men är ett JSON-liknande objekt (tekniskt BSON). Varje dokument har en unik identifierare `_id`. Dokument i samma samling kan ha olika struktur.
- Fält (Field) – Motsvarar en kolumn i SQL. Ett fält är ett nyckel-värde-par i ett dokument. Värdet kan vara en sträng, tal, boolean, array, eller till och med ett inbäddat dokument.

Ett MongoDB-dokument ser ut som ett JSON-objekt:

```
{
  "_id": ObjectId("64a3f2c1b5e8f123456789ab"),
  "name": "Anna Svensson",
  "email": "anna@example.com",
  "age": 28,
  "address": {
    "street": "Storgatan 12",
    "city": "Stockholm",
    "postalCode": "111 22"
  },
  "hobbies": ["programmering", "löpning", "matlagning"],
  "createdAt": ISODate("2024-07-03T10:30:00.000Z")
}
```

Notera hur dokumentet kan innehålla inbäddade objekt (address) och arrayer (hobbies). Det är en av MongoDB:s styrkor – vi kan modellera komplex data naturligt utan att behöva dela upp den i flera tabeller.

ObjectId – den unika identifieraren

Varje MongoDB-dokument har automatiskt ett `_id`-fält med ett ObjectId-värde om vi inte anger ett eget. ObjectId är en 12-byte identifierare som garanteras vara unik. Den är baserad på en kombination av tidsstämpel, ett maskin-ID, ett process-ID och en inkrementel räknare – vilket innebär att ObjectIds genererade nära varandra i tid sorteras kronologiskt.

En viktig konsekvens för oss som backendutvecklare: när vi skickar ett dokumentets id i en URL eller returnerar det i ett API-svar representeras ObjectId vanligtvis som en 24-tecken

hexadecimalsträng, till exempel '64a3f2c1b5e8f123456789ab'. När vi söker efter ett dokument med Mongoose måste vi ibland konvertera strängen tillbaka till ett ObjectId-objekt, men Mongoose hanterar det åt oss i de flesta fall.

MongoDB Atlas – vår molnbaserade databas

MongoDB Atlas är MongoDB:s molnbaserade databastjänst – en fullt hanterad plattform för att köra MongoDB-databaser i molnet utan att behöva installera och konfigurera något på en egen server. Atlas är ett utmärkt val för oss i utbildningssyfte och för att snabbt komma igång, men det används också brett i produktion av allt från startups till stora företag.

Atlas erbjuder en gratis nivå (Free Tier med en M0-kluster) som är mer än tillräcklig för utbildningsändamål. Den fria nivån inkluderar 512 MB lagringsutrymme, vilket räcker bra för de projekt vi bygger under kursen.

Komma igång med MongoDB Atlas

Följ dessa steg för att sätta upp er databas inför kursveckan:

1. Skapa ett konto på mongodb.com/atlas och logga in.
2. Skapa ett nytt projekt och välj sedan 'Create a cluster'. Välj gratisnivån M0 och en region nära er (exempelvis Frankfurt).
3. Skapa en databasanvändare under 'Database Access'. Notera användarnamn och lösenord – dessa behöver vi i anslutningssträngen.
4. Konfigurera nätverksåtkomst under 'Network Access'. Under utveckling kan vi tillåta anslutningar från alla IP-adresser (0.0.0.0/0). I produktion ska vi begränsa detta.
5. Hämta anslutningssträngen under 'Connect' > 'Connect your application'. Den ser ut ungefär så här: `mongodb+srv://[användarnamn]:[lösenord]@cluster0.xxxxx.mongodb.net/[databasnamn]`

Anslutningssträngen är känslig information – den innehåller lösenordet till vår databas. Vi ska aldrig skriva den direkt i koden eller committa den till Git. Hur vi hanterar den säkert behandlas i nästa avsnitt om miljövariabler.

Miljövariabler och dotenv

Miljövariabler är ett standardiserat sätt att konfigurera applikationer utan att hårdkoda känslig eller miljöspecifik information i koden. Typisk information som hanteras via miljövariabler inkluderar databasanslutningssträngar, API-nycklar, JWT-hemligheter och portnummer.

Principen är enkel: vi lagrar konfigurationen i en fil (.env) som inte checkas in i versionshanteringen, och läser den i koden via process.env. Det innebär att vi kan ha olika konfiguration för utveckling, testning och produktion utan att ändra en enda rad kod.

Sätta upp dotenv

Vi installerar dotenv-paketet som hanterar inläsning av .env-filen:

```
npm install dotenv
```

Sedan skapar vi en .env-fil i projektets rotkatalog:

```
# .env
PORT=3000
MONGODB_URI=mongodb+srv://adminUser:mittLösenord@cluster0.abc123.mongodb.net/
minDatabas
NODE_ENV=development
```

Och lägger till .env i .gitignore:

```
# .gitignore
node_modules/
.env
dist/
```

Slutligen läser vi in miljövariablerna tidigt i vår applikation – i server.js, allra längst upp:

```
// server.js
require('dotenv').config(); // Läs in .env-filen

const app = require('./app');

const PORT = process.env.PORT || 3000;
const MONGODB_URI = process.env.MONGODB_URI;

if (!MONGODB_URI) {
  console.error('MONGODB_URI saknas i miljövariablerna!');
  process.exit(1); // Avsluta processen om konfiguration saknas
}

app.listen(PORT, () => {
  console.log(`Servern lyssnar på port ${PORT}`);
});
```

Notera kontrollen av MONGODB_URI – om den saknas avslutar vi processen med en tydlig felbeskrivning. Det är ett bra mönster: vi föredrar att applikationen misslyckas snabbt och tydligt (fail fast) istället för att starta och sedan krascha på ett svårfelsökt sätt längre in i körningen.

En annan bra vana är att skapa en .env.example-fil som committas till Git och visar vilka miljövariabler som behövs – men utan verkliga värden. Den fungerar som dokumentation för andra utvecklare som klonar projektet:

```
# .env.example - committas till Git, men innehåller inga riktiga värden
PORT=3000
MONGODB_URI=mongodb+srv://[användarnamn]:[lösenord]@cluster0.xxxxx.mongodb.net/
[databas]
NODE_ENV=development
```

Mongoose – Object Document Mapper

Vi skulle tekniskt sett kunna kommunicera med MongoDB direkt via det officiella MongoDB Node.js-drivrutinen, men det är ganska lågnivå och kräver mycket kod för att validera data och strukturera frågor. Mongoose är ett ODM (Object Document Mapper) – ett bibliotek som lägger ett lager av abstraktion ovanpå MongoDB och ger oss ett kraftfullt och expressivt API.

Mongoose tillför tre centrala saker:

- Schemas och modeller – Vi definierar strukturen för våra dokument i förväg, precis som vi definierar typer i TypeScript. Mongoose ser till att data som sparas matchar schemat.
- Validering – Vi kan definiera valideringsregler direkt i schemat: att ett fält är obligatoriskt, att ett tal måste ligga inom ett visst intervall, att en sträng måste följa ett visst mönster.
- Query-API – Mongoose ger oss ett kraftfullt och kedjbart API för databasfrågor: `Book.find({ genre: 'fiction' }).sort({ title: 1 }).limit(10)` är ett tydligt exempel på hur expressivt det kan vara.

Vi installerar Mongoose med:

```
npm install mongoose
```

Ansluta till MongoDB med Mongoose

Vi ansluter till MongoDB i vår `server.js` eller i en separat databasfil. Det är god praxis att separera databasanslutningslogiken i en egen fil:

```
// src/config/database.js
const mongoose = require('mongoose');

async function connectToDatabase() {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log('Ansluten till MongoDB');
  } catch (error) {
    console.error('Kunde inte ansluta till MongoDB:', error.message);
    process.exit(1);
  }
}

module.exports = { connectToDatabase };
```

Sedan anropar vi funktionen i `server.js` och startar servern:

```
// server.js
require('dotenv').config();

const app = require('./app');
const { connectToDatabase } = require('./src/config/database');

const PORT = process.env.PORT || 3000;
```

```
async function startServer() {  
  await connectToDatabase();  
  app.listen(PORT, () => {  
    console.log(`Servern lyssnar på port ${PORT}`);  
  });  
}  
  
startServer();
```

Notera att vi väntar på att databasanslutningen ska etableras innan vi startar servern. Det är viktigt: om servern tar emot requests innan databasen är ansluten kommer alla databasoperationer att misslyckas. Genom att ansluta först och sedan starta servern undviker vi det problemet.

Schemas och modeller med Mongoose och TypeScript

En av Mongoose:s viktigaste funktioner är möjligheten att definiera ett schema för våra dokument. Schemat beskriver dokumentets struktur: vilka fält det har, vilka typer de har, om de är obligatoriska, och eventuella valideringsregler. I kombination med TypeScript får vi dubbel typsäkerhet – TypeScript fångar fel vid kompilering och Mongoose fångar fel vid körning.

Notera att vi använder TypeScript specifikt för schemas och modeller, precis som vi tränade på i vecka 3. TypeScript-typning ger oss autokomplettering och felupptäckning i editorn, medan Mongoose-schemat ger oss validering när vi faktiskt försöker spara data i databasen.

Definiera ett schema

Vi skapar en fil för varje modell i src/models/-mappen. Här är ett komplett exempel med TypeScript för en bok-modell:

```
// src/models/Book.ts
import mongoose, { Schema, Document, Model } from 'mongoose';

// TypeScript-interface för bokens datastruktur
export interface IBook extends Document {
  title: string;
  author: string;
  genre: string;
  publishedYear: number;
  isbn?: string;      // ? = valfritt fält
  inStock: boolean;
  createdAt: Date;
  updatedAt: Date;
}

// Mongoose-schemat definierar struktur och validering
const BookSchema = new Schema<IBook>({
  {
    title: {
      type: String,
      required: [true, 'Titel är obligatorisk'],
      trim: true,      // Ta bort ledande/avslutande mellanslag
      maxlength: [200, 'Titeln får inte vara längre än 200 tecken'],
    },
    author: {
      type: String,
      required: [true, 'Författare är obligatorisk'],
      trim: true,
    },
    genre: {
      type: String,
      required: [true, 'Genre är obligatorisk'],
      enum: {
        values: ['fiction', 'non-fiction', 'science', 'history', 'biography'],
        message: '{VALUE} är inte en giltig genre',
      },
    },
  },
},
```



```

    publishedYear: {
      type: Number,
      required: [true, 'Utgivningsår är obligatoriskt'],
      min: [1000, 'Utgivningsår måste vara minst 1000'],
      max: [new Date().getFullYear(), 'Utgivningsår kan inte vara i framtiden'],
    },
    isbn: {
      type: String,
      unique: true,          // Inga dubletter tillåtna
      sparse: true,         // Tillåt att fältet saknas (null ignoreras i unique-
index)
    },
    inStock: {
      type: Boolean,
      default: true,        // Standardvärde om fältet utelämnas
    },
  },
  {
    timestamps: true,      // Lägger automatiskt till createdAt och updatedAt
  }
);

// Skapa och exportera modellen
export const Book: Model<IBook> = mongoose.model<IBook>('Book', BookSchema);

```

Låt oss gå igenom de viktigaste delarna av schemat:

- `required: [true, 'meddelande']` – Gör fältet obligatoriskt. Om det saknas returnerar Mongoose ett valideringsfel med det angivna meddelandet.
- `trim: true` – Tar automatiskt bort whitespace i början och slutet av strängar. Förhindrar att ' Anna' och 'Anna' behandlas som olika värden.
- `enum` – Begränsar de tillåtna värdena för ett fält. Mongoose kastar ett valideringsfel om ett värde som inte finns i listan används.
- `min` och `max` – Anger minsta och högsta tillåtna värde för numeriska fält.
- `unique: true` – Skapar ett unikt index för fältet. Inga två dokument får ha samma värde för det fältet.
- `default` – Anger ett standardvärde som används om fältet utelämnas när ett dokument skapas.
- `timestamps: true` – En mycket praktisk inställning som automatiskt lägger till och uppdaterar `createdAt` och `updatedAt`-fält. Vi slipper hantera det manuellt.

Modellen – vår nyckel till databasen

Modellen är det vi faktiskt använder i vår applikationskod. Den skapas från schemat med `mongoose.model()` och ger oss ett gränssnitt mot MongoDB-samlingen. Mongoose namnger samlingen automatiskt baserat på modellnamnet: modellen 'Book' använder samlingen 'books' (pluraliserat och i lowercase).

Via modellen kan vi utföra alla databasoperationer: skapa, läsa, uppdatera och ta bort dokument. Vi tittar nu på de grundläggande operationerna för att skapa och läsa dokument – de fullständiga CRUD-operationerna kommer vi att gå djupare in på nästa vecka.

Grundläggande databasoperationer

Nu när vi har en modell är vi redo att använda den i våra route handlers. Alla Mongoose-operationer är asynkrona – de returnerar Promises – så vi använder async/await genomgående.

Skapa ett dokument

Det finns två sätt att skapa ett nytt dokument med Mongoose:

Alternativ 1 – Instansiera och spara:

```
const newBook = new Book({
  title: 'Clean Code',
  author: 'Robert C. Martin',
  genre: 'non-fiction',
  publishedYear: 2008,
});

const savedBook = await newBook.save();
console.log(savedBook._id); // MongoDB:s tilldelade ObjectId
```

Alternativ 2 – Använda Model.create() (kortare och vanligast):

```
const savedBook = await Book.create({
  title: 'Clean Code',
  author: 'Robert C. Martin',
  genre: 'non-fiction',
  publishedYear: 2008,
});
```

Båda alternativen validerar mot schemat och kastar ett `ValidationError` om data inte är giltig. Här är hur vi integrerar det i en route handler:

```
// src/routes/books.js
const express = require('express');
const router = express.Router();
const { Book } = require('../models/Book');

// POST /api/v1/books - skapa en ny bok
router.post('/', async (req, res, next) => {
  try {
    const book = await Book.create(req.body);
    res.status(201).json(book);
  } catch (error) {
    next(error); // Skicka vidare till felhanteringsmiddleware
  }
});
```

Läsa dokument

Mongoose erbjuder flera metoder för att hämta dokument. Här är de viktigaste:

Hämta alla dokument:

```
// Hämta alla böcker
const books = await Book.find();

// Hämta alla böcker med filter
const fictionBooks = await Book.find({ genre: 'fiction' });

// Hämta böcker och välj specifika fält (projection)
const titlesOnly = await Book.find({}, 'title author -_id');
```

Hämta ett specifikt dokument:

```
// Hämta ett dokument med ObjectId
const book = await Book.findById(req.params.id);

// Hämta ett dokument med ett specifikt fält
const book = await Book.findOne({ isbn: '978-0-13-468599-1' });
```

`findById()` returnerar null om inget dokument hittas. Vi måste alltid kontrollera för null och returnera ett lämpligt 404-svar:

```
// GET /api/v1/books/:id - hämta en specifik bok
router.get('/:id', async (req, res, next) => {
  try {
    const book = await Book.findById(req.params.id);

    if (!book) {
      return res.status(404).json({ message: 'Boken hittades inte' });
    }

    res.json(book);
  } catch (error) {
    next(error);
  }
});
```

Ett vanligt fel är att skicka ett ogiltigt ObjectId-format som id-parameter, till exempel `/api/v1/books/ettOgiltigtId`. Mongoose kastar då ett `CastError` eftersom strängen inte kan konverteras till ett giltigt ObjectId. Vi hanterar sådana fel centralt i vår felhanteringsmiddleware – mer om det i vecka 6.

En komplett route-fil

Här är en komplett route-fil med POST och GET-endpoints:

```
// src/routes/books.js
const express = require('express');
const router = express.Router();
```

```
const { Book } = require('../models/Book');

// GET /api/v1/books - hämta alla böcker
router.get('/', async (req, res, next) => {
  try {
    const books = await Book.find();
    res.json(books);
  } catch (error) {
    next(error);
  }
});

// GET /api/v1/books/:id - hämta en specifik bok
router.get('/:id', async (req, res, next) => {
  try {
    const book = await Book.findById(req.params.id);
    if (!book) {
      return res.status(404).json({ message: 'Boken hittades inte' });
    }
    res.json(book);
  } catch (error) {
    next(error);
  }
});

// POST /api/v1/books - skapa en ny bok
router.post('/', async (req, res, next) => {
  try {
    const book = await Book.create(req.body);
    res.status(201).json(book);
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

Rekommenderad projektstruktur

Nu när vi introducerar databaser och modeller är det ett bra tillfälle att titta på hur projektstrukturen bör se ut. En tydlig och konsekvent struktur gör det enkelt att hitta relevant kod och håller projektet underhållbart när det växer.

```
mitt-api/
  src/
    config/
      database.js      # Mongoose-anslutningslogik
    models/
      Book.ts          # Mongoose-schema + TypeScript-interface
      User.ts          # Ytterligare modeller
    routes/
      books.js         # Route handlers för /api/v1/books
      users.js
    middleware/
      errorHandler.js  # Centraliserad felhantering
    app.js             # Express-applikation (middleware + routes)
  server.js           # Serverstart (dotenv + DB + listen)
  .env                # Miljövariabler (ALDRIG i Git!)
  .env.example        # Mall för miljövariabler (committas)
  .gitignore
  package.json
  tsconfig.json       # TypeScript-konfiguration
```

Så här ser app.js ut när vi inkluderar databasanslutning och routes:

```
// src/app.js
const express = require('express');
const cors = require('cors');
const booksRouter = require('./routes/books');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/v1/books', booksRouter);

// Felhanteringsmiddleware (registreras SIST)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    message: 'Något gick fel på servern',
  });
});

module.exports = app;
```


Vanliga misstag och hur vi undviker dem

Det finns ett antal vanliga misstag som nybörjare gör när de börjar arbeta med MongoDB och Mongoose. Här går vi igenom de viktigaste och hur vi undviker dem.

Glömma att invänta asynkrona operationer

Alla Mongoose-operationer är asynkrona. Att glömma `await` är ett av de vanligaste felen och ger ett svar med ett Promise-objekt istället för faktisk data:

```
// FEL - returnerar ett Promise-objekt, inte datan
router.get('/', (req, res) => {
  const books = Book.find(); // Glömde await!
  res.json(books);           // Skickar ett Promise, inte en array
});

// RÄTT - inväntar operationen med async/await
router.get('/', async (req, res, next) => {
  try {
    const books = await Book.find();
    res.json(books);
  } catch (error) {
    next(error);
  }
});
```

Hantera null-resultat

`findById()` och `findOne()` returnerar null om inget dokument hittas. Om vi inte kontrollerar för null och försöker komma åt fält på resultatet kraschar servern:

```
// FEL - kraschar om book är null
const book = await Book.findById(id);
res.json({ title: book.title }); // TypeError: Cannot read properties of null

// RÄTT - kontrollera för null
const book = await Book.findById(id);
if (!book) {
  return res.status(404).json({ message: 'Hittades inte' });
}
res.json(book);
```

Committa .env till Git

Det är ett allvarligt säkerhetsproblem att committa `.env`-filen med känsliga uppgifter till Git – speciellt om repositoryet är publikt. Databasanslutningssträngar, lösenord och API-nycklar kan då läcka ut. Kontrollera alltid att `.env` finns i `.gitignore` innan ni gör en commit.

Om vi av misstag har committat känslig information till Git räcker det inte att ta bort filen och committa igen – historiken finns kvar. I det fallet behöver vi byta lösenord och API-nycklar omedelbart, och sedan städa upp Git-historiken (vilket är komplicerat). Det bästa är att aldrig hamna i den situationen från första början.

Checkpoint: Förberedelse inför kursvecka 4

Inför boiler room måndag och workshop onsdag-torsdag ska ni ha studerat detta material och förberett er lokala miljö. Kontrollera att ni har följande klart:

6. Skapat ett konto på MongoDB Atlas och en gratis M0-kluster.
7. Skapat en databasanvändare med lösenord och konfigurerat nätverksåtkomst.
8. Kopierat anslutningssträngen och lagt in den i en .env-fil i ert projekt.
9. Installerat mongoose och dotenv i ert projekt och verifierat att anslutningen fungerar.

Ta med er dessa frågor och reflektioner till workshopen: När är en dokumentdatabas ett bättre val än en relationsdatabas? Vad är fördelen med att definiera ett Mongoose-schema när MongoDB i sig är schemafritt? Hur ser TypeScript-interfacet och Mongoose-schemat på varandra – varför behöver vi båda?

Sammanfattning

Under kursvecka 4 har vi tagit det avgörande steget att koppla vår Express-backend till en riktig databas. Vi förstår nu varför persistent datalagring är nödvändig – in-memory-lösningar förlorar all data vid omstart och kan inte hantera data som är större än tillgängligt RAM-minne.

Vi har granskat de grundläggande skillnaderna mellan relationsdatabaser och dokumentdatabaser. Relationsdatabaser organiserar data i tabeller med strikta schemas och används via SQL – de är utmärkta för komplex, väldefinierad data med starka konsistenskrav. Dokumentdatabaser som MongoDB lagrar data som JSON-liknande dokument i samlingar, erbjuder flexibla schemas och passar bra för snabb iteration och data som naturligt representeras som dokument. Inget av dem är objektivt bättre – valet beror på kravbilden.

MongoDB Atlas ger oss en molnbaserad MongoDB-instans utan att vi behöver installera och konfigurera en databasserver lokalt. Gratisnivån är mer än tillräcklig för utbildningens projekt, och Atlas används också brett i produktion. Nyckeln till att ansluta vår applikation till Atlas är anslutningssträngen, som vi hanterar säkert via miljövariabler med dotenv – vi committar aldrig .env-filen till Git.

Mongoose är vår brygga till MongoDB. Som ODM ger det oss schemas för att definiera dokumentstrukturen, validering för att säkerställa datakvaliteten, och ett expressivt query-API. Vi kombinerar Mongoose schemas med TypeScript-interfaces för dubbel typsäkerhet: TypeScript fångar typfel vid kompilering och Mongoose fångar valideringsfel vid körning. En Mongoose-modell skapas från ett schema och är det objekt vi använder i vår applikationskod för alla databasoperationer.

Vi har sett hur man skapar dokument med `Book.create()` och hämtar dokument med `Book.find()` och `Book.findById()`. Alla operationer är asynkrona och vi använder `async/await` med `try/catch` och `next(error)` för att hantera fel korrekt. Vi kontrollerar alltid för null-resultat från `findById()` och `findOne()` och returnerar ett 404-svar om dokumentet inte finns.

Nästa vecka fördjupar vi oss i CRUD-operationer – vi lägger till uppdatering och borttagning, lär oss filtrera, sortera och paginera sökresultat, och börjar utforska relationer mellan dokument. Vi introducerar också testning av API:er mot en testdatabas. Boiler room-projektet ska nu börja använda en riktig databas – mer information i [BoilerRoomProjekt.pdf](#).

Resurser för fördjupning

Följande resurser rekommenderas för de som vill fördjupa sina kunskaper inom MongoDB och Mongoose.

Officiell dokumentation

MongoDB officiella dokumentation – fullständig referens för MongoDB:

<https://www.mongodb.com/docs/>

Mongoose officiella dokumentation – guide och API-referens för Mongoose:

<https://mongoosejs.com/docs/>

MongoDB Atlas dokumentation – guide för att komma igång med Atlas:

<https://www.mongodb.com/docs/atlas/getting-started/>

dotenv dokumentation – hur man hanterar miljövariabler:

<https://github.com/motdotla/dotenv>

Rekommenderade guider

"MongoDB University" erbjuder gratis onlinekurser direkt från MongoDB. Kursen "MongoDB for JavaScript Developers" är särskilt relevant och ger en grundlig genomgång av MongoDB från ett JavaScript-perspektiv.

<https://learn.mongodb.com/>

"Mongoose Crash Course" av Traversy Media på YouTube är en bra introduktion till Mongoose för de som föredrar video. Den täcker schemas, modeller och grundläggande CRUD-operationer på ett pedagogiskt sätt.

Säkerhet och bästa praxis

"The Twelve-Factor App" är ett etablerat ramverk för att bygga moderna applikationer. Faktor tre (Config) behandlar hantering av konfiguration via miljövariabler och är direkt relevant för det vi gjort i veckan:

<https://12factor.net/config>

GitGuardian publicerar regelbundet statistik och guider om hemlighetsskydd i kodrepositorier – ett problem som är vanligare än man kan tro. Deras guide om att förhindra att hemligheter läcker till Git är välskriven och praktisk:

<https://blog.gitguardian.com/secrets-credentials-api-git/>

Studiematerial

CRUD-operationer, relationer och testning

Kursvecka 5

Introduktion

Under vecka 4 lade vi grunden för databasarbete: vi satte upp MongoDB Atlas, lärde oss vad Mongoose är och varför vi använder det, definierade våra första schemas och modeller, och anslöt vår Express-server till databasen. Vi genomförde även de allra enklaste databasoperationerna – att skapa och hämta dokument. Nu tar vi det steget vidare och bygger ut vår förståelse till att täcka hela CRUD-spektrumet.

CRUD är en akronym för Create, Read, Update och Delete – de fyra grundläggande operationerna som nästan all databasinteraktion kan reduceras till. Att behärska fullständiga CRUD-operationer är kärnan i backendutveckling: det är det vi gör när vi bygger ett API som en frontend kan kommunicera med. Under den här veckan lär vi oss inte bara hur man implementerar dessa operationer, utan också hur man gör det på ett strukturerat och testbart sätt.

En viktig ny dimension som vi introducerar den här veckan är relationer. Verkliga applikationer har sällan en enda typ av data – de har användare som skriver inlägg, produkter som tillhör kategorier, beställningar som kopplas till kunder. MongoDB hanterar relationer annorlunda än relationsdatabaser, och det är viktigt att vi förstår de avvägningar som finns. Vi utforskar de två huvudstrategierna – embedding och referencing – och lär oss när man ska välja vad.

Den andra stora nyheten den här veckan är testning. Att bygga ett API som fungerar är en sak; att kunna bevisa att det fungerar – och fortsätter fungera – är en annan. Vi introducerar Jest och Supertest, som tillsammans ger oss ett kraftfullt verktyg för att skriva automatiserade tester för våra Express-routes och Mongoose-operationer. Tester är inte ett lyxprojekt – de är ett professionellt verktyg som skyddar oss mot regressioner och ger oss trygghet att refaktorera kod.

Det är en innehållsrik vecka, men allt hänger ihop: fullständig CRUD ger oss operationerna, relationer ger oss datamodellen, och testning ger oss verifieringen. Tillsammans bildar de en solid grund för det professionella API-bygget som vi arbetar mot under resten av kursen.

Fullständiga CRUD-operationer med Mongoose

Mongoose erbjuder ett rikt API för att interagera med MongoDB. Vi arbetar med metoder direkt på modellen (`Model.find()`, `Model.findById()` osv.) och på enskilda dokument-instanser (`document.save()`, `document.deleteOne()`). Det är viktigt att vi förstår skillnaden: statiska metoder på modellen används för att söka och hämta, medan instansmetoder används för att manipulera ett specifikt dokument vi redan hämtat.

Vi använder en enkel bokmodell som exempel genomgående. Vi antar att modellen är definierad i en separat fil och exporterad – precis som vi lärde oss under vecka 4.

Skapa dokument – Create

Vi har redan sett hur man skapar dokument med `new Model()` och `.save()`. Det finns även en kortform med `Model.create()` som vi kan använda:

```
// Metod 1: new + save
const book = new Book({
  title: 'The Pragmatic Programmer',
  author: 'David Thomas',
  genre: 'technology',
  year: 1999
});
await book.save();

// Metod 2: Model.create() - kortare och skapar direkt
const book = await Book.create({
  title: 'The Pragmatic Programmer',
  author: 'David Thomas',
  genre: 'technology',
  year: 1999
});

// Skapa flera dokument på en gång
const books = await Book.insertMany([
  { title: 'Clean Code', author: 'Robert C. Martin', year: 2008 },
  { title: 'Refactoring', author: 'Martin Fowler', year: 1999 }
]);
```

`Model.create()` är en bekvämlighetsmetod som skapar en instans och anropar `.save()` – den är ekvivalent med `new + save` men kortare. `insertMany()` är effektivt när vi vill skapa flera dokument på en gång eftersom det gör ett enda databasanrop istället för ett per dokument.

Läsa dokument – Read

Mongoose erbjuder flera metoder för att hämta data:

```
// Hämta alla dokument
const allBooks = await Book.find();

// Hämta med filter (query-objekt)
const techBooks = await Book.find({ genre: 'technology' });
```

```
// Hämta ett enskilt dokument med _id
const book = await Book.findById('64a3c1f2e8b4a12345678901');

// Hämta ett dokument baserat på valfritt fält
const book = await Book.findOne({ title: 'Clean Code' });

// Välj specifika fält (projection) - exkludera __v och inkludera title och author
const books = await Book.find({}, { title: 1, author: 1, _id: 0 });
```

`findById()` är en genväg för `findOne({ _id: id })`. Om dokumentet inte hittas returnerar båda `null` – det är viktigt att vi hanterar det fallet i vår route handler och returnerar ett 404-svar.

Uppdatera dokument – Update

Det finns flera sätt att uppdatera dokument i Mongoose. Det viktigaste valet är om vi vill hämta dokumentet först (och sedan spara det) eller uppdatera direkt i databasen. Båda har sina fördelar.

```
// Metod 1: Hämta, modifiera, spara - triggar Mongoose validators och middleware
const book = await Book.findById(id);
if (!book) return res.status(404).json({ message: 'Boken hittades inte' });

book.title = req.body.title;
book.author = req.body.author;
await book.save();

// Metod 2: findByIdAndUpdate - ett databasanrop, returnerar dokumentet
const book = await Book.findByIdAndUpdate(
  id,
  { $set: { title: req.body.title } }, // $set uppdaterar bara angivna fält
  { new: true, runValidators: true } // new: true returnerar det uppdaterade dokumentet
);

// updateOne - uppdaterar utan att returnera dokumentet
await Book.updateOne({ _id: id }, { $set: { year: 2024 } });
```

Notera MongoDB-operatörn `$set` – den uppdaterar bara de angivna fälten och lämnar resten orörda. Om vi skickar ett helt objekt utan `$set` ersätts hela dokumentet. Det finns fler operatörer: `$push` lägger till ett element i en array, `$pull` tar bort ett element, `$inc` ökar ett numeriskt värde. Vi återkommer till dessa i praktiken.

När vi använder `findByIdAndUpdate` är det kritiskt att vi skickar med `{ new: true }` – annars returneras dokumentet som det såg ut innan uppdateringen, vilket kan leda till förvirrande buggar. `{ runValidators: true }` säkerställer att Mongoose-validators körs även vid uppdateringsoperationer.

Radera dokument – Delete

```
// findByIdAndDelete - hämtar och raderar i ett steg, returnerar det raderade dokumentet
const book = await Book.findByIdAndDelete(id);
```



```

if (!book) return res.status(404).json({ message: 'Boken hittades inte' });

// deleteOne - raderar utan att returnera dokumentet
const result = await Book.deleteOne({ _id: id });
// result.deletedCount är 1 om raderat, 0 om inget hittades

// deleteMany - raderar flera dokument
await Book.deleteMany({ genre: 'outdated' });

```

findByIdAndDelete är generellt att föredra framför deleteOne i route handlers, eftersom det returnerar det raderade dokumentet. Det låter oss bekräfta att dokumentet faktiskt existerade och skicka tillbaka ett meningsfullt svar. Om vi bara vill ta bort utan att returnera data är deleteOne effektivare.

Sätta ihop en fullständig CRUD-router

Nedan visas hur en komplett CRUD-router för böcker kan se ut. Notera hur vi hanterar fel konsekvent och returnerar rätt statuskoder:

```

// routes/books.js
const express = require('express');
const router = express.Router();
const Book = require('../models/Book');

// GET /books - Hämta alla böcker
router.get('/', async (req, res) => {
  try {
    const books = await Book.find();
    res.json(books);
  } catch (err) {
    res.status(500).json({ message: 'Serverfel' });
  }
});

// GET /books/:id - Hämta en specifik bok
router.get('/:id', async (req, res) => {
  try {
    const book = await Book.findById(req.params.id);
    if (!book) return res.status(404).json({ message: 'Boken hittades inte' });
    res.json(book);
  } catch (err) {
    res.status(500).json({ message: 'Serverfel' });
  }
});

// POST /books - Skapa en ny bok
router.post('/', async (req, res) => {
  try {
    const book = await Book.create(req.body);
    res.status(201).json(book);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

```

```
});

// PUT /books/:id - Uppdatera en bok
router.put('/:id', async (req, res) => {
  try {
    const book = await Book.findByIdAndUpdate(
      req.params.id,
      { $set: req.body },
      { new: true, runValidators: true }
    );
    if (!book) return res.status(404).json({ message: 'Boken hittades inte' });
    res.json(book);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// DELETE /books/:id - Radera en bok
router.delete('/:id', async (req, res) => {
  try {
    const book = await Book.findByIdAndDelete(req.params.id);
    if (!book) return res.status(404).json({ message: 'Boken hittades inte' });
    res.status(204).send();
  } catch (err) {
    res.status(500).json({ message: 'Serverfel' });
  }
});

module.exports = router;
```

Notera att DELETE returnerar statuskod 204 No Content – det är den korrekta koden när en operation lyckades men det inte finns något att returnera. Vi kan inte skicka en body med 204, därav `res.status(204).send()` utan innehåll.

Filtrera, sortera och paginera data

I verkliga applikationer vill vi sällan hämta alla dokument ur en samling på en gång – det kan röra sig om tusentals eller miljontals poster. Mongoose ger oss kraftfulla verktyg för att filtrera (hitta rätt subset), sortera (bestämma ordning) och paginera (dela upp i sidor).

Filtrera med queries

MongoDB-queries är objekt där vi anger villkor för varje fält vi vill filtrera på. Mongoose stödjer hela MongoDB query-språket:

```
// Exakt matchning
const books = await Book.find({ genre: 'technology' });

// Jämförelseoperatorer
// $gt (greater than), $gte, $lt, $lte, $ne (not equal)
const modernBooks = await Book.find({ year: { $gte: 2000 } });

// $in - matchar ett av värdena i arrayen
const books = await Book.find({ genre: { $in: ['technology', 'fiction'] } });

// Söka i strängar med reguljära uttryck (case-insensitive)
const books = await Book.find({
  title: { $regex: req.query.search, $options: 'i' }
});

// Kombinera villkor - AND (implicit)
const books = await Book.find({
  genre: 'technology',
  year: { $gte: 2010 }
});

// OR-villkor
const books = await Book.find({
  $or: [
    { author: 'Robert C. Martin' },
    { author: 'Martin Fowler' }
  ]
});
```

Sortering

```
// Sortera stigande (1) på title
const books = await Book.find().sort({ title: 1 });

// Sortera fallande (-1) på year
const books = await Book.find().sort({ year: -1 });

// Sortera på flera fält
const books = await Book.find().sort({ genre: 1, year: -1 });

// Hantera dynamisk sortering från query parameters
```

```
// GET /books?sort=year&order=desc
const sortField = req.query.sort || 'createdAt';
const sortOrder = req.query.order === 'desc' ? -1 : 1;
const books = await Book.find().sort({ [sortField]: sortOrder });
```

Paginering

Paginering är processen att dela upp ett stort resultat i mindre sidor. Det finns två vanliga mönster: offset-baserad paginering (sida 1, 2, 3...) och cursor-baserad paginering. Vi fokuserar på offset-baserad paginering som är vanligast och enklast att förstå.

```
// GET /books?page=2&limit=10

const page = parseInt(req.query.page) || 1; // Standardsida: 1
const limit = parseInt(req.query.limit) || 10; // Standardgräns: 10
const skip = (page - 1) * limit; // Hoppa över tidigare sidor

const [books, total] = await Promise.all([
  Book.find().sort({ createdAt: -1 }).skip(skip).limit(limit),
  Book.countDocuments()
]);

res.json({
  data: books,
  pagination: {
    total,
    page,
    limit,
    totalPages: Math.ceil(total / limit),
    hasNextPage: page < Math.ceil(total / limit),
    hasPrevPage: page > 1
  }
});
```

Vi använder `Promise.all()` för att köra de två databasanropen parallellt – ett för att hämta dokumenten och ett för att räkna totalt antal. Det är effektivare än att köra dem sekventiellt med `await`. Svaret inkluderar pagineringsmetadata som frontendar behöver för att rendera navigeringsknappar.

Notera att vi konverterar query parameters till heltal med `parseInt()`. Query parameters är alltid strängar, och om vi skickar en sträng till `skip()` eller `limit()` kan det ge oväntade resultat. Det är ett bra exempel på den princip vi fördjupar oss i under vecka 6: lita aldrig på indata från klienten utan validering.

Relationer i MongoDB: Embedding vs Referencing

En av de viktigaste designbesluten vi tar när vi modellerar data i MongoDB handlar om relationer: hur vi kopplar samman data av olika typer. Till skillnad från relationsdatabaser (SQL) som har ett enda svar – tabeller med främmande nycklar och JOIN-operationer – erbjuder MongoDB två huvudstrategier med sina respektive styrkor och svagheter.

Embedding – Inbäddning

Embedding innebär att vi lagrar relaterad data direkt inuti ett dokument, som ett underobjekt eller en array. Det är det enklaste och snabbaste alternativet när datan alltid används tillsammans.

```
// Exempel: En bloggpost med kommentarer inbäddade
const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  author: String,
  comments: [           // Array av underdokument
    {
      text: String,
      author: String,
      createdAt: { type: Date, default: Date.now }
    }
  ]
});

// Lägg till en kommentar - allt i ett databasanrop
const post = await Post.findById(postId);
post.comments.push({ text: 'Bra artikel!', author: 'Anna' });
await post.save();
```

Embedding passar bra när:

- Datan är tätt kopplad och alltid används tillsammans (t.ex. kommentarer och deras bloggpost)
- Datan på "barn"-sidan inte har för många poster (max ett par hundra)
- Datan sällan uppdateras separat från föräldradokumentet
- Vi vill ha atomiska uppdateringar (ändra allt på en gång)

Embedding passar dåligt när:

- Barndata växer obegränsat (MongoDB-dokument har en maxstorlek på 16 MB)
- Samma barndata delas mellan flera föräldradokument (t.ex. en bok som tillhör flera genrer som visas separat)

Referencing – Referenslagring

Referencing innebär att vi lagrar en referens (ett ObjectId) till ett dokument i en annan samling, precis som en främmande nyckel i SQL. Datan lagras i separata samlingar och kopplas samman vid behov.

```
// Bokmodell med referens till en author
```

```

const bookSchema = new mongoose.Schema({
  title: { type: String, required: true },
  genre: String,
  year: Number,
  // Referens till Author-samlingen med ObjectId
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Author',      // Namnet på modellen vi refererar till
    required: true
  }
});

// Authormodell
const authorSchema = new mongoose.Schema({
  name: { type: String, required: true },
  nationality: String,
  birthYear: Number
});

```

Referencing passar bra när:

- Datan är lös kopplad och används oberoende av varandra (vi vill kanske lista alla authors separat)
- Barndata kan vara obegränsad i storlek (t.ex. beställningar för en kund)
- Samma data refereras av flera dokument (en author har skrivit många böcker)
- Vi vill kunna uppdatera refererad data en gång och se uppdateringen överallt

Tumregeln för val av strategi

En enkel tumregel: om du konsekvent hämtar data A när du hämtar data B, och B inte existerar utan A, bör B vara inbäddad i A. Om du ibland hämtar A utan B, eller B kan existera självständigt, bör du använda referering.

Det finns också en hybrid-strategi som är vanlig i praktiken: vi lagrar grundläggande information om det refererade dokumentet inbäddat (t.ex. authorens namn) och en fullständig referens för när vi behöver mer information. Det minskar antalet databasanrop för de vanliga fallen.

```

// Hybrid: lagra grunddata + referens
const bookSchema = new mongoose.Schema({
  title: String,
  author: {
    _id: { type: mongoose.Schema.Types.ObjectId, ref: 'Author' },
    name: String // Inbäddad för snabb läsning utan populate
  }
});

```

Populate i Mongoose

När vi använder referenslagring och vill hämta refererad data tillsammans med huvuddokumentet använder vi Mongooses `populate()`-metod. Den ersätter ObjectId-referensen med det faktiska dokumentet från den refererade samlingen.

```
// Utan populate - vi får bara ObjectId
const book = await Book.findById(id);
// book.author = '64a3c1f2e8b4a12345678901' (bara ett id)

// Med populate - vi ersätter id:t med hela author-dokumentet
const book = await Book.findById(id).populate('author');
// book.author = { _id: '...', name: 'Robert C. Martin', nationality:
'American', ... }
```

`populate()` gör ett extra databasanrop för att hämta de refererade dokumenten. Det är viktigt att förstå denna kostnad – om vi hämtar hundra böcker och `populate`:ar `author` för var och en blir det 101 databasanrop (1 + 100). I praktiken batcher Mongoose dessa effektivt, men vi bör vara medvetna om att `populate` alltid innebär extra arbete.

Avancerad populate

```
// Populera specifika fält av den refererade resursen
const book = await Book.findById(id).populate('author', 'name nationality');
// Hämtar bara name och nationality från Author

// Populera flera fält
const order = await Order.findById(id)
  .populate('customer', 'name email')
  .populate('products', 'name price');

// Populera nästlade relationer (nested populate)
const post = await Post.findById(id).populate({
  path: 'comments',
  populate: {
    path: 'author', // Populera author inne i varje kommentar
    select: 'name'
  }
});

// Populate med filter - bara godkända kommentarer
const post = await Post.findById(id).populate({
  path: 'comments',
  match: { approved: true },
  select: 'text author createdAt'
});
```

Nästlad `populate` är kraftfullt men kan snabbt leda till många databasanrop. I produktionskritiska applikationer överväger man alternativa strategier som aggregation-pipelines eller denormalisering, men för de flesta projektet i den här utbildningen är `populate` ett utmärkt och lättbegripligt verktyg.

Indexering i MongoDB

Ett index i MongoDB fungerar precis som ett register i en bok – istället för att läsa varje sida för att hitta ett visst ord kan vi slå upp det direkt i registret. Utan index måste MongoDB göra en "collection scan" och titta på varje dokument i en samling, vilket blir allt långsammare ju fler dokument vi har.

MongoDB skapar automatiskt ett index på `_id`-fältet. För alla andra fält vi söker på regelbundet behöver vi skapa egna index.

Skapa index i Mongoose schemas

```
const bookSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    index: true           // Enkelt index på title
  },
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Author',
    index: true           // Index för populate och filtrering
  },
  genre: String,
  year: Number
});

// Sammansatt index (compound index) - optimerar queries som filtrerar på båda
bookSchema.index({ genre: 1, year: -1 });

// Unikt index - säkerställer att inga dubletter skapas
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true           // unique: true skapar automatiskt ett unikt index
  }
});

// Textindex - möjliggör MongoDB text search
bookSchema.index({ title: 'text', description: 'text' });
```

Index förbättrar lästider dramatiskt men har en kostnad: de tar upp diskutrymme och gör skrivoperationer (insert, update, delete) något långsammare eftersom index måste uppdateras. Den grundläggande tumregeln är att indexera fält som vi ofta söker, sorterar eller filtrerar på.

`unique: true` på ett fält är mer än ett prestandaverktyg – det är en dataintegritetskontroll som säkerställer att värdet är unikt i hela samlingen. Om vi försöker spara ett dokument med ett email som redan finns får vi ett `MongoServerError` med kod 11000. Det är viktigt att vi hanterar det felet i vår felhantering.

```
// Hantera duplicate key error
```

```
router.post('/users', async (req, res) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json(user);
  } catch (err) {
    if (err.code === 11000) {
      // Duplicate key error
      return res.status(409).json({ message: 'E-postadressen används redan' });
    }
    res.status(500).json({ message: 'Serverfel' });
  }
});
```

Introduktion till testning av API:er

Testning är ett av de ämnen som ofta nedprioriteras i början av en programmerares karriär – det känns som extra arbete när man bara vill bygga grejer. Men tester är en investering som betalar sig snabbt. Varje gång vi ändrar kod riskerar vi att introducera en regression – ett fel i något som fungerade tidigare. Utan automatiserade tester måste vi manuellt klicka igenom alla funktioner för att verifiera att ingenting gick sönder. Med tester kör vi en enda kommando och får svaret på sekunder.

Det finns olika typer av tester med olika scope och kostnad:

- Enhetstester (unit tests) – Testar en enskild funktion eller modul isolerat. Snabba att köra, lätta att förstå. Beroenden ersätts med mocks.
- Integrationstester (integration tests) – Testar att flera delar av systemet fungerar tillsammans. I vår kontext innebär det att testa en hel HTTP-route från anrop till databasoperation.
- End-to-end-tester (E2E) – Testar hela flödet från frontend till databas. Mest realistiska men också långsammaste och dyraste att underhålla.

Under den här kursen fokuserar vi på integrationstester för våra Express-routes, vilket är det mest värdefulla testnivån för en backend. Vi testar att rätt HTTP-statuskod returneras, att response body:n har rätt format, och att databasoperationerna faktiskt genomförs korrekt.

Jest och Supertest

Jest är det dominerande testramverket för JavaScript och det vi använde i kursen React, TypeScript och React Native. Det ger oss ett komplett testverktyg med testrunner, assertions och mocking-kapabiliteter i ett paket. Supertest är ett kompletterande bibliotek som låter oss göra HTTP-anrop mot vår Express-applikation i tester – utan att behöva starta en riktig server.

Installation och konfiguration

```
npm install --save-dev jest supertest

// package.json - lägg till test script
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  },
  "jest": {
    "testEnvironment": "node"
  }
}
```

Jest hittar automatiskt testfiler som slutar på .test.js eller .spec.js, eller filer i en __tests__-mapp. Vi placerar våra testfiler bredvid koden de testar, exempelvis books.test.js bredvid books.js.

Grundläggande Jest-syntax

Innan vi skriver integrationstester är det bra att känna till grunderna i Jest:

```
// grundläggande test-struktur
describe('Böcker API', () => {
  // describe grupperar relaterade tester

  test('ska returnera 200 och en array', () => {
    // test (alias: it) definierar ett enskilt testfall
    expect(2 + 2).toBe(4);
    expect([1, 2, 3]).toHaveLength(3);
    expect({ name: 'Anna' }).toMatchObject({ name: 'Anna' });
  });
});

// Vanliga Jest matchers
expect(value).toBe(exact);           // Strikt likhet (===)
expect(value).toEqual(deep);         // Djup likhet (objekt, arrayer)
expect(value).toBeTruthy();           // Truthy-värde
expect(value).toBeNull();            // null
expect(value).toBeUndefined();        // undefined
expect(value).toContain(item);        // Array innehåller element
expect(value).toHaveLength(n);        // Array/sträng har längd n
expect(fn).toThrow();                // Funktion kastar ett fel
```

```
expect(value).toMatchObject(obj); // Objekt matchar delvis
```

Integrationstester med Supertest

Supertest importerar vår Express-applikation (app.js) och låter oss skicka riktiga HTTP-anrop mot den utan att starta en server. Det är viktigt att vi separerat app.js från server.js (som vi lärde oss i vecka 1) – Supertest behöver app-objektet, inte servern.

```
// __tests__/books.test.js
const request = require('supertest');
const app = require('../app'); // Express-appen utan server.listen()

describe('GET /api/v1/books', () => {

  test('ska returnera statuskod 200', async () => {
    const response = await request(app).get('/api/v1/books');
    expect(response.status).toBe(200);
  });

  test('ska returnera en array', async () => {
    const response = await request(app).get('/api/v1/books');
    expect(Array.isArray(response.body)).toBe(true);
  });
});

describe('POST /api/v1/books', () => {

  test('ska skapa en ny bok och returnera 201', async () => {
    const newBook = {
      title: 'Test Book',
      author: 'Test Author',
      genre: 'technology'
    };

    const response = await request(app)
      .post('/api/v1/books')
      .send(newBook) // Skicka JSON body
      .set('Content-Type', 'application/json');

    expect(response.status).toBe(201);
    expect(response.body).toMatchObject({ title: 'Test Book' });
    expect(response.body._id).toBeDefined();
  });
});

describe('GET /api/v1/books/:id', () => {

  test('ska returnera 404 för okänt id', async () => {
    const fakeId = '000000000000000000000000'; // Giltigt ObjectId-format
    const response = await request(app).get('/api/v1/books/' + fakeId);
    expect(response.status).toBe(404);
  });
});
```

```
});
```

Supertest gör det enkelt att testa alla delar av en HTTP-respons: status kod, body, headers och mer. Vi kan kedja metoder för att bygga upp anropet: `.get()`, `.post()`, `.put()`, `.delete()` för metod, `.send()` för body, `.set()` för headers, och så vidare.

Testa mot en in-memory-databas

Det finns ett problem med integrationstester som interagerar med en riktig databas: testerna förorenar produktions- eller utvecklingsdatabasen med testdata, de är beroende av att databasen är tillgänglig, och de kan påverka varandra om de delar data. Lösningen är att använda en in-memory-databas som startas och stoppas under testsviten.

mongodb-memory-server är ett paket som startar en riktig MongoDB-instans i minnet, utan att skriva data till disk. Det ger oss en isolerad, snabb testmiljö som alltid börjar med en ren databas.

Installation och konfiguration

```
npm install --save-dev mongodb-memory-server
```

Vi skapar en hjälpfil för databasanslutningen i tester som vi importerar i varje testfil:

```
// __tests__/helpers/db.js
const mongoose = require('mongoose');
const { MongoMemoryServer } = require('mongodb-memory-server');

let mongoServer;

// Starta in-memory-databas och koppla Mongoose
async function connect() {
  mongoServer = await MongoMemoryServer.create();
  const uri = mongoServer.getUri();
  await mongoose.connect(uri);
}

// Rensa alla samlingar mellan tester
async function clearDatabase() {
  const collections = mongoose.connection.collections;
  for (const key in collections) {
    await collections[key].deleteMany();
  }
}

// Stäng Mongoose-anslutningen och stoppa in-memory-servern
async function disconnect() {
  await mongoose.connection.dropDatabase();
  await mongoose.connection.close();
  await mongoServer.stop();
}

module.exports = { connect, clearDatabase, disconnect };
```

Kompletta integrationstester med in-memory-databas

Nu kan vi skriva kompletta tester som faktiskt sparar och hämtar data från databasen – men utan att röra vår riktiga databas:

```
// __tests__/books.test.js
const request = require('supertest');
const app = require('../app');
const Book = require('../models/Book');
const db = require('../helpers/db');

// Jest hooks - körs vid rätt tillfällen i testsviten
beforeAll(async () => {
  await db.connect(); // Starta databas innan alla tester
});

afterEach(async () => {
  await db.clearDatabase(); // Rensa data mellan varje test
});

afterAll(async () => {
  await db.disconnect(); // Stäng databas efter alla tester
});

describe('Böcker CRUD', () => {

  describe('GET /api/v1/books', () => {
    test('returnerar tom array när inga böcker finns', async () => {
      const res = await request(app).get('/api/v1/books');
      expect(res.status).toBe(200);
      expect(res.body).toHaveLength(0);
    });

    test('returnerar alla böcker', async () => {
      // Skapa testdata direkt i databasen
      await Book.create([
        { title: 'Bok A', author: 'Autor A', genre: 'fiction' },
        { title: 'Bok B', author: 'Autor B', genre: 'technology' }
      ]);

      const res = await request(app).get('/api/v1/books');
      expect(res.status).toBe(200);
      expect(res.body).toHaveLength(2);
    });
  });

  describe('POST /api/v1/books', () => {
    test('skapar en ny bok med giltiga data', async () => {
      const bookData = { title: 'Ny Bok', author: 'Ny Autor', genre:
'fiction' };

      const res = await request(app)
        .post('/api/v1/books')
        .send(bookData);

      expect(res.status).toBe(201);
      expect(res.body.title).toBe('Ny Bok');
      expect(res.body._id).toBeDefined();
    });
  });
});
```



```

    // Verifiera att boken faktiskt sparades i databasen
    const dbBook = await Book.findById(res.body._id);
    expect(dbBook).not.toBeNull();
  });
});

describe('DELETE /api/v1/books/:id', () => {
  test('raderar en befintlig bok', async () => {
    const book = await Book.create({ title: 'Ta bort mig', author: 'X', genre:
'fiction' });

    const res = await request(app).delete('/api/v1/books/' + book._id);
    expect(res.status).toBe(204);

    // Verifiera att boken verkligen raderades
    const dbBook = await Book.findById(book._id);
    expect(dbBook).toBeNull();
  });

  test('returnerar 404 för ett id som inte finns', async () => {
    const res = await
request(app).delete('/api/v1/books/000000000000000000000000');
    expect(res.status).toBe(404);
  });
});
})

```

Jest-hooks (beforeAll, afterEach, afterAll) ger oss kontroll över testlivscykeln. Vi använder beforeAll för att starta databasen en gång, afterEach för att rensa data mellan varje enskilt test (vilket säkerställer att tester inte påverkar varandra), och afterAll för att stänga anslutningen när alla tester är klara.

Lägg märke till att vi i vissa tester verifierar databasen direkt med Mongoose-anrop (Book.findById()). Det är ett viktigt mönster: vi vill inte bara testa att API:et returnerar rätt svar, vi vill också testa att datan faktiskt manipulerades korrekt i databasen.

Köra testerna

```

npm test          # Kör alla tester en gång
npm run test:watch # Kör tester i watch-läge (kör om vid filändringar)
npm run test:coverage # Kör tester och generera kodtäckningsrapport

```

Kodtäckningsrapporten (coverage) visar vilka rader kod som exekveras av testerna. Hög täckning är bra men är inte ett mål i sig – det är bättre att ha meningsfulla tester med 70% täckning än meningslösa tester med 100%. Fokusera på att testa de kritiska flödena: lyckade operationer, felhantering vid ogiltiga input, och 404-svar när resurser inte hittas.

Projektstruktur för ett testbart API

En testbar kodbas kräver en tydlig separation av ansvarsområden. Här är en rekommenderad mappstruktur som fungerar väl med Jest och Supertest:

```
mitt-api/  
  src/  
    routes/  
      books.js  
      books.test.js      # Testfil bredvid routen  
    controllers/  
      booksController.js  
    models/  
      Book.js  
  app.js                  # Express-app utan server.listen()  
  server.js               # Startar servern  
  __tests__/  
    helpers/  
      db.js               # Databashjälpare för tester  
  jest.config.js  
  package.json
```

Det är viktigt att vår app.js-fil inte anropar `mongoose.connect()` direkt – anslutningslogiken bör vara separat så att tester kan ansluta till in-memory-databasen istället. En vanlig lösning är att ha en `connect()`-funktion som anropas från `server.js` (vid körning) eller från testets `beforeAll` (vid testning).

```
// src/app.js - Exporterar bara Express-applikationen  
const express = require('express');  
const booksRouter = require('./routes/books');  
  
const app = express();  
app.use(express.json());  
app.use('/api/v1/books', booksRouter);  
  
module.exports = app;  
  
// src/server.js - Ansluter till databas och startar servern  
const mongoose = require('mongoose');  
const app = require('./app');  
  
const PORT = process.env.PORT || 3000;  
const MONGO_URI = process.env.MONGO_URI;  
  
mongoose.connect(MONGO_URI).then(() => {  
  app.listen(PORT, () => {  
    console.log('Server körs på port ' + PORT);  
  });  
});
```

Checkpoint: Förberedelse inför kursvecka 5

Den här veckan tar vi det vi lärt oss om MongoDB och Mongoose ett stort steg vidare. För att vi ska kunna använda workshoptiden väl är det viktigt att ni har läst igenom detta studiematerial, kör era projekt från vecka 4, och har en fungerande databasanslutning att bygga vidare på.

Kontrollera att ni har följande klart inför workshopen:

1. Projektet från vecka 4 körs utan fel – Express-server ansluten till MongoDB Atlas
2. Ni har minst en Mongoose-modell definierad med ett schema
3. `npm install --save-dev jest supertest mongodb-memory-server` är klart i projektet
4. Test-scriptet är definierat i `package.json`

Ta med er dessa frågor och reflektioner till workshopen: Hur hade ni modellerat relationen mellan böcker och författare i en SQL-databas? Vilka skillnader ser ni jämfört med embedding och referencing i MongoDB? Varför tror ni det är viktigt att testa mot en separat testdatabas och inte den riktiga?

Sammanfattning

Under kursvecka 5 har vi tagit MongoDB och Mongoose från grunderna till ett komplett och testbart CRUD-API. Vi har täckt de fyra grundläggande operationerna – Create, Read, Update och Delete – med Mongooses fullständiga metoduppsättning: `Model.create()`, `Model.find()`, `Model.findById()`, `Model.findByIdAndUpdate()` och `Model.findByIdAndDelete()`.

Vi har lärt oss att filtrera resultat med MongoDB query-operatorer (`$gt`, `$in`, `$regex` och fler), sortera data med `.sort()` och paginera stora datamängder med `.skip()` och `.limit()`. Dessa verktyg är grundläggande för att bygga API:er som skalas bra och inte returnerar onödigt mycket data till klienten.

Vi utforskade de två huvudstrategierna för relationer i MongoDB: embedding och referencing. Embedding passar för tätt kopplad data som alltid används tillsammans och inte växer obegränsat. Referencing passar för löst kopplad data som existerar självständigt eller delas mellan många dokument. `Populate()`-metoden låter oss ersätta ObjectId-referenser med fullständiga dokument i ett enkelt metदानrop.

Indexering är ett viktigt prestandaverktyg: index förbättrar sökhastigheten drastiskt men har en kostnad i form av diskutrymme och något långsammare skrivoperationer. Unique index är dessutom ett dataintegritetsskydd som förhindrar dubletter, och vi lärde oss att hantera `duplicate key errors` (felkod 11000) korrekt.

Slutligen introducerade vi testning med Jest och Supertest. Integrationstester mot Express-routes ger oss trygghet att funktionaliteten fungerar som förväntat, och mongodb-memory-server ger oss en isolerad, snabb testmiljö utan att röra vår riktiga databas. Jest-hooks (`beforeAll`, `afterEach`, `afterAll`) ger oss kontroll över testlivscykeln.

Nästa vecka fördjupar vi oss i inputvalidering och robust felhantering – de lager av säkerhet som skyddar vår databas och våra API-svar från felaktig och skadlig data. Boiler room-projektet fortsätter, och vi förväntar oss att ni nu implementerar fullständig CRUD med relationer i ert gemensamma projekt.

Resurser för fördjupning

Följande resurser rekommenderas för de som vill fördjupa sina kunskaper om CRUD-operationer, relationer och testning.

Officiell dokumentation

Mongoose-dokumentation – fullständig referens för queries, populate, indexering och mycket mer:

<https://mongoosejs.com/docs/>

MongoDB Query Operators – referens för alla query-operatorer (\$gt, \$in, \$regex osv.):

<https://www.mongodb.com/docs/manual/reference/operator/query/>

Jest-dokumentation – komplett referens för alla matchers, hooks och konfiguration:

<https://jestjs.io/docs/getting-started>

Supertest – GitHub-repository med exempel och API-dokumentation:

<https://github.com/ladjs/supertest>

mongodb-memory-server – dokumentation för konfiguration och användning i tester:

<https://github.com/nodkz/mongodb-memory-server>

Fördjupning

"6 Rules of Thumb for MongoDB Schema Design" av MongoDB: En djupdykning i hur man tar beslut om embedding vs referencering med konkreta exempel. Rekommenderas starkt för att befästa förståelsen för relationsmodellering:

<https://www.mongodb.com/blog/post/6-rules-of-thumb-for-mongodb-schema-design>

"Testing Node.js + Mongoose with an in-memory database" är en artikel som visar exakt det mönster vi använde under den här veckan. Bra att ha som referens när ni sätter upp testmiljön i ert projekt:

<https://dev.to/paulasantamaria/testing-node-js-mongoose-with-an-in-memory-database-32np>

Videomaterial

"Mongoose Crash Course" av Traversy Media på YouTube är en kompakt genomgång av Mongoose som täcker både CRUD och relationer. Den är ett bra komplement till detta studiematerial och rekommenderas för de som vill se koden i aktion.

"Jest Crash Course" av Traversy Media är en utmärkt introduktion till Jest som testar verktyg. Den täcker grunderna vi gick igenom i detta material och hjälper er att komma igång snabbt med att skriva egna tester.

Studiematerial

Inputvalidering och felhantering

Kursvecka 6

Introduktion

Vi har nu fem veckor bakom oss i kursen. Vi behärskar Node.js och Express, kan designa REST API:er med tydlig struktur, har lagt till TypeScript för säkrare typning och kopplat vår backend till MongoDB via Mongoose. Vi kan hämta, skapa, uppdatera och ta bort data – och vi har börjat testa vår kod. Det är en imponerande grund.

Men det finns ett hål i det vi byggt hittills. Vi har i stor utsträckning litat på att klienten skickar korrekt data. Vi har antagit att request body innehåller rätt fält, att params är i rätt format och att query-parametrar har rimliga värden. Det är ett antagande vi aldrig kan göra i ett riktigt system.

Kursvecka 6 handlar om att täppa till det hålet på två sätt: genom robust inputvalidering och genom genomtänkt felhantering. Dessa två ämnen hänger tätt samman – vi validerar input för att fånga fel tidigt, och vi hanterar felen på ett sätt som är säkert, konsekvent och användbart för den som anropar vårt API.

Vi introducerar Zod – ett kraftfullt valideringsbibliotek som låter oss deklarera exakt vad vi förväntar oss av inkommande data och automatiskt generera meningsfulla felmeddelanden när data inte stämmer. Vi bygger också ett mönster för centraliserad felhantering i Express som gör att alla fel, oavsett var de uppstår i applikationen, hanteras på ett enhetligt sätt.

Det är viktigt att förstå att detta inte bara handlar om att göra API:et trevligare att använda. Det handlar om säkerhet. Okontrollerad input är en av de vanligaste vägarna in för attacker mot webbapplikationer. Att validera och städa indata är ett av de grundläggande försvarsmönstren vi alltid ska tillämpa.

Principen: Don't trust the client

Den kanske viktigaste principen inom backendutveckling och webbsäkerhet kan sammanfattas i fyra ord: don't trust the client. Det innebär att vi aldrig kan förlita oss på att data som skickas till vår server är korrekt, komplett, välformaterad eller ens välmenande.

Det kan tyckas självklart att en användare kan skicka fel data av misstag. Det som är viktigare att förstå är att en angripare aktivt kan skicka skadlig data med avsikt att bryta vår applikation, kringgå vår affärslogik eller komma åt data de inte har rätt till. Och det behövs inte avancerad kunskap – det räcker med ett verktyg som Postman eller curl för att skicka precis vilket HTTP-anrop som helst till vår server.

Vad kan en angripare skicka?

Låt oss konkretisera vad vi menar. Om vi har en endpoint för att registrera en användare:

```
POST /api/users
{ "name": "Anna", "email": "anna@example.com", "age": 25 }
```

En angripare kan skicka:

- Felaktiga typer: { "age": "tjugofem" } – ett textvärde istället för ett tal.
- Extremt långa strängar: { "name": "A".repeat(1000000) } – för att försöka orsaka minnesöverflöd.
- Saknade obligatoriska fält: {} – ett tomt objekt för att se hur servern reagerar.
- Extra fält: { "name": "Anna", "role": "admin" } – för att försöka tilldela sig admin-rättigheter.
- Skadlig kod i strängar: { "name": "<script>alert(1)</script>" } – för att testa om vår backend lagrar och returnerar data osanerad.
- Specialtecken och operatorer: { "email": { "\$gt": "" } } – ett klassiskt mönster för NoSQL-injektion i MongoDB.

Varför räcker inte frontend-validering?

Vi har i kurs 2 och 3 arbetat med formulärvalidering på klientsidan. Det är en viktig del av användarupplevelsen – det ger omedelbar återkoppling och förhindrar onödiga nätverksanrop. Men det är inte ett säkerhetsskydd.

Det finns tre fundamentala skäl till varför frontend-validering aldrig är tillräcklig:

1. Klienten kan kringgå. Vem som helst kan använda Postman, curl eller ett eget skript för att skicka HTTP-anrop direkt till vår server, utan att gå via vår React-app.
2. JavaScript kan stängas av. En angripare kan enkelt inaktivera JavaScript i webbläsaren och skicka formulär utan validering.
3. Vi kontrollerar inte klienten. I ett API som används av flera klienter (webb, mobilapp, tredjepartsintegrationer) har vi ingen garanti om att alla klienter validerar korrekt.

Slutsatsen är enkel men viktig: all validering som har säkerhetsmässig betydelse måste göras på servern. Frontend-validering är ett UX-verktyg, inte ett säkerhetsverktyg. Servervalidering är det enda vi kan kontrollera och det enda vi kan lita på.

Manuell validering i Express

Innan vi introducerar Zod är det värdefullt att förstå hur validering ser ut utan externa bibliotek. Det ger oss en känsla för problemet och varför ett strukturerat verktyg är så hjälpsamt.

Anta att vi har en route för att skapa en produkt:

```
app.post('/api/products', (req, res) => {
  const { name, price, category } = req.body;

  // Manuell validering
  if (!name || typeof name !== 'string' || name.trim().length === 0) {
    return res.status(400).json({ message: 'name är obligatoriskt och måste vara en sträng' });
  }

  if (typeof price !== 'number' || price <= 0) {
    return res.status(400).json({ message: 'price måste vara ett positivt tal' });
  }

  if (!category || ![ 'electronics', 'clothing', 'food' ].includes(category)) {
    return res.status(400).json({ message: 'category måste vara electronics, clothing eller food' });
  }

  // Skapa produkten om allt ser bra ut
  const product = { name: name.trim(), price, category };
  res.status(201).json(product);
});
```

Det här fungerar, men det har uppenbara problem. Valideringskoden tar mer plats än affärslogiken. Felmeddelanden är ojämna – ibland returnerar vi ett fel, ibland ett annat. Om vi har tio endpoints med likartad validering måste vi upprepa samma mönster tio gånger. Och om vi vill ändra felformatet måste vi uppdatera varje endpoint för sig.

Det finns ytterligare ett problem: om flera fält är felaktiga rapporterar vi bara det första felet vi stöter på. Klienten måste skicka om anropet om och om igen för att hitta alla fel. Det är dålig upplevelse och ineffektivt.

Det är här valideringsbibliotek som Zod kommer in. De löser exakt dessa problem: de gör validering deklarativ, konsekvent och möjlig att återanvända.

Introduktion till Zod

Zod är ett TypeScript-first valideringsbibliotek med en elegant och läsbar API. Det låter oss definiera schemas – beskrivningar av hur data ska se ut – och sedan validera verklig data mot dessa schemas. Zod är TypeScript-first men fungerar alldeles utmärkt med vanligt JavaScript, och det är hur vi primärt kommer att använda det i den här kursen.

Vi installerar Zod i projektet:

```
npm install zod
```

Grundläggande Zod-schemas

Vi importerar Zod och skapar våra första schemas:

```
const { z } = require('zod');

// Enkla primitiva schemas
const nameSchema = z.string();
const ageSchema = z.number();
const activeSchema = z.boolean();

// Validera ett värde
const result = nameSchema.safeParse('Anna');
console.log(result); // { success: true, data: 'Anna' }

const badResult = nameSchema.safeParse(42);
console.log(badResult);
// { success: false, error: ZodError { issues: [...] } }
```

Vi ser direkt ett viktigt mönster: vi använder `safeParse` istället för `parse`. Skillnaden är att `parse` kastar ett undantag om valideringen misslyckas, medan `safeParse` alltid returnerar ett objekt med antingen `{ success: true, data }` eller `{ success: false, error }`. I en Express-route vill vi alltid använda `safeParse` för att undvika okontrollerade undantag.

Strängregler och constraints

Zod erbjuder en rik uppsättning metoder för att sätta regler på schemas:

```
const { z } = require('zod');

// Strängar med regler
const nameSchema = z
  .string()
  .min(2, 'Namn måste ha minst 2 tecken')
  .max(100, 'Namn får inte överstiga 100 tecken')
  .trim(); // Ta bort whitespace automatiskt

const emailSchema = z
  .string()
  .email('Ogiltig e-postadress');

const urlSchema = z
```

```

.string()
.url('Ogiltig URL');

// Tal med regler
const priceSchema = z
  .number()
  .positive('Pris måste vara positivt')
  .max(999999, 'Pris får inte överstiga 999 999');

const quantitySchema = z
  .number()
  .int('Antal måste vara ett heltal')
  .min(1, 'Antal måste vara minst 1');

// Enum - ett av de tillåtna värdena
const roleSchema = z.enum(['admin', 'user', 'moderator']);

// Valfritt fält
const descriptionSchema = z.string().optional();

```

Objektschemas med z.object()

Det mest användbara Zod-verktyget för API-validering är `z.object()`. Det låter oss definiera ett helt objekt med alla sina fält och deras respektive regler:

```

const { z } = require('zod');

const createProductSchema = z.object({
  name: z.string().min(2).max(100).trim(),
  price: z.number().positive(),
  category: z.enum(['electronics', 'clothing', 'food']),
  description: z.string().max(500).optional(),
  inStock: z.boolean().default(true),
});

// Testa validering
const result = createProductSchema.safeParse({
  name: 'Laptop',
  price: 8999,
  category: 'electronics',
});

if (result.success) {
  console.log(result.data);
  // { name: 'Laptop', price: 8999, category: 'electronics', inStock: true }
} else {
  console.log(result.error.issues);
}

```

Notera att Zod automatiskt fyller i standardvärdet för `inStock` (`true`) eftersom vi använde `.default(true)`. Det gör att vi kan ha vettiga standardvärden utan att behöva hantera dem manuellt i route-logiken.

En annan viktig sak att känna till: `z.object()` tillåter som standard extra fält i det inkommande objektet. Om klienten skickar `{ name: 'Laptop', price: 8999, category: 'electronics', role: 'admin' }` kommer `role` att ignoreras i `result.data`. Det är ett viktigt säkerhetsmönster – vi slipper oroa oss för att extra fält smyger sig in i vår databaslagring. Men vi kan också vara explicita med det via `.strict()` om vi vill att extra fält ska ge ett fel.

Felmeddelanden från Zod

En av Zods styrkor är att den samlar alla valideringsfel och rapporterar dem simultant. Om klienten skickar ett objekt med tre felaktiga fält får de tillbaka alla tre felen på en gång, inte bara det första:

```
const result = createProductSchema.safeParse({
  name: 'A', // För kort
  price: -100, // Negativt
  category: 'furniture', // Inte ett tillåtet värde
});

if (!result.success) {
  console.log(result.error.issues);
  // [
  //   { path: ['name'], message: 'String must contain at least 2
character(s)' },
  //   { path: ['price'], message: 'Number must be greater than 0' },
  //   { path: ['category'], message: "Invalid enum value..." }
  // ]
}
```

`result.error.issues` är en array av felobjekt. Varje objekt har en `path` (sökvägen till det felaktiga fältet) och ett `message` (en beskrivning av felet). Vi kan använda det här för att bygga ett konsekvent felrespons-format som vi returnerar till klienten.

Validera body, params och query

I ett Express-API finns det tre platser data kan komma från: request body (req.body), URL-parametrar (req.params) och query-parametrar (req.query). Vi behöver schemas för alla tre.

Validera request body

Body-validering är vanligast och enklast – vi validerar ett JSON-objekt direkt mot ett `z.object()`-schema:

```
const { z } = require('zod');

const createUserSchema = z.object({
  name: z.string().min(2).max(50).trim(),
  email: z.string().email(),
  age: z.number().int().min(13).max(120).optional(),
});

app.post('/api/users', (req, res) => {
  const result = createUserSchema.safeParse(req.body);

  if (!result.success) {
    return res.status(400).json({
      message: 'Valideringsfel',
      errors: result.error.issues.map(issue => ({
        field: issue.path.join('.'),
        message: issue.message,
      })),
    });
  }

  // result.data är nu validerad och säker data
  const { name, email, age } = result.data;
  // Skapa användaren...
  res.status(201).json({ name, email, age });
});
```

Det är viktigt att vi alltid använder `result.data` (den validerade och eventuellt transformerade datan) och inte `req.body` direkt efter validering. Zod kan ha gjort transformationer – som `.trim()` på strängar eller `.default()` för saknade fält – och det är `result.data` som reflekterar det slutliga värdet.

Validera URL-parametrar (req.params)

URL-parametrar är alltid strängar – Express konverterar dem inte. En anrop till `/api/products/42` ger `req.params.id === '42'` (sträng), inte 42 (tal). Det innebär att vi måste hantera konvertering i våra schemas:

```
const { z } = require('zod');

// Schema för att validera och konvertera ett ID-param
const idParamSchema = z.object({
  id: z.string().regex(/^([0-9a-fA-F]{24})$/, 'Ogiltigt MongoDB ObjectId'),
```

```
});

// Alternativt, för numeriska ID:n
const numericIdSchema = z.object({
  id: z.string().transform(val => parseInt(val, 10)).pipe(
    z.number().int().positive()
  ),
});

app.get('/api/products/:id', (req, res) => {
  const result = idParamSchema.safeParse(req.params);

  if (!result.success) {
    return res.status(400).json({ message: 'Ogiltigt ID-format' });
  }

  const { id } = result.data;
  // Hämta produkt med validerat ID...
  res.json({ id });
});
```

Validera query-parametrar (req.query)

Query-parametrar är också alltid strängar. Om klienten anropar `/api/products?page=2&limit=10` är `req.query.page === '2'` och `req.query.limit === '10'`. Vi behöver konvertera till tal och sätta rimliga gränsvärden:

```
const { z } = require('zod');

const paginationSchema = z.object({
  page: z
    .string()
    .optional()
    .default('1')
    .transform(val => parseInt(val, 10))
    .pipe(z.number().int().min(1)),
  limit: z
    .string()
    .optional()
    .default('20')
    .transform(val => parseInt(val, 10))
    .pipe(z.number().int().min(1).max(100)),
  search: z.string().optional(),
});

app.get('/api/products', (req, res) => {
  const result = paginationSchema.safeParse(req.query);

  if (!result.success) {
    return res.status(400).json({ message: 'Ogiltiga query-parametrar' });
  }
});
```

```
const { page, limit, search } = result.data;
// page och limit är nu riktiga tal, ej strängar
const skip = (page - 1) * limit;
res.json({ page, limit, skip, search });
});
```

Kombinationen av `.optional().default().transform().pipe()` kan se komplex ut vid första anblicken, men det är ett välstrukturerat mönster. Vi säger: 'Om värdet saknas, använd standardvärdet. Konvertera sedan strängen till ett tal. Validera att talet uppfyller kraven.' Det är deklarativt och läsbart när man väl förstår mönstret.

Återanvändbara valideringsmiddleware

Det vi gjort hittills har ett problem: vi upprepar valideringsmönstret i varje route. Om vi har tio routes som behöver validering har vi tio ställen att underhålla. Lösningen är att abstrahera valideringen till en återanvändbar middleware-funktion.

Vi kan skapa en generisk middleware-fabrik – en funktion som tar ett Zod-schema och returnerar en middleware-funktion:

```
const { z } = require('zod');

// En fabriksfunktion som skapar validerings-middleware
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);

    if (!result.success) {
      return res.status(400).json({
        message: 'Valideringsfel',
        errors: result.error.issues.map(issue => ({
          field: issue.path.join('.'),
          message: issue.message,
        })),
      });
    }

    // Spara validerad data på req-objektet
    req.validatedBody = result.data;
    next();
  };
}

// Schemas
const createProductSchema = z.object({
  name: z.string().min(2).max(100).trim(),
  price: z.number().positive(),
  category: z.enum(['electronics', 'clothing', 'food']),
});

const updateProductSchema = z.object({
  name: z.string().min(2).max(100).trim().optional(),
  price: z.number().positive().optional(),
  category: z.enum(['electronics', 'clothing', 'food']).optional(),
});

// Användning i routes
app.post('/api/products', validate(createProductSchema), (req, res) => {
  // req.validatedBody innehåller validerad data
  const { name, price, category } = req.validatedBody;
  // Skapa produkt...
  res.status(201).json({ name, price, category });
});
```

```
app.patch('/api/products/:id', validate(updateProductSchema), (req, res) => {
  const updates = req.validatedBody;
  // Uppdatera produkt...
  res.json({ updated: true, data: updates });
});
```

Det här mönstret är kraftfullt. Vi separerar valideringslogiken från affärslogiken, vi återanvänder middleware-koden, och varje route-handler kan förlita sig på att `req.validatedBody` innehåller korrekt och säker data. Route-handlens enda ansvar är affärslogiken.

Validera fler delar av requesten

Vi kan utöka vår middleware-fabrik för att validera params och query också:

```
function validateRequest(schemas) {
  return (req, res, next) => {
    const errors = [];

    if (schemas.body) {
      const result = schemas.body.safeParse(req.body);
      if (!result.success) {
        errors.push(...result.error.issues.map(i => ({
          location: 'body',
          field: i.path.join('.'),
          message: i.message,
        })));
      } else {
        req.validatedBody = result.data;
      }
    }

    if (schemas.params) {
      const result = schemas.params.safeParse(req.params);
      if (!result.success) {
        errors.push(...result.error.issues.map(i => ({
          location: 'params',
          field: i.path.join('.'),
          message: i.message,
        })));
      } else {
        req.validatedParams = result.data;
      }
    }

    if (schemas.query) {
      const result = schemas.query.safeParse(req.query);
      if (!result.success) {
        errors.push(...result.error.issues.map(i => ({
          location: 'query',
          field: i.path.join('.'),
          message: i.message,
        })));
      }
    }
  }
}
```

```
    }));
  } else {
    req.validatedQuery = result.data;
  }
}

if (errors.length > 0) {
  return res.status(400).json({ message: 'Valideringsfel', errors });
}

next();
};
}

// Exempel: validera params och body
const idSchema = z.object({
  id: z.string().regex(/^([0-9a-fA-F]{24})$/, 'Ogiltigt ID'),
});

app.put('/api/products/:id',
  validateRequest({ params: idSchema, body: updateProductSchema }),
  (req, res) => {
    const { id } = req.validatedParams;
    const updates = req.validatedBody;
    // Uppdatera produkt med id och updates...
    res.json({ id, updates });
  }
);
```

Mongoose-validering vs applikationsvalidering

Vi har nu två ställen där validering sker: i Mongoose-schemas (databassidan) och i Zod-schemas (applikationssidan). Det kan verka som dubbelt arbete, men de fyller faktiskt olika syften och kompletterar varandra.

Mongoose-valideringens roll

Från vecka 4 minns vi att vi kan sätta valideringsregler direkt i Mongoose-schemas:

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Namn är obligatoriskt'],
    minlength: [2, 'Namn måste ha minst 2 tecken'],
    maxlength: [100, 'Namn får inte överstiga 100 tecken'],
    trim: true,
  },
  price: {
    type: Number,
    required: true,
    min: [0, 'Pris kan inte vara negativt'],
  },
  category: {
    type: String,
    enum: ['electronics', 'clothing', 'food'],
    required: true,
  },
});
```

Mongoose-validering är det sista försvaret – det händer precis innan data skrivs till databasen. Det är ett viktigt skyddsnät, men det är inte den primära validerings-lagret vi ska förlita oss på.

Varför vi behöver båda

De två valideringsnivåerna fyller fundamentalt olika roller:

- **Zod (applikationsnivå):** Validerar inkommande HTTP-data innan den ens bearbetas. Ger omedelbara, API-vänliga felmeddelanden med rätt HTTP-statuskoder. Kan validera saker som Mongoose inte förstår, exempelvis format på query-parametrar eller headers. Det är vår frontlinje.
- **Mongoose (databasnivå):** Validerar data precis innan den lagras. Är ett säkerhetsnät som fångar programmeringsfel i vår egna kod. Garanterar dataintegritet oavsett hur datan hamnade i databasen (direkt databasmanipulation, seeds, migrationer). Det är vår sista linje.

Problemet med att bara förlita sig på Mongoose-validering är att vi får ett Mongoose `ValidationError` kastas som ett JavaScript-undantag, och om vi inte fångar det kraschar servern (eller om vi fångar det måste vi parse Mongoose-felstrukturen, som ser annorlunda

ut än vad klienter förväntar sig). Med Zod hanterar vi valideringsfel proaktivt, innan någon databasoperation ens försöks.

Tumregeln vi ska följa: validera inkommande data med Zod på API-nivå, och behåll Mongoose-validering som ett säkerhetsnät. Det är djupförsvar – vi har inte ett enda skyddslager utan flera.

Felhantering i Express

Felbetantering är lika viktig som validering och hänger tätt samman med den. Valideringen fångar förväntade fel i klientens data – men det finns många andra typer av fel som kan uppstå: resursen som efterfrågas finns inte, databasanslutningen tappar, en extern tjänst svarar inte, eller ett oförutsett programmeringsfel inträffar.

I Express finns ett särskilt middleware-mönster för felhantering: en middleware med fyra parametrar (err, req, res, next). Express identifierar detta som en felhanterare och anropar den automatiskt när en route eller middleware anropar next() med ett fel-argument, eller när ett undantag kastas i ett synkront context.

Problemet med ad-hoc felhantering

Utan centraliserad felhantering ser vår kod ut ungefär så här:

```
app.get('/api/products/:id', async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);

    if (!product) {
      return res.status(404).json({ message: 'Produkten hittades inte' });
    }

    res.json(product);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Internt serverfel' });
  }
});

app.delete('/api/products/:id', async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);

    if (!product) {
      return res.status(404).json({ message: 'Produkten hittades inte' });
    }

    await product.deleteOne();
    res.json({ message: 'Produkt borttagen' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Serverfel' }); // Annat format!
  }
});
```

Vi ser problemen direkt: try-catch upprepas i varje route, felformaten är inkonsekventa (ibland 'Internt serverfel', ibland 'Serverfel'), och vi loggar felen på olika ställen. Om vi vill ändra hur vi hanterar 500-fel måste vi uppdatera varje route.

Skicka fel till Express felhanterare

Istället för att hantera fel direkt i varje route kan vi skicka dem vidare med `next(error)`. Express kommer då att anropa vår centraliserade felhanterare:

```
app.get('/api/products/:id', async (req, res, next) => {
  try {
    const product = await Product.findById(req.params.id);

    if (!product) {
      // Vi skapar ett fel och skickar det vidare
      const error = new Error('Produkten hittades inte');
      error.statusCode = 404;
      return next(error);
    }

    res.json(product);
  } catch (error) {
    // Alla oväntade fel skickas vidare
    next(error);
  }
});
```

Centraliserad error handler middleware

Nu skapar vi en centraliserad felhanterare som registreras sist i middleware-kedjan:

```
// middleware/errorHandler.js
function errorHandler(err, req, res, next) {
  // Logga felet för intern diagnostik
  console.error({
    message: err.message,
    stack: err.stack,
    path: req.path,
    method: req.method,
  });

  // Bestäm statuskod
  const statusCode = err.statusCode || 500;

  // Bygg ett konsekvent felrespons
  const response = {
    message: err.message || 'Ett oväntat fel inträffade',
    errors: err.errors || undefined,
  };

  // I produktion, exponera INTE stack trace eller interna detaljer
  if (process.env.NODE_ENV === 'development') {
    response.stack = err.stack;
  }

  // 500-fel ska inte avslöja interna detaljer till klienten
  if (statusCode === 500) {
```

```
    response.message = 'Ett oväntat serverfel inträffade';  
  }  
  
  res.status(statusCode).json(response);  
}  
  
module.exports = errorHandler;  
  
// app.js  
const errorHandler = require('./middleware/errorHandler');  
  
// ... övriga middleware och routes ...  
  
// Felhanteraren MÅSTE registreras SIST  
app.use(errorHandler);
```


Egna felklasser

I exemplet ovan satte vi `error.statusCode` direkt på ett vanligt `Error`-objekt. Det fungerar men är inte idealiskt – det ger oss ingen typstruktur och gör det svårt att skilja olika typer av fel från varandra. En mer elegant lösning är att skapa egna felklasser.

JavaScript har fullt stöd för att ärva från den inbyggda `Error`-klassen:

```
// errors/AppError.js

// Basfelsklassen för alla förväntade applikationsfel
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.name = this.constructor.name;
    this.statusCode = statusCode;
    this.isOperational = true; // Skiljer appfel från programmeringsfel
    Error.captureStackTrace(this, this.constructor);
  }
}

// 404 - Resursen hittades inte
class NotFoundError extends AppError {
  constructor(message = 'Resursen hittades inte') {
    super(message, 404);
  }
}

// 400 - Felaktig indata från klienten
class ValidationError extends AppError {
  constructor(message = 'Valideringsfel', errors = []) {
    super(message, 400);
    this.errors = errors; // Array med detaljerade felbeskrivningar
  }
}

// 401 - Kräver autentisering
class UnauthorizedError extends AppError {
  constructor(message = 'Autentisering krävs') {
    super(message, 401);
  }
}

// 403 - Autentiserad men saknar behörighet
class ForbiddenError extends AppError {
  constructor(message = 'Åtkomst nekad') {
    super(message, 403);
  }
}

// 409 - Konflikt, t.ex. e-post redan registrerad
class ConflictError extends AppError {
  constructor(message = 'Resursen existerar redan') {
```

```

    super(message, 409);
  }
}

module.exports = {
  AppError,
  NotFoundError,
  ValidationError,
  UnauthorizedError,
  ForbiddenError,
  ConflictError,
};

```

Egenskapen `isOperational` är ett viktigt mönster. Det skiljer förväntade applikationsfel (`NotFoundError`, `ValidationError` – saker vi förutsett och kan hantera elegant) från oförutsedda programmeringsfel (t.ex. att vi försöker läsa en property på `undefined`). Vår felhanterare kan använda den här egenskapen för att bestämma hur aggressivt vi ska logga ett fel.

Använda egna felklasser i routes

Med felklasserna på plats blir route-koden avsevärt mer läsbar:

```

const { NotFoundError, ConflictError } = require('../errors/AppError');

// Hämta produkt
app.get('/api/products/:id', async (req, res, next) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) throw new NotFoundError('Produkten hittades inte');
    res.json(product);
  } catch (error) {
    next(error);
  }
});

// Registrera användare
app.post('/api/users', async (req, res, next) => {
  try {
    const existing = await User.findOne({ email: req.validatedBody.email });
    if (existing) throw new ConflictError('E-postadressen är redan registrerad');

    const user = await User.create(req.validatedBody);
    res.status(201).json(user);
  } catch (error) {
    next(error);
  }
});

```

Uppdaterad felhanterare med felklasser

Nu kan vi uppdatera vår errorHandler för att dra nytta av felklasserna:

```
const { AppError } = require('../errors/AppError');

function errorHandler(err, req, res, next) {
  // Logga felet
  if (!err.isOperational) {
    // Oförutsett programmeringsfel - logga aggressivt
    console.error('ALLVARLIGT FEL:', err);
  } else {
    // Förväntat applikationsfel - logga på lägre nivå
    console.warn('Applikationsfel:', err.message);
  }

  // Om felet är ett AppError, använd dess statuskod
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      message: err.message,
      errors: err.errors,
    });
  }

  // Hantera Mongoose ValidationError
  if (err.name === 'ValidationError') {
    const errors = Object.values(err.errors).map(e => ({
      field: e.path,
      message: e.message,
    }));
    return res.status(400).json({ message: 'Valideringsfel', errors });
  }

  // Hantera Mongoose CastError (ogiltigt ObjectId-format)
  if (err.name === 'CastError') {
    return res.status(400).json({ message: 'Ogiltigt ID-format' });
  }

  // Oförutsett fel - exponera aldrig detaljer i produktion
  const message = process.env.NODE_ENV === 'development'
    ? err.message
    : 'Ett oväntat serverfel inträffade';

  res.status(500).json({ message });
}
```

Konsekvent felrespons-format

En underskattad aspekt av API-design är konsistens i felrespons. Om klienten ska kunna hantera fel automatiskt – visa rätt felmeddelanden för användaren, logga fel på rätt sätt, försöka igen vid tillfälliga fel – måste felen följa ett förutsägbart format.

Det finns ingen universell standard, men ett väldesignat felrespons brukar innehålla:

- **message:** En läsbar beskrivning av felet. Lämplig att visa för utvecklare och ibland för slutanvändaren.
- **errors:** (valfri) En array med specifika valideringsfel när det handlar om felaktig indata. Varje objekt bör ha **field** och **message**.
- **HTTP-statuskod i svaret:** Alltid korrekt statuskod – det är HTTP-protokollets sätt att kommunicera felets natur.

Här är ett exempel på konsekventa felresponser:

```
// 400 Bad Request - valideringsfel
{
  "message": "Valideringsfel",
  "errors": [
    { "field": "email", "message": "Ogiltig e-postadress" },
    { "field": "name", "message": "Namn måste ha minst 2 tecken" }
  ]
}

// 404 Not Found - resursen saknas
{
  "message": "Produkten hittades inte"
}

// 409 Conflict - dubblettdata
{
  "message": "E-postadressen är redan registrerad"
}

// 500 Internal Server Error - oväntat fel
{
  "message": "Ett oväntat serverfel inträffade"
}
```

Notera att 500-felet inte avslöjar vad som faktiskt gick fel. Det är ett medvetet säkerhetsbeslut som vi tittar närmre på i nästa avsnitt.

Undvika känslig information i felmeddelanden

Information är makt. Detaljerade felmeddelanden ger angripare värdefull information om hur vår applikation är byggd, vilka tekniker vi använder och var svagheter kan finnas. Det är därför vi måste vara noga med vad vi inkluderar i felrespons som skickas till klienten.

Vad vi aldrig ska exponera

- Stack traces: En fullständig stack trace avslöjar filsystemsstrukturen, biblioteksversioner och exakt var i koden ett fel uppstod. Det är extremt värdefull information för en angripare.
- Databasfelmeddelanden: Mongoose och MongoDB kan generera felmeddelanden som innehåller schema-detalljer, collection-namn och query-struktur. Dessa ska aldrig nå klienten.
- Interna sökvägar: Felmeddelanden som `/home/ubuntu/app/src/models/User.js` avslöjar serverstrukturen.
- Teknologiversion: Versioner av Node.js, Express eller Mongoose ger angripare möjlighet att söka efter kända säkerhetshål i just den versionen.
- Autentiseringsdetaljer: Felmeddelanden som 'Fel lösenord' avslöjar att användarnamnet existerar. Bättre är 'Felaktiga inloggningsuppgifter' för att inte ge angriparen information om vilken del som var fel.

Produktion vs utveckling

En vanlig och effektiv strategi är att ha olika beteenden beroende på miljön. Under utveckling vill vi se all information för att kunna felsöka. I produktion ska vi vara minimalistiska:

```
function errorHandler(err, req, res, next) {
  const isDevelopment = process.env.NODE_ENV === 'development';

  // Intern loggning - alltid fullständig
  console.error({
    timestamp: new Date().toISOString(),
    method: req.method,
    path: req.path,
    error: err.message,
    stack: err.stack,
  });

  // Klientrespons - beroende på miljö
  if (err.isOperational) {
    // Förväntade fel: visa meddelandet (det är säkert)
    return res.status(err.statusCode).json({
      message: err.message,
      errors: err.errors,
      ...(isDevelopment && { stack: err.stack }),
    });
  }

  // Övåntade fel: minimalt svar i produktion
  res.status(500).json({
    message: isDevelopment
```

```
    ? err.message // Full info i dev
    : 'Ett oväntat serverfel inträffade', // Generiskt i prod
    ...(isDevelopment && { stack: err.stack }),
  });
}
```

Säkerhet vs användbarhet i felmeddelanden

Det finns en naturlig spänning mellan säkerhet och användbarhet i felmeddelanden. Om vi är för vaga ('Ett fel inträffade') är det frustrerande för legitima användare och svårt för frontendutvecklare att integrera mot vårt API. Om vi är för detaljerade riskerar vi att läcka information.

En rimlig avvägning är:

- Valideringsfel (400): Var specifika. Berätta exakt vilket fält som är fel och varför. Det är inte känslig information – klienten behöver veta detta för att kunna rätta till sin data.
- Inte hittad (404): Var specifik om vad som inte hittades ('Produkten hittades inte'), men avslöja inte om resursen finns men är skyddad – det kan ibland motivera att returnera 404 istället för 403.
- Autentiseringsfel (401/403): Var vag av säkerhetsskäl. 'Autentisering krävs' och 'Åtkomst nekad' räcker.
- Serverfel (500): Var alltid vag mot klienten. Alla detaljer ska bara synas i logs.

Att sätta ihop helheten

Låt oss se hur alla delar samverkar i ett komplett exempel. Vi har en Express-router för produkter med validering, egna felklasser och centraliserad felhantering:

```
// Filstruktur
// src/
//   errors/AppError.js
//   middleware/validate.js
//   middleware/errorHandler.js
//   schemas/productSchemas.js
//   routes/products.js
//   app.js

// src/schemas/productSchemas.js
const { z } = require('zod');

const createProductSchema = z.object({
  name: z.string().min(2).max(100).trim(),
  price: z.number().positive(),
  category: z.enum(['electronics', 'clothing', 'food']),
  description: z.string().max(500).optional(),
});

const updateProductSchema = createProductSchema.partial();

const idParamSchema = z.object({
  id: z.string().regex(/^([0-9a-fA-F]{24})$/, 'Ogiltigt ID-format'),
});

module.exports = { createProductSchema, updateProductSchema, idParamSchema };

// src/routes/products.js
const express = require('express');
const router = express.Router();
const { NotFoundError } = require('../errors/AppError');
const { validate } = require('../middleware/validate');
const {
  createProductSchema,
  updateProductSchema,
  idParamSchema,
} = require('../schemas/productSchemas');
const Product = require('../models/Product');

// GET /api/products
router.get('/', async (req, res, next) => {
  try {
    const products = await Product.find();
    res.json(products);
  } catch (error) {
    next(error);
  }
});
```

```

    }
  });

  // GET /api/products/:id
  router.get('/:id',
    validate({ params: idParamSchema }),
    async (req, res, next) => {
      try {
        const { id } = req.validatedParams;
        const product = await Product.findById(id);
        if (!product) throw new NotFoundError('Produkten hittades inte');
        res.json(product);
      } catch (error) {
        next(error);
      }
    }
  );

  // POST /api/products
  router.post('/',
    validate({ body: createProductSchema }),
    async (req, res, next) => {
      try {
        const product = await Product.create(req.validatedBody);
        res.status(201).json(product);
      } catch (error) {
        next(error);
      }
    }
  );

  // PATCH /api/products/:id
  router.patch('/:id',
    validate({ params: idParamSchema, body: updateProductSchema }),
    async (req, res, next) => {
      try {
        const { id } = req.validatedParams;
        const product = await Product.findByIdAndUpdate(
          id,
          req.validatedBody,
          { new: true, runValidators: true }
        );
        if (!product) throw new NotFoundError('Produkten hittades inte');
        res.json(product);
      } catch (error) {
        next(error);
      }
    }
  );

  module.exports = router;

```


Lägg märke till hur ren route-koden har blivit. Valideringen sköts av middleware, felen hanteras av errorHandler, och varje route-handler fokuserar enbart på sin affärslogik: hämta data, skapa data, returnera svar.

Användningen av `.partial()` på `updateProductSchema` är ett elegant Zod-mönster. Det skapar ett nytt schema där alla fält från `createProductSchema` är valfria – perfekt för PATCH-endpoints där klienten kanske bara vill uppdatera ett eller ett fåtal fält.

Checkpoint: Förberedelse inför kursvecka 6

Inför checkpointen tisdag ska vi ha läst igenom detta studiematerial och ha en grundläggande förståelse för de koncept som presenteras. Vi ska också ha boiler room-projektet från vecka 5 med oss som utgångspunkt för att applicera inputvalidering och felhantering.

Kontrollera att ni har följande klart inför boiler room måndag:

4. Zod är installerat i projektet: `npm install zod`
5. Ni har identifierat vilka endpoints i ert boiler room-projekt som saknar validering.
6. Ni har skissat på en mappstruktur för schemas/, errors/ och uppdaterad middleware/.

Ta med er följande tankar till checkpointen: Var i ert boiler room-projekt finns det fält som en angripare skulle kunna missbruka? Finns det endpoints som just nu saknar felhantering helt och kraschar servern vid oväntad input? Hur ser ert nuvarande felrespons ut – är det konsekvent?

Det är vanligt att känna sig överväldigad av mängden mönster i detta material. Det viktigaste är att förstå varför de finns, inte att memorera exakt syntax. Under workshopen arbetar vi praktiskt med att implementera validering och felhantering i era egna projekt, och mönstren faller på plats naturligt genom praktik.

Sammanfattning

Under kursvecka 6 har vi adresserat ett av de mest kritiska aspekterna av säker backendutveckling: att validera all inkommande data och hantera fel på ett robust och konsekvent sätt.

Vi har etablerat principen 'don't trust the client' som ett grundläggande förhållningssätt. All data som skickas till vår server måste behandlas som potentiellt skadlig tills vi har verifierat att den uppfyller våra förväntningar. Frontend-validering är ett UX-verktyg, aldrig ett säkerhetsverktyg – den enda validering vi kan lita på är den vi utför på servern.

Vi har introducerat Zod som verktyg för schemabaserad validering. Zod låter oss deklarativt beskriva vilken data vi förväntar oss med hjälp av ett kedjebart, läsbart API. Med `safeParse` validerar vi data utan att riskera undantag, och vi får tillgång till antingen den validerade och transformerade datan eller en detaljerad lista med alla valideringsfel – simultant, för alla fält.

Vi har sett hur vi validerar request body, URL-parametrar och query-parametrar med Zod-schemas, och hur vi abstraherar valideringen till återanvändbara middleware-funktioner. Det separerar valideringslogiken från affärslogiken och gör varje route-handler enklare och mer fokuserad.

Vi har också klargjort rollfördelningen mellan Zod och Mongoose. Zod validerar inkommande HTTP-data på API-nivå och ger omedelbara, API-vänliga felmeddelanden. Mongoose validerar data precis innan den lagras i databasen som ett sista skyddsnät. Tillsammans utgör de djupförsvaret – flera lager av skydd.

Den centraliserade felhanteringen i Express löser problemet med upprepat try-catch och inkonsekvent felrespons. Genom att skicka alla fel till `next(error)` och ha en dedikerad `ErrorHandler`-middleware hanteras alla fel på ett ställe med ett enhetligt format. Egna felklasser (`NotFoundError`, `ValidationError`, `UnauthorizedError` och fler) gör koden mer uttrycksfull och gör det enkelt att skilja förväntade applikationsfel från oväntade programmeringsfel.

Slutligen har vi diskuterat vad vi aldrig ska exponera i felmeddelanden: stack traces, databasfelmeddelanden, interna sökvägar och versionsinformation. I produktion ska 500-felen alltid visa ett generiskt meddelande till klienten – all detaljerad information ska bara synas i serverloggar.

Nästa vecka tar vi steget in i autentisering och säkerhet på en djupare nivå. Vi introducerar OWASP Top 3, lär oss att hasha lösenord med `bcrypt` och implementerar JWT-baserad autentisering. Mycket av det vi byggt den här veckan – felklasserna, valideringsmiddleware och centraliserad felhantering – kommer att vara direkt återanvändbart.

Resurser för fördjupning

Nedan följer resurser för de som vill fördjupa sina kunskaper om inputvalidering och felhantering.

Officiell dokumentation

Zod officiell dokumentation – fullständig referens med exempel för alla schema-typer och metoder:

<https://zod.dev>

Express - felhantering - den officiella guiden för error handling middleware i Express:

<https://expressjs.com/en/guide/error-handling.html>

MDN Web Docs - HTTP statuskoder - fullständig referens för alla HTTP-statuskoder och när de används:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

Säkerhetsresurser

OWASP Top Ten – organisationen OWASP (Open Web Application Security Project) publicerar en lista med de tio vanligaste säkerhetshoten mot webbapplikationer. Kursvecka 7 fokuserar på de tre första, men det är värdefullt att bekanta sig med hela listan redan nu:

<https://owasp.org/www-project-top-ten/>

"Node.js Best Practices" av Yoni Goldberg – ett community-drivet repository med rekommenderade mönster för Node.js. Avsnitten om felhantering och säkerhet är direkt relevanta för det vi arbetar med den här veckan:

<https://github.com/goldbergystony/nodebestpractices>

Videomaterial

"Zod Tutorial" av Matt Pocock på YouTube ger en utmärkt och praktisk genomgång av Zod, från grunderna till mer avancerade mönster. Matt är en av de mest uppskattade TypeScript-pedagogerna i communityt och förklarar Zod med stor tydlighet.

"Node.js Error Handling Best Practices" finns i flera välgjorda format på YouTube. Sök på termen och välj ett nyare videoklipp (2023 eller senare) för att säkerställa att exemplen är relevanta för moderna versioner av Express och Node.js.

Studiematerial

Autentisering och OWASP Top 3

Kursvecka 7

Introduktion

Välkomna till kursvecka 7. Vi har under de senaste sex veckorna byggt upp ett komplett API: vi har lagt grunden i Express, designat REST-endpoints, lärt oss TypeScript på servern, kopplat in MongoDB med Mongoose, implementerat fullständig CRUD med tester, och förra veckan stärkte vi systemet med Zod-validering och centraliserad felhantering. Vi har ett API som fungerar – men det är ett API som vem som helst på internet kan använda hur de vill.

Den veckan vi nu påbörjar handlar om att bygga ett API som vi kan släppa ut i världen utan att vakna kallsvettiga på natten. Vi ska lära oss att tänka som en angripare: vad skulle någon kunna göra för att skada vårt system, läsa data de inte ska se, eller utge sig för att vara någon de inte är? Och framför allt – hur skyddar vi oss?

Vi ska prata om OWASP Top 3, en lista över de allvarligaste säkerhetsriskerna i moderna webbapplikationer. Vi kommer att förstå vad de innebär, hur de utnyttjas och hur vi designar bort dem från start. Vi ska implementera autentisering: möjligheten för en användare att registrera sig, logga in och därefter bevisa sin identitet via en token vid varje anrop. Vi använder bcrypt för att hasha lösenord och JSON Web Tokens (JWT) för att hantera autentiserade sessioner i en stateless värld.

Säkerhet är inte ett "feature" vi lägger till sist. Det är en disciplin som påverkar varje rad kod vi skriver. Att tänka säkert blir aldrig fel, men att tänka säkert för sent kan bli väldigt fel. Vi börjar därför med principerna och bygger sedan implementationen ovanpå dem.

Den här veckan tar vi de första stora stegen mot ett produktionsdugligt API. Nästa vecka bygger vi vidare med auktorisering och rollbaserad åtkomstkontroll (RBAC), och om två veckor pratar vi om GDPR och säker loggning. Allt hänger ihop – och allt börjar här.

Varför säkerhet är vårt ansvar

När vi byggde frontend i kurs 2 och 3 var säkerhet något vi rörde vid på avstånd. Vi konsumerade API:er, hanterade tokens som vi fick utdelade, och visste att inloggning skulle fungera "någonstans där bakom". Den där "någonstans" är vad vi ska bygga nu. Och för första gången är det vi som bär ansvaret om något går fel.

Backenden är platsen där känslig data lever. Användares e-postadresser, lösenord, personnummer, betalningsuppgifter, privata meddelanden – allt passerar genom våra API:er och lagras i våra databaser. När en frontend har en bugg ser en användare fel data. När en backend har en säkerhetsbugg kan tusentals användares personliga data hamna hos en angripare. Skalan på konsekvenserna är fundamentalt annorlunda.

Det moderna hotlandskapet

Det är värt att förstå vad vi faktiskt skyddar oss mot. Idag är inte den största risken en specifik person som riktar in sig på just vårt system. Det är automatiserade verktyg som skannar hela internet kontinuerligt och letar efter sårbarheter. När vi driftsätter ett API till en publik adress får det inom några minuter sina första anropsförsök från botar som testar vanliga endpoints, vanliga lösenord och kända exploits.

Det innebär att vi inte kan lita på "genom obscurity". Vi kan inte tänka "vem skulle bry sig om vårt lilla API?" – för svaret är: alla automatiserade angreppsverktyg på internet bryr sig, eftersom de testar allt. Vårt API behöver klara sig på egen hand, oavsett hur litet eller okänt det är.

Den goda nyheten är att de flesta angrepp använder välkända tekniker som det finns välbeprövade skydd mot. När vi följer etablerade säkerhetsprinciper – hashar lösenord, validerar input, använder parametriserade queries, skyddar våra routes med autentisering – stänger vi dörren för den absoluta majoriteten av angrepp. Säkerhet handlar mindre om att vara genial och mer om att vara disciplinerad.

Vårt ansvar som backendutvecklare

Som backendutvecklare har vi ett juridiskt och etiskt ansvar för den data vi hanterar. GDPR (som vi går djupare in på vecka 9) ställer specifika krav på hur personuppgifter får hanteras. Men även utan lagkrav har vi ett ansvar mot våra användare: när de litar på oss med sin information har vi en skyldighet att skydda den.

Det här ansvaret innebär konkret att vi alltid ska:

- Behandla all input från klienten som potentiellt fientlig och validera den noggrant.
- Aldrig lagra känslig information som lösenord i klartext.
- Aldrig skicka känslig information över okrypterade kanaler.
- Aldrig läcka information om systemet via felmeddelanden eller stack traces i produktion.
- Aldrig anta att frontend har gjort jobbet åt oss – varje endpoint står ensam mot omvärlden.

Det här är inte bara en checklista. Det är ett tankesätt vi bär med oss i varje rad kod vi skriver.

OWASP Top 10 och vårt fokus

OWASP står för Open Worldwide Application Security Project och är en ideell organisation som arbetar med att förbättra säkerheten i webbapplikationer. Deras mest kända publikation är OWASP Top 10 – en lista över de tio allvarligaste säkerhetsriskerna i webbapplikationer, baserad på data från tusentals organisationer världen över. Listan uppdateras regelbundet och är en av de viktigaste referenserna för säkerhet inom branschen.

OWASP Top 10 är inte en checklista i bemärkelsen "botta av och ni är säkra". Den är en utgångspunkt – om vi inte hanterar de tio största riskerna har vi sannolikt grava problem. Men en applikation som hanterar dem alla kan fortfarande ha andra säkerhetsbrister. Säkerhet är ett kontinuerligt arbete.

Den senaste versionen (OWASP Top 10:2021) listar dessa risker, sorterade efter allvarlighet:

1. Broken Access Control – auktoriseringsbrister.
2. Cryptographic Failures – brister i kryptering och datahantering.
3. Injection – inklusive SQL-, NoSQL- och kommandoinjektion.
4. Insecure Design – arkitektoniska säkerhetsbrister.
5. Security Misconfiguration – felaktig konfiguration.
6. Vulnerable and Outdated Components – osäkra beroenden.
7. Identification and Authentication Failures – brister i autentisering.
8. Software and Data Integrity Failures – brister i integritet.
9. Security Logging and Monitoring Failures – brister i loggning.
10. Server-Side Request Forgery (SSRF) – manipulerade serveranrop.

Vi kommer i den här kursen att fokusera på de tre översta – Broken Access Control, Cryptographic Failures och Injection. Det är de risker som rankats högst flera år i rad och som varje backendutvecklare måste behärska. När vi förstår dem och vet hur vi skyddar oss mot dem har vi en stabil grund att stå på.

Vi ska gå igenom var och en, förstå vad det innebär konkret, och se hur vi designar vårt API för att vara skyddat mot dem från start. Resten av OWASP Top 10 berör vi i kursen också – auktoriseringsbrister återkommer i vecka 8 (RBAC), loggningsfrågor i vecka 9 (GDPR), och konfigurationsfrågor löpande – men huvudfokus den här veckan ligger på de tre översta.

OWASP #1: Broken Access Control

Broken Access Control – brister i åtkomstkontroll – handlar om att en användare kan utföra åtgärder eller läsa data som de inte borde ha rätt till. Det är den enskilt vanligaste och mest allvarliga risken i moderna API:er. Det har klättrat till första platsen i OWASP-listan, och det av en god anledning: de flesta API:er har någon form av åtkomstkontroll, men många implementerar den fel.

Det grundläggande misstaget är att förlita sig på frontend för att kontrollera vad användare ser och kan göra. Frontend-koden döljer en knapp för användare som inte är admin – men knappen utför ändå ett anrop till `/api/admin/delete-user` när den klickas. Om backend bara kollar att klienten är inloggad, men inte vad de har rätt att göra, kan vem som helst skicka samma anrop manuellt och utföra åtgärden.

Konkreta exempel

Här är några klassiska scenarier där access control brister:

- En användare anropar `GET /api/users/42/orders` och får tillbaka beställningar för användare 42, även om de själva är inloggade som användare 17. Backend kollar att användaren är inloggad men inte att de äger resursen.
- En vanlig användare anropar `DELETE /api/products/99` och kan radera produkter. Backend har ingen kontroll på vilken roll användaren har.
- En användare ändrar sitt eget user-id i en JWT-payload och får plötsligt tillgång till en annan användares konto. Tokenens signatur verifieras inte.
- Ett admin-API exponeras på en "hemlig" URL som `/admin-panel-x7q2` utan riktig auktorisering – tanken är att ingen ska hitta den. Verktyg som scannar kataloger hittar den ändå.

Hur vi skyddar oss

Den fundamentala principen för åtkomstkontroll är: kontrollera alltid på servern, för varje request, både att användaren är autentiserad och att de är auktoriserade för just den åtgärden. Det här är två separata frågor:

- Autentisering – vem är användaren? (Frågan vi besvarar denna vecka med JWT.)
- Auktorisering – vad får den här användaren göra? (Frågan vi fördjupar oss i nästa vecka med RBAC.)

En enkel grundregel: deny by default. Allting är förbjudet tills det är uttryckligen tillåtet. Vi ska aldrig ha en endpoint där vi "glömmer" att lägga på autentisering och som därför är öppen. Vår middleware-pipeline ska se till att inget kommer förbi utan att kontrolleras.

Den här veckan löser vi den första halvan – autentiseringen. Nästa vecka bygger vi den andra halvan – rollbaserad auktorisering. Men redan nu ska vi tänka på resursägarskap: när en användare hämtar `/api/orders/42` ska vi kontrollera att beställning 42 faktiskt tillhör den användare som är inloggad.

OWASP #2: Cryptographic Failures

Cryptographic Failures – kryptografiska brister – handlar om att känslig data hanteras eller skyddas felaktigt. Tidigare hette den här kategorin "Sensitive Data Exposure", och det beskriver fortfarande vad det handlar om: data som inte borde vara läsbar för obehöriga blir det, antingen för att vi inte krypterat den, för att vi använt fel kryptografiska metoder, eller för att vi lagrat den på ett sätt som inte borde vara reversibelt.

Det klassiska exemplet är lösenord lagrade i klartext i databasen. Om någon får tag i en kopia av databasen – via SQL-injection, en intern läcka, en stulen backup – så kan de logga in som vilken användare som helst. Vad värre är: eftersom människor ofta återanvänder lösenord, kan angriparen logga in som samma användare på Gmail, banken, Facebook och andra tjänster där samma e-post och lösenord används.

Vad räknas som känslig data?

Mer än vi tror. När vi tänker säkerhet är det lätt att fokusera på lösenord och betalningskort, men listan är längre:

- Lösenord och autentiseringstokens.
- Personuppgifter: namn, e-postadress, telefonnummer, personnummer, adress.
- Betalningsuppgifter: kortnummer, IBAN, transaktionshistorik.
- Hälsouppgifter och annan känslig kategoridata enligt GDPR.
- API-nycklar, databasanslutningssträngar och andra hemligheter som ger åtkomst till våra system.

Tre nivåer av skydd

Vi måste tänka kryptografi på tre nivåer:

- Data in transit – data som transporteras över nätverket. Skyddas med HTTPS (TLS). I produktion ska vårt API alltid köras över HTTPS, aldrig HTTP.
- Data at rest – data som lagras i databasen eller filsystemet. Lösenord ska aldrig kunna återställas till klartext (vi använder hashing, inte kryptering – mer om det strax). Annan känslig data kan krypteras vid behov.
- Data i loggar – data som hamnar i applikationsloggar. En vanlig miss är att logga hela request-objekt, inklusive lösenord eller tokens, vilket gör loggarna lika känsliga som databasen.

Den här veckan fokuserar vi på det första problemet: hur vi hanterar lösenord på rätt sätt. Vi kommer att djupdyka i hashning, salting och bcrypt om en stund. Men först ska vi reda ut den tredje OWASP-risken.

OWASP #3: Injection

Injection-attacker handlar om att en angripare lyckas få vårt system att tolka data som de skickat in som kod. Det klassiska exemplet är SQL-injection: en angripare skriver in en sträng i ett formulärfält som – om vi bygger vår SQL-query genom att klistra ihop strängar – tolkas som en del av SQL-frågan när den körs mot databasen.

Föreställ er en inloggning där vi bygger SQL-frågan så här (illustrativt – vi ska aldrig göra så här):

```
// FARLIGT exempel - aldrig så här i verkligheten!  
const query = "SELECT * FROM users WHERE email = '" + email + "' AND password = '" + password + "'";
```

Om en användare skriver in följande som e-post: ' OR '1'='1 så blir den faktiska SQL-frågan `SELECT * FROM users WHERE email = ' OR '1'='1' AND password = '...'`. Eftersom '1'='1' alltid är sant matchar frågan alla rader i tabellen, och angriparen är inloggad som den första användaren i tabellen – ofta en administratör.

Det här är ett 25 år gammalt angrepp och det fungerar fortfarande mot system som inte använder rätt teknik. Lyckligtvis har vi rätt teknik.

Parametriserade queries

Lösningen heter parametriserade queries. Istället för att sätta ihop en sträng skickar vi frågan och parametrarna separat till databasen. Databasen vet då vad som är frågans struktur (kod) och vad som är data (värden), och behandlar dem helt olika. Användarens input kan inte ändra frågans struktur, oavsett vad de skriver.

I vårt fall – när vi använder Mongoose mot MongoDB – är det här inbyggt. När vi skriver:

```
const user = await User.findOne({ email: req.body.email });
```

... så bygger Mongoose en BSON-fråga där email-värdet behandlas som ett strängvärde, inte som en del av frågans struktur. Det går inte att "bryta sig ut" ur strängen, oavsett vad användaren skickar. Det är därför Mongoose-baserade applikationer i stor utsträckning är skyddade mot injection by default.

NoSQL injection finns ändå

Det betyder dock inte att MongoDB-applikationer är immuna. NoSQL injection är ett verkligt fenomen och uppstår framför allt när vi inte validerar input ordentligt. MongoDB:s frågespråk har operatorer som `$ne` (not equal), `$gt` (greater than), `$where` (kör JavaScript) – och dessa operatorer skickas som JSON-objekt.

Om vi inte validerar att `req.body.email` är en sträng kan en angripare skicka denna JSON i sitt request body:

```
{  
  "email": { "$ne": null },  
  "password": { "$ne": null }  
}
```

Om vi sedan, utan validering, gör `User.findOne(req.body)` söker vi efter en användare där email inte är null OCH password inte är null – alltså den första användaren i tabellen. Igen – ofta en administratör.

Det här är det perfekta exemplet på varför vecka 6:s tema – inputvalidering – är så tätt sammankopplat med vecka 7:s tema – säkerhet. Om vi använder Zod för att validera att email är en sträng (`z.string()`) så stoppas attacken redan i validerings-middleware, innan den någonsin når databasen. Validering och säkerhet är samma sak.

Andra typer av injection

Injection är en bred kategori. Förutom SQL/NoSQL injection finns:

- Command injection – en angripare lyckas få servern att köra OS-kommandon. Uppstår när vi anropar `exec()` eller liknande med användardata.
- Cross-Site Scripting (XSS) – en angripare lyckas få webbläsaren att köra JavaScript hos en annan användare. Mer ett frontend-problem, men vi som backend har ansvar att inte returnera oescaped användardata i HTML-renderade svar.
- LDAP injection, XPath injection, etc. – samma princip i andra teknologier.

Den röda tråden är alltid samma: blanda inte ihop data och kod. När data från en oberoende källa (klienten) ska användas i en kontext där det kan tolkas som kod (en query, ett kommando, HTML), måste vi se till att den behandlas som data – aldrig som en del av strukturen. Mongoose hjälper oss långt, men vi måste fortfarande validera input.

Verklig säkerhet vs "security by obscurity"

Innan vi går in i den praktiska implementationen behöver vi reda ut ett vanligt missförstånd. Många utvecklare förväxlar verklig säkerhet med vad som kallas "security by obscurity" – säkerhet genom hemlighållande. Det är inte samma sak.

Security by obscurity innebär att man förlitar sig på att en angripare inte ska känna till hur systemet fungerar. Klassiska exempel: "vi lägger admin-panelen på /xqz9-secret-admin så ingen hittar den", "vi använder ett eget krypteringsalgoritm som vi inte berättar för någon", "vi gömmer API-nyckeln genom att obfuskerar koden". I alla dessa fall kollapsar säkerheten i samma sekund som hemligheten avslöjas – och hemligheter avslöjas alltid förr eller senare.

Verklig säkerhet bygger på det motsatta: systemet ska vara säkert även om angriparen vet exakt hur det fungerar. Vår JWT-implementation är säker just för att kryptografin är offentlig och välbeprövad – signaturalgoritmen, datastrukturen och hela protokollet är dokumenterat öppet. Säkerheten ligger i den hemliga signeringsnyckeln, inte i att gömma hur tokens ser ut.

Kerckhoffs princip

Auguste Kerckhoffs formulerade redan på 1880-talet ett kryptografiskt grundprincip som fortfarande gäller: ett kryptosystem ska vara säkert även om allt om det är offentligt utom den hemliga nyckeln. Det betyder att vår säkerhet ska bygga på två saker:

- Kraftfulla, välbeprövade och offentligt granskade algoritmer.
- Hemlig nyckelhantering – nycklar och lösenord som hålls säkra.

Det betyder att vi aldrig ska försöka uppfinna våra egna kryptografiska algoritmer. Vi ska använda etablerade bibliotek (bcrypt, jsonwebtoken, etc.) som har granskats av tusentals säkerhetsforskare under många år. Att försöka skriva en egen "säkrare" hashning är det enskilt vanligaste sättet att göra ett system mindre säkert.

När obscurity har en plats

Det här betyder inte att hemlighållande aldrig är värdefullt. Det är till exempel rimligt att inte exponera mer information än nödvändigt – vi ska inte returnera version på vår Express-server i HTTP-headers, vi ska inte returnera detaljerade stack traces i felmeddelanden, och vi ska inte avslöja om en e-postadress är registrerad i systemet vid inloggning.

Skillnaden är: dessa åtgärder är extra lager på toppen av ett system som redan är säkert utan dem. De gör det svårare för angripare att kartlägga systemet, men de är inte den enda försvarslinjen. Tyvärr är det vanligt att utvecklare lägger på obscurity-lager och tror att de är klara, utan att grundsäkerheten är på plats. Det är där det går fel.

Vi ska tänka i termer av "defense in depth" – flera försvarslager. Om ett lager fallerar finns det andra som tar vid. Verklig säkerhet är fundamentet, obscurity kan vara ett extra lager – men aldrig tvärtom.

Lösenordshantering: hashning med bcrypt

Nu går vi från principer till praktik. Det första vi ska implementera är säker lösenordshantering. Det är basen för all autentisering och därför vår första hörnsten.

Den enkla regeln: vi lagrar aldrig lösenord i klartext. Aldrig. Inte i databasen, inte i loggar, inte i felmeddelanden. När en användare skapar ett konto med lösenordet hemligt123 ska det aldrig finnas något ställe i vårt system där strängen "hemligt123" kan läsas av någon, oavsett om de är admin, har rotaccess eller har stulit en backup. Det enda som ska finnas lagrat är ett envägstransformerat värde – en hash – som inte går att räkna baklänges.

Skillnaden mellan hashing och kryptering

Det är värt att förstå skillnaden mellan dessa två begrepp, eftersom de ibland förväxlas:

- Kryptering är reversibelt. Vi krypterar något med en nyckel och kan dekryptera det med samma (eller en relaterad) nyckel. Klartexten är fortfarande tillgänglig om man har nyckeln.
- Hashing är envägs. Vi tar en input och får en hash som output, men det går inte att räkna fram inputet ur hashen. Hashen är inte lösenordet, men varje gång vi hashar samma lösenord får vi samma hash – det är det som gör att vi kan jämföra vid inloggning.

För lösenord vill vi alltid ha hashing, inte kryptering. Vi har ingen anledning att kunna dekryptera ett lösenord – när användaren glömmer det, ska vi skicka en återställningslänk, inte mejla över deras gamla lösenord. Om vi någonsin känner att vi behöver dekryptera ett lösenord har vi designat något fel.

Varför vanlig hashing inte räcker

Naivt skulle man kunna tro att en SHA-256-hash av lösenordet räcker. SHA-256 är en kryptografiskt säker hash, det går inte att räkna ut inputet ur hashen. Tyvärr räcker det inte, av två skäl.

Det första problemet är att SHA-256 är extremt snabb. Modern hårdvara kan beräkna miljarder SHA-256-hashar per sekund. Det betyder att en angripare som har en lista över hashade lösenord (efter en databasläcka) kan testa miljarder lösenord per sekund mot hashen. Vanliga lösenord knäcks på sekunder.

Det andra problemet är så kallade rainbow tables – förberäknade tabeller där en angripare har hashat miljarder vanliga lösenord i förväg. När en angripare får tag på din hash kan de slå upp den i tabellen direkt. Om två användare i databasen har samma lösenord får de samma hash, och båda knäcks på samma gång.

Vi behöver två extra ingredienser: salt och en långsam algoritm. Det är vad bcrypt ger oss.

Salt: unika hashar per användare

Ett salt är ett slumpmässigt värde som blandas in i lösenordet innan det hashas. Varje användare får ett unikt salt, vilket innebär att:

- Två användare med samma lösenord får olika hashar – knäckning av en användare hjälper inte angriparen att knäcka den andra.
- Förberäknade rainbow tables blir oanvändbara – angriparen skulle behöva en separat tabell för varje möjligt salt, vilket är beräkningsmässigt orimligt.

Saltet behöver inte vara hemligt – det lagras tillsammans med hashen. Men det måste vara unikt per användare. När vi använder bcrypt sköts saltet automatiskt åt oss; det blir inbäddat i den slutgiltiga hashsträngen.

Långsam hashing: bcrypt:s hemlighet

bcrypt är designat för att vara avsiktligt långsamt. Det baseras på Blowfish-chiffret och har en "cost factor" som styr hur långsamt det är – ju högre cost, desto fler iterationer av algoritmen, desto långsammare blir både hashning och knäckningsförsök.

En typisk cost factor på 10 (default i de flesta bibliotek) gör att en hash tar runt 100 millisekunder att beräkna på modern hårdvara. För oss som hashar ett lösenord vid registrering eller inloggning är 100ms en fullständigt okej fördröjning – användaren märker det inte. För en angripare som vill testa miljarder lösenord blir det däremot ett oöverstigligt hinder. Tio miljoner försök tar tio miljoner sekunder – över hundra dagar. Det förändrar matematiken helt.

Cost factor är konfigurerbar för att kunna ökas allt eftersom hårdvaran blir snabbare. Det som var långsamt nog år 2010 är inte långsamt nog 2030. bcrypt:s smarthet är att den lagrar cost factor i hashen, så vi kan stegvis öka kostnaden över tid.

bcrypt i praktiken

Vi installerar bcrypt-paketet:

```
npm install bcrypt
```

Det finns två huvudfunktioner vi behöver: hash() för att hasha ett lösenord och compare() för att jämföra ett lösenord med en hash:

```
const bcrypt = require('bcrypt');

// Hasha ett lösenord (vid registrering)
const SALT_ROUNDS = 10;
const password = 'användarens-lösenord';
const hash = await bcrypt.hash(password, SALT_ROUNDS);
// hash blir en sträng som:
$2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhwy

// Jämföra ett lösenord med en hash (vid inloggning)
const inkomstLösenord = 'användarens-lösenord';
const ärRätt = await bcrypt.compare(inkomstLösenord, hash);
// ärRätt blir true om lösenordet stämmer, annars false
```

Notera att vi inte själva genererar något salt. bcrypt.hash() gör det automatiskt och bygger in saltet i den returnerade hashsträngen. När vi senare anropar bcrypt.compare() läser den ut saltet ur hashen, hashar inkommande lösenord med samma salt och cost factor, och jämför resultatet.

Det här gör att hashsträngen blir lite märklig att titta på. En typisk bcrypt-hash ser ut så här:

```
$2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhwy
```

Strängen är uppbyggd av flera delar separerade med dollartecken: algoritmsidentifierare (\$2b\$ = bcrypt), cost factor (10), salt (de första 22 tecknen efter cost) och själva hashen (resten). All information som behövs för att verifiera ett lösenord finns i strängen, så vi behöver bara lagra den i databasen.

Användarmodell och registrering

Med bcrypt på plats kan vi börja bygga den faktiska autentiseringen. Det första vi behöver är en användarmodell. I vårt Mongoose-schema lägger vi de fält som behövs för en användare: e-post, lösenord (hashat) och eventuellt namn eller andra fält. Den centrala principen: lösenord-fältet ska bara innehålla hashen, aldrig klartexten.

Här är en minimal användarmodell att utgå ifrån:

```
// models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true,
  },
  password: {
    type: String,
    required: true,
  },
  name: {
    type: String,
    trim: true,
  },
}, {
  timestamps: true,
});

module.exports = mongoose.model('User', userSchema);
```

Lägg märke till två detaljer i schemat. `unique: true` på email säkerställer att vi inte kan ha två användare med samma e-postadress – Mongoose skapar ett databasindex som gör att försök att skapa duplikat misslyckas på databasnivå. `lowercase: true` gör att alla e-postadresser konverteras till gemener innan de sparas, vilket gör jämförelser konsekventa.

Validering av lösenord vid registrering

Innan vi hashar ett lösenord ska vi validera att det är acceptabelt. Vi använder Zod (från vecka 6) för att kräva en minimilängd och eventuella andra regler. Lösenordsregler är ett område där det finns motstridiga råd, men en rimlig grundnivå är:

- Minst 8 tecken – kortare lösenord knäcks för lätt även med bcrypt.
- Maximalt 72 tecken – bcrypt har en gräns på 72 byte och tystar längre lösenord. Vi vill inte att en användare tror de har skydd för långa lösenord när bcrypt egentligen ignorerar resten.
- Inga onödiga komplexitetsregler – moderna säkerhetsråd (NIST) avråder från att kräva specialtecken, blandade gemener/versaler etc. Det leder oftast till sämre lösenord ("Password1!") snarare än bättre. En lång, lättkommad mening är bättre än en kort, krånglig sträng.

Ett enkelt Zod-schema för registrering:

```
const { z } = require('zod');

const registerSchema = z.object({
  email: z.string().email('Ogiltig e-postadress'),
  password: z.string()
    .min(8, 'Lösenordet måste vara minst 8 tecken')
    .max(72, 'Lösenordet får vara maximalt 72 tecken'),
  name: z.string().trim().min(1).optional(),
});
```

Registreringsendpunkten

Med modell och valideringsschema på plats kan vi skriva registreringsendpunkten. Logiken är:

11. Validera input mot Zod-schemat (sker i middleware).
12. Kolla om e-posten redan finns i databasen.
13. Hasha lösenordet med bcrypt.
14. Spara användaren med hashat lösenord.
15. Returnera ett svar – men aldrig innehållande lösenordet eller hashen.

En implementation kan se ut så här:

```
// controllers/authController.js
const bcrypt = require('bcrypt');
const User = require('../models/User');

const SALT_ROUNDS = 10;

async function register(req, res, next) {
  try {
    const { email, password, name } = req.body;

    // Kolla om e-posten redan finns
    const existing = await User.findOne({ email });
    if (existing) {
      return res.status(409).json({
        message: 'En användare med denna e-post finns redan'
      });
    }

    // Hasha lösenordet
    const hash = await bcrypt.hash(password, SALT_ROUNDS);

    // Skapa användaren
    const user = await User.create({
      email,
      password: hash,
      name,
    });
  } catch (error) {
    next(error);
  }
}
```

```

// Returnera svaret UTAN lösenordet
res.status(201).json({
  id: user._id,
  email: user.email,
  name: user.name,
});
} catch (err) {
  next(err);
}
}

module.exports = { register };

```

Det viktiga med svaret: vi konstruerar ett objekt manuellt med just de fält vi vill returnera, snarare än att bara skicka tillbaka user-dokumentet direkt. Om vi gör `res.json(user)` skickar vi tillbaka hela dokumentet, inklusive password-fältet (hashen). Hashen är inte hemlig på samma sätt som klartextlösenordet, men det finns ingen anledning att exponera den för klienten. Principen "minimera vad vi exponerar" är alltid rätt.

En annan teknik är att använda Mongoose-funktionen `toJSON()` på schemat för att automatiskt utesluta känsliga fält. Det är ett bra mönster när modellen blir mer komplex:

```

userSchema.set('toJSON', {
  transform: function(doc, ret) {
    delete ret.password;
    delete ret.__v;
    return ret;
  }
});

```

Med den inställningen kan vi skicka `res.json(user)` tryggt – password-fältet sällas bort automatiskt vid serialisering.

Inloggning: verifiera lösenordet

Inloggningen är spegelbilden av registreringen. Användaren skickar e-post och lösenord. Vi:

16. Letar upp användaren via e-postadressen.
17. Jämför inkommande lösenord med den lagrade hashen via `bcrypt.compare()`.
18. Om lösenordet stämmer: utfärdar en JWT som klienten kan använda i framtida anrop (mer om JWT i nästa kapitel).
19. Om lösenordet inte stämmer: returnerar ett generiskt felmeddelande.

Generiska felmeddelanden vid inloggning

Det här är en viktig säkerhetsdetalj som ofta missas. När en inloggning misslyckas, ska vi aldrig avslöja om det var e-postadressen eller lösenordet som var fel. Vi ska alltid svara med samma generiska meddelande: "Felaktig e-post eller lösenord".

Anledningen är att en angripare som testar en lista över e-postadresser annars kan bygga upp en lista över registrerade konton. Om vi svarar "E-postadressen är inte registrerad" för okända e-poster och "Felaktigt lösenord" för rätt e-poster, har vi just gett angriparen ett verktyg för att kartlägga vilka e-postadresser som är registrerade hos oss. Det är värdefull information för en angripare – de kan nu rikta in sina lösenordsförsök på just de adresserna.

Samma princip gäller för registrering, men med ett tydligare avvägningsproblem. Om vi inte signalerar att en e-post redan finns när användaren försöker registrera sig, blir användarupplevelsen sämre. Här finns en kompromiss: vi kan acceptera att registreringen avslöjar dubletter (eftersom användaren har ett legitimt behov av informationen), men vi behöver inte avslöja det vid inloggning.

Implementation av inloggning

Vi behöver först ett Zod-schema för inloggning. Det är enklare än registrering: bara e-post och lösenord, utan extra valideringsregler (det räcker att svara nej om något är fel).

```
const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(1, 'Lösenord krävs'),
});
```

Och inloggningsfunktionen:

```
// controllers/authController.js (forts.)
async function login(req, res, next) {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({
        message: 'Felaktig e-post eller lösenord'
      });
    }

    const isValid = await bcrypt.compare(password, user.password);
```

```
if (!isValid) {  
  return res.status(401).json({  
    message: 'Felaktig e-post eller lösenord'  
  });  
}  
  
// TODO: utfärda JWT - kommer i nästa avsnitt  
res.json({ message: 'Inloggning lyckades' });  
} catch (err) {  
  next(err);  
}  
}
```

Notera att vi använder samma felmeddelande och samma statuskod (401 Unauthorized) i båda misslyckandefallen. Det går till och med att argumentera för att man borde göra båda kontrollerna även när användaren inte hittas, eller åtminstone fördröja svaret, så att det inte går att skilja på de två fallen genom att mäta svarstiden. För vår kurs är det tillräckligt att vi använder samma meddelande – det är 90% av skyddet.

Hur håller vi en användare inloggad?

Vi har nu en användare som kan registrera sig och logga in. Men HTTP är, som vi lärde oss vecka 1, ett stateless protokoll. Servern minns inget mellan requests. Hur kan en användare förbli inloggad mellan anrop, om servern inte minns dem? Det här är ett av de viktigaste problemen som autentisering löser, och det finns två huvudsakliga lösningar.

Sessioner – serversidans minne

Det traditionella sättet är sessioner. När en användare loggar in skapar servern en sessionspost i en intern lagring (vanligen en databas eller in-memory cache som Redis). Sessionsposten innehåller information om vem användaren är. Klienten får tillbaka ett session-ID, ofta i form av en cookie, och skickar med detta ID i varje framtida request. Servern slår upp sessionsposten utifrån ID:t och vet då vem användaren är.

Sessioner har sina fördelar: vi kan invalidera dem omedelbart (radera sessionsposten ur databasen), vi kan lagra mycket information per session, och de fungerar utmärkt för traditionella webbapplikationer där servern renderar HTML.

Men för REST API:er har sessioner några nackdelar. De gör servern stateful – den behöver komma ihåg sessioner. Det blir besvärligt när vi vill skala horisontellt (köra flera serverinstanser bakom en lastbalanserare), eftersom alla instanser måste dela tillgång till sessionslagringen. Och de fungerar dåligt för mobilappar och fristående klienter där cookies är svårare att hantera.

Tokens – stateless autentisering

Den moderna lösningen för REST API:er är tokens. Istället för att servern minns vem klienten är, ger den klienten en token som klienten själv bevarar. Tokenen är signerad av servern, så ingen kan förfälska den. När klienten skickar tokenen tillbaka kan servern verifiera signaturen och därmed lita på innehållet utan att behöva slå upp något i en databas.

Den stora fördelen med tokens är att de är stateless. Vår server behöver inte komma ihåg någon session – den verifierar bara tokens vid varje request. Det skalar naturligt: vi kan ha tio servrar bakom en lastbalanserare och alla kan verifiera samma tokens utan att dela på något state. Det här passar perfekt med REST-principerna.

Standarden för signerade tokens i webbappar heter JSON Web Tokens (JWT). Det är vad vi ska använda.

JSON Web Tokens (JWT)

En JWT är en signerad sträng som innehåller ett JSON-objekt. Den är utformad så att vem som helst kan läsa innehållet (det är inte krypterat) men bara den som har den hemliga signeringsnyckeln kan skapa eller modifiera en giltig token. Det här är ett vanligt missförstånd – JWT:er är inte hemliga utåt, men de går inte att förfälska.

Strukturen i en JWT är tre delar separerade med punkter:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2NDg5...VnFmF8.s5CCl8zM-V...
```

De tre delarna är:

- Header – metadata om tokenen, som vilken algoritm som använts för att signera den. Base64-kodat JSON.
- Payload – själva innehållet, det vi vill bära med oss. Också Base64-kodat JSON. Det är här vi lägger användarens ID och eventuella andra fält.
- Signature – en kryptografisk signatur av header och payload, beräknad med vår hemliga nyckel. Det är signaturen som garanterar att tokenen inte modifierats.

Eftersom header och payload är base64-kodat (inte krypterat) kan vem som helst läsa innehållet. Prova att klistra in en JWT på jwt.io – ni ser direkt vad som finns i payloaden. Detta är medvetet och okej, så länge vi inte lagrar något hemligt i payloaden.

Hur signaturen skyddar oss

Signaturen beräknas så här (förenklat): servern tar header och payload, lägger till sin hemliga nyckel, och kör resultatet genom en kryptografisk algoritm (typiskt HMAC-SHA256). Resultatet är signaturen, som bifogas tokenen som tredje del.

När tokenen kommer tillbaka till servern senare kan servern göra om beräkningen: ta header och payload, lägga till sin hemliga nyckel, köra samma algoritm – och jämföra resultatet med signaturen i tokenen. Om de stämmer överens vet servern att payloaden inte har modifierats sedan tokenen utfärdades. Om de inte stämmer överens vet vi att någon har försökt manipulera tokenen, eller att tokenen är från en annan utfärdare.

Den hemliga nyckeln måste hållas hemlig på servern. Om någon får tag i den kan de skapa giltiga tokens med vilket innehåll som helst – och vi har förlorat all autentisering. Det är därför vi alltid ska lagra signeringsnyckeln i miljövariabler, aldrig direkt i koden, och aldrig committa den till Git. Vi pratar mer om miljövariabler i vecka 8, men principen gäller redan nu.

Vad ska vi lägga i payloaden?

En vanlig fråga: vad ska egentligen finnas i en JWT-payload? Som tumregel: så lite som möjligt, och inget som är hemligt.

Det vanliga är att lagra användarens ID (sub – subject) och eventuellt roll eller andra ofta-behövda fält. Då kan auth-middleware läsa ut user-ID direkt ur tokenen utan att behöva slå upp användaren i databasen vid varje request. Det är prestandavinsten med stateless tokens.

En typisk payload kan se ut så här:

```
{
```

```
"sub": "6489abc123def456", // User-ID (subject)
"email": "anna@example.com",
"role": "user",
"iat": 1730000000,          // Issued at (timestamp)
"exp": 1730086400          // Expiration time
}
```

Vi ska aldrig lagra:

- Lösenord, tokenhashar eller andra autentiseringsuppgifter.
- Personuppgifter som personnummer eller adress (eftersom payloaden är läsbar för alla som har tokenen).
- Information som kan komma att ändras frekvent – tokenen utfärdas vid inloggning och en användare loggar inte in på nytt vid varje rolländring.

Tokens utgångstid

En av de viktigaste detaljerna är att tokens ska ha en utgångstid (exp – expiration). En JWT som aldrig går ut är en säkerhetsrisk: om tokenen läcker ut är angriparen inloggad för all framtid. Vi sätter därför alltid en exp.

Vad är rimlig livslängd? Det beror på applikationen, men för en typisk webbapp är 1 timme till 24 timmar vanligt. Kortare livslängd är säkrare men kräver att användaren loggar in oftare. För vår kurs sätter vi typiskt 1 dag (24 timmar) – det är en bra avvägning för utbildningsprojekt.

Det finns ett mer avancerat mönster med refresh tokens, där vi har två tokens: en kort access token (15 minuter) och en längre refresh token (dagar eller veckor). När access-tokenen går ut används refresh-tokenen för att få en ny. Det här är produktionsstandard men ligger utanför kursens scope. Vi nöjer oss med en token för enkelhetens skull.

Implementera JWT i Express

Vi använder paketet jsonwebtoken som är de facto standard i Node.js-ekosystemet:

```
npm install jsonwebtoken
```

Det finns två huvudfunktioner: `jwt.sign()` för att skapa en token och `jwt.verify()` för att verifiera och avkoda en. Båda kräver vår hemliga nyckel.

Hemligheten i miljövariabel

Vi placerar JWT-hemligheten i `.env`-filen, precis som vi gjort med MongoDB-anslutningssträngen. `.env` ska aldrig committas till Git – vi har den med i `.gitignore`. För vår utvecklingsmiljö kan en typisk `.env` se ut så här:

```
# .env
MONGODB_URI=mongodb+srv://...
JWT_SECRET=mitt-utvecklings-superhemliga-värde-byt-i-prod
JWT_EXPIRES_IN=1d
```

Hemligheten i utveckling kan vara enkel, men i produktion ska den vara en lång slumpmässig sträng – minst 32 byte slumpmässig data. Vi kan generera en med:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Det ger oss en sträng som ser ut ungefär så här:

a3f4d8c92e1b67ad8f9c4e0d1b3a7f5c8e9d2a6b1f4c7e0a3d6b9c2e5f8d1a4. Den styrkan gör det praktiskt omöjligt för en angripare att gissa nyckeln, även om de hade obegränsad beräkningstid.

Skapa en JWT vid inloggning

Nu kan vi färdigställa inloggningsfunktionen. När lösenordet stämmer skapar vi en JWT och returnerar den till klienten:

```
const jwt = require('jsonwebtoken');

async function login(req, res, next) {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({
        message: 'Felaktig e-post eller lösenord'
      });
    }

    const isValid = await bcrypt.compare(password, user.password);
    if (!isValid) {
      return res.status(401).json({
        message: 'Felaktig e-post eller lösenord'
      });
    }

    const token = jwt.sign({ email }, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
    return res.json({ token });
  } catch (error) {
    next(error);
  }
}
```

```

    });
  }

  // Skapa JWT
  const payload = {
    sub: user._id.toString(),
    email: user.email,
  };

  const token = jwt.sign(
    payload,
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '1d' }
  );

  res.json({
    token,
    user: {
      id: user._id,
      email: user.email,
      name: user.name,
    }
  });
} catch (err) {
  next(err);
}
}

```

Notera att vi konverterar `user._id` till sträng innan vi lägger den i payloaden. ObjectId i MongoDB är ett objekt, men i en JSON-payload vill vi ha en ren sträng. Det gör tokenen lättare att hantera.

Vi sätter `expiresIn` med ett alternativ till `jwt.sign()`. Värdet kan vara en sträng ("1d", "15m", "1h") eller ett antal sekunder. `jwt.sign()` konverterar det till en `exp-claim` i payloaden.

Auth-middleware: skydda routes

Vi har nu en fungerande inloggning som returnerar en JWT. Nästa steg är att använda tokenen för att skydda routes. När klienten gör ett anrop till en skyddad route ska de skicka med tokenen, och vi ska verifiera den innan vi släpper igenom anropet. Det här är ett perfekt arbete för en middleware.

Authorization-headern och Bearer-formatet

Standardplatsen att skicka en JWT är i Authorization-headern, med formatet "Bearer <token>":

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Bearer betyder "innehavare" – principen är att den som visar upp tokenen anses vara innehavare av rättigheterna. Det är därför det är viktigt att tokens hålls säkra på klientsidan, eftersom vem som helst som har tokenen kan utge sig för att vara innehavaren.

Vår middleware ska läsa headern, plocka ut tokenen, verifiera den och – om allt stämmer – lägga information om användaren på request-objektet så att efterföljande route handlers vet vem som gör anropet.

Implementera middlewären

Här är en grundimplementation:

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader) {
    return res.status(401).json({
      message: 'Autentisering krävs'
    });
  }

  // Förväntat format: 'Bearer <token>'
  const parts = authHeader.split(' ');
  if (parts.length !== 2 || parts[0] !== 'Bearer') {
    return res.status(401).json({
      message: 'Felaktigt autentiseringsformat'
    });
  }

  const token = parts[1];

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);

    // Lägg användarinformationen på req för senare middleware/routes
    req.user = {
```

```

      id: payload.sub,
      email: payload.email,
    };

    next();
  } catch (err) {
    return res.status(401).json({
      message: 'Ogiltig eller utgången token'
    });
  }
}

module.exports = { authenticate };

```

Nyckeln i den här koden är `jwt.verify()`. Den gör fyra saker samtidigt:

20. Kontrollerar att signaturen är giltig (att tokenen är utfärdad av oss).
21. Kontrollerar att tokenen inte har gått ut (`exp-claimet`).
22. Avkodar payloaden från base64.
23. Kastar ett fel om något inte stämmer.

Om verifieringen lyckas har vi payloaden och kan plocka ut `user-id` och annan information. Vi sätter den på `req.user` så att efterföljande handlers har tillgång till den. Om verifieringen misslyckas svarar vi med 401 Unauthorized.

Använda middlewären på routes

Vi kan nu använda `authenticate` på routes som ska vara skyddade. Tre vanliga mönster:

```

const { authenticate } = require('./middleware/auth');

// 1. Skydda en enskild route
app.get('/api/profile', authenticate, (req, res) => {
  // req.user finns här
  res.json({ id: req.user.id, email: req.user.email });
});

// 2. Skydda en hel router
const ordersRouter = require('./routes/orders');
app.use('/api/orders', authenticate, ordersRouter);

// 3. Skydda en grupp routes globalt utom vissa
app.post('/api/auth/register', register);
app.post('/api/auth/login', login);
app.use(authenticate); // Allt efter detta kräver auth
app.use('/api/orders', ordersRouter);
app.use('/api/products', productsRouter);

```

Vilket mönster vi väljer beror på applikationen. Att deklarera `authenticate` på varje skyddad route är mer explicit och tydligt – när man läser routen ser man direkt att den är skyddad. Att skydda hela routrar eller använda `app.use()` är mer bekvämt och svårare att glömma. För

nybörjare rekommenderar vi det första mönstret tills man fått känsla för hur middlewären beter sig.

Använda req.user i route handlers

När en authenticate-middleware har körts vet vi att req.user är satt. Det öppnar möjligheter att skriva routes som beter sig olika beroende på vilken användare som gör anropet. Det här är central pusselbiten för åtkomstkontroll på resursnivå.

Föreställ er en endpoint som hämtar "mina beställningar":

```
router.get('/my-orders', authenticate, async (req, res, next) => {
  try {
    const orders = await Order.find({ owner: req.user.id });
    res.json(orders);
  } catch (err) {
    next(err);
  }
});
```

Här finns inget url-parameter för user-id – vi tar det från tokenen istället. Det betyder att en användare bara kan hämta sina egna beställningar, oavsett vad de skickar i URL:en. Det är säkrare än att låta klienten skicka user-id som parameter, eftersom klienten kan ljuga om vad som finns i URL:en, men inte om vad som finns i en signerad token.

När en route däremot har en URL-parameter, som GET /orders/:id, behöver vi göra en extra kontroll: hämta beställningen och kolla att den ägs av inloggad användare. Annars kan vem som helst hämta vilken beställning som helst genom att gissa ID:n.

```
router.get('/:id', authenticate, async (req, res, next) => {
  try {
    const order = await Order.findById(req.params.id);

    if (!order) {
      return res.status(404).json({ message: 'Beställning hittades inte' });
    }

    // Kontrollera att inloggad användare äger beställningen
    if (order.owner.toString() !== req.user.id) {
      return res.status(403).json({ message: 'Åtkomst nekad' });
    }

    res.json(order);
  } catch (err) {
    next(err);
  }
});
```

Notera skillnaden mellan 401 och 403. 401 Unauthorized betyder att vi inte vet vem användaren är (de är inte autentiserade) – det löser authenticate-middlewareen. 403 Forbidden betyder att vi vet vem användaren är, men de har inte rätt att utföra åtgärden. Den distinktionen är värd att hålla rätt på i sina svars-koder.

Skydd mot injection i Mongoose

Vi diskuterade tidigare i materialet att Mongoose i grunden skyddar oss mot injection genom att alltid skicka frågor och parametrar separat till databasen. Men det finns ett par vanliga fallgropar som vi ska förstå för att verkligen vara säkra.

Fallgrop 1: skicka req.body direkt till en query

Det här är den största risken med Mongoose. När vi gör så här ser det praktiskt ut, men öppnar för NoSQL injection:

```
// FARLIGT - skickar otestad input direkt
const user = await User.findOne(req.body);
```

Som vi diskuterade tidigare kan en angripare skicka { email: { "\$ne": null } } i request body och därmed manipulera frågan. Lösningen är dubbel: vi ska alltid validera input med Zod (vecka 6:s lärdomar gäller fortfarande), och vi ska alltid extrahera fält explicit, inte skicka in hela objektet:

```
// SÄKERT - extrahera explicit och förlita på Zod-validering
const { email } = req.body;
const user = await User.findOne({ email });
```

När email är validerat som z.string() kan det inte vara ett objekt – Zod stoppar requesten redan i middleware om någon försöker. Det är skälet till att vi sa att validering och säkerhet är samma sak.

Fallgrop 2: dynamiska fältnamn

Något subtilare fallgrop är när vi tillåter klienten att ange vilket fält de vill söka på. Föreställ er en sökendpoint som tar field och value från klienten:

```
// FARLIGT - klienten kan söka på vilket fält som helst
const query = { [req.query.field]: req.query.value };
const result = await User.find(query);
```

Här kan en angripare söka på password-fältet (eller någon annan känslig fält) och få ut hashar de inte borde ha tillgång till. Lösningen är att alltid validera vilka fält som är tillåtna:

```
// SÄKERT - vitlista tillåtna fält
const ALLOWED_FIELDS = ['email', 'name'];
const field = req.query.field;

if (!ALLOWED_FIELDS.includes(field)) {
  return res.status(400).json({
    message: `Otillåtet sökfält: ${field}`
  });
}

const query = { [field]: req.query.value };
const result = await User.find(query);
```

Mönstret "vitlista, inte svartlista" är en universell säkerhetsprincip. Vi listar vad som är tillåtet och nekar allt annat – snarare än att lista vad som är förbjudet och tillåta allt annat. Vitlistor är säkra by default; svartlistor missar saker vi inte tänkt på.

Fallgrop 3: \$-operatorer i input

Även med Zod-validering på topp-nivå kan vissa attacker komma igenom om vi inte är försiktiga. MongoDB:s frågespråk använder operatorer som börjar med \$ (\$eq, \$gt, \$where, \$regex). Om vi någon gång konstruerar en query där värden från användaren hamnar som operatorvärden måste vi vara extra försiktiga.

För grundläggande användning, där vi alltid extraherar fält explicit och låter Zod validera dem, är det här inget direkt problem. Men det är värt att veta att paket som mongo-sanitize finns – de strippar bort \$-prefixade keys ur objekt som ett extra skyddslager. Vi behöver det inte i den här kursen, men det är ett verktyg att känna till för produktion.

Att sätta ihop helheten

Vi har nu gått igenom alla byggstenar: registrering, lösenordshashning, inloggning, JWT-utfärdning, auth-middleware och routeskydd. Låt oss sätta ihop dem till en komplett bild av hur ett autentiseringsflöde ser ut i ett Express-API.

Filstrukturen i en typisk applikation kan se ut så här efter denna vecka:

```
mitt-api/
  src/
    routes/
      auth.js          # /register, /login
      orders.js        # Skyddade beställningsroutes
    controllers/
      authController.js
      ordersController.js
    middleware/
      auth.js          # authenticate-middleware
      validate.js       # Zod-valideringsmiddleware (från v6)
      errorHandler.js  # Centraliserad felhanterare (från v6)
    models/
      User.js
      Order.js
    schemas/
      authSchemas.js  # Zod-schemas för auth
  app.js
  server.js
  .env                 # JWT_SECRET, MONGODB_URI
  .gitignore
  package.json
```

Auth-routern

Med valideringen från vecka 6 och vår nya auth-logik kan en komplett auth-router se ut så här:

```
// routes/auth.js
const express = require('express');
const { register, login } = require('../controllers/authController');
const { validate } = require('../middleware/validate');
const { registerSchema, loginSchema } = require('../schemas/authSchemas');

const router = express.Router();

router.post('/register', validate(registerSchema), register);
router.post('/login', validate(loginSchema), login);

module.exports = router;
```

Notera middleware-kedjan: först validering, sedan controller. Om Zod hittar ett valideringsfel kommer requesten aldrig fram till kontrollern. Det betyder att vi i kontrollern kan utgå ifrån att data är giltig och slipper skriva defensiv kod för dåliga inputs.

Skyddade routes med ägarskapskontroll

En skyddad routegrupp kombinerar autentisering med ägarskapskontroll:

```
// routes/orders.js
const express = require('express');
const { authenticate } = require('../middleware/auth');
const Order = require('../models/Order');

const router = express.Router();

// Alla routes nedan kräver autentisering
router.use(authenticate);

// Hämta inloggad användares beställningar
router.get('/', async (req, res, next) => {
  try {
    const orders = await Order.find({ owner: req.user.id });
    res.json(orders);
  } catch (err) {
    next(err);
  }
});

// Skapa en beställning för inloggad användare
router.post('/', async (req, res, next) => {
  try {
    const order = await Order.create({
      ...req.body,
      owner: req.user.id, // Sätt ägaren från tokenen
    });
    res.status(201).json(order);
  } catch (err) {
    next(err);
  }
});

// Hämta en specifik beställning - bara om den ägs av användaren
router.get('/:id', async (req, res, next) => {
  try {
    const order = await Order.findById(req.params.id);

    if (!order) {
      return res.status(404).json({ message: 'Hittades inte' });
    }

    if (order.owner.toString() !== req.user.id) {
      return res.status(403).json({ message: 'Åtkomst nekad' });
    }

    res.json(order);
  } catch (err) {
    next(err);
  }
});
```

```

    }
  });

  module.exports = router;

```

Mönstret är konsekvent: vi sätter owner från req.user.id (tokenen), aldrig från klientens input. Och vi kontrollerar ägarskap innan vi returnerar eller modifierar resurser med URL-parameter.

app.js – sammanställning

Slutligen kopplas allt ihop i app.js:

```

// app.js
const express = require('express');
const cors = require('cors');
const authRouter = require('./routes/auth');
const ordersRouter = require('./routes/orders');
const errorHandler = require('./middleware/errorHandler');

const app = express();

app.use(cors());
app.use(express.json());

app.use('/api/auth', authRouter);
app.use('/api/orders', ordersRouter);

app.use(errorHandler);

module.exports = app;

```

Testa flödet end-to-end

När allt är på plats kan vi testa flödet med Thunder Client eller Postman:

24. POST /api/auth/register med email, password och name. Förväntat svar: 201 Created med användarens info (utan lösenord).
25. POST /api/auth/login med email och password. Förväntat svar: 200 OK med en token-sträng.
26. GET /api/orders utan Authorization-header. Förväntat svar: 401 Unauthorized.
27. GET /api/orders med Authorization: Bearer <token>. Förväntat svar: 200 OK med en lista (eventuellt tom).
28. POST /api/orders med valid token och body, sedan GET /api/orders för att se beställningen.

Om alla dessa steg fungerar har vi byggt vårt första autentiserade API. Det är en milstolpe i kursen – vi har gått från ett öppet API som vem som helst kan använda, till ett API där användare har egna identiteter, lösenord skyddas korrekt, och resurser endast är tillgängliga för sina ägare.

Praktiska säkerhetstips

Innan vi avrundar, några ytterligare detaljer som hör hemma i en seriös autentiseringsimplementation.

Logga aldrig lösenord eller tokens

En vanlig miss är att logga req.body i debug-syfte. Det innebär att man loggar lösenord vid varje registrering och inloggning, och tokens vid varje skyddat anrop. Loggar är ofta mindre skyddade än databaser och kan hamna i tredjepartstjänster (som loggaggregatorer). Vi ska aldrig logga något som finns i Authorization-headern eller i password-fältet.

När vi loggar requests ska vi vara explicita om vad vi loggar. Om vi använder ett färdigt loggningsbibliotek som morgan eller pino kan vi konfigurera det att hoppa över känsliga fält. Vi pratar mer om strukturerad loggning under vecka 9, men principen är: tänk innan du loggar.

Rate limiting på autentisering

En enskild användare ska inte kunna gissa sig till lösenord genom att skicka tusentals försök i sekunden. Det löser vi med rate limiting – en gräns för hur många anrop en IP-adress får göra inom en viss tidsperiod. Paketet express-rate-limit är standard för detta:

```
npm install express-rate-limit

const rateLimit = require('express-rate-limit');

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minuter
  max: 5,                    // 5 försök per IP
  message: 'För många inloggningsförsök, försök igen senare'
});

router.post('/login', authLimiter, validate(loginSchema), login);
```

Vi pratar mer om rate limiting i vecka 10 i samband med driftsättning, men det är värt att känna till redan nu att det här är en viktig komplementär skyddsåtgärd.

HTTPS i produktion

Vår JWT är base64-kodat JSON, inte krypterat. Om en angripare kan se nätverkstrafiken kan de stjäla tokenen och utge sig för att vara användaren. Det är därför produktion-API:er alltid ska köras över HTTPS. Under utveckling på localhost är det okej att köra HTTP, men så fort vi släpper systemet ut på internet ska all trafik vara krypterad.

Vi pratar mer om HTTPS och driftsättning under vecka 10. Lyckligtvis sköter de flesta moderna driftsättningsplattformar (Render, Railway, Vercel) HTTPS automatiskt – vi behöver inte göra något särskilt själva.

Tokenförvaring på klientsidan

En sista detalj som ligger lite utanför vår backend men ändå är viktig att förstå: hur ska klienten lagra sin token? De vanligaste alternativen i en webbläsarklient är:

- localStorage – enkelt att använda men sårbart för XSS-attacker (om en angripare får in JavaScript på sidan kan de läsa tokenen).
- httpOnly-cookies – säkrare mot XSS men kräver att frontend och backend ligger på samma domän eller att vi konfigurerar CORS noggrant.
- Memory (variabel i appen) – säkrast mot XSS men kräver återinloggning vid sidladdning.

Det finns inte ett perfekt svar – det är en avvägning mellan säkerhet och användarvänlighet. För boiler room-projekten i den här kursen är localStorage en pragmatisk lösning, men i produktion skulle vi typiskt välja httpOnly-cookies med korrekt CORS-konfiguration.

Checkpoint: Förberedelse inför kursvecka 7

Inför workshopen i vecka 7 ska vi ha läst igenom detta material och förberett oss på följande sätt. Förberedelsen är extra viktig den här veckan eftersom vi täcker både teori och praktik – och teorin kring säkerhet är något vi vill diskutera tillsammans, inte bara hasta igenom.

Praktisk förberedelse:

29. Säkerställ att vårt API från vecka 6 fungerar: vi har en ren felhanterare, Zod-validering på alla endpoints och en MongoDB-koppling.
30. Skapa en .env-fil i projektroten om vi inte redan har en. Lägg till JWT_SECRET med ett långt slumpmässigt värde. Säkerställ att .env finns i .gitignore.
31. Installera dependencies inför workshopen: bcrypt, jsonwebtoken och dotenv om vi inte redan har dem.
32. Bekanta er med jwt.io – klistra in en exempel-JWT där och se hur strukturen ser ut.

Reflektioner att ta med till workshopen:

- Vad är skillnaden mellan autentisering och auktorisering, med era egna ord?
- Varför går det inte att "dekryptera" en bcrypt-hash, och varför är det önskvärt?
- Om en angripare får tag i vår JWT_SECRET, vad är konsekvenserna och hur skulle vi hantera det?
- Hur ändras balansen mellan säkerhet och användarvänlighet beroende på vilken applikation vi bygger? En banktjänst vs en blogg?

För boiler room-projektet är denna vecka en viktig milstolpe: vi ska implementera autentisering för vårt projekt. Tänk redan nu igenom vilka resurser som ska kräva inloggning, och vilka som kan vara öppna. Skissa gärna upp det innan workshopen.

Sammanfattning

Under kursvecka 7 har vi tagit första stora steget in i säkerhetsdomänen. Vi har lärt oss att tänka som en angripare och förstå att backenden är där känslig data lever – och därmed också där det egentliga säkerhetsansvaret ligger. Det perspektivet förändrar hur vi designar varje endpoint.

Vi har gått igenom OWASP Top 3: Broken Access Control (vem får göra vad), Cryptographic Failures (hur skyddar vi känslig data) och Injection (hur hindrar vi att klientens data blandas ihop med vår kod). Vi har förstått att Mongoose i grunden skyddar oss mot injection, så länge vi också gör vår del med Zod-validering och inte skickar in obearbetad input direkt till queries. Vi har också reflekterat kring skillnaden mellan verklig säkerhet och "security by obscurity", och varför vi alltid ska bygga på beprövad teknik snarare än hemlighållande.

Konkret har vi implementerat säker lösenordshantering med bcrypt: aldrig klartext, alltid hashat med automatiskt salt och en cost factor som gör knäckningsförsök opraktiska. Vi har skrivit både registrering och inloggning, och förstått varför inloggningssvaret aldrig får avslöja om det var e-postadressen eller lösenordet som var fel.

Vi har djupdykt i JSON Web Tokens som mekanism för stateless autentisering. Vi förstår strukturen `header.payload.signature`, vad signaturen skyddar mot, varför payloaden inte är hemlig, och vad vi får och inte får lägga i den. Vi använder `jwt.sign()` för att utfärda tokens vid inloggning och `jwt.verify()` i en middleware för att verifiera dem på inkommande requests. Tokens har en utgångstid – de ska aldrig vara eviga.

Auth-middlewareen läser Authorization-headern, plockar ut Bearer-tokenen, verifierar den och placerar användarinformationen på `req.user` för efterföljande handlers. Det här mönstret kommer vi att bygga vidare på nästa vecka när vi lägger till rollbaserad åtkomstkontroll – då kan vi ha en kedja av middleware som först autentiserar och sedan auktoriserar för en specifik roll.

Vi har också rört vid praktiska detaljer som rate limiting, HTTPS, tokenförvaring i frontend och vikten av att aldrig logga lösenord eller tokens. Det är detaljer som inte är centrala för veckan men som vi behöver känna till för att förstå helheten.

Nästa vecka går vi vidare till auktorisering: skillnaden mellan vad en användare och en admin får göra, hur vi designar roller och behörigheter, och hur vi använder middleware för att kontrollera rollkrav på enskilda routes. Vi gräver också djupare i miljövariabler och konfigurationshantering – ett ämne som har blivit viktigare för oss nu när vi har en JWT-hemlighet att skydda. Allt vi gör de kommande veckorna bygger på det vi lagt grunden för i dag.

Resurser för fördjupning

Säkerhet är ett område där det alltid finns mer att lära sig. Här är några resurser vi rekommenderar för den som vill gå djupare.

Officiell dokumentation

OWASP Top 10 – officiell dokumentation av de tio största riskerna med detaljerade exempel och motåtgärder:

<https://owasp.org/Top10>

OWASP Cheat Sheet Series – praktiska checklistor för specifika säkerhetsutmaningar (autentisering, lösenordshantering, JWT, etc.):

<https://cheatsheetseries.owasp.org>

jsonwebtoken på npm – paketets dokumentation med alla alternativ till sign() och verify():

<https://www.npmjs.com/package/jsonwebtoken>

bcrypt på npm – officiell dokumentation och prestandanoteringar:

<https://www.npmjs.com/package/bcrypt>

jwt.io – interaktiv debugger där vi kan klistra in tokens och se deras innehåll, samt en sammanfattning av JWT-standarderna:

<https://jwt.io>

Säkerhetsresurser

OWASP Authentication Cheat Sheet – mycket konkreta råd om hur autentisering ska implementeras, från lösenordsregler till session management. Mer omfattande än vad vi går igenom i kursen och ett bra nästa steg:

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP JSON Web Token Cheat Sheet – specifik vägledning kring säker JWT-hantering, inklusive vanliga misstag att undvika:

https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

Have I Been Pwned – databas över kända databasläckor. Användbart både för att kolla om egna konton komprometterats och för att förstå vilka kategorier av läckor som faktiskt sker i verkligheten:

<https://haveibeenpwned.com>

Videomaterial

"What is JWT?" och "JWT Authentication Tutorial" på YouTube av Web Dev Simplified – välproducerade introduktionsvideor som kompletterar materialet med visuella förklaringar av JWT-strukturen och autentiseringsflödet.

"Password Storage Tier List" på YouTube av Bytebytego – en mycket pedagogisk genomgång av varför olika sätt att lagra lösenord rangordnas som de gör, från klartext (sämst) till bcrypt och argon2 (bäst). Hjälper till att befästa varför vi gör som vi gör.

"OWASP Top 10: A Quick Look" – det finns många videor på YouTube under den titeln. Välj någon som är från senare år (2021 eller senare) eftersom listan har uppdaterats. Det ger en visuell sammanfattning av samma material vi gått igenom här.

Studiematerial

Auktorisering, RBAC och miljövariabler

Kursvecka 8

Introduktion

Vi har nu sju veckor bakom oss i kursen och har byggt en imponerande grund. Vi behärskar Node.js och Express, designar väl strukturerade REST API:er, har lagt till TypeScript för säkrare typning, kopplat backend till MongoDB via Mongoose med fullständiga CRUD-operationer, validerar all inkommande data med Zod, har robust felhantering och har precis lärt oss att autentisera användare med bcrypt och JWT. Vi har en användare som kan registrera sig, logga in och få ett token som skyddar våra routes.

Men säkerhet handlar inte bara om vem du är – det handlar lika mycket om vad du får göra. En inloggad användare ska kanske kunna se sina egna ordrar, men inte andras. En vanlig användare ska kanske kunna läsa produktinformation, men bara en admin ska kunna lägga till nya produkter eller ta bort användarkonton. Den här distinktionen – mellan att veta vem någon är och att avgöra vad den får göra – är en av de mest fundamentala i all säkerhetsdesign.

Kursvecka 8 handlar om auktorisering: hur vi bestämmer vad inloggade användare får göra. Vi går igenom rollbaserad åtkomstkontroll (RBAC), den vanligaste och mest beprövade modellen för auktorisering. Vi designar roller, lagrar dem i databasen, kopplar dem till användare och bygger middleware som skyddar routes baserat på roll. Vi tittar också på hur RBAC förhåller sig till mer finkornig behörighetshantering.

Den andra halvan av veckan handlar om något som löper parallellt med all säkerhet vi byggt: hanteringen av hemligheter. Vi har redan använt miljövariabler i tidigare veckor – för att lagra databasanslutningssträngen och JWT-hemligheten – men nu fördjupar vi oss på riktigt. Vi går igenom hur en seriös backend hanterar konfiguration över olika miljöer (utveckling, test, produktion), vad som aldrig får checkas in till Git och hur vi kan validera att alla nödvändiga miljövariabler finns när applikationen startar.

Slutligen knyter vi ihop det vi byggt under hela kursen genom att se på hela bilden: hur frontend, backend och databas samverkar i ett fullstacksystem. Vi har byggt komponenter, men nu zoomar vi ut och förstår hur ett komplett dataflöde ser ut – från användarens klick i React-appen, genom HTTP-anropet till vår Express-server, autentisering och auktorisering, vidare till databasen och hela vägen tillbaka. Det är den helhetsbilden som gör skillnaden mellan att kunna skriva backendkod och att förstå hur fullstack faktiskt fungerar.

Autentisering vs auktorisering

De två orden låter lika, börjar på samma bokstav och hänger så tätt ihop att de ofta blandas ihop i vardagligt tal. Men de svarar på två helt olika frågor, och att förstå skillnaden är grundläggande för att kunna bygga säkra system.

Autentisering (engelska: authentication, ofta förkortat authn) svarar på frågan: vem är du? Det är processen där en användare bevisar sin identitet, vanligtvis genom att skicka in ett användarnamn och lösenord. Vi verifierar lösenordet mot en hashad version i databasen, och om det stämmer skapar vi en token (JWT) som användaren kan visa upp i framtida anrop. Det är vad vi byggde under kursvecka 7.

Auktorisering (engelska: authorization, ofta förkortat authz) svarar på frågan: vad får du göra? Det är processen där vi, för en redan autentiserad användare, avgör om de har behörighet att utföra en specifik åtgärd – läsa en viss resurs, skapa något, ta bort något. Auktorisering förutsätter att autentiseringen redan har ägt rum: vi kan inte besluta vad någon får göra utan att först veta vem den är.

En vardaglig analogi

Tänk dig att du kommer till ett kontor. Vid receptionen visar du upp ditt körkort – det är autentisering. Receptionen verifierar att personen på fotot är du och att kortet är giltigt. Du har bevisat vem du är.

Sedan vill du ta dig in i serverhallen. Vakten kollar ditt passerkort och ser att du är registrerad som vaktmästare. Vaktmästare har inte tillträde till serverhallen – det är auktorisering. Att du är legitimerad och inloggad i byggnaden räcker inte; det krävs en specifik behörighet. En tekniker, som också autentiserat sig med körkort, hade fått passera.

Samma sak händer i vårt API. När en request kommer in med ett JWT verifierar vi tokenet (autentisering): är den giltig, signerad av oss, inte utgången? Om det stämmer vet vi vem användaren är. Sedan tittar vi på vad användaren försöker göra och om dennes roll tillåter det (auktorisering).

Två distinkta felscenarier

Det är viktigt att vi reflekterar dessa två steg i våra HTTP-statuskoder, eftersom de faktiskt motsvarar två olika typer av fel:

- 401 Unauthorized – returneras när autentiseringen misslyckas. Trots det missvisande namnet (det borde egentligen heta Unauthenticated) signalerar 401 att klienten inte har lagt fram en giltig identitet. Token saknas, är ogiltig, eller har gått ut.
- 403 Forbidden – returneras när autentiseringen lyckats men auktoriseringen misslyckas. Vi vet vem användaren är, och vi accepterar deras token, men deras roll eller behörighet räcker inte för det de försöker göra.

Det är en distinktion vi alltid ska respektera. En klient som ser 401 vet att den behöver logga in på nytt. En klient som ser 403 vet att inloggningen i sig är okej, men att åtgärden inte är tillåten – det är ett helt annat användarflöde att hantera.

Ordningen spelar roll

I praktiken körs autentisering alltid före auktorisering. I våra Express-applikationer betyder det att auth-middlewären vi byggde under vecka 7 (den som verifierar JWT och lägger användaren på req.user) körs först. Sedan kommer en RBAC-middleware som tittar på

`req.user.role` och avgör om åtgärden är tillåten. Den ordningen är inte slumpmässig – det är en logisk konsekvens av att auktorisering kräver en känd identitet.

Vi kommer att se den här kedjan upprepas genom hela veckan: först `authenticate`, sedan `authorize`, sedan affärslogiken. Det är ett av de viktigaste mönstren i all backendutveckling och det vi nu bygger på.

Rollbaserad åtkomstkontroll (RBAC)

Det finns flera modeller för auktorisering. Vi kunde tilldela varje enskild användare specifika rättigheter (t.ex. "Anna får läsa, Erik får läsa och skriva, Sofia får läsa, skriva och ta bort"). Det fungerar för små system men blir snabbt ohanterligt: om vi har hundra användare och tjugo olika åtgärder behöver vi hantera tvåtusen kombinationer, och varje gång en användare byter avdelning eller en ny åtgärd läggs till måste vi gå igenom dem en och en.

Rollbaserad åtkomstkontroll (RBAC) löser det här problemet med en enkel insikt: i de flesta organisationer har människor liknande arbetsuppgifter och behöver liknande behörigheter. Istället för att tilldela rättigheter till individer skapar vi roller (admin, redaktör, kund, support) och tilldelar rättigheter till rollerna. Sedan tilldelar vi roller till användare. När en ny användare anställs sätts deras roll till exempelvis "redaktör" och de får automatiskt alla rättigheter som rollen innebär.

De tre nivåerna i RBAC

RBAC-modellen vilar på tre lager som tillsammans bildar systemet:

- Användare (users) – de personer eller tjänster som använder systemet. I vårt fall lagras de som dokument i en MongoDB-collection.
- Roller (roles) – namngivna grupper av rättigheter, exempelvis "admin", "user" eller "moderator". En roll är inte en person – det är en samling tillåtna åtgärder som beskriver en typisk arbetsroll i systemet.
- Behörigheter (permissions) – konkreta åtgärder eller resurser, exempelvis "products:create", "users:delete" eller "orders:read". Behörigheter knyts till roller, inte till individer.

Genom att tilldela en användare en roll får den indirekt alla behörigheter som rollen innehåller. Den indirekta kopplingen är hela poängen – den gör systemet flexibelt och underhållbart.

Varför RBAC och inte bara individuella rättigheter?

Tänk att en organisation har 50 redaktörer som alla behöver samma rättigheter: skapa artiklar, redigera sina egna artiklar, se publiceringskön. Med individuella rättigheter skulle vi behöva tilldela alla 50 dessa tre rättigheter manuellt – och om vi senare vill lägga till en ny rättighet ("se kommentarsstatistik") måste vi uppdatera alla 50 användare.

Med RBAC har vi en roll "redaktör" som har dessa tre rättigheter. De 50 redaktörerna har alla samma roll. När den fjärde rättigheten ska läggas till uppdaterar vi rollen en gång – alla 50 användare får automatiskt den nya rättigheten. Det är inte bara mindre arbete, det är också säkrare: det går inte att råka glömma någon.

När en redaktör slutar och en ny anställs är det också enkelt: ny användare, samma roll. När ett team ändrar sina arbetsuppgifter ändrar vi rollen en gång och alla i teamet påverkas konsekvent.

Designprinciper för roller

När vi designar roller i ett system finns det några principer vi bör hålla oss till:

- Roller speglar verkliga arbetsroller – inte tekniska detaljer. "Admin", "kund", "support" är bra rollnamn. "CanReadProducts" är inte en roll, det är en behörighet.

- Principen om minst privilegium (least privilege) – varje roll ska bara ha de behörigheter som är absolut nödvändiga för rollens arbetsuppgifter. Vi ger inte admin-behörighet till någon som inte behöver det.
- Tydlig hierarki när det är meningsfullt – ofta är admin en superuser som kan allt, user kan göra grundläggande saker och moderator/redaktör har en delmängd av admins rättigheter. Hierarkin behöver dock inte vara strikt linjär – två roller kan ha överlappande men inte identiska behörigheter.
- Få roller är ofta bättre än många – tio noggrant designade roller är vanligtvis mer hanterbara än hundra hyperspecifika. Om vi behöver mer finkornig kontroll är det ofta bättre att lägga till behörigheter än att skapa fler roller.

Vanliga rolldesigner i webbapplikationer

För många typer av applikationer räcker det med ett fåtal grundläggande roller:

- En e-handelsapplikation kan ha rollerna user (kund), staff (kundsupport som kan se ordrar) och admin (full kontroll över produkter, ordrar och användare).
- En blogg eller nyhetssajt kan ha reader (läsare), author (kan skapa egna inlägg), editor (kan publicera och redigera andras inlägg) och admin.
- En SaaS-tjänst kan ha owner (företagsägaren med fakturering), member (vanlig anställd) och guest (extern medlem med begränsade rättigheter).

För boiler room-projekten i kursen kommer en enkel uppdelning som user och admin att räcka i de flesta fall – grupperna kan utöka det om deras specifika scenario kräver det.

Lagra roller i databasen

För att kunna kontrollera roller behöver vi först lagra dem. Den naturliga platsen är i vår User-modell – varje användare har en eller flera roller, och rollen följer användaren över tiden.

Det finns två huvudvarianter av rollagring: enstaka roll per användare (en sträng) eller flera roller per användare (en lista med strängar). Båda är giltiga val. Enstaka roll är enklare och räcker för många applikationer; flera roller ger mer flexibilitet när användare kan tillhöra flera grupper samtidigt.

Enstaka roll med Mongoose

Det enklaste fallet är när varje användare har exakt en roll. Vi utökar User-schemat vi byggde under vecka 7 med ett role-fält:

```
// models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const userSchema = new mongoose.Schema(
  {
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
    },
    passwordHash: {
      type: String,
      required: true,
    },
    role: {
      type: String,
      enum: ['user', 'admin'],
      default: 'user',
      required: true,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('User', userSchema);
```

Två viktiga detaljer i schemat: enum begränsar vilka värden som är giltiga – Mongoose kommer att avvisa alla försök att spara en användare med rollen 'superhacker'. default: 'user' säkerställer att nya användare alltid får den minst privilegierade rollen, vilket ligger i linje med principen om minst privilegium. Ingen användare får admin-rollen av misstag.

Flera roller per användare

Om vi behöver att en användare kan ha flera roller (exempelvis en kund som också är moderator i forum-delen) använder vi en array istället:

```
roles: {  
  type: [String],  
  enum: ['user', 'moderator', 'admin'],  
  default: ['user'],  
},
```

Logiken i vår middleware blir då en aning annorlunda: istället för `user.role === 'admin'` använder vi `user.roles.includes('admin')`. Vi går igenom båda varianterna senare i materialet.

Hur skapas den första admin-användaren?

Om default är 'user' kommer ingen att registrera sig som admin via det vanliga registreringsflödet, vilket är precis vad vi vill. Men då uppstår frågan: hur får vi den allra första admin-användaren?

Det finns flera vanliga lösningar:

- Skapa admin direkt i databasen – via MongoDB Atlas-konsolen eller en mongosh-session uppdaterar vi en befintlig användare med `db.users.updateOne({ email: 'admin@example.com' }, { $set: { role: 'admin' } })`.
- Seed-script – en separat fil som körs en gång och skapar en admin med uppgifter från miljövariabler. Användbart för att sätta upp utvecklings- och testmiljöer.
- Beforndringsmekanism – en befintlig admin har en separat endpoint för att utse nya admins. Det är hur det fungerar i de flesta produktionssystem.

Det vi aldrig ska göra är att ha en publik endpoint som låter vem som helst registrera sig som admin. Det är ett privilegieskaleringsskäl av första rang.

Roller i JWT-payloaden

När en användare loggar in genererar vi ett JWT med en payload som innehåller information om användaren. Under vecka 7 lade vi typiskt in user-id (sub eller userId). Nu lägger vi även till rollen, så att vi kan utföra auktoriseringskontroller utan att behöva slå upp användaren i databasen vid varje request.

```
// I login-controllern
const jwt = require('jsonwebtoken');

function generateToken(user) {
  const payload = {
    sub: user._id.toString(),
    email: user.email,
    role: user.role,
  };

  return jwt.sign(payload, process.env.JWT_SECRET, {
    expiresIn: '1h',
  });
}
```

När token sedan kommer in i en framtida request verifierar vår auth-middleware den och lägger payloaden på req.user:

```
// middleware/authenticate.js
const jwt = require('jsonwebtoken');

function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({
      error: 'Authentication required',
    });
  }

  const token = authHeader.split(' ')[1];

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = payload; // { sub, email, role }
    next();
  } catch (error) {
    return res.status(401).json({
      error: 'Invalid or expired token',
    });
  }
}

module.exports = authenticate;
```

En viktig säkerhetsdetalj

Att lägga in rollen i JWT-payloaden är bekvämt eftersom vi då slipper en databasfråga vid varje request. Men det innebär också en viktig sak: om vi ändrar en användares roll i databasen kommer deras gamla, redan utgivna token fortfarande att innehålla den gamla rollen tills den går ut.

Det här är en känd avvägning med JWT. Om en användares behörighet höjs syns det inte förrän nästa inloggning. Om en användares behörighet sänks – allvarigare ur ett säkerhetsperspektiv – kan en aktiv token fortfarande ge dem den högre behörigheten i upp till tokens livslängd. Det är ett av skälen till att JWT-tokens vanligtvis har relativt korta livslängder (15 minuter till någon timme) och kombineras med refresh tokens. För kursens scope är det viktigaste att vi förstår avvägningen, även om vi inte implementerar refresh tokens.

För en alternativ design kan vi istället slå upp användaren från databasen i auth-middlewareen och hämta rollen därifrån. Det ger alltid uppdaterade roller men kostar en databasfråga per request. Båda mönstren förekommer i verkliga system, och valet beror på krav på prestanda och konsekvens.

RBAC-middleware i Express

Nu när vi har rollen tillgänglig på req.user kan vi bygga en middleware som blockerar requests som inte har rätt behörighet. Eftersom vi ska kunna återanvända samma logik för olika roller på olika routes använder vi en factory-funktion: en funktion som returnerar en middleware-funktion baserat på vilka roller som ska tillåtas.

En enkel rollkontroll

Den enklaste varianten godkänner en specifik roll:

```
// middleware/authorize.js

function requireRole(role) {
  return function (req, res, next) {
    if (!req.user) {
      return res.status(401).json({
        error: 'Authentication required',
      });
    }

    if (req.user.role !== role) {
      return res.status(403).json({
        error: 'Forbidden: insufficient permissions',
      });
    }

    next();
  };
}

module.exports = { requireRole };
```

Notera att vi även här gör en kontroll mot req.user. Det är ett skyddsnät – om någon glömmer att lägga authenticate-middlewareen före authorize-middlewareen, ska vi fortfarande svara med 401 istället för att krascha. Defensiv programmering betalar sig.

Stöd för flera tillåtna roller

I praktiken vill vi ofta tillåta flera roller åt en route – exempelvis ska både admin och moderator kunna ta bort olämpliga inlägg. Vi gör factory-funktionen mer flexibel genom att låta den ta emot en lista:

```
function requireRole(...allowedRoles) {
  return function (req, res, next) {
    if (!req.user) {
      return res.status(401).json({
        error: 'Authentication required',
      });
    }

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
```

```

        error: 'Forbidden: insufficient permissions',
      });
    }

    next();
  };
}

```

Genom att använda rest-parameter (...allowedRoles) kan vi anropa den på två sätt: `requireRole('admin')` för en enskild roll eller `requireRole('admin', 'moderator')` för flera. Det är ett rent och flexibelt API.

Användning i routes

Det fina med Express middleware-kedjor är hur tydliga de blir när vi sätter ihop dem. Här är ett exempel där vi skyddar olika endpoints med olika krav:

```

// routes/products.js
const express = require('express');
const authenticate = require('../middleware/authenticate');
const { requireRole } = require('../middleware/authorize');
const productsController = require('../controllers/productsController');

const router = express.Router();

// Publikt - ingen autentisering
router.get('/', productsController.list);
router.get('/:id', productsController.getById);

// Inloggade användare - bara autentisering
router.post('/:id/reviews',
  authenticate,
  productsController.addReview
);

// Endast admin - autentisering + auktorisering
router.post('/',
  authenticate,
  requireRole('admin'),
  productsController.create
);

router.delete('/:id',
  authenticate,
  requireRole('admin'),
  productsController.remove
);

// Admin eller moderator
router.patch('/:id/feature',
  authenticate,
  requireRole('admin', 'moderator'),

```

```
    productsController.toggleFeature  
  );  
  
  module.exports = router;
```

När en request kommer in körs middleware i ordning. Om authenticate misslyckas kommer requireRole aldrig att köras, och om requireRole misslyckas kommer controller-funktionen aldrig att köras. Felhanteringen sker explicit och tydligt: 401 om identitet saknas, 403 om behörigheten inte räcker, och vi når aldrig affärslogiken om något gick fel tidigare i kedjan.

Resursägarskap - när rollen inte räcker

RBAC är kraftfullt men inte alltid tillräckligt. Tänk på en blogg där vanliga användare kan skapa egna inlägg. Logiken är inte "alla användare får uppdatera vilka inlägg som helst" – det är "alla användare får uppdatera sina egna inlägg, och admins får uppdatera alla inlägg".

Den här typen av kontroll – där behörigheten beror på relationen mellan användaren och resursen – kallas resursägarskap eller objektnivåauktorisering. Det är en av de vanligaste säkerhetsbristerna i webbapplikationer (OWASP klassar det som "Broken Object Level Authorization").

Den klassiska sårbarheten

Föreställ dig ett API där användarens egna ordrar finns på `/api/orders/:id`. Vår auth-middleware kontrollerar att användaren är inloggad. Vår controller hämtar orderet och returnerar det. Vad händer om en angripare gissar ett order-id som inte är deras eget?

```
// SÅRBAR KOD - hämtar utan ägarskapskontroll
router.get('/orders/:id',
  authenticate,
  async (req, res) => {
    const order = await Order.findById(req.params.id);
    if (!order) return res.status(404).json({ error: 'Not found' });
    res.json(order); // Returneras oavsett vem som äger den!
  }
);
```

En inloggad användare som skickar GET `/api/orders/abc123` där `abc123` tillhör någon annan får ut hela det orderet med alla detaljer. Det är en allvarlig säkerhetsbrist trots att vi har autentisering på plats. Roll-kontroll skulle inte ha hjälpt heller – båda användarna har rollen "user".

Lösningen: kontrollera ägarskap

Vi måste kontrollera att den inloggade användaren faktiskt äger resursen, eller att de har en roll som ger dem rätt att se andras resurser:

```
// SÄKER KOD - kontrollerar ägarskap
router.get('/orders/:id',
  authenticate,
  async (req, res, next) => {
    try {
      const order = await Order.findById(req.params.id);

      if (!order) {
        return res.status(404).json({ error: 'Not found' });
      }

      const isOwner = order.userId.toString() === req.user.sub;
      const isAdmin = req.user.role === 'admin';

      if (!isOwner && !isAdmin) {
        return res.status(403).json({ error: 'Forbidden' });
      }
    } catch (error) {
      next(error);
    }
  }
);
```

```
    }  
  
    res.json(order);  
  } catch (error) {  
    next(error);  
  }  
}  
};
```

Vi kombinerar nu två sorters auktorisering: rollkontroll (är användaren admin?) och ägarskapskontroll (äger användaren resursen?). Båda är nödvändiga för att täcka alla legitima fall.

404 vs 403 - en subtil säkerhetsfråga

En subtil men viktig fråga: ska vi returnera 404 (Not Found) eller 403 (Forbidden) när användaren försöker komma åt någon annans resurs?

Att returnera 403 är teknisk korrekt – resursen finns, men du saknar behörighet. Problemet är att svaret avslöjar att resursen existerar. För känsliga resurser (privata profiler, dolda dokument) kan det vara informationsläckage som hjälper en angripare att kartlägga systemet. För sådana fall kan 404 vara säkrare även om resursen i verkligheten finns – det är samma svar som om resursen inte fanns alls, så angriparen lär sig ingenting.

För publika resurser, exempelvis ett ordernummer som användaren själv kan se i sin orderhistorik, är 403 oftast bättre eftersom det ger användaren tydlig återkoppling. Som princip: om existensen av resursen är hemlig, returnera 404. Annars är 403 mer informativt.

Finkornig behörighetskontroll

För större system räcker det ofta inte med en handfull roller. Vi vill kunna säga "redaktören för Sport-sektionen får publicera sportinlägg, men inte nyhetsinlägg" – det är inte en ren rollfråga utan en kombination av roll och resurstyp. Vi går inte in på det i detalj i kursen, men det är värt att känna till de mer avancerade modellerna eftersom de förekommer i verkliga system.

Permission-baserad RBAC

En vanlig vidareutveckling är att introducera explicita behörigheter (permissions) som hänger på rollen. Istället för att kontrollera roller direkt i koden kontrollerar vi behörigheter:

```
const rolePermissions = {
  user: ['orders:read:own', 'reviews:create'],
  moderator: ['orders:read:own', 'reviews:create', 'reviews:delete'],
  admin: [
    'orders:read:any', 'orders:delete',
    'reviews:create', 'reviews:delete',
    'products:create', 'products:delete', 'users:delete',
  ],
};

function requirePermission(permission) {
  return function (req, res, next) {
    if (!req.user) {
      return res.status(401).json({ error: 'Auth required' });
    }

    const userPermissions = rolePermissions[req.user.role] || [];

    if (!userPermissions.includes(permission)) {
      return res.status(403).json({ error: 'Forbidden' });
    }

    next();
  };
}
```

Användning i routes blir då `requirePermission('products:create')` istället för `requireRole('admin')`. Fördelen är att routen blir oberoende av exakt vilken roll som har behörigheten – om vi senare flyttar produkthanteringen från admin till en ny roll "merchant" behöver vi inte ändra route-koden. Vi uppdaterar bara mappningen mellan roller och behörigheter.

ABAC och policy-baserad auktorisering

För riktigt komplexa system finns Attribute-Based Access Control (ABAC) där beslutet baseras på flera attribut: vem är användaren, vilken resurs är det, vad är förhållandet mellan dem, vilken tid på dygnet är det, från vilken IP-adress kommer requesten? Det förekommer i större organisationer, men är överkill för de allra flesta applikationer. För kursen och de boiler room-projekt vi bygger räcker RBAC mer än väl.

Vanliga fallgropar i auktorisering

Auktorisering är ett område där små misstag får stora konsekvenser. OWASP rankar Broken Access Control som det allvarligaste säkerhetshotet – inte för att det är svårt att förstå, utan för att det är så lätt att missa. Här är de fallgropar vi ska se upp för.

Glömd middleware

Den absolut vanligaste sårbarheten är att man helt enkelt glömmet att lägga till auth-middleware på en route. Det kan hända när man kopierar en befintlig route som ett mönster, modifierar den och råkar missa autentiseringsraden. Eller när nya features läggs till sent i en sprint och man inte hinner tänka igenom säkerheten.

Skyddet mot detta är medvetna mönster och rutiner: gå alltid igenom alla nya routes och fråga "vem ska få göra detta?" innan koden mergas. Centralisera så mycket som möjligt – kan en hel router monteras med authenticate som default? Skriv tester som specifikt verifierar att skyddade endpoints returnerar 401 utan token.

Att lita på klientsidans kontroller

En vanlig nybörjarmiss är att tänka "jag visar bara admin-knappar för admin-användare i React, så det är säkert". Det är inte säkert. En angripare bryr sig inte om hur frontenden ser ut – de skickar HTTP-anrop direkt med Postman eller curl. Frontenden kan dölja saker för bekvämlighetens skull, men aldrig för säkerheten.

Allt som ska skyddas måste skyddas på servern. Punkt slut. Det är samma princip som "don't trust the client" från vecka 6, applicerad på auktorisering.

Förlitar sig på obfuskering

"Vi använder UUID:er istället för löpande nummer, så det går inte att gissa ID:er." Det är inte säkerhet – det är security by obscurity. Visserligen är UUID:er omöjliga att gissa, men de läcker ut när användaren ser en, delar en länk eller den syns i en logg. När det händer finns ingen åtkomstkontroll alls.

UUID:er är bra för helt andra skäl (de är globalt unika och kan genereras utan koordinering), men de ersätter aldrig korrekt auktorisering.

Roll i request body

En klassisk privilegeskaleringsbugg uppstår när man tar emot rollen från klientens request body, exempelvis vid en uppdatering av användarens profil. Om vi gör `User.findByIdAndUpdate(id, req.body)` och låter den exakta request body-strukturen bestämma vilka fält som uppdateras, kan en angripare skicka `{ role: 'admin' }` och plötsligt vara admin.

Skyddet är att alltid välja explicit vilka fält som får uppdateras. Med Zod definierar vi ett uppdateringsschema som inte innehåller role-fältet. Med Mongoose kan vi använda projektion eller uttryckligen bygga upp uppdateringsobjektet i kod. Aldrig att blint kopiera in hela `req.body`.

```
// FARLIGT - massvis-tilldelning
await User.findByIdAndUpdate(id, req.body);

// SÄKERT - explicit kontroll över vad som uppdateras
const { name, email } = req.validatedBody; // Zod tog bort role
```

```
await User.findByIdAndUpdate(id, { name, email });
```

Inkonsekvens mellan endpoints

Ibland kan vi ha en korrekt skyddad GET-endpoint men en oskyddad PATCH-endpoint för samma resurs. En angripare som gör reconnaissance hittar snart den svagaste länken. När vi tänker på säkerhet ska vi alltid tänka på resurser, inte enskilda endpoints – vilka åtgärder är möjliga på den här resursen, och har varje åtgärd korrekt kontroll?

Miljövariabler och separerad konfiguration

Vi har redan använt miljövariabler tidigare i kursen – `process.env.MONGODB_URI` för databasanslutningen och `process.env.JWT_SECRET` för token-signering. Nu ska vi förstå varför det görs så och hur vi gör det rätt.

Konfiguration är all information vår applikation behöver för att köra men som inte är en del av själva koden: databasanslutningssträngar, API-nycklar för externa tjänster, hemliga signeringsnycklar, port-nummer, cookie-domäner, e-postservrar, och mer. Konfiguration delar två viktiga egenskaper: den varierar mellan miljöer (utveckling, test, produktion har olika databaser) och den innehåller ofta hemligheter (lösenord, nycklar, tokens).

The Twelve-Factor App

En av de mest inflytelserika texterna om hur moderna webbapplikationer bör byggas heter The Twelve-Factor App. Den slår fast en princip som blivit standard: konfiguration ska lagras i miljövariabler, inte i koden. Skälen är flera:

- Hemligheter ska aldrig hamna i versionshanteringen – om en API-nyckel checkas in i Git finns den kvar i historiken för alltid och kan läcka på otaliga sätt. Miljövariabler hålls utanför koden och Git.
- Samma kod ska kunna köras i olika miljöer – exakt samma kod körs på utvecklarens dator, på testservern och i produktion. Den enda skillnaden är miljövariablerna. Det förenklar både utveckling och driftsättning.
- Olika personer ska ha olika åtkomst – produktionsdatabasens lösenord behöver inte (och bör inte) finnas på alla utvecklarens datorer. Med miljövariabler kan varje utvecklare ha sin lokala konfiguration.
- Verktyg och plattformar förstår miljövariabler – Render, Railway, Vercel, Heroku, Docker och alla andra moderna driftsättningsplattformar har stöd för att sätta miljövariabler säkert utanför koden.

process.env i Node.js

Node.js exponerar alla miljövariabler via det globala `process.env`-objektet. Om vi har en miljövariabel `PORT=3000` i vår miljö, kan vi läsa den i koden:

```
const port = process.env.PORT;
console.log('Servern startar på port', port);
```

En sak som är viktig att komma ihåg: alla värden i `process.env` är strängar. Även om vi sätter en variabel som ser ut som ett tal eller en boolean kommer Node.js att leverera den som en sträng. Vi måste själva konvertera om vi vill ha andra typer:

```
const port = parseInt(process.env.PORT, 10);
const debugEnabled = process.env.DEBUG === 'true';
const maxRetries = Number(process.env.MAX_RETRIES);
```

Det är också viktigt att hantera fallet då en miljövariabel saknas. `process.env.SAKNAS` är `undefined`, vilket kan ge svårdiagnostiserade fel långt senare i applikationen om vi inte fångar det tidigt. Vi återkommer till det när vi pratar om validering.

dotenv - lokal hantering av miljövariabler

I produktion sätts miljövariabler av plattformen vi kör på (Render, Railway osv). Men under utveckling vill vi kunna sätta dem lokalt utan att behöva exportera dem i terminalen varje gång vi startar applikationen. Det löser dotenv – ett populärt npm-paket som läser en .env-fil och lägger värdena i process.env.

Installation och grundläggande användning

Vi installerar dotenv som en vanlig dependency:

```
npm install dotenv
```

Sedan skapar vi en fil som heter .env i projektets rotmapp. Filen har formatet KEY=value, en variabel per rad:

```
# .env
PORT=3000
MONGODB_URI=mongodb+srv://anvandare:losenord@cluster.mongodb.net/myapp
JWT_SECRET=det-har-ar-en-superlang-och-slumpmassig-strang
NODE_ENV=development
```

För att .env-filen ska läsas måste vi anropa dotenv tidigt i applikationens livscykel – allra helst som första rad i ingångsfilen, innan något annat importeras som kan behöva miljövariablerna:

```
// server.js eller index.js
require('dotenv').config(); // ABSOLUT FÖRSTA RADEN

const app = require('./app');
const port = process.env.PORT || 3000;

app.listen(port, () => {
  console.log('Servern lyssnar på port', port);
});
```

Anledningen till ordningen är att dotenv lägger till miljövariablerna i process.env. Om vi importerar app innan dotenv har körts, kommer kod inuti app som läser process.env att se odefinierade värden eller gamla värden från terminalen. Det är en av de vanligaste konfigurationsbuggarna och alltid surt att felsöka.

.env i .gitignore - ALLTID

Den absolut viktigaste regeln om .env-filer: de får aldrig checkas in i Git. Aldrig. .env-filen innehåller hemligheter – databasnycklar, JWT-secrets, externa API-nycklar – och det allra första vi gör efter att ha skapat .env är att lägga till den i .gitignore:

```
# .gitignore
node_modules/
.env
.env.local
.env.*.local
```

Om en .env-fil av misstag har checkats in räcker det inte att ta bort den i en ny commit – den finns kvar i Git-historiken och kan hittas av vem som helst som klonar repot. Hemligheterna måste betraktas som komprometterade och bytas ut omedelbart. Det är ett av de mest stressande misstagen man kan göra som utvecklare och det händer regelbundet.

.env.example som dokumentation

Eftersom .env inte finns i Git, hur ska andra utvecklare veta vilka variabler som behövs? Lösningen är en fil som heter .env.example som innehåller alla nödvändiga variabler men med dummy-värden eller tomma värden:

```
# .env.example - checkas IN i Git som dokumentation
PORT=3000
MONGODB_URI=mongodb+srv://anvandare:losenord@cluster.mongodb.net/myapp
JWT_SECRET=skapa-en-superlang-slumpmassig-strang-har
NODE_ENV=development
```

Den här filen checkas in. När en ny utvecklare klonar repot kopierar de .env.example till .env och fyller i de riktiga värdena. Det är ett enkelt och beprövat mönster.

Olika miljöer - utveckling, test och produktion

En modern backend körs typiskt i flera olika miljöer som tjänar olika syften:

- Utveckling (development) – på utvecklarnas datorer. Här kan vi köra med detaljerad loggning, koppla till en lokal databas och tillåta CORS från localhost. Det är okej att saker går sönder här – det är hela poängen.
- Test (test) – när automatiska tester körs, oftast i CI-systemet eller lokalt. Här använder vi en separat testdatabas (eller MongoDB Memory Server som vi såg i vecka 5) så att tester inte påverkar verklig data.
- Staging – en miljö som är så lik produktion som möjligt men där vi kan testa nya features innan de släpps till riktiga användare. Egen databas, egen domän, egna nycklar.
- Produktion (production) – där riktiga användare interagerar med systemet. Här är driftsäkerhet och prestanda viktigast, loggning ska vara minimal och säker, och alla hemligheter ska vara så skyddade som möjligt.

Skiljelinjen mellan miljöerna sätts av miljövariabeln `NODE_ENV`. Det är en konvention i Node.js-världen: `NODE_ENV=production` indikerar att vi kör i produktion, vilket även påverkar beteendet hos många npm-paket (Express slår exempelvis av detaljerade felmeddelanden i produktion).

Att anpassa beteende till miljö

Vi kan låta vår applikation bete sig olika beroende på miljö. Det är extremt användbart för loggning, felmeddelanden och CORS-konfiguration:

```
const isProduction = process.env.NODE_ENV === 'production';
const isDevelopment = process.env.NODE_ENV === 'development';

// Använd morgan för loggning, mer verbose i development
if (isDevelopment) {
  app.use(morgan('dev'));
} else {
  app.use(morgan('combined'));
}

// Felhantering avslöjar mer i development
app.use((err, req, res, next) => {
  res.status(err.statusCode || 500).json({
    error: err.message,
    stack: isDevelopment ? err.stack : undefined,
  });
});
```

Notera den centrala principen: i utveckling vill vi se så mycket information som möjligt för att felsöka snabbt. I produktion vill vi avslöja så lite som möjligt för att inte ge angripare information. Stack traces, fildatabaser, intern logik – inget av det ska komma ut till klienten i produktion.

Flera .env-filer för olika miljöer

Det är vanligt att ha olika .env-filer för olika miljöer på sin egen dator: .env.development, .env.test, kanske en .env.staging. dotenv har stöd för detta och vi kan välja vilken som ska laddas baserat på NODE_ENV. För kursens syften räcker det med en .env-fil för utveckling och att låta plattformen sätta värdena i produktion.

Hantering av hemligheter

Vissa miljövariabler är hemligheter – information som om den hamnar i fel händer kan kompromettera systemet eller läcka användardata. Det är värt att tänka extra noga på dessa. De vanligaste hemligheterna i en typisk backend är:

- `JWT_SECRET` – nyckeln vi signerar våra tokens med. Om någon får tag på den kan de skapa egna giltiga tokens och utge sig för att vara vilken användare som helst, inklusive admin.
- Databasanslutningssträngar – innehåller användarnamn och lösenord till databasen. Den som har dem kan läsa, ändra eller radera all data.
- API-nycklar för externa tjänster – Stripe för betalningar, SendGrid för e-post, OpenAI för AI-tjänster. De kostar oftast pengar att använda, så missbruk kan kosta riktigt mycket.
- OAuth client secrets – används vid inloggning med Google, GitHub och andra. Den hemligheten verifierar att vår applikation är vår applikation.

Att skapa en stark JWT-hemlighet

En vanlig misstag är att använda en kort, gissningsbar sträng som `JWT_SECRET`. "hemlig123" eller "my-secret-key" är inte secret – med tillgång till tokenet och en lista över vanliga lösenord kan en angripare brute-force fram nyckeln på minuter.

En bra JWT-hemlighet är minst 32 tecken lång och helt slumpmässig. Vi kan generera en med Node.js inbyggda crypto-modul:

```
// Kör i terminalen en gång för att generera
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"

// Resultat (kopiera till .env):
// 7c4a8d09ca3762af61e59520943dc26494f8941b1c5e7fe...etc
```

Den här strängen är 128 tecken lång hexadecimal, vilket motsvarar 512 bitar äkta slumpmässighet. Det är väsentligt mer än vad någon kommer brute-forca i den här världen. Olika applikationer ska alltid ha olika hemligheter, och de ska ändras om vi misstänker att de läckt.

Vad händer om en hemlighet läcker?

Det är inte frågan om utan när ett team kommer att stöta på problemet. Någon checkar in en .env-fil av misstag, en repo-server hackas, eller en bärbar dator stjäls. När det händer ska teamet veta vad som ska göras.

- Behandla hemligheten som komprometterad – anta att den finns hos en angripare. Det går oftast inte att bevisa motsatsen.
- Roter hemligheten omedelbart – generera en ny och uppdatera överallt där den används. För `JWT_SECRET` innebär det att alla aktiva tokens blir ogiltiga, vilket är en känd kostnad men det är nödvändigt.
- Granska loggar för misstänkt aktivitet – har det hänt något ovanligt sedan läckaget kan ha skett?
- Lär av det - hur kunde det hända, och vilken process eller verktyg kan hindra att det händer igen?

Verktyg som git-secrets, gitleaks och liknande kan installeras som pre-commit hooks och vägrar att låta utvecklare commita filer som ser ut som hemligheter (långa hex-strängar, AWS-nyckelmönster och så vidare). De är värda att installera för riktiga projekt.

Validera konfiguration vid uppstart

En av de mest frustrerande typerna av fel är när applikationen startar utan problem men kraschar tre dagar senare när någon försöker logga in, eftersom JWT_SECRET aldrig var satt. Felet uppstår precis i ögonblicket när ingen är på plats och loggen säger något kryptiskt om jwt.sign-funktionen.

Lösningen är att validera all konfiguration när applikationen startar. Om något saknas eller är fel formaterat ska applikationen vägra starta överhuvudtaget – det är mycket bättre att veta direkt än att upptäcka det när en användare hittar buggen.

Enkel validering

Den enklaste valideringen är en kontroll av att alla nödvändiga variabler finns:

```
// config.js
require('dotenv').config();

const requiredVars = [
  'PORT',
  'MONGODB_URI',
  'JWT_SECRET',
  'NODE_ENV',
];

for (const varName of requiredVars) {
  if (!process.env[varName]) {
    console.error('Saknad miljövariabel:', varName);
    process.exit(1); // Avsluta applikationen
  }
}

module.exports = {
  port: parseInt(process.env.PORT, 10),
  mongoUri: process.env.MONGODB_URI,
  jwtSecret: process.env.JWT_SECRET,
  nodeEnv: process.env.NODE_ENV,
};
```

Genom att exportera ett konfigurationsobjekt från en central config.js får vi flera fördelar: applikationen vägrar starta om något saknas, vi typkonverterar värdena en gång (alla portvärden är numbers från och med nu), och resten av koden importerar config istället för att läsa process.env överallt. Det är enklare att refaktorera och enklare att överblicka.

Validering med Zod

Eftersom vi redan använder Zod för request-validering kan vi använda det även för miljövariabler. Det ger oss starkare typkontroll och mer detaljerade felmeddelanden:

```
const { z } = require('zod');
require('dotenv').config();

const envSchema = z.object({
```

```
PORT: z.string().regex(/^\d+$/).transform(Number),
MONGODB_URI: z.string().url(),
JWT_SECRET: z.string().min(32,
  'JWT_SECRET måste vara minst 32 tecken'
),
NODE_ENV: z.enum(['development', 'test', 'production']),
});

const result = envSchema.safeParse(process.env);

if (!result.success) {
  console.error(
    'Felaktig miljökonfiguration:',
    result.error.format()
  );
  process.exit(1);
}

module.exports = result.data;
```

Här ser vi flera saker hända: PORT konverteras automatiskt från sträng till nummer, MONGODB_URI valideras som en giltig URL, JWT_SECRET kontrolleras för att vara tillräckligt långt, och NODE_ENV måste vara ett av de tillåtna värdena. Alla fel rapporteras tillsammans, och applikationen vägrar starta om något inte stämmer.

Hur frontend, backend och databas samverkar

Vi har nu byggt så mycket att det är värt att zooma ut och se helheten. När en användare interagerar med en fullstack-applikation – klickar på en knapp i React-appen för att ta bort en produkt – händer mycket mer än vad som syns. Att förstå hela kedjan av händelser är vad som skiljer en utvecklare som kan koda från en utvecklare som förstår systemet.

Det kompletta flödet

Låt oss spåra ett konkret scenario: en admin-användare loggar in och tar bort en produkt. Vi går igenom det steg för steg så att vi förstår vart varje del passar in.

1. Användaren öppnar React-appen i webbläsaren. Frontend hämtas som statiska filer (HTML, JavaScript, CSS) från en webbserver eller CDN. Inget anrop går till backenden ännu.
2. Användaren skriver in e-post och lösenord och klickar 'Logga in'. React-appen skickar POST /api/auth/login med uppgifterna i request body till vår Express-server.
3. Express tar emot anropet. Det går genom CORS-middleware, JSON-parsing-middleware, eventuell loggning och slutligen till routen för login.
4. Login-controllern validerar request body med Zod. Om något saknas eller är fel formaterat skickas 400 tillbaka direkt.
5. Vi söker efter användaren i MongoDB med User.findOne({ email }). MongoDB returnerar dokumentet om det finns. Vi jämför det inskickade lösenordet med passwordHash via bcrypt.compare.
6. Om allt stämmer skapar vi en JWT som innehåller user-id, e-post och rollen ('admin'). Tokenet signeras med JWT_SECRET från miljövariablerna och returneras till React-appen i responsen.
7. React-appen tar emot tokenet och sparar det (i memory eller localStorage beroende på säkerhetsmodell). UI:t uppdateras till inloggat läge.
8. Användaren navigerar till en produkt och klickar 'Ta bort'. React skickar DELETE /api/products/abc123 med headern Authorization: Bearer <token>.
9. I Express körs först authenticate-middleware. Den verifierar tokenet med jwt.verify och JWT_SECRET. Om allt är okej läggs payload (sub, email, role) på req.user.
10. Sedan körs requireRole('admin'). Eftersom req.user.role === 'admin' fortsätter requesten vidare. (Hade en vanlig user försökt hade det stoppats med 403.)
11. Controllern kör Product.findByIdAndDelete(id). Mongoose översätter det till en MongoDB-query och tar bort dokumentet i databasen.
12. Servern svarar med 204 No Content. React tar emot svaret och uppdaterar sin lokala state, vilket gör att produkten försvinner från listan i UI:t.

Det är tolv steg, och varje steg involverar lager från det vi byggt under hela kursen: routing och middleware (vecka 1-2), TypeScript och Mongoose (vecka 3-5), validering och felhantering (vecka 6), autentisering (vecka 7) och nu auktorisering (vecka 8). Det är så ett verkligt fullstack-system fungerar.

Frontend och backend - var sköts vad?

En vanlig fråga är var olika ansvar ska ligga. Det finns inga absoluta regler, men en bra mental modell är denna: backend är auktoriteten och frontend är upplevelsen.

Allt som handlar om sanning, säkerhet och bestående tillstånd hör hemma på backend. Vem har behörighet? Vad är det riktiga lagersaldot? Är e-postadressen redan registrerad? Backend har det slutgiltiga ordet i alla sådana frågor och bestämmer vad som faktiskt sker.

Frontend ansvarar för hur saker presenteras och vilka åtgärder användaren kan välja mellan. Frontend kan dölja admin-knappen för en vanlig user (för att UX:n ska vara ren), men det är inte säkerhet – det är bara bekvämlighet. Backend måste fortfarande blockera försöket om någon ändå försöker.

Validering är ett bra exempel på dubbelarbete med olika syften: vi validerar i frontend för snabb feedback ("e-post är inte giltig" syns innan användaren skickar formuläret), och vi validerar i backend för säkerhet (klienten kan vara komprometterad eller manipulerad).

Var ligger vad i en typisk driftsättning?

När vi senare driftsätter (vecka 10) kommer komponenterna att ligga på olika ställen i världen:

- Frontend (React-appen) – byggd som statiska filer och lagd på en CDN eller en webbserver som Vercel eller Netlify. Hämtas av webbläsaren och körs där.
- Backend (Express-API:et) – körs på en Node.js-server hos en plattform som Render, Railway eller Heroku. Tar emot HTTP-anrop från frontend.
- Databas (MongoDB) – körs hos MongoDB Atlas i molnet, oftast i samma region som backenden för låg latency.
- Hemligheter - lagras i miljövariabler hos den plattform där backenden körs. Aldrig i koden, aldrig i Git, aldrig hos frontenden.

Den fysiska separationen mellan komponenterna är en av styrkorna i den moderna webb-arkitekturen. Varje komponent kan skalas oberoende, uppdateras oberoende och ersättas med en alternativ implementation utan att de andra påverkas. Vi kan exempelvis byta ut React mot en mobilapp utan att ändra backend, eller byta ut MongoDB mot PostgreSQL utan att frontenden märker det.

Checkpoint: Förberedelse inför kursvecka 8

Inför checkpointen tisdag ska vi ha läst igenom detta studiematerial och ha en grundläggande förståelse för auktorisering, RBAC och miljövariabler. Vi ska också ha boiler room-projektet från vecka 7 med oss som utgångspunkt - det är där vi byggde autentisering, och nu ska vi lägga till auktorisering ovanpå.

Kontrollera att ni har följande klart inför boiler room måndag:

13. Autentisering med JWT från vecka 7 fungerar i ert projekt - registrering, inloggning och en authenticate-middleware som lägger användaren på req.user.
14. Ni har dotenv installerat och en .env-fil som hanterar MONGODB_URI och JWT_SECRET. Kontrollera att .env finns i .gitignore.
15. Ni har skissat på vilka roller ert projekt behöver - troligen 'user' och 'admin', men beroende på domän kanske ni har fler.
16. Ni har identifierat vilka endpoints i projektet som ska kräva admin-roll och vilka som ska vara öppna för alla inloggade användare.

Ta med er följande tankar och frågor till checkpointen: Var i ert boiler room-projekt finns endpoints som idag är öppna för alla inloggade men som egentligen borde vara begränsade? Vilka resurser i ert projekt har ägarskapskontroll - där en användare bara ska se sina egna saker? Hur planerar ni att den första admin-användaren skapas?

En vanlig fråga vi kan diskutera tillsammans är hur ni vill hantera scenariot när en admin loggar in i frontenden - ska ni visa olika menyer för olika roller, och hur skickar React-appen vidare information om rollen utan att göra ett extra anrop? Det är en designfråga med flera giltiga svar och vi tittar gärna på era tankar.

Sammanfattning

Under kursvecka 8 har vi tagit ett stort steg framåt i hur vi tänker på säkerhet i våra API:er. Där vecka 7 handlade om att etablera identitet, handlar vecka 8 om att kontrollera vad olika identiteter får göra.

Vi har klargjort skillnaden mellan autentisering (vem är du?) och auktorisering (vad får du göra?). De är två distinkta steg som tjänar olika syften och som speglas i olika HTTP-statuskoder: 401 när autentiseringen misslyckas, 403 när auktoriseringen misslyckas. I vår middleware-kedja körs de alltid i ordningen authenticate först, authorize sedan, eftersom auktorisering kräver en känd identitet.

Rollbaserad åtkomstkontroll (RBAC) är den vanligaste auktoriseringsmodellen och löser ett verkligt problem: att tilldela rättigheter till hundratals individer är ohanterligt, men att tilldela rättigheter till några få roller och sedan ge användarna roller är skalbart. Vi har sett hur vi designar roller, lagrar dem i Mongoose User-modellen, lägger in dem i JWT-payloaden och kontrollerar dem med en factory-funktion-baserad RBAC-middleware.

Vi har också gått djupare in på resursägarskap - att RBAC i sig ofta inte räcker. När varje användare har sina egna resurser måste vi kontrollera ägarskapet, inte bara rollen. Det är en av de vanligaste säkerhetsbristerna i webbapplikationer och listas högst upp i OWASP Top 10.

När det gäller miljövariabler har vi gått från att bara använda dem till att förstå varför vi använder dem. Twelve-Factor App-principen om att hålla konfiguration utanför koden ger oss flexibilitet mellan miljöer, säkrar hemligheter från versionshantering och möjliggör de moderna driftsättningsmodeller vi senare ska använda. Vi har sett hur dotenv hjälper oss lokalt, hur .env aldrig får checkas in, och hur .env.example dokumenterar vilka variabler som behövs.

Hantering av hemligheter förtjänar särskild uppmärksamhet. JWT_SECRET, databasnycklar och API-nycklar är inga vanliga konfigurationsvärden - de är makten i systemet. Vi ska generera dem starka, lagra dem säkert, och veta exakt vad vi gör om en sådan hemlighet läcker.

Slutligen har vi kopplat ihop hela kursen genom att se på det fullständiga dataflödet i ett fullstack-system - från användarens klick i React, genom autentisering och auktorisering i Express, till databasoperationen i MongoDB och hela vägen tillbaka. Det är ett sammanhang som binder ihop alla våra åtta veckor och visar varför varje del finns där den finns.

Nästa vecka går vi vidare till GDPR, säker loggning och dataflöden. Vi tittar på hur lagar och regler påverkar vår API-design, hur vi loggar utan att läcka känslig information, och hur vi dokumenterar systemflöden så att andra kan förstå dem. Det är den dokumentations- och regelefterlevnadsbild som kompletterar säkerhetspusslet vi byggt under de senaste veckorna.

Resurser för fördjupning

Nedan följer resurser för de som vill fördjupa sina kunskaper om auktorisering, RBAC och hantering av miljövariabler.

Officiell dokumentation

dotenv på GitHub - dokumentation och exempel för hur man arbetar med .env-filer i Node.js-projekt:

<https://github.com/motdotla/dotenv>

OWASP - Broken Access Control - översikt över vanliga sårbarheter inom auktorisering och vad man ska tänka på:

https://owasp.org/Top10/A01_2021-Broken_Access_Control/

OWASP Authorization Cheat Sheet - en mer praktisk guide om hur man implementerar auktorisering korrekt i webbapplikationer:

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

The Twelve-Factor App - kapitlet om Config (faktor III) sammanfattar varför konfiguration ska ligga i miljön och inte i koden:

<https://12factor.net/config>

Säkerhetsresurser

NIST RBAC Reference - NIST (National Institute of Standards and Technology) har en referensmodell för RBAC som har påverkat hur de flesta moderna implementationer ser ut. Det är en akademisk text men ger djupare förståelse för varför RBAC är designat som det är.

OWASP API Security Top 10 - för den som vill förstå mer specifika risker som rör just API:er, vilket är vad vi bygger:

<https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

Videomaterial och artiklar

"Authentication vs Authorization" - det finns många bra introduktionsvideor på YouTube om skillnaden mellan dessa. Sök på termen och välj en kortare video (under 10 minuter) för en effektiv genomgång av distinktionen.

"Node.js Best Practices" av Yoni Goldberg - GitHub-repository med rekommenderade mönster för Node.js. Avsnitten om säkerhet och konfiguration är direkt relevanta för veckans tema:

<https://github.com/goldbergystony/nodebestpractices>

"How to Store JWT Tokens Securely" - en återkommande diskussion i community är var JWT:er ska lagras i frontend (localStorage, sessionStorage, cookies). Det är ett ämne värt att läsa på för den som vill förstå avvägningarna mellan XSS-risker och CSRF-risker.

"The Secrets Game" - flera artiklar och videor om hur stora företag hanterar hemligheter i produktion. Sök på termer som "secrets management", "HashiCorp Vault" eller "AWS Secrets Manager" för att se hur det skalas i större organisationer.

Studiematerial

GDPR, säker loggning och dataflöden

Kursvecka 9

Introduktion

Vi har nu åtta veckor bakom oss och ett API som börjar bli imponerande. Vi har Node.js och Express som plattform, MongoDB som datalager, TypeScript för typning, Zod för inputvalidering, JWT för autentisering och RBAC för auktorisering. Vi har täppt till de mest uppenbara säkerhetshålen och vårt API kan skydda sina endpoints från obehöriga användare. Det är en stark teknisk grund.

Men det finns en aspekt av backendutveckling som vi ännu inte har berört på allvar – och det är ansvaret som följer med att hantera människors data. Varje gång vi sparar en e-postadress, ett lösenord eller en personlig anteckning i vår databas tar vi på oss ett ansvar. Användaren litar på att vi inte missbrukar datan, att vi skyddar den från obehöriga och att vi raderar den när den inte längre behövs. Det är inte bara god sed – det är lag.

Kursvecka 9 handlar om det ansvaret. Vi går igenom GDPR och vad det innebär konkret för hur vi designar våra API:er och våra databaser. Vi lär oss att logga händelser i systemet på ett strukturerat sätt utan att samtidigt skapa en oavsiktlig databas över våra användares personuppgifter i loggfilerna. Vi tittar på mjuk radering och hur vi hanterar livscykeln för data, från det ögonblick den skapas till det ögonblick den ska tas bort eller anonymiseras.

Vi tar också ett kort steg åt sidan från MongoDB för att titta på relationsdatabaser och SQL. Det är inte för att vi ska byta plattform – kursen kommer fortsatt att basera sig på MongoDB – utan för att förstå skillnaderna mellan de två modellerna och kunna fatta ett medvetet beslut nästa gång vi börjar ett nytt projekt. Många riktiga system använder dessutom båda parallellt, och som backendutvecklare ska vi förstå när det ena passar bättre än det andra.

Veckan avslutas med dokumentation. Vi visualiserar dataflöden mellan klient, server och databas med sekvensdiagram, vi använder Mermaid och draw.io för att rita dem, och vi introducerar Swagger/OpenAPI för att dokumentera vårt API på ett standardiserat och maskinläsbart sätt. Bra dokumentation är inte ett tillval – det är en grundförutsättning för att andra utvecklare (och vi själva, om sex månader) ska kunna förstå och underhålla systemet.

Det här är en vecka som väver ihop juridik, etik, dokumentation och teknik. För många utvecklare är det de här ämnena som är skillnaden mellan att skriva kod som fungerar i en demo och att bygga system som faktiskt kan användas i produktion av riktiga företag och organisationer.

GDPR och dataskydd i praktiken

GDPR (General Data Protection Regulation) är EU:s allmänna dataskyddsförordning som trädde i kraft den 25 maj 2018. Den ersatte tidigare nationella lagar (i Sverige personuppgiftslagen, PUL) och skapade ett gemensamt regelverk i hela EU för hur personuppgifter får samlas in, lagras och behandlas. Som backendutvecklare måste vi känna till GDPR i tillräcklig utsträckning för att kunna designa system som följer regelverket – det är inte bara juristernas ansvar.

GDPR berör i princip varje webbapplikation som hanterar information om identifierbara personer. Det räcker inte med att vi har en sida med kakor och en länk till en integritetspolicy. Förordningen ställer krav på vad vi får samla in, hur vi får använda det, hur länge vi får behålla det och vilka rättigheter användaren har över sin egen data. Brott mot GDPR kan leda till böter på upp till 4 % av den globala omsättningen eller 20 miljoner euro – vad som är högst – för det företag vi bygger system för.

Vad räknas som personuppgifter?

Personuppgifter är all information som direkt eller indirekt kan kopplas till en levande, identifierbar fysisk person. Det är en bredare definition än många tror och det är här de flesta utvecklare gör sina första misstag.

Direkta personuppgifter är de som omedelbart identifierar en person:

- Namn (för- och efternamn)
- Personnummer eller samordningsnummer
- E-postadress kopplad till en specifik person
- Telefonnummer
- Postadress
- Foton, ljudinspelningar och videor där personen kan kännas igen

Indirekta personuppgifter är information som i kombination med annan data kan identifiera en person, även om den enskilda uppgiften i sig inte räcker:

- IP-adresser - kan kopplas till en internetabonnemang och därigenom till en hushåll eller person
- Användar-ID:n och session-ID:n i ett system
- Cookie-ID:n och device fingerprints
- Geografiska positioner (även approximativa, t.ex. via IP)
- Användarnamn och pseudonymer som kan kopplas till en person via andra system

Det är värt att stanna upp vid IP-adresser. Många utvecklare loggar IP-adressen tillsammans med varje API-anrop "för debugging". GDPR betraktar IP-adressen som en personuppgift, vilket betyder att vi måste ha en laglig grund för att lagra den och att vi måste kunna motivera varför vi behåller den. Det är inte förbjudet att logga IP-adresser, men vi måste tänka igenom varför vi gör det och hur länge vi sparar dem.

En särskild kategori är så kallade känsliga personuppgifter. Dessa har starkare skydd och får i princip inte behandlas alls utan särskilt samtycke eller annan stark laglig grund. Hit räknas information om hälsa, etniskt ursprung, politiska åsikter, religiös övertygelse, fackligt medlemskap, sexuell läggning och biometriska uppgifter. Om vårt system hanterar något av detta måste vi vara extra försiktiga och i de flesta fall konsultera jurist.

De grundläggande principerna

GDPR bygger på sex grundläggande principer. Det är värt att ha med dem som mental checklista varje gång vi designar ett nytt API eller en ny databastabell. Som backendutvecklare påverkar vi hur dessa principer realiseras i praktiken.

- Laglighet, korrekthet och öppenhet - vi måste ha en laglig grund för att behandla personuppgifter (t.ex. samtycke, avtal, laglig förpliktelse), och användaren ska få veta hur datan används.
- Ändamålsbegränsning - vi får bara använda datan för det specifika syfte vi samlade in den för. Att samla in en e-post för att skicka kvitto och sedan sälja den till annonsörer är ett tydligt brott mot principen.
- Uppgiftsminimering - vi får bara samla in den data som faktiskt behövs för det aktuella syftet. Mer om det i nästa avsnitt.
- Korrekthet - datan ska vara riktig och uppdaterad. Användaren har rätt att få felaktiga uppgifter rättade.
- Lagringsminimering - personuppgifter får inte sparas längre än vad som är nödvändigt för syftet. När syftet är uppfyllt ska datan raderas eller anonymiseras.
- Integritet och konfidentialitet - datan måste skyddas mot obehörig åtkomst, läckage och förlust genom lämpliga tekniska och organisatoriska åtgärder. Det är denna princip som ligger bakom mycket av det vi gjort i tidigare veckor: hashade lösenord, autentisering, auktorisering och säker konfiguration.

Användarens rättigheter

GDPR ger användaren flera konkreta rättigheter över sin egen data. Som backendutvecklare måste vårt system kunna stödja dessa rättigheter, vilket påverkar hur vi designar databasen och vilka API-endpoints vi behöver bygga.

- Rätt till information - vi ska tydligt informera om vilken data vi samlar in och varför.
- Rätt till tillgång - användaren har rätt att få ut all data vi har om dem, ofta i ett maskinläsbart format.
- Rätt till rättelse - användaren ska kunna korrigera felaktig information.
- Rätt till radering ("rätten att bli glömd") - användaren har under vissa förutsättningar rätt att få sin data raderad. Vi ska kunna utföra detta tekniskt.
- Rätt till begränsning av behandling - användaren kan be oss att frysa behandlingen av deras data.
- Rätt till dataportabilitet - användaren ska kunna ta med sig sin data till en annan tjänst, ofta som JSON eller CSV.
- Rätt att invända - användaren kan invända mot behandlingar som baseras på berättigat intresse.

För oss innebär det här att vi i praktiken ofta behöver bygga API-endpoints som GET /api/users/me/data (för rätten till tillgång och dataportabilitet) och DELETE /api/users/me (för rätten att bli glömd). Det är funktionalitet som ofta missas i den första versionen av ett system och som sedan måste byggas i efterhand under stress – det är mycket bättre att tänka på det från början.

Dataminimering i praktiken

Dataminimering är en av de viktigaste principerna i GDPR och samtidigt den som är lättast att tillämpa felaktigt. Den säger i grunden: samla bara in den data du faktiskt behöver, för det specifika syfte du samlar in den för. Det låter självklart, men i praktiken är det förvånansvärt vanligt att utvecklare lägger till fält i en datamodell "för säkerhets skull" eller "för att vi kanske vill använda det senare". Det är ett tankesätt som GDPR förbjuder.

En bra mental övning är att för varje fält i en datamodell fråga sig: vilken funktion i applikationen kräver att detta fält finns? Om svaret är otydligt, eller om svaret är "vi kanske använder det till något i framtiden", är det en stark signal om att fältet inte borde finnas.

Exempel: minimerad användarmodell

Tänk dig att vi bygger en enkel anteckningsapp. En vanlig första instinkt är att skapa en användarmodell med så mycket information som möjligt:

Ej minimerad modell:

```
const userSchema = new Schema({
  email: { type: String, required: true },
  password: { type: String, required: true },
  firstName: { type: String, required: true },
  lastName: { type: String, required: true },
  birthDate: { type: Date, required: true },
  phoneNumber: String,
  address: {
    street: String,
    city: String,
    postalCode: String,
    country: String
  },
  gender: String,
  occupation: String,
  ipAddress: String,
  lastLoginIp: String
});
```

Frågan vi måste ställa: vilka av dessa fält behöver appen för att fungera? För en anteckningsapp som bara låter användaren spara och läsa egna anteckningar räcker det med ett identifikationsfält och något att autentisera mot. Allt annat är överskott.

Minimerad modell:

```
const userSchema = new Schema({
  email: { type: String, required: true, unique: true },
  passwordHash: { type: String, required: true },
  displayName: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});
```

Notera att vi tagit bort födelsedatum, adress, telefonnummer, kön och yrke. Inget av detta är nödvändigt för att appen ska fungera. Vi har också ersatt password med passwordHash,

vilket vi redan vet från vecka 7 är ett krav. Och displayName istället för för- och efternamn räcker för att hälsa användaren välkommen i gränssnittet.

Detta innebär inte att vi aldrig får samla in mer data – men varje extra fält måste ha ett specifikt, dokumenterat syfte. Om appen senare ska kunna skicka påminnelser via SMS, då har vi en laglig grund för att lägga till telefonnummer. Om vi behöver verifiera att användaren är myndig för en viss funktion, då kan vi lägga till födelsedatum. Men först då.

Minimering på API-nivå

Dataminimering gäller inte bara vad vi sparar – det gäller också vad vi returnerar i API-svar. När vi returnerar användarinformation från en endpoint ska vi bara inkludera den data som klienten faktiskt behöver. En klient som visar en lista över andra användare i en kommentarstråd behöver kanske bara namn och avatar, inte e-postadress och kontoinställningar.

En vanlig miss är att returnera hela mongoose-dokumentet rakt av. Det leder ofta till att känslig information som passwordHash, interna flagger och fält avsedda för administrativt bruk läcker ut till klienten. En ren lösning är att explicit välja vilka fält vi vill returnera, antingen i Mongoose-frågan eller via en transformations-funktion:

```
// Med Mongoose-projection (säkrare default)
const user = await User.findById(id)
  .select('email displayName createdAt');

// Eller via en mappningsfunktion
function toPublicUser(user) {
  return {
    id: user._id,
    displayName: user.displayName,
    createdAt: user.createdAt
  };
}
```

Det första alternativet med .select() begränsar redan på databasnivå vilka fält som hämtas, vilket är effektivt. Det andra alternativet ger oss mer kontroll och är användbart när olika endpoints ska returnera olika nivåer av detalj (en användare som ser sin egen profil får mer information än en annan användare som ser samma profil utifrån).

Det är också en god idé att markera känsliga fält som select: false i schemat. Det innebär att fältet inte hämtas som standard – vi måste explicit be om det. Det gör att vi inte oavsiktligt råkar returnera passwordHash i ett API-svar:

```
const userSchema = new Schema({
  email: { type: String, required: true },
  passwordHash: { type: String, required: true, select: false },
  displayName: String
});

// Hashen hämtas inte:
const user = await User.findById(id);

// Hashen hämtas explicit (vid inloggning):
const user = await User.findOne({ email })
```

```
.select('+passwordHash');
```

Det här är ett enkelt skydd mot ett mycket vanligt fel: att lösenordshashar oavsiktligt skickas tillbaka till klienten i API-svar. Vi har inte sett detta mönster tidigare i kursen och det är värt att lägga till i våra projekt.

Säker loggning

Loggning är en ovärderlig del av all backendutveckling. När något går fel i produktion är loggarna ofta det enda vi har att gå på för att förstå vad som hände. När vi vill veta hur ofta en endpoint anropas, när en bug först uppstod, eller vilka användare som påverkats av en incident, är loggarna vår källa till sanning. Utan bra loggning bygger vi system i blindo.

Men loggning har en mörk baksida: det är extremt lätt att av misstag bygga upp en sekundär databas full av personuppgifter i loggfilerna. En logg som registrerar varje inkommande request med fullständig request body, headers och URL kommer att samla in lösenord, e-postadresser, IP-adresser, autentiseringstokens och allt annat användaren skickar. Loggfilen blir en GDPR-katastrof i väntan på att hända.

Den här veckan handlar lika mycket om vad vi inte ska logga som vad vi ska logga. Vi behöver disciplin och struktur, och vi behöver verktyg som hjälper oss att skriva loggar som är användbara utan att vara farliga.

Varför vi loggar

Innan vi pratar om hur vi loggar är det viktigt att vara tydlig med varför. Det finns flera goda skäl att logga, och varje typ av logg har sitt eget syfte:

- Felsökning och diagnostik - när buggar uppstår vill vi veta vilken kodväg som exekverades, vilka värden variabler hade och vilka externa anrop som gjordes.
- Driftövervakning - vi vill veta om servern är hälsosam, hur svarstiderna ser ut, vilken belastning vi har och om något börjar gå fel innan det blir kritiskt.
- Säkerhet och revision - vi vill ha spår av kritiska händelser: inloggningar, behörighetsändringar, dataraderingar, misslyckade autentiseringsförsök. Det här kallas ibland audit logs.
- Affärsanalys - vi vill ibland veta hur tjänsten används: vilka funktioner som används mest, hur många nya användare som registrerar sig per dag. Notera att den här kategorin är den mest känsliga ur GDPR-perspektiv.

Varje typ av logg har olika krav. En audit log behöver vara svår att manipulera och måste behållas länge för att uppfylla revisionskrav. En diagnoslogg är ofta värdefull bara i några dagar. En affärsanalyslogg ska helst vara aggregerad och anonymiserad. Att ha tydligt klart för sig vilken sorts logg vi skriver gör det lättare att fatta rätt beslut om innehåll och retention.

Vad vi inte ska logga

Det är lättast att börja med en lista över vad vi aldrig ska logga, oavsett hur frestande det än kan kännas i en stunds frustration över en svår bugg:

- Lösenord i klartext - aldrig, oavsett om det är vid inloggning, registrering eller misslyckade försök.
- Lösenordshashar - även om de är hashade ger de en angripare något att försöka knäcka offline om loggen läcker.
- Autentiseringstokens (JWT, refresh tokens, API-nycklar) - dessa är effektivt nycklar till kontot och måste behandlas som lösenord.
- Sessionscookies
- Personnummer och bankkontonummer
- Kreditkortsnummer (PCI-DSS förbjuder det dessutom)
- Hela request bodies för endpoints som hanterar känslig data

- Fullständiga URL:er om de innehåller känsliga query-parametrar (t.ex. /reset-password?token=xxx)

Det vi däremot bör logga är information som hjälper oss förstå vad som hände utan att exponera personlig information. Istället för att logga "User john.doe@example.com logged in" loggar vi "User <userId> logged in". Användar-ID:t är en intern identifierare som bara går att koppla till en person via vår egen databas, och den identifieringen kan vi göra när vi behöver.

Loggnivåer

Loggar bör kategoriseras efter allvarlighetsgrad. Det är en gemensam konvention som de flesta loggningsbibliotek implementerar och som gör det möjligt att filtrera loggar baserat på hur intressanta de är. De vanligaste nivåerna, från minst till mest allvarlig, är:

- trace - extremt detaljerad information för djup felsökning. Används sällan i produktion.
- debug - information användbar under utveckling och felsökning. Avslås oftast i produktion.
- info - normala händelser som är värda att notera (server startad, användare registrerad, batch-jobb klart).
- warn - något oväntat händelse men systemet kunde fortsätta. En signal att något bör undersökas.
- error - ett fel inträffade som påverkade en specifik operation eller request.
- fatal - ett fel som gör att systemet inte längre kan fungera korrekt. Kräver omedelbar uppmärksamhet.

En bra tumregel är att i utvecklingsmiljö logga från debug och uppåt, medan vi i produktion typiskt loggar från info och uppåt. Tracenivå sparas oftast bara när vi specifikt felsöker en svår bugg och produktionsloggning på den nivån snabbt blir ohanterligt stora och kostsamma att lagra.

Strukturerad loggning

När vi använder `console.log()` i utveckling skriver vi loggar som rader av text. Det fungerar bra när vi tittar på dem manuellt, men blir ohanterligt så fort applikationen växer. Hur söker vi efter alla loggrader som rör en specifik användare? Hur räknar vi hur många 500-fel vi haft det senaste dygnet? Hur korrelerar vi en loggrad i webbservern med en loggrad i databasen?

Strukturerad loggning är svaret. Istället för att skriva loggar som fri text skriver vi dem som JSON-objekt med tydligt namngivna fält. Varje loggrad blir då maskinläsbar, sökbar och filtrerbar. När vi senare skickar loggarna till ett centralt system som ELK Stack, Datadog eller Grafana Loki, kan systemet automatiskt indexera fälten och vi kan söka efter "alla error-loggar för `userId=abc123` inom senaste timmen" på ett ögonblick.

Skillnaden är dramatisk. Jämför följande två varianter:

Ostrukturerad loggning:

```
[INFO] 2026-04-28 10:32:15 - User abc123 logged in from 192.168.1.5
[ERROR] 2026-04-28 10:32:18 - Failed to fetch user profile: Database timeout
```

Strukturerad loggning (JSON):

```
{"level":"info","time":"2026-04-28T10:32:15Z",
  "event":"user.login","userId":"abc123",
  "requestId":"req-9f8e..."}

{"level":"error","time":"2026-04-28T10:32:18Z",
  "event":"user.profile.fetch_failed",
  "userId":"abc123","error":"DatabaseTimeout",
  "durationMs":5000,"requestId":"req-9f8e..."}

```

Lägg märke till att vi inte loggar IP-adressen som en del av användardata och att vi inte loggar e-postadressen alls. `requestId` är ett ID som genereras för varje inkommande request och som följer med i alla loggrader som hör till den requesten – det gör det lätt att rekonstruera vad som hände vid en specifik händelse. Det är en teknik som kallas correlation ID och är central i moderna logg- och övervakningssystem.

Pino - snabb och välintegrerad

Pino är ett av de mest populära loggningsbiblioteken för Node.js. Det är extremt snabbt (en av Pinos uttalade designprinciper är låg overhead), producerar JSON-loggar by default och har god integration med Express. Vi installerar det med:

```
npm install pino pino-http
npm install -D pino-pretty # läsbart format under utveckling
```

En grundläggande konfiguration ser ut så här:

```
// logger.js
const pino = require('pino');

const logger = pino({
```

```

    level: process.env.LOG_LEVEL || 'info',
    // I utveckling vill vi ha läsbart format
    transport: process.env.NODE_ENV !== 'production'
      ? { target: 'pino-pretty' }
      : undefined,
    // Vissa fält bör aldrig hamna i loggar
    redact: {
      paths: [
        'req.headers.authorization',
        'req.headers.cookie',
        'req.body.password',
        'req.body.token',
        '*.password',
        '*.passwordHash'
      ],
      censor: '[REDACTED]'
    }
  });

module.exports = logger;

```

Det fina med redact-konfigurationen är att även om vi av misstag skulle försöka logga ett objekt som innehåller authorization-headern eller ett password-fält, ersätter Pino värdet med [REDACTED] innan loggen skrivs ut. Det är ett skyddsnät som fångar misstag innan de blir GDPR-incidenter.

Vi använder loggern på samma sätt som vi använt console.log, men nu med strukturerade data:

```

const logger = require('./logger');

// Information om en händelse
logger.info({ userId, event: 'user.login' }, 'User logged in');

// Varning
logger.warn(
  { userId, attempts: 3 },
  'Multiple failed login attempts'
);

// Fel - skicka error-objektet som har en speciell hantering
logger.error(
  { err, userId, operation: 'fetchProfile' },
  'Failed to fetch user profile'
);

```

Pino känner igen err-fältet specifikt och formaterar Error-objektet med stack trace, namn och meddelande på ett strukturerat sätt. Det är en konvention som flera andra bibliotek också följer.

HTTP-loggning som middleware

För att logga inkommande HTTP-requests och deras svar använder vi pino-http, som integrerar Pino som en Express-middleware:

```
const express = require('express');
const pinoHttp = require('pino-http');
const logger = require('./logger');

const app = express();

app.use(pinoHttp({
  logger,
  // Anpassad logg-meddelande baserat på resultatet
  customSuccessMessage: function (req, res) {
    return `${req.method} ${req.url} ${res.statusCode}`;
  },
  customErrorMessage: function (req, res, err) {
    return `${req.method} ${req.url} failed: ${err.message}`;
  },
  // Lägg till requestId så vi kan korrelera
  genReqId: (req) => req.headers['x-request-id'] || crypto.randomUUID()
}));

// Nu kan vi använda req.log inuti våra handlers
app.get('/api/users/:id', async (req, res, next) => {
  req.log.info({ userId: req.params.id }, 'Fetching user');
  // ...
});
```

Den största fördelen med req.log är att Pino automatiskt inkluderar requestId i alla loggrader som skrivs via det objektet. Vi får alltså korrelerings-ID:n gratis. Om en request orsakar fem loggrader på olika ställen i koden, kan vi söka efter den specifika requestId:n och få ut alla fem som en sammanhängande historia.

Winston - flexibelt alternativ

Winston är ett annat populärt alternativ. Det är något äldre och något långsammare än Pino, men har en mer flexibel transport-modell som gör det enkelt att skicka loggar till flera destinationer samtidigt (filer, konsolen, externa tjänster). För många projekt är båda biblioteken bra val – det viktiga är att välja ett av dem och vara konsekvent.

En grundläggande Winston-konfiguration kan se ut så här:

```
const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: 'api' },
```

```
    transports: [  
      new winston.transports.Console(),  
      new winston.transports.File({  
        filename: 'error.log',  
        level: 'error'  
      })  
    ]  
  });  
  
  logger.info('Server started', { port: 3000 });  
  logger.error('Database connection failed', {  
    err: error,  
    attempt: 3  
  });
```

Notera att Winston inte har samma inbyggda redact-funktionalitet som Pino. Om vi använder Winston behöver vi själva se till att inte logga känsliga fält, eller skriva en egen format-funktion som filtrerar bort dem innan loggen skrivs.

Mjuk radering och datalivscykel

När en användare ber oss radera ett dokument – eller när systemet automatiskt ska rensa gammal data – står vi inför ett val. Ska vi faktiskt ta bort raden från databasen (hård radering), eller ska vi markera den som raderad utan att fysiskt avlägsna den (mjuk radering)? Båda har sin plats, och som backendutvecklare ska vi veta när vi ska välja vilken.

Hård vs mjuk radering

Hård radering är vad som händer när vi kör `Model.deleteOne()` eller `Model.findByIdAndDelete()` i Mongoose. Dokumentet försvinner från databasen och kan inte återställas (utöver via backupar). Det är enkelt att förstå och enkelt att implementera, men det finns situationer där det är ett dåligt val.

Mjuk radering innebär att vi behåller dokumentet men markerar det som raderat med ett extra fält, exempelvis `deletedAt`. Alla framtida queries måste då filtrera bort dokument där `deletedAt` finns satt. Dokumentet är fortfarande tillgängligt för administratörer och kan återställas, men för den vanliga klienten ser det ut som att det är borta.

Argument för mjuk radering:

- Vi kan återställa data om en användare ångrar sig eller om en raderingsoperation gjorts av misstag.
- Relationer i databasen behåller sin integritet (referenser till raderade dokument bryts inte).
- Vi kan logga och granska raderingar genom att veta vem som raderade vad och när.
- Vi behöver inte ta bort referenser i andra dokument omedelbart - det kan göras i bakgrunden.

Argument för hård radering:

- Det är enklare och mindre felbenäget - varje query behöver inte tänka på `deletedAt`-filter.
- Databasen växer inte med data som ingen längre använder.
- Det är vad GDPR ofta förväntar sig när användaren utövar rätten att bli glömd. Att hålla data "i hemlighet" är inte att radera den.

Det finns alltså en spänning här. Mjuk radering är pragmatisk för många funktioner i en applikation (en användare som av misstag raderat ett blogginlägg uppskattar att kunna återställa det), men det krockar med GDPR:s krav på faktisk radering. Lösningen är ofta att använda mjuk radering som ett mellansteg, men ha en automatisk process som efter en viss tid (typiskt 30-90 dagar) konverterar mjukt raderade dokument till hård radering eller anonymisering.

Implementation av mjuk radering

Vi börjar med att lägga till ett `deletedAt`-fält i schemat. Värdet är null för aktiva dokument och ett datum för raderade:

```
const noteSchema = new Schema({
  title: String,
  content: String,
  ownerId: { type: Schema.Types.ObjectId, ref: 'User' },
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now },
```

```

    deletedAt: { type: Date, default: null }
  });

```

Istället för att ta bort dokumentet vid radering uppdaterar vi `deletedAt`:

```

// Mjuk radering
router.delete('/notes/:id', auth, async (req, res, next) => {
  try {
    const note = await Note.findOneAndUpdate(
      { _id: req.params.id, ownerId: req.user.id, deletedAt: null },
      { deletedAt: new Date() },
      { new: true }
    );
    if (!note) throw new NotFoundError('Anteckning hittades inte');
    res.status(204).send();
  } catch (err) { next(err); }
});

```

Vi måste sedan se till att alla våra queries filtrerar bort raderade dokument. En vanlig miss är att glömma detta och få med raderade poster i sökningar. Det finns flera sätt att hantera det. Det enklaste är att alltid lägga till `deletedAt: null` i queries:

```

const notes = await Note.find({
  ownerId: req.user.id,
  deletedAt: null
});

```

Det är dock lätt att glömma. En mer robust lösning är att använda Mongoose middleware (även kallat hooks) för att automatiskt lägga till filtret på alla find-queries:

```

// Lägg till filter på alla find* queries
noteSchema.pre(/^find/, function (next) {
  // Skippa filtret om vi explicit vill ha med raderade
  if (this.getOptions().includeDeleted) {
    return next();
  }
  this.where({ deletedAt: null });
  next();
});

// Vanliga queries filtrerar automatiskt
const activeNotes = await Note.find({ ownerId: id });

// Admin kan inkludera raderade
const allNotes = await Note.find(
  { ownerId: id },
  null,
  { includeDeleted: true }
);

```

Det här mönstret är kraftfullt eftersom det gör mjuk radering till ett standardbeteende: vi måste aktivt välja att inkludera raderade dokument, inte aktivt välja att exkludera dem. Det är säkrare default.

Datalivscykel och retention

GDPR säger att vi inte får spara personuppgifter längre än nödvändigt. Det här är en fråga om datalivscykel: vi behöver tänka igenom från det ögonblick datan skapas till det ögonblick den ska tas bort. För varje typ av data ska vi ha ett svar på frågan: "hur länge sparar vi det här, och varför?"

En retention-policy är en explicit beskrivning av hur länge olika typer av data sparas. Här är ett exempel på vad det kan innehålla:

- Aktiva användarkonton - sparas så länge användaren är aktiv. Inaktiveras efter 24 månaders inaktivitet.
- Inaktiverade konton - mjukt raderade och permanent raderade efter ytterligare 90 dagar.
- Audit logs (säkerhetsloggar) - sparas 12 månader för att möjliggöra incidentutredning.
- Diagnostiska loggar - sparas 30 dagar och raderas sedan automatiskt.
- Användarens innehåll (anteckningar, dokument) - sparas så länge kontot är aktivt, raderas med kontot.
- Aggregerad användningsstatistik - sparas obegränsat efter att personidentifierare avlägsnats.

Hur en sådan policy implementeras i praktiken är beroende av systemet. Ofta är det ett schemalagt jobb som körs nattligen och rensar data baserat på reglerna. I Node.js-världen kan vi använda node-cron eller liknande för att schemalägga sådana jobb:

```
const cron = require('node-cron');

// Hård radera dokument som varit mjukt raderade > 90 dagar
cron.schedule('0 3 * * *', async () => { // varje natt 03:00
  const cutoff = new Date();
  cutoff.setDate(cutoff.getDate() - 90);

  const result = await Note.deleteMany(
    { deletedAt: { $lt: cutoff } },
    { includeDeleted: true }
  );
  logger.info(
    { count: result.deletedCount, type: 'note' },
    'Permanent deletion of soft-deleted documents'
  );
});
```

Notera att vi loggar antalet raderade dokument men inte vilka konkreta dokument det var. Det är en bra balans: vi får statistik som är användbar för övervakning, men vi skapar inte en logg som i sig blir en samling av personuppgifter.

Anonymisering som alternativ

Ibland vill vi inte radera all data. En kommentar som en användare lämnat i ett forum kan vara värdefull för andra användare att läsa, även efter att den ursprunglige skribenten lämnat tjänsten. Lösningen är då anonymisering: vi tar bort kopplingen mellan kommentaren och personen, men låter själva innehållet stå kvar.

Konkret kan det innebära att vi vid radering av ett konto utför följande steg:

1. Ersätter användarens displayName med 'Anonym användare' eller liknande.
2. Ersätter e-postadressen med ett genererat värde (t.ex. deleted-{userId}@example.invalid).
3. Tar bort eventuella profilbilder, biografier och andra identifierande uppgifter.
4. Behåller användar-ID:t i databasen så att foreign keys i andra dokument inte bryts, men ID:t kan inte längre kopplas till en specifik person.

Det är värt att notera att anonymisering bara fungerar om den verkligen är oåterställbar. Om vi sparar en separat tabell över "vem var detta innan vi anonymiserade" har vi inte anonymiserat – vi har bara obfuskat. Riktig anonymisering innebär att kopplingen är permanent borta.

Relationsdatabaser i korthet

Hittills i kursen har vi arbetat med MongoDB, en dokumentdatabas som lagrar data i flexibla JSON-liknande dokument. Det är en av två stora familjer av databaser. Den andra – och historiskt sett mest använda – är relationsdatabaser, som använder SQL (Structured Query Language) för att fråga och manipulera data. Den här veckan tar vi en kort introduktion till SQL och relationsmodellen så att vi förstår skillnaderna.

Relationsdatabaser har funnits sedan 1970-talet och bygger på relationsmodellen som formulerades av matematikern Edgar F. Codd. Modellen organiserar data i tabeller med rader och kolumner, där varje tabell representerar en entitet (användare, ordrar, produkter) och varje rad är en specifik instans av den entiteten. Tabeller kopplas samman via relationer, ofta uttryckta som foreign keys.

Vanliga relationsdatabaser inkluderar PostgreSQL, MySQL/MariaDB, SQLite och Microsoft SQL Server. PostgreSQL är ett av de mest älskade open source-alternativen och en utmärkt utgångspunkt för den som vill lära sig SQL i praktiken.

SQL-grunder

SQL är ett deklarativt språk: vi beskriver vad vi vill ha, inte hur databasen ska hämta det. Här är de vanligaste konstruktionerna i mycket korthet.

Skapa en tabell:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  display_name VARCHAR(100) NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Infoga en rad:

```
INSERT INTO users (email, display_name, password_hash)  
VALUES ('alice@example.com', 'Alice', '$2b$12$...');
```

Hämta data:

```
SELECT id, email, display_name FROM users WHERE id = 5;  
  
SELECT * FROM users WHERE created_at > '2026-01-01'  
ORDER BY created_at DESC LIMIT 10;
```

Uppdatera och radera:

```
UPDATE users SET display_name = 'Alice Smith' WHERE id = 5;  
  
DELETE FROM users WHERE id = 5;
```

Relationer mellan tabeller:

```

CREATE TABLE notes (
  id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  content TEXT,
  owner_id INTEGER NOT NULL REFERENCES users(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- JOIN för att hämta anteckningar med deras ägare
SELECT n.id, n.title, u.display_name AS owner
FROM notes n
INNER JOIN users u ON u.id = n.owner_id
WHERE u.email = 'alice@example.com';

```

Notera att relationsdatabaser har ett fast schema. Vi måste i förväg definiera vilka kolumner en tabell har, vilka typer de är och vilka regler som gäller (NOT NULL, UNIQUE, foreign keys, etc.). Det skiljer sig markant från MongoDB där dokument i samma collection kan ha helt olika struktur.

Jämförelse med MongoDB

Skillnaderna mellan relationsdatabaser och MongoDB är inte bara syntaktiska – de speglar olika filosofier kring hur data bäst organiseras. Det finns områden där den ena har tydliga fördelar över den andra.

Relationsdatabaser har styrkor i:

- Komplexa relationer mellan entiteter där JOIN-frågor är vanliga.
- Strikta datastrukturer där schemat sällan ändras och datan måste vara konsekvent.
- Transaktioner med ACID-garantier över flera tabeller (t.ex. ekonomisk bokföring där delvis lyckade operationer skulle vara katastrofalt).
- Mogen verktygskedja - decennier av erfarenhet, dokumentation och tooling.

MongoDB har styrkor i:

- Snabb iterativ utveckling där datamodellen inte är slutgiltigt fastställd.
- Hierarkiska data som naturligt representeras som inbäddade dokument.
- Hög skalbarhet horisontellt över många servrar (sharding).
- Naturlig passform med JavaScript-ekosystemet - JSON in, JSON ut.

Många moderna system använder båda. Ett företag kan ha PostgreSQL för bokföring och kundregister (där konsistens och transaktioner är kritiskt) och MongoDB för logghändelser och produktkatalog (där flexibilitet och skala är viktigare). Som backendutvecklare ska vi förstå båda tillräckligt väl för att kunna välja rätt verktyg när vi designar ett nytt system.

I Node.js-ekosystemet finns moderna ORM- och query-bibliotek som Prisma, Drizzle och Knex som låter oss arbeta med relationsdatabaser på ett sätt som påminner om hur vi använder Mongoose. Vi går inte djupt in i dem under den här kursen, men de är värda att känna till för framtida projekt.

Visualisera dataflöden

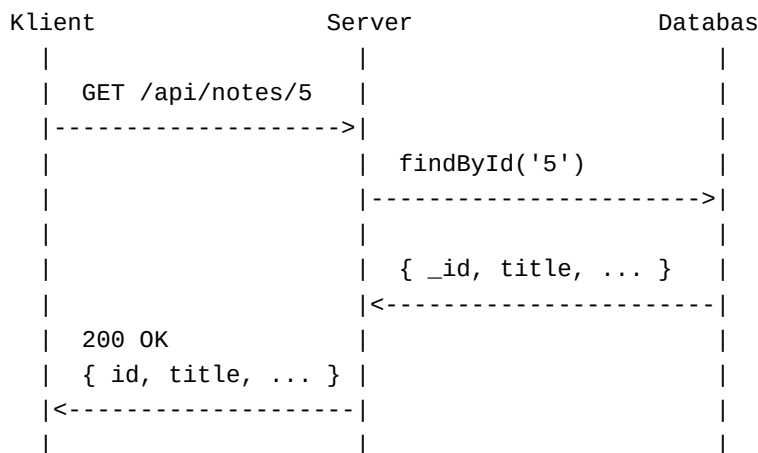
När vi designar ett system börjar vi ofta med att rita upp det. Vi tar en penna eller en whiteboard och skissar hur klienten kommunicerar med servern, hur servern pratar med databasen, vilka externa tjänster vi anropar och hur datan flödar genom systemet. Det är ett kraftfullt sätt att tänka och kommunicera. Vi går från abstrakta begrepp till konkreta bilder, och bilderna avslöjar ofta saker som rena ord inte gör.

För backendutvecklare är det här särskilt viktigt. Vi arbetar med system som inte är direkt synliga – ingenting på skärmen visar hur en HTTP-request bearbetas internt. Diagram blir vårt sätt att göra det osynliga synligt. När en ny utvecklare ansluter till teamet är ett bra arkitekturdiagram värdefullare än timmar av muntlig genomgång. När vi själva återvänder till en gammal kodbas påminner diagrammen oss om hur det hela hänger ihop.

Klient, server, databas

Det enklaste och mest grundläggande diagrammet är klient-server-databas-flödet. Det visar hur en HTTP-request från klienten blir till en databas-query och hur svaret hittar tillbaka.

Som ASCII-diagram (vilket vi ofta använder i kodkommentarer) ser ett enkelt flöde ut så här:



Pilarna visar riktningen på kommunikationen, och tiden flyter uppifrån och ned. Diagrammet är extremt enkelt men kommunicerar viktig information: klienten kontaktar inte databasen direkt, det är alltid servern som är mellanhand, och varje pil representerar ett HTTP-anrop eller en databas-query.

Sekvensdiagram för komplexa flöden

När flödena blir mer komplicerade räcker inte de enkla diagrammen längre. Då tar vi till sekvensdiagram – en formaliserad notation från UML-världen som visar hur olika aktörer (klient, API, databas, externa tjänster) interagerar över tid. Sekvensdiagram är ett av de mest användbara dokumentationsverktygen vi som backendutvecklare har.

Ett bra exempel är ett inloggningsflöde med JWT. Det involverar minst tre aktörer: klienten, vår Express-server och vår MongoDB-databas. Och det innehåller flera steg: hämta användare baserat på e-post, jämföra lösenordshash, generera JWT, returnera token till klienten.

Som sekvensdiagram skulle det kunna se ut så här (i pseudoformat):

Aktörer: Klient, API, Database

```
1. Klient -> API: POST /api/auth/login { email, password }
2. API -> Database: User.findOne({ email })
3. Database -> API: { _id, email, passwordHash, ... }
4. API: bcrypt.compare(password, passwordHash)
5. API: jwt.sign({ userId }, JWT_SECRET)
6. API -> Klient: 200 OK { token }
```

Felfall (fel lösenord):

```
4'. API: bcrypt.compare returnerar false
5'. API -> Klient: 401 Unauthorized
```

När vi ritar det här som ett riktigt diagram (vilket vi gör i nästa avsnitt) blir det ännu tydligare. Sekvensdiagram är särskilt värdefulla när vi vill diskutera ett flöde med en kollega eller en kund – ett gemensamt diagram skapar gemensam förståelse på ett sätt som ord ofta misslyckas med.

Mermaid - diagram som kod

Mermaid är ett textbaserat sätt att skriva diagram. Vi beskriver diagrammet med ett enkelt syntax-språk och Mermaid renderar det till en bild. Det låter inte särskilt revolutionerande tills vi inser fördelarna: diagrammen kan versionshanteras i Git tillsammans med koden, ändringar visas som diff:ar, och vi behöver aldrig leta efter senaste versionen av en .png-fil i någons inkorg.

Mermaid stöds direkt i flera moderna verktyg: GitHub renderar Mermaid-diagram inuti README-filer, GitLab gör samma sak, Notion och Obsidian har stöd inbyggt. Det betyder att ett Mermaid-diagram i ett README-dokument blir en bild när någon läser det på GitHub – utan att vi behöver bädda in en separat bildfil.

Sekvensdiagram i Mermaid

Här är inloggningsflödet ovan uttryckt i Mermaid-syntax:

```

```mermaid
sequenceDiagram
 participant C as Klient
 participant A as API
 participant DB as Database

 C->>A: POST /api/auth/login
 A->>DB: User.findOne({ email })
 DB-->>A: User-dokument
 A->>A: bcrypt.compare(password, hash)

 alt Korrekt lösenord
 A->>A: jwt.sign({ userId })
 A-->>C: 200 OK { token }
 else Fel lösenord
 A-->>C: 401 Unauthorized
 end
end
```

```

Notera ett par detaljer i syntaxen. ->> är en heldragen pil (synkront anrop), -->> är en streckad pil (svar). alt och else representerar förgreningar i flödet, så att vi kan visa både lyckade och misslyckade scenarier i samma diagram. participant-deklarationerna i början definierar aktörerna och ger dem korta alias för att hålla själva diagrammet kompakt.

Flödesdiagram i Mermaid

Sekvensdiagram är inte den enda diagramtypen Mermaid stöder. Flödesdiagram (flowcharts) är användbara för att visa beslutsträd och processflöden:

```

```mermaid
flowchart TD
 Start([Inkommande request]) --> Auth{Har giltig JWT?}
 Auth -->|Nej| Reject[401 Unauthorized]
 Auth -->|Ja| Role{Har rätt roll?}
 Role -->|Nej| Forbidden[403 Forbidden]
 Role -->|Ja| Validate[Validera input]

```

```

 Validate -->|Ogiltig| BadRequest[400 Bad Request]
 Validate -->|Giltig| Process[Bearbeta requesten]
 Process --> Response[200 OK]

```

Det här är en utmärkt sätt att dokumentera middleware-kedjan i ett Express-API. Varje rektangel representerar ett steg, varje romb ett beslut. När en ny utvecklare ska förstå hur våra requests flödar genom systemet är ett sådant diagram värt mycket mer än textbeskrivningar.

## ER-diagram för datamodeller

En tredje typ av diagram som är ovärderlig för backendutvecklare är ER-diagrammet (Entity-Relationship). Det visar vilka entiteter (tabeller eller collections) som finns i systemet och hur de relaterar till varandra. Mermaid har inbyggt stöd även för dessa:

```

```mermaid
erDiagram
    USER ||--o{ NOTE : skapar
    USER ||--o{ COMMENT : skriver
    NOTE ||--o{ COMMENT : har

    USER {
        string id PK
        string email
        string displayName
        datetime createdAt
    }
    NOTE {
        string id PK
        string ownerId FK
        string title
        string content
        datetime deletedAt
    }
  ```

```

Notationen `||--o{` är crow's foot-notation och uttrycker kardinalitet: en USER har noll eller flera NOTES (en-till-många-relation). PK står för primary key, FK för foreign key. Det är ett mycket kompakt sätt att kommunicera databasstruktur.

## draw.io som komplement

Mermaid är fantastiskt för diagram som är en del av kod- och dokumentationsflödet. Men ibland behöver vi något mer flexibelt: arkitekturdiagram med ikoner för olika tjänster, mer kontroll över layout, eller diagram som ska skrivas ut i en presentation. Då är draw.io (numera diagrams.net) ett utmärkt komplement.

draw.io är en gratis, webbaserad ritapplikation som kan användas direkt i webbläsaren utan registrering. Den har stora ikonbibliotek för molntjänster (AWS, Azure, GCP), nätverksdiagram, UML och mycket mer. Filer kan sparas i Google Drive, OneDrive, lokalt eller i GitHub. Det är ett av de mest använda verktygen för att rita systemarkitekturer i industrin.

En vanlig arbetsfördelning är att använda Mermaid för diagram som hör till koden (sekvensdiagram för specifika flöden, ER-diagram för datamodeller) och draw.io för översiktsdiagram (systemarkitektur, deployment-topologi). Båda har sina platser, och båda är värda att lära sig.

## API-dokumentation med Swagger och OpenAPI

Diagrammen vi pratat om dokumenterar systemet på en hög nivå. Men för det API vi själva bygger behöver vi också dokumentation på en mer detaljerad nivå: vilka endpoints finns, vilka parametrar accepterar de, vilket format har request body, vilka statuskoder returneras, vilka fält finns i svaret. Det är information som klientutvecklare (eller andra backend-utvecklare) behöver för att kunna integrera mot vårt API.

OpenAPI Specification (tidigare kallat Swagger Specification) är en industristandard för att beskriva REST API:er. En OpenAPI-specifikation är en JSON- eller YAML-fil som strukturerat beskriver vårt API: vilka resurser som finns, vilka operationer som stöds, vilka datamodeller som används. Filen kan sedan användas av många olika verktyg för att automatiskt generera dokumentation, klientkod, mockservrar och tester.

Swagger är en samling verktyg som arbetar med OpenAPI-spec. Det mest kända är Swagger UI – ett webbgränssnitt som genererar en interaktiv dokumentationssida från en OpenAPI-fil. Användare kan läsa beskrivningar, se exempel på request och response, och till och med köra requests direkt från dokumentationssidan.

### OpenAPI-spec i grundform

En OpenAPI-spec kan skrivas som ett JSON- eller YAML-objekt. YAML är ofta mer läsbart för människor, så vi använder det här. Ett mycket enkelt exempel:

```
openapi: 3.0.3
info:
 title: Anteckningsapp API
 version: 1.0.0
 description: API för en enkel anteckningsapp

paths:
 /api/notes:
 get:
 summary: Hämta alla anteckningar för inloggad användare
 security:
 - bearerAuth: []
 responses:
 '200':
 description: Lyckad respons
 content:
 application/json:
 schema:
 type: array
 items:
 $ref: '#/components/schemas/Note'
 '401':
 description: Ej inloggad

components:
 securitySchemes:
 bearerAuth:
 type: http
 scheme: bearer
 bearerFormat: JWT
```

```
schemas:
 Note:
 type: object
 properties:
 id: { type: string }
 title: { type: string }
 content: { type: string }
 createdAt: { type: string, format: date-time }
```

Specifikationen är hierarkisk. Vi deklarerar info om API:et, sedan paths som beskriver varje endpoint och slutligen components som innehåller återanvändbara delar (datamodeller, säkerhetsmekanismer). Notera \$ref-syntaxen – den fungerar som en pekare till en annan del av filen och låter oss undvika duplicering.

## Swagger UI i Express

Vi kan integrera Swagger UI direkt i vår Express-applikation med hjälp av paketen swagger-ui-express och js-yaml:

```
npm install swagger-ui-express js-yaml

const express = require('express');
const swaggerUi = require('swagger-ui-express');
const yaml = require('js-yaml');
const fs = require('fs');

const app = express();

// Läs in OpenAPI-spec från fil
const openApiSpec = yaml.load(
 fs.readFileSync('./openapi.yaml', 'utf8')
);

// Servera Swagger UI på /api-docs
app.use('/api-docs',
 swaggerUi.serve,
 swaggerUi.setup(openApiSpec)
);
```

När servern startar kan vi besöka <http://localhost:3000/api-docs> i en webbläsare och se en interaktiv dokumentationssida med alla endpoints från vår spec. Vi kan klicka på en endpoint, se exempel på request och response och till och med köra anrop direkt från sidan med inloggad token.

## Kodgenererad dokumentation

Att skriva OpenAPI-spec för hand fungerar för små projekt, men blir snabbt tungt när API:et växer. En vanlig lösning är att generera spec:en från koden. Det finns flera vägar att gå.

Ett populärt alternativ är swagger-jsdoc, som låter oss skriva OpenAPI-fragment som JSDoc-kommentarer ovanför våra route-handlers. Spec:en byggs sedan automatiskt:

```

/**
 * @openapi
 * /api/notes/{id}:
 * get:
 * summary: Hämta en specifik anteckning
 * parameters:
 * - in: path
 * name: id
 * required: true
 * schema:
 * type: string
 * responses:
 * 200:
 * description: Lyckat svar
 * 404:
 * description: Anteckning hittades inte
 */
router.get('/:id', auth, async (req, res, next) => {
 // ...
});

```

Det här mönstret håller dokumentation och kod nära varandra, vilket gör det mer sannolikt att de hålls i synk. När vi ändrar en endpoint kommer vi sannolikt också ändra dokumentationen som ligger precis ovanför.

Ett annat alternativ, om vi använder Zod för validering, är att använda zod-to-openapi som kan generera OpenAPI-schemas direkt från våra Zod-scheman. Det är ett kraftfullt mönster eftersom det innebär att samma definition används för validering och dokumentation – de kan inte komma ur synk.

## Code-along: Strukturerad loggning och API-dokumentation

Under workshopen sätter vi ihop det vi gått igenom till ett komplett exempel. Vi tar utgångspunkt i ett enkelt Express-API med ett par endpoints och lägger på strukturerad loggning med Pino samt OpenAPI-dokumentation med Swagger UI. Det här kommer att vara den centrala code-alongen för veckan.

Strukturen vi utgår från:

```
api/
 src/
 index.js // applikationsstart
 logger.js // Pino-konfiguration
 middleware/
 auth.js // JWT-autentisering
 errorHandler.js // central felhantering
 routes/
 auth.js // /api/auth/*
 notes.js // /api/notes/*
 models/
 User.js
 Note.js
 openapi.yaml // API-spec
 package.json
```

### Steg 1: Logger med redact

Vi börjar med en konfiguration av Pino som inkluderar redact-regler för känsliga fält. Det är vår första försvarslinje mot att personuppgifter hamnar i loggarna:

```
// src/logger.js
const pino = require('pino');

const isProduction = process.env.NODE_ENV === 'production';

const logger = pino({
 level: process.env.LOG_LEVEL || (isProduction ? 'info' : 'debug'),
 transport: !isProduction
 ? { target: 'pino-pretty', options: { colorize: true } }
 : undefined,
 redact: {
 paths: [
 'req.headers.authorization',
 'req.headers.cookie',
 'req.body.password',
 'req.body.passwordConfirm',
 'req.body.token',
 'res.headers["set-cookie"]',
 '*.password',
 '*.passwordHash',
 '*.token'
],
 },
 censor: '[REDACTED]'
});
```

```

 },
 // Lägg till några standardfält i alla loggrader
 base: {
 service: 'notes-api',
 env: process.env.NODE_ENV || 'development'
 }
 });

 module.exports = logger;

```

## Steg 2: HTTP-loggningsmiddleware

Vi sätter upp pino-http som en middleware som loggar varje inkommande request, sätter ett requestId och gör logger tillgänglig som req.log:

```

// src/index.js
const express = require('express');
const pinoHttp = require('pino-http');
const crypto = require('crypto');
const logger = require('./logger');

const app = express();

app.use(express.json());

app.use(pinoHttp({
 logger,
 genReqId: (req) =>
 req.headers['x-request-id'] || crypto.randomUUID(),
 customLogLevel: function (req, res, err) {
 if (err || res.statusCode >= 500) return 'error';
 if (res.statusCode >= 400) return 'warn';
 return 'info';
 }
}));

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/notes', require('./routes/notes'));

app.use(require('./middleware/errorHandler'));

const port = process.env.PORT || 3000;
app.listen(port, () => {
 logger.info({ port }, 'Server listening');
});

```

Den anpassade customLogLevel-funktionen sätter loggnivån baserat på resultatet: 5xx-svar loggas som error, 4xx som warn och övriga som info. Det är en bra balans som ger oss tillräcklig signal utan att översvämma loggarna.



### Steg 3: Säker loggning i route-handlers

Inuti våra route-handlers använder vi req.log för att logga händelser. Vi tänker noga på vad vi loggar och loggar aldrig användarens e-post, lösenord eller andra direkta personuppgifter:

```
// src/routes/auth.js
const router = require('express').Router();
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

router.post('/login', async (req, res, next) => {
 try {
 const { email, password } = req.body;

 const user = await User
 .findOne({ email })
 .select('+passwordHash');

 if (!user) {
 // Ingen e-post i loggen, bara att en login misslyckades
 req.log.warn(
 { event: 'auth.login.failed', reason: 'user_not_found' },
 'Login attempt for unknown user'
);
 return res.status(401).json({ error: 'Felaktiga uppgifter' });
 }

 const ok = await bcrypt.compare(password, user.passwordHash);
 if (!ok) {
 req.log.warn(
 { event: 'auth.login.failed', userId: user.id,
 reason: 'wrong_password' },
 'Login attempt with wrong password'
);
 return res.status(401).json({ error: 'Felaktiga uppgifter' });
 }

 const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET);

 req.log.info(
 { event: 'auth.login.success', userId: user.id },
 'User logged in'
);
 res.json({ token });
 } catch (err) { next(err); }
});

module.exports = router;
```

Lägg märke till hur vi loggar event-namn som `user.login.failed` och `user.login.success`. Att använda strukturerade event-namn gör det enkelt att senare räkna och söka efter specifika typer av händelser i loggarna.

## Steg 4: OpenAPI-spec

Vi skriver en OpenAPI-spec för vårt API. Vi börjar enkelt och utökar efter hand. Här är en grundläggande spec som dokumenterar inloggning och listning av anteckningar:

```
openapi.yaml
openapi: 3.0.3
info:
 title: Anteckningsapp API
 version: 1.0.0

servers:
 - url: http://localhost:3000
 description: Lokal utveckling

components:
 securitySchemes:
 bearerAuth:
 type: http
 scheme: bearer
 bearerFormat: JWT
 schemas:
 LoginRequest:
 type: object
 required: [email, password]
 properties:
 email: { type: string, format: email }
 password: { type: string, minLength: 8 }
 LoginResponse:
 type: object
 properties:
 token: { type: string }
 Note:
 type: object
 properties:
 id: { type: string }
 title: { type: string }
 content: { type: string }
 createdAt: { type: string, format: date-time }
 Error:
 type: object
 properties:
 error: { type: string }

paths:
 /api/auth/login:
 post:
 summary: Logga in användare
 requestBody:
```

```

 required: true
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/LoginRequest'
 responses:
 '200':
 description: Lyckad inloggning
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/LoginResponse'
 '401':
 description: Felaktiga uppgifter
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/Error'

/api/notes:
 get:
 summary: Hämta inloggad användares anteckningar
 security:
 - bearerAuth: []
 responses:
 '200':
 description: Lista med anteckningar
 content:
 application/json:
 schema:
 type: array
 items:
 $ref: '#/components/schemas/Note'
 '401':
 description: Ej inloggad

```

## Steg 5: Swagger UI i Express

Vi monterar Swagger UI på /api-docs:

```

// src/index.js (tillägg)
const swaggerUi = require('swagger-ui-express');
const yaml = require('js-yaml');
const fs = require('fs');
const path = require('path');

const openApiSpec = yaml.load(
 fs.readFileSync(path.join(__dirname, '../openapi.yaml'), 'utf8')
);

// Swagger UI bara i utveckling och staging - inte i produktion
if (process.env.NODE_ENV !== 'production') {

```

```

app.use(
 '/api-docs',
 swaggerUi.serve,
 swaggerUi.setup(openApiSpec)
);
logger.info('Swagger UI available at /api-docs');
}

```

Vi väljer att inte exponera Swagger UI i produktion av två skäl. Det första är att den ger en angripare värdefull information om vårt API:s struktur. Det andra är att det för publika API:er ofta är bättre att hosta dokumentationen separat från själva API:et. För interna API:er räcker det ofta att exponera den bakom inloggning, men då behöver vi lägga på autentisering på dokumentationssidan, vilket är extra arbete.

## Steg 6: Sekvensdiagram för inloggningen

Slutligen dokumenterar vi inloggningsflödet med ett Mermaid-sekvensdiagram. Vi sparar det i en docs-mapp i repot:

```
docs/sequence-login.md
```

```
Inloggningsflöde
```

Det här diagrammet beskriver hur en inloggning går till.

```

```mermaid
sequenceDiagram
    participant C as Klient
    participant API as API (Express)
    participant DB as MongoDB

    C->>API: POST /api/auth/login { email, password }
    API->>DB: User.findOne({ email })
    alt Användare finns
        DB-->>API: User-dokument
        API->>API: bcrypt.compare(password, hash)
        alt Korrekt lösenord
            API->>API: jwt.sign({ userId })
            API-->>C: 200 OK { token }
        else Fel lösenord
            API-->>C: 401 Unauthorized
        end
    end
    else Användare finns inte
        DB-->>API: null
        API-->>C: 401 Unauthorized
    end
end
```

```

När det här filen läggs in i ett GitHub-repo renderas diagrammet automatiskt när någon tittar på den i webbgöransnittet. Det är dokumentation som lever tillsammans med koden – när vi ändrar inloggningsflödet i koden ser vi också att diagrammet behöver uppdateras, eftersom det ligger i samma repo.



## Checkpoint: Förberedelse inför kursvecka 9

Inför check-in tisdag förväntas vi ha läst igenom det här studiematerialet och ha en grundläggande förståelse för veckans koncept. Vi ska också titta över vårt boiler room-projekt och fundera över hur veckans ämnen kan tillämpas på det vi byggt hittills.

Ta med er följande till checkpoint-mötet:

5. En lista över alla fält i era datamodeller där ni hanterar potentiella personuppgifter. Vad behövs verkligen och vad kan tas bort?
6. En genomgång av era nuvarande loggar (om ni har några). Loggar ni något som ni inte borde logga?
7. Tankar kring vilka delar av ert API som behöver mest dokumentation. Var skulle ett sekvensdiagram tillföra mest värde?
8. Ett första utkast till en datalivscykel-policy: hur länge sparas olika typer av data i ert system?

Det är inte meningen att ni ska ha allt klart inför checkpointen – syftet är att ni ska ha tänkt på frågorna och kunna diskutera dem konstruktivt. Många av dessa beslut är inte tekniska utan handlar om policy och avvägningar, och de blir bättre när hela gruppen tänker tillsammans.

En vanlig insikt under den här veckan är att vi i tidigare veckor sparat mer data än vi behöver – inte av illvilja utan för att det var lätt att lägga till fält i scheman när man ändå höll på. Det är helt normalt och en del av lärandeprocessen. Veckans uppgift är delvis att rensa, inte bara att lägga till.

## Sammanfattning

Under kursvecka 9 har vi lagt till en dimension till vår backendutveckling som handlar om mer än bara teknik: ansvarsfullt hanterande av människors data, transparent dokumentation och medveten loggning. Det är ämnen som skiljer en hobbyutvecklare från en professionell backendutvecklare.

Vi har gått igenom GDPR och de sex grundläggande principerna: laglighet, ändamålsbegränsning, uppgiftsminimering, korrekthet, lagringsminimering samt integritet och konfidentialitet. Vi har sett vad som räknas som personuppgifter, varför IP-adresser och användar-ID:n är inkluderade och varför känsliga personuppgifter har ännu starkare skydd. Vi har också gått igenom användarens rättigheter och vad de innebär för vår API-design – konkret behöver vi ofta endpoints för att exportera och radera användardata.

Dataminimering är den princip vi i praktiken oftast brister i. Vi har sett hur en användarmodell kan slimmas från ett dussintal fält till bara de som verkligen behövs, och hur vi via Mongoose-projection och select: false skyddar oss mot att oavsiktligt exponera känsliga fält som passwordHash i API-svar.

Säker loggning kräver disciplin: lösenord, tokens, hela request bodies och fullständiga URL:er med känsliga query-parametrar ska aldrig hamna i loggarna. Strukturerad loggning med JSON-format gör loggarna sökbara och filtrerbara, och loggnivåer (trace, debug, info, warn, error, fatal) ger oss kontroll över hur mycket information vi sparar. Pino och Winston är två starka val i Node.js-ekosystemet, och Pinos redact-funktionalitet är ett särskilt värdefullt skyddsnät. Correlation ID:n via req.log gör det möjligt att spåra en specifik request genom hela systemet i loggarna.

Mjuk radering ger oss flexibilitet och möjlighet att återställa data, men måste balanseras mot GDPR:s krav på faktisk radering. Lösningen är ofta en datalivscykel där mjukt raderade dokument efter en viss tid hård-raderas eller anonymiseras. Vi har sett konkreta exempel på hur Mongoose middleware automatiserar filtreringen av raderade dokument och hur node-cron kan schemalägga rensningsjobb.

Den korta introduktionen till relationsdatabaser och SQL har givit oss perspektiv på MongoDB. Båda modellerna har sina styrkor: relationsdatabaser excellerar i komplexa relationer, strikta scheman och transaktioner, medan dokumentdatabaser excellerar i flexibilitet, hierarkiska data och horisontell skalning. Många moderna system använder båda.

Slutligen har vi sett hur vi visualiserar och dokumenterar våra system. Sekvensdiagram, flödesdiagram och ER-diagram med Mermaid är kraftfulla verktyg som lever tillsammans med koden i vårt versionskontrollsystem. draw.io kompletterar för översiktsdiagram. OpenAPI och Swagger UI ger oss en standardiserad och interaktiv API-dokumentation som klientutvecklare uppskattar.

Nästa vecka kopplar vi ihop allt vi byggt med en frontend och tittar på driftsättning till en molnmiljö. Mycket av det vi gjort i denna vecka – strukturerad loggning, säker konfiguration via miljövariabler, dokumentation – kommer att vara direkt tillämpbart när systemet ska gå från utvecklingsdator till produktionsserver.

## Resurser för fördjupning

Nedan följer resurser för de som vill fördjupa sina kunskaper inom GDPR, säker loggning, dataflöden och API-dokumentation.

### GDPR och dataskydd

Integritetsskyddsmyndigheten (IMY) är den svenska tillsynsmyndigheten för dataskyddsfrågor. Deras webbplats innehåller god vägledning på svenska för hur GDPR ska tolkas i praktiken:

<https://www.imy.se>

GDPR.eu - en utbildningsresurs som förklarar GDPR-artiklarna i mer tillgängligt språk. Bra utgångspunkt för att förstå förordningen i sin helhet:

<https://gdpr.eu>

### Loggningsbibliotek

Pino - officiell dokumentation med fullständig API-referens och konfigurationsexempel:

<https://getpino.io>

Winston - dokumentation med transports, format och konfiguration:

<https://github.com/winstonjs/winston>

"The Twelve-Factor App" - klassiskt dokument om hur moderna webbappar ska byggas. Faktor XI handlar specifikt om loggar och behandling av loggar som dataströmmar:

<https://12factor.net/logs>

### Diagram och dokumentation

Mermaid - officiell dokumentation med exempel för alla diagramtyper:

<https://mermaid.js.org>

Mermaid Live Editor - prova Mermaid direkt i webbläsaren utan installation:

<https://mermaid.live>

draw.io (numera diagrams.net) - gratis online-ritapplikation:

<https://app.diagrams.net>

OpenAPI Specification - officiell spec, mest värdefull som referens:

<https://spec.openapis.org/oas/latest.html>



Swagger Tools - översikt över Swaggers verktygsekosystem inklusive Swagger UI och Swagger Editor:

<https://swagger.io/tools/>

## Säkerhet och Best Practices

"Node.js Best Practices" av Yoni Goldberg - kapitlet om loggning och övervakning är direkt relevant för veckans innehåll:

<https://github.com/goldbergyon/nodebestpractices>

OWASP Logging Cheat Sheet - praktiska riktlinjer för säker loggning, vad som ska och inte ska loggas:

[https://cheatsheetseries.owasp.org/cheatsheets/Logging\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html)

## Videomaterial

"GDPR Explained in 5 Minutes" och liknande videoklipp på YouTube ger snabba översikter av GDPR. Sök på "GDPR for developers" för innehåll specifikt riktat mot utvecklare.

"Logging in Node.js with Pino" finns i flera bra varianter på YouTube och kompletterar dokumentationen med praktiska genomgångar.

"How to Document Your API with Swagger" på YouTube ger en visuell genomgång av Swagger UI och OpenAPI som kan vara lättare att ta till sig än ren textdokumentation, särskilt för den som lär sig bäst genom att se.

# **Studiematerial**

## **Fullstack-integration och driftsättning**

Kursvecka 10

## Introduktion

Vi har under nio veckor byggt upp ett komplett backend-API. Vi har designat REST-endpoints, lagt till TypeScript-typning, kopplat till MongoDB via Mongoose, implementerat CRUD-operationer med tester, byggt robust inputvalidering med Zod, lagt till autentisering med JWT och bcrypt, infört rollbaserad åtkomstkontroll, och förra veckan tittade vi på GDPR, säker loggning och hur vi dokumenterar dataflöden. Allt detta har vi gjort lokalt på våra egna datorer. Nu är det dags att ta nästa kliv: vi ska koppla ihop vårt API med en frontend och vi ska driftsätta det till en molnmiljö så att andra människor kan använda det.

Det här är veckan då allt vi byggt börjar kännas som en produkt. Tidigare har vi testat våra endpoints med Thunder Client eller Postman, men i ett verkligt system är det en annan applikation som anropar vårt API – en React-app i webbläsaren, en mobilapp eller någon annan tjänst. Den här veckan handlar om att förstå hur den kopplingen faktiskt fungerar i praktiken: hur en webbläsare hanterar HTTP-anrop, varför CORS finns, hur JWT-tokens flödar mellan klient och server, och vilka problem som uppstår när allt inte längre körs på localhost.

Driftsättning är samtidigt ett område där många backendutvecklare först stöter på överraskningar. Något som fungerade perfekt på den egna datorn slutar plötsligt att fungera när det körs i molnet: portar är låsta, miljövariabler saknas, databasanslutningen vägras, eller CORS-policy blockerar frontenden. Vi går igenom dessa fallgropar i förväg så att vi förstår vad vi behöver göra istället för att gissa när det väl smäller.

Vi använder Render som vår primära molnplattform i kursen eftersom den har en gratisnivå, är enkel att komma igång med och täcker det vi behöver. MongoDB Atlas använder vi sedan vecka 4 och är redan en molntjänst, så den biten är vi vana vid. På frontend-sidan visar vi hur vi driftsätter en React-app till Vercel, men huvudfokus den här veckan ligger på backend och hur de två kopplas ihop.

Det är värt att vara tydlig med en sak: produktionsdriftsättning är ett enormt område som hela team i större företag arbetar heltid med. Vi täcker grunderna – det som krävs för att vi ska kunna leverera ett fungerande projekt och förstå huvudprinciperna. Avsnitten om HTTPS, CORS-policy och rate limiting är inte uttömmande, men de ger oss en stabil bas att stå på och en känsla för vad vi behöver tänka på även om vi inte behärskar varje detalj.

## Fullstack-arkitektur i praktiken

Innan vi gräver ner oss i detaljer är det värt att ta ett steg tillbaka och se hela bilden. En modern fullstackapplikation består typiskt av tre lager som körs på olika ställen och kommunicerar via nätverket: en frontend (React-appen i användarens webbläsare), en backend (vårt Express-API på en server) och en databas (MongoDB Atlas i molnet). Var och en av dessa har sina egna ansvarsområden och sina egna säkerhetsegenskaper.

Frontend körs på klientens dator i deras webbläsare. Det innebär att vi inte har kontroll över miljön: användaren kan öppna utvecklarverktygen, läsa vår kod, ändra vår JavaScript och skicka godtyckliga anrop till vårt API. Allt som ligger i frontenden – inklusive vilka hemligheter den känner till – är synligt för användaren. Det här är grunden till principen "don't trust the client" som vi pratade om i vecka 6. Frontend ska vara optimerad för användarupplevelse: snabb, responsiv och informativ. Den får aldrig vara den som validerar säkerhet.

Backend körs på en server vi kontrollerar. Här lever den affärslogik som vi inte kan eller vill exponera, och här lever också kopplingen till databasen. Backend ansvarar för validering på riktigt, för auktorisering, för att hantera känslig information och för att se till att data inte kan manipuleras på ett otillåtet sätt. När vi driftsätter backenden är det den första gången vår kod körs i en miljö som är beständig och åtkomlig från internet.

Databasen är vår persistenta lagring. Den körs typiskt också i molnet (i vårt fall MongoDB Atlas) och bör inte vara åtkomlig direkt från frontenden – endast backenden ska kunna prata med databasen. Det är därför vi har en backend i första hand: utan den skulle vi vara tvungna att ge frontenden direkt tillgång till databasen, vilket skulle innebära att alla användare kunde läsa och skriva godtyckligt i den.

### Var sker vad?

En typisk användarinteraktion i en fullstackapplikation följer ett mönster vi behöver kunna i sömnen. Säg att en användare loggar in i vår app:

1. Användaren skriver in e-post och lösenord i ett formulär i React-appen och klickar på "Logga in".
2. React skickar en POST-request till vårt backend-API:s /auth/login-endpoint med uppgifterna i request body.
3. Backenden tar emot anropet, validerar inputformatet med Zod, hämtar användaren ur databasen via Mongoose, jämför lösenordet med bcrypt och – om allt stämmer – genererar en JWT.
4. Backenden svarar med JWT:n och eventuellt användarens grunddata.
5. React tar emot svaret, sparar token (i localStorage eller en HTTP-only cookie, beroende på strategi) och navigerar användaren vidare.
6. För efterföljande anrop som kräver autentisering inkluderar React token i Authorization-headern, backenden verifierar den och returnerar relevant data.

Att kunna rita upp ett sådant flöde i huvudet är en av de viktigaste färdigheterna för en fullstackutvecklare. När något inte fungerar är frågan alltid: var i flödet bryts det? Frontend-konsolen, backend-loggar och databasens tillstånd är våra tre primära felsökningsplatser, och de motsvarar de tre lagren ovan.

## **Två separata kodbaser, två separata driftsättningar**

En central insikt är att frontend och backend i en modern fullstackapp är två separata applikationer. De har egna repository (eller åtminstone egna mappar i ett monorepo), egna package.json-filer, egna beroenden och driftsätts ofta till olika tjänster. Frontend kan ligga på Vercel medan backend ligger på Render. De känner till varandra endast genom URL:er och det API-kontrakt som backenden exponerar.

Det här är annorlunda från äldre arkitekturer (exempelvis traditionella PHP-applikationer eller serverrenderade .NET-applikationer) där frontend och backend var samma kodbas och samma process. Den moderna separationen ger oss flexibilitet – vi kan byta ut frontenden utan att röra backenden, vi kan driftsätta dem oberoende, och vi kan låta samma backend serva både en webbapp och en mobilapp – men den introducerar också utmaningar som CORS, autentiseringshantering och versioneringsproblematik som vi behöver hantera.

## CORS i praktiken

CORS – Cross-Origin Resource Sharing – är ett av de mest missförstådda och frustrerande fenomenen för utvecklare som möter det för första gången. Vi har sett "Access-Control-Allow-Origin"-felen i konsolen redan i tidigare kurser, och i vecka 1 lade vi till `app.use(cors())` som en magisk besvärjelse som fick allt att fungera. Nu ska vi förstå vad CORS faktiskt är, varför det finns och hur vi konfigurerar det korrekt – inte bara med ett tillåtande "låt allt komma igenom".

### Same-Origin Policy – grunden till CORS

För att förstå CORS måste vi först förstå Same-Origin Policy. Webbläsaren har en grundläggande säkerhetsregel: JavaScript som körs på en sida från ett ursprung (origin) får inte göra anrop till ett annat ursprung. Ett ursprung definieras av kombinationen protokoll, domän och port – exempelvis `https://min-app.com:443`. Om någon av de tre skiljer sig är det ett annat ursprung.

Anledningen till Same-Origin Policy är säkerhet. Föreställ dig att vi är inloggade på vår bank i en flik. I en annan flik öppnar vi en skadlig webbplats. Utan Same-Origin Policy skulle den skadliga sidans JavaScript kunna göra anrop till banken med våra inloggningscookies och göra överföringar i vårt namn. Webbläsaren förhindrar detta genom att blockera anrop från en sida till en annan domäns API.

Problemet är att i en modern fullstackapplikation är det här precis vad vi vill göra: vår React-app på `localhost:5173` ska anropa vårt API på `localhost:3000`, eller i produktion ska `app.minprodukt.com` anropa `api.minprodukt.com`. Det är två olika ursprung, så Same-Origin Policy skulle blockera anropen. CORS är mekanismen för att tillåta detta på ett kontrollerat sätt.

### Hur CORS fungerar tekniskt

När en webbläsare upptäcker ett cross-origin-anrop gör den något smart. Innan det "riktiga" anropet skickas (om det är ett "non-simple request", exempelvis med JSON-body eller anpassade headers) skickas en preflight-request: en OPTIONS-request till samma URL där webbläsaren frågar servern "får denna typ av anrop göras?". Servern svarar med headers som beskriver vad som är tillåtet – vilka ursprung, vilka metoder, vilka headers. Om svaret tillåter det aktuella anropet skickar webbläsaren det riktiga anropet. Annars blockeras det och vi får CORS-felet.

Det är viktigt att förstå att CORS är en kontroll som görs av webbläsaren, inte av servern. Servern svarar villigt på alla anrop oavsett ursprung – det är webbläsaren som vägrar att låta JavaScript läsa svaret om CORS-headers inte tillåter det. Det är därför Postman och Thunder Client aldrig får CORS-fel: de är inte webbläsare och bryr sig inte om Same-Origin Policy.

De viktigaste CORS-headrarna som servern skickar i svaret är:

- `Access-Control-Allow-Origin` – Vilket eller vilka ursprung som tillåts. Antingen en specifik URL eller asterisken (\*) som tillåter alla ursprung.
- `Access-Control-Allow-Methods` – Vilka HTTP-metoder som tillåts (GET, POST, PUT, DELETE, etc.).
- `Access-Control-Allow-Headers` – Vilka headers klienten får skicka med (exempelvis Authorization för JWT).
- `Access-Control-Allow-Credentials` – Om cookies och autentiseringsinformation får skickas med (true/false).

## Konfigurera CORS i Express

I Express använder vi cors-paketet som vi installerade i vecka 1. Den enklaste konfigurationen tillåter alla ursprung:

```
const cors = require('cors');

app.use(cors()); // Tillåter ALLA ursprung
```

Det här fungerar i utveckling och är vad vi använt hittills, men det är inte säkert för produktion. Att tillåta alla ursprung innebär att vilken webbplats som helst kan göra anrop till vårt API från en användares webbläsare. För ett offentligt API kan det vara okej, men för ett API som hanterar känslig data eller autentiserade användare bör vi vara mer restriktiva.

En produktionsmässig CORS-konfiguration ser ofta ut så här:

```
const cors = require('cors');

const allowedOrigins = [
 'http://localhost:5173', // React dev
 'https://min-app.vercel.app', // Frontend i produktion
];

app.use(cors({
 origin: (origin, callback) => {
 // Tillåt anrop utan origin (t.ex. Postman, server-till-server)
 if (!origin) return callback(null, true);

 if (allowedOrigins.includes(origin)) {
 return callback(null, true);
 }

 return callback(new Error('Inte tillåten av CORS-policy'));
 },
 credentials: true, // Tillåt cookies/auth
 methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
 allowedHeaders: ['Content-Type', 'Authorization'],
}));
```

Notera att vi listar specifika ursprung istället för att använda asterisken. När vi behöver stödja credentials (cookies eller Authorization-header) kan vi inte använda asterisken alls – CORS-specifikationen tillåter inte den kombinationen. Det är en av många fallgropar som dyker upp i praktiken.

## Vanliga CORS-fel och vad de betyder

När något går fel med CORS kommer felmeddelandet i webbläsarens konsol. Här är de vanligaste och vad de betyder:

- "No 'Access-Control-Allow-Origin' header is present" – Servern svarar utan CORS-headers. Antingen är cors-middlwaren inte registrerad, eller så registrerades den efter routes (ordningen spelar roll!).

- "The 'Access-Control-Allow-Origin' header has a value '...' that is not equal to the supplied origin" – Servern tillåter ett specifikt ursprung men anropet kommer från ett annat. Vi har glömt att lägga till frontendens URL i listan över tillåtna ursprung.
- "Method ... is not allowed" – Vi har begränsat methods och anropet använder en metod som inte är med.
- "Request header ... is not allowed" – Klienten skickar en header (ofta Authorization) som inte finns i allowedHeaders.

Det är värt att veta att CORS-fel ofta visas i konsolen tillsammans med ett annat fel som ser ut som ett anslutningsfel. Det beror på att webbläsaren först gör preflight-anropet (som misslyckas) och sedan rapporterar att det "riktiga" anropet inte kunde göras. När vi felsöker bör vi alltid läsa den första meddelanderaden – det är där den verkliga orsaken finns.



## Proxy-konfiguration i utvecklingsmiljön

Ett alternativt sätt att hantera CORS i utvecklingsmiljön – och ett som ofta är bekvämare – är att använda en proxy i utvecklingsservern. Idén är enkel: istället för att React-appen på localhost:5173 anropar API:et på localhost:3000 direkt (vilket kräver CORS-konfiguration), låter vi React-appens dev-server vidarebefordra alla API-anrop till backenden. Webbläsaren ser bara att den anropar samma origin – utvecklingsservern – och Same-Origin Policy aktiveras aldrig.

Vite, som är det vanligaste verktyget för moderna React-projekt, har inbyggt stöd för detta. Vi konfigurerar det i vite.config.js (eller vite.config.ts):

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
 plugins: [react()],
 server: {
 proxy: {
 '/api': {
 target: 'http://localhost:3000',
 changeOrigin: true,
 },
 },
 },
});
```

Med den här konfigurationen kan React-appen anropa /api/books istället för http://localhost:3000/api/books. Vite ser att sökvägen börjar med /api och vidarebefordrar anropet till backenden. För webbläsaren är det ett same-origin-anrop, så CORS aktiveras inte. För backenden ser det ut som vilket anrop som helst.

Fördelen är att vi i frontend-koden alltid kan använda relativa URL:er, oavsett om vi är i utvecklingsläge eller produktionsläge. I utveckling vidarebefordras de via proxyn, och i produktion (om frontend och backend ligger på samma domän) hanteras de naturligt av webbservern.

Det här löser dock bara CORS i utvecklingsläge. När vi driftsätter måste frontend och backend antingen ligga på samma domän, eller så måste vi konfigurera CORS korrekt på backenden. Vi kommer i nästan alla fall att vilja använda samma CORS-konfiguration som i förra avsnittet i produktion, så proxyn ersätter inte CORS – den gör bara utvecklingsupplevelsen smidigare.

## Anropa API:et från React

När vi väl har proxy eller CORS på plats kan vi anropa API:et från React med fetch eller axios som vi gjort tidigare. En enkel hook för att hämta data kan se ut så här:

```
// hooks/useBooks.js
import { useState, useEffect } from 'react';

export function useBooks() {
 const [books, setBooks] = useState([]);
```

```
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
 fetch('/api/books')
 .then(res => {
 if (!res.ok) throw new Error('Kunde inte hämta böcker');
 return res.json();
 })
 .then(data => setBooks(data))
 .catch(err => setError(err.message))
 .finally(() => setLoading(false));
}, []);

return { books, loading, error };
}
```

En sak vi vill undvika är att ha API-URL:en hårdkodad i flera filer. Det är bra att samla alla anrop i en separat modul (api.js eller liknande) eller att använda en biblioteksabstraktion. Vid driftsättning behöver vi då bara ändra på ett ställe när URL:en till backenden ändras.

## JWT i fullstack – hela flödet

I vecka 7 implementerade vi JWT-baserad autentisering i backenden: vi skapade tokens vid inloggning och verifierade dem i en auth-middleware. Vi testade allt med Thunder Client. Nu ska vi se hur biten på klientsidan ser ut och hur frontend och backend tillsammans bildar ett komplett autentiseringsflöde.

### Flödet steg för steg

Det vanligaste mönstret för JWT-baserad autentisering ser ut så här:

7. Inloggning – React skickar e-post och lösenord till backenden via en POST till /auth/login. Backend returnerar en JWT som svar.
8. Lagring – React sparar token. De vanligaste alternativen är localStorage (lättast att jobba med, men sårbart för XSS) eller HTTP-only cookies (säkrare, men kräver mer konfiguration).
9. Autentiserade anrop – För varje anrop som kräver autentisering inkluderar React token i Authorization-headern: "Authorization: Bearer <token>".
10. Verifiering – Backend tar emot anropet, läser headern, verifierar token med vår JWT-hemlighet och – om allt stämmer – tillåter anropet att fortsätta till route handlaren.
11. Utloggning – React tar bort token från lagringen. Backend behöver inte involveras (med ren JWT-strategi har servern inget tillstånd att rensa).

Här är en grov skiss av en login-funktion på React-sidan:

```
// services/auth.js
export async function login(email, password) {
 const res = await fetch('/api/auth/login', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ email, password }),
 });

 if (!res.ok) {
 throw new Error('Inloggning misslyckades');
 }

 const { token, user } = await res.json();
 localStorage.setItem('token', token);
 return user;
}
```

Och en hjälpfunktion som lägger till token automatiskt i alla anrop:

```
// services/api.js
export async function apiFetch(url, options = {}) {
 const token = localStorage.getItem('token');

 const headers = {
 'Content-Type': 'application/json',
 ...options.headers,
```

```

};

if (token) {
 headers.Authorization = `Bearer ${token}`;
}

const res = await fetch(url, { ...options, headers });

// Hantera 401 globalt - logga ut användaren om token är ogiltig
if (res.status === 401) {
 localStorage.removeItem('token');
 window.location.href = '/login';
}

return res;
}

```

Det här är ett enkelt mönster som fungerar väl för vårt projekt. I större applikationer används ofta bibliotek som axios med interceptors, eller kraftfullare lösningar som React Query med en gemensam fetcher, men principen är densamma: token läses från lagringen och läggs till automatiskt.

## localStorage vs HTTP-only cookies – avvägningar

Var vi lagrar token är en säkerhetsavvägning som det är värt att förstå. localStorage är enklast: vi sätter och läser strängar som vilken nyckel som helst. Nackdelen är att JavaScript på sidan kan läsa localStorage, vilket innebär att om en angripare lyckas injicera JavaScript via en XSS-sårbarhet (Cross-Site Scripting) kan de stjäla token och utge sig för att vara användaren.

HTTP-only cookies är cookies som JavaScript inte kan läsa – endast webbläsaren och servern kan se dem. Det skyddar mot XSS, men introducerar andra utmaningar: cookies skickas automatiskt till servern, vilket öppnar för CSRF-attacker (Cross-Site Request Forgery), och de fungerar inte över olika domäner utan extra konfiguration. Det är därför många moderna API:er kombinerar HTTP-only cookies med CSRF-skydd.

För kursens projekt är localStorage tillräckligt och det vi rekommenderar att börja med. Vi behöver vara medvetna om begränsningarna men kan fokusera på att få helheten att fungera. I ett verkligt produktionssystem skulle vi göra en mer noggrann säkerhetsanalys och sannolikt landa i en cookie-baserad lösning, men det är utanför kursens omfattning.

## Skydda routes i React

Det räcker inte att skicka med token vid anrop – vi vill också kunna kontrollera vilka sidor i appen som kräver inloggning och dirigera om användaren till login-sidan om de inte är inloggade. Det här är klientsidesvalidering och får inte ersätta serverns auth-middleware, men det förbättrar användarupplevelsen avsevärt.

En enkel ProtectedRoute-komponent kan se ut så här:

```

// components/ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';

export function ProtectedRoute({ children }) {

```

```
const token = localStorage.getItem('token');

if (!token) {
 return <Navigate to="/login" replace />;
}

return children;
}
```

Vi måste komma ihåg: det här är bara UI-skydd. Backendens måste fortfarande verifiera token vid varje skyddat anrop. En användare som är skicklig kan ändra React-koden i webbläsaren och kringgå ProtectedRoute, men de kan inte kringgå backendens kontroll. Det är därför båda lagren är nödvändiga.

## Visualisera det kompletta dataflödet

Förra veckan introducerade vi sekvensdiagram och dokumentation av API-flöden. Den här veckan tar vi det vidare och tittar på det kompletta flödet i en fullstackapplikation – från användarklick i React, genom nätverket, in i Express, vidare till MongoDB, och tillbaka samma väg. Att kunna visualisera detta är inte bara en dokumentationsövning utan ett verktyg för att tänka klart om systemet.

### Sekvensdiagram för en login-funktion

Ett sekvensdiagram visar deltagare (vanligtvis tjänster eller komponenter) i kolumner, med tidens gång nedåt och meddelanden som pilar mellan kolumnerna. Här är hur ett login-flöde kan beskrivas i Mermaid (samma syntax vi använde i vecka 9):

```
sequenceDiagram
 participant U as Användare
 participant R as React-app
 participant E as Express-API
 participant D as MongoDB

 U->>R: Fyller i e-post och lösenord
 R->>E: POST /api/auth/login
 E->>D: User.findOne({ email })
 D-->>E: Användarobjekt med hash
 E->>E: bcrypt.compare(lösenord, hash)
 E->>E: jwt.sign(payload, secret)
 E-->>R: 200 OK { token, user }
 R->>R: localStorage.setItem('token')
 R-->>U: Visar startside
```

Sådana här diagram fyller flera funktioner: de tvingar oss att tänka noggrant på alla steg i flödet, de blir utmärkta i dokumentationen för andra utvecklare, och de fungerar som ett gemensamt språk när vi diskuterar systemet med teamet. När det dyker upp en bugg är ett sekvensdiagram ofta det första vi tar fram för att lokalisera vilket steg som gick fel.

### Komponentdiagram för hela systemet

Utöver sekvensdiagram är det värdefullt att ha ett översiktsdiagram som visar systemets komponenter och hur de hänger ihop. Det visar inte tidsförloppet utan strukturen: vilka delar finns, var körs de och vilka anslutningar finns mellan dem.

```
graph LR
 U[Användare]
 R[React-app
Vercel]
 E[Express-API
Render]
 D[MongoDB
Atlas]

 U -->|HTTPS| R
 R -->|HTTPS / JWT| E
 E -->|TLS / connection string| D
```

En sådan här bild ska kunna förklara systemet på 30 sekunder för någon som aldrig sett det förut. Var körs vad? Vad pratar med vad? Vilka säkerhetslager finns mellan komponenterna? När vi börjar dokumentera för boiler room-projektet är det här ett bra första diagram att skapa.

## Felsökning genom dataflödet

När vi felsöker en buggig fullstackfunktion följer vi flödet steg för steg och kontrollerar att varje steg gör rätt sak. Vi kallar det här för "divide and conquer" – vi delar problemet i mindre delar för att lokalisera det. En typisk felsökningssession kan se ut så här:

12. Klickar användaren på rätt knapp? Skriv ut i React-koden vid onClick.
13. Skickas anropet alls? Öppna nätverksfliken i webbläsarens utvecklarverktyg.
14. När anropet backenden? Kontrollera backend-loggen.
15. Tolkas datan rätt? Logga req.body i route handlern.
16. Hittas användaren i databasen? Logga resultatet av findOne.
17. Returneras rätt svar? Logga responsobjektet innan res.json().
18. Tar React emot svaret korrekt? Logga svaret i .then-blocken.

Det viktiga är att inte hoppa till slutsatser. Det är frestande att tro att man redan vet var felet är, men i praktiken är det nästan alltid ett annat steg än man trodde. Att gå systematiskt igenom flödet sparar tid på sikt även om det känns långsamt i stunden.

## Driftsättning – vad krävs för att köra i produktion?

Att driftsätta innebär att vi tar vår kod, lägger den på en server någonstans, och gör den tillgänglig på en URL som vem som helst kan anropa. Det låter enkelt – och tack vare moderna molntjänster är det enklare än någonsin – men det finns ett antal saker som måste på plats för att en applikation ska fungera korrekt i produktion.

### Skillnader mellan utveckling och produktion

När vi utvecklar lokalt har vi total kontroll över miljön. Vi kör en process i terminalen, vi har direkt tillgång till loggarna, vi kan starta om servern när som helst, och vi vet exakt vilka filer som finns var. I produktion är allt detta annorlunda.

De viktigaste skillnaderna att vara medveten om:

- Andra portar – I utveckling kör vi ofta på port 3000 eller 5173. I produktion är det vanligtvis port 80 (HTTP) eller 443 (HTTPS), och dessa hanteras av plattformens infrastruktur. Vi läser den port vi ska lyssna på från en miljövariabel (`process.env.PORT`).
- Inga utvecklarverktyg – I produktion finns ingen nodemon, inga lokala filer att redigera. Koden är paketerad och körs som en bakgrundsprocess. Att göra en ändring kräver en ny deploy.
- Annan databas – Vi byter från en utvecklingsdatabas till en produktionsdatabas. Connection-strängen ändras och databasen kan ha mer data och högre belastning.
- Andra hemligheter – JWT-secrets, API-nycklar och databaslösenord ska aldrig vara samma i utveckling och produktion. Produktionshemligheter är känsliga och måste hanteras separat.
- HTTPS istället för HTTP – I produktion vill vi alltid använda HTTPS. Det krypterar trafiken mellan klient och server och förhindrar avlyssning.
- Aggregerad loggning – I utveckling läser vi konsoloutput direkt. I produktion behöver vi ett sätt att samla in och söka i loggarna i efterhand. Plattformen vi använder ger oss ofta en webbaserad logg-vy.

### NODE\_ENV och miljöer

Det är konvention i Node.js-världen att ha en miljövariabel som heter `NODE_ENV`. Den brukar ha ett av tre värden: `development`, `test` eller `production`. Många bibliotek (inklusive Express) ändrar sitt beteende baserat på den – exempelvis visar Express stack traces i felmeddelanden i utveckling men inte i produktion.

Vi kan använda `NODE_ENV` i vår egen kod för att skilja på vad som sker i olika miljöer:

```
// app.js
const isProduction = process.env.NODE_ENV === 'production';

if (isProduction) {
 // Strikt CORS, säker loggning, etc.
 app.use(cors({ origin: process.env.FRONTEND_URL }));
} else {
 // Mer tillåtande i utveckling
 app.use(cors());
}
```



Plattformen vi driftsätter på sätter typiskt NODE\_ENV till production automatiskt. När vi kör lokalt är den vanligtvis odefinierad eller satt till development.

## Vad behöver vara på plats?

En checklista över vad vi behöver innan vi driftsätter:

- Koden i ett Git-repository – De flesta plattformar driftsätter direkt från GitHub, GitLab eller Bitbucket.
- Ett start-script i package.json – Plattformen kör npm start för att starta vår server. Vi behöver ett "start"-script som pekar på rätt fil.
- Port från miljövariabel – Vår server måste lyssna på den port plattformen anger via process.env.PORT, inte en hårdkodad port.
- Miljövariabler konfigurerade – Alla nycklar och hemligheter måste sättas i plattformens UI innan första deploy.
- En produktionsdatabas – MongoDB Atlas-databas separat från utvecklingsdatabasen, med korrekt connection-sträng.
- CORS-konfiguration som tillåter produktions-frontend – Vi måste lägga till frontendens produktions-URL i listan över tillåtna ursprung.

Vi går igenom var och en av dessa i de följande avsnitten.

## Molnmiljöer för Node.js-backend

Det finns ett brett spektrum av tjänster för att hosta en Node.js-backend, från enkla "git push och kör"-tjänster till fullständiga molnplattformar som AWS och Google Cloud. För kursen fokuserar vi på enklare alternativ som passar utbildningssyfte och mindre projekt: Render, Railway och Vercel.

### Render

Render är vår primära rekommendation. Den har en gratisnivå för webbtjänster, är enkel att komma igång med och stödjer Node.js-applikationer med minimal konfiguration. Render kopplar mot ett GitHub-repository, övervakar grenen vi väljer (typiskt main) och bygger om och driftsätter automatiskt vid varje push.

Stegen för att driftsätta vår backend till Render ser ut så här:

19. Skapa konto på render.com – det går att logga in med GitHub-kontot direkt.
20. Klicka "New +" och välj "Web Service".
21. Anslut GitHub-kontot och välj repositoryet med vår backend.
22. Konfigurera tjänsten: namn, region, gren (oftast main), build command (npm install) och start command (npm start).
23. Lägg till miljövariabler under Environment-fliken: MONGODB\_URI, JWT\_SECRET, FRONTEND\_URL och eventuella andra.
24. Klicka "Create Web Service". Render bygger och driftsätter, vilket tar några minuter. Vi får en URL som vår backend är tillgänglig på, exempelvis <https://min-backend.onrender.com>.

En sak att vara medveten om med Render:s gratisnivå är att tjänsten somnar efter en period av inaktivitet. Första anropet efter en sömn kan ta 30–60 sekunder att svara medan tjänsten startar. För kursprojekt är det inget problem, men för en produktionstjänst skulle vi vilja ha en betalplan.

### Railway

Railway är ett liknande alternativ till Render. Det har historiskt haft en gratisnivå, men den har förändrats över tiden – det är värt att kontrollera aktuella villkor. Railway är något snabbare att driftsätta än Render och har en intuitiv UI för att hantera flera tjänster i samma projekt.

Stegen liknar Renders: koppla GitHub, välj repository, konfigurera miljövariabler, deploya. Skillnaderna är mest kosmetiska från ett utvecklingsperspektiv. Båda fungerar utmärkt för kursprojektet.

### Vercel

Vercel är en plattform som primärt riktar sig mot frontend och serverless functions. Det går att köra Node.js-backender på Vercel via deras serverless-modell, men det fungerar lite annorlunda än ett traditionellt Express-API. Vercel kör ingen långlivad serverprocess utan startar en ny instans för varje anrop, vilket har implikationer för exempelvis databasanslutningar och bakgrundsjobb.

Vi rekommenderar att använda Vercel för frontend (det är det den är byggd för) och Render eller Railway för backend. Då får vi det bästa av båda världarna: snabb och gratis frontend-hosting, och en traditionell long-running Node.js-process för backenden.

## Att läsa porten från miljön

En av de absolut vanligaste anledningarna till att en första deploy misslyckas är att servern lyssnar på fel port. Plattformer som Render och Railway sätter en miljövariabel PORT med den port servern ska lyssna på. Om vi har hårdkodat port 3000 kommer plattformen att stänga av processen som "unhealthy" eftersom den inte svarar där den ska.

Lösningen är enkel och bör finnas i all kod från början:

```
// server.js
const app = require('./app');

// Använd PORT från miljön, fall tillbaka på 3000 i utveckling
const port = process.env.PORT || 3000;

app.listen(port, () => {
 console.log(`Servern lyssnar på port ${port}`);
});
```

Den här pattern ska vi alltid använda. Det fungerar lika bra lokalt (PORT är då undefined och vi faller tillbaka på 3000) som i produktion (där PORT sätts av plattformen).

## MongoDB Atlas i produktion

Vi har använt MongoDB Atlas sedan vecka 4 så grunderna är på plats. Men för att gå från en utvecklingsdatabas till en produktionsdatabas finns ett antal saker vi bör tänka på. Det handlar inte om att byta till en helt annan tjänst – Atlas räcker fint för kursprojektet – utan om att konfigurera den korrekt för produktionsanvändning.

### Separata databaser för olika miljöer

En av de viktigaste produktionsprinciperna är att aldrig använda samma databas för utveckling, test och produktion. Anledningen är uppenbar när vi tänker på det: en bugg i utvecklingskoden kan radera produktionsdata, och tester som rensar databasen kan av misstag radera riktiga användares konton.

I MongoDB Atlas skapar vi separata databaser inom samma kluster (det räcker bra för kursen), eller separata kluster om vi har resurser till det. Connection-strängen pekar på olika databaser:

```
.env.development
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/min-app-dev

.env.production (sätts i Render, inte i en fil!)
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/min-app-prod
```

Notera att `.env.production` aldrig ska finnas i Git-repositoriet. Produktionshemligheter ska ligga i plattformens (Render/Railway) miljövariabel-konfiguration, inte i kodbasen. Vi återkommer till det i nästa avsnitt.

### IP-whitelisting och nätverksåtkomst

MongoDB Atlas kräver att vi uttryckligen tillåter vilka IP-adresser som får ansluta till databasen. I utveckling har vi typiskt lagt till `0.0.0.0/0` (alla IP-adresser), men det är inte idealiskt för produktion. Vi vill helst bara tillåta IP-adresserna för våra produktionstjänster (Render, Railway).

Problemet är att Render och liknande tjänster inte alltid har fasta IP-adresser, och det blir då praktiskt svårt att whitelisting. Render erbjuder fasta IP:er som ett betalalternativ. För kursen är det acceptabelt att ha `0.0.0.0/0` även i produktion, men databasen ska då skyddas av starka lösenord och eventuellt tvåfaktorsautentisering på Atlas-kontot.

### Connection pooling

När Mongoose ansluter till MongoDB skapar den automatiskt en connection pool – en samling av återanvändbara anslutningar. Det är vad vi vill: att öppna en ny anslutning för varje request skulle vara extremt långsamt. Standardinställningarna är vettiga för de flesta fall, men det är värt att veta att de finns och kan justeras vid behov.

En sak som ofta orsakar problem första gången man driftsätter är att databasanslutningen inte etableras innan första anropet kommer in. Det är därför vi väntar på `connect()` innan vi börjar lyssna på port:

```
// server.js
const mongoose = require('mongoose');
const app = require('./app');
```

```
const port = process.env.PORT || 3000;

mongoose
 .connect(process.env.MONGODB_URI)
 .then(() => {
 console.log('Ansluten till MongoDB');
 app.listen(port, () => {
 console.log(`Servern lyssnar på port ${port}`);
 });
 })
 .catch(err => {
 console.error('Kunde inte ansluta till MongoDB:', err);
 process.exit(1);
 });
```

Att avsluta processen vid ett anslutningsfel (`process.exit(1)`) är ett medvetet val. Plattformen kommer att märka att processen avslutas och starta om den – ofta är ett anslutningsproblem temporärt och en omstart räcker. En server som körs utan databasanslutning är ändå värdelös.

## Miljövariabler i produktionsmiljö

I vecka 8 introducerade vi miljövariabler och .env-filer. Vi lärde oss att hemligheter aldrig ska committas till Git. Nu ska vi se hur miljövariabler hanteras i produktion – för där har vi ingen .env-fil att läsa från.

### Plattformens miljövariabel-konfiguration

Alla seriösa hostingplattformar har ett gränssnitt där vi kan sätta miljövariabler. På Render finns det under Environment-fliken på vår webbtjänst, på Railway under Variables, och i Vercel under Settings → Environment Variables. Vi anger nyckel och värde, sparar, och vid nästa deploy är variablerna tillgängliga via process.env i koden.

Det fina med detta är att hemligheterna aldrig finns i koden. De ligger krypterade hos plattformen och injiceras i processens miljö när tjänsten startar. Även om någon får tag i vår källkod kan de inte läsa hemligheterna – de finns helt enkelt inte där.

Typiska miljövariabler vi sätter för vårt API i produktion:

- NODE\_ENV=production – Aktiverar produktionsläge i Express och bibliotek.
- MONGODB\_URI – Connection-sträng till produktionsdatabasen.
- JWT\_SECRET – Hemlighet för att signera JWT. Ska vara en lång slumpmässig sträng, helt annorlunda från utvecklingsmiljön.
- FRONTEND\_URL – URL till frontenden, används i CORS-konfigurationen.
- PORT – Plattformen sätter denna automatiskt; vi ska inte sätta den manuellt.

### Validera miljövariabler vid uppstart

En vanlig och otäck buggklass är att en miljövariabel saknas i produktion utan att man märker det förrän applikationen kraschar vid första anropet. Det är därför en bra praxis att validera att alla nödvändiga miljövariabler finns vid uppstart, och avsluta processen direkt om något saknas:

```
// config/env.js
const requiredVars = [
 'MONGODB_URI',
 'JWT_SECRET',
 'FRONTEND_URL',
];

function validateEnv() {
 const missing = requiredVars.filter(v => !process.env[v]);

 if (missing.length > 0) {
 console.error(
 `Saknade miljövariabler: ${missing.join(', ')}\`
);
 process.exit(1);
 }
}

module.exports = { validateEnv };
```

Vi anropar `validateEnv()` tidigt i `server.js`, innan vi laddar resten av applikationen. Om något saknas kraschar vi direkt med ett tydligt felmeddelande, istället för att kraschka senare med en obegriplig stack trace.

## Generera säkra hemligheter

En vanlig nybörjarmiss är att använda korta eller enkla strängar som JWT-secret. "min-hemlighet" eller "abc123" är inte hemligheter – de kan gissas eller knäckas på sekunder. En riktig JWT-secret ska vara en lång slumpmässig sträng som ingen kan gissa.

Vi kan generera en bra hemlighet med Node.js inbyggda crypto-modul:

```
Kör i terminalen för att generera en 64-byte hex-sträng
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

Den genererade strängen är 128 tecken lång och praktiskt taget omöjlig att gissa. Vi använder olika sådana strängar i utveckling och produktion, och vi roterar dem regelbundet i en riktig produktionssetting (om vi misstänker att en hemlighet läckt ut).

## Grundläggande produktionsförberedelser

När vår backend väl ligger live är den exponerad för internet. Det betyder att den kommer att få anrop – och inte bara från våra användare. Bots scannar konstant nätet efter sårbara tjänster, och utan grundläggande skydd kan vår tjänst missbrukas, överbelastas eller exponera mer än vi tänkt oss. Vi går igenom de viktigaste skydden vi bör ha på plats.

### HTTPS

HTTPS – HTTP över TLS – krypterar all kommunikation mellan klient och server. Utan HTTPS skickas all trafik i klartext, inklusive lösenord, JWT-tokens och persondata, och kan läsas av vem som helst som lyckas avlyssna nätverket (exempelvis på ett öppet WiFi). I dagsläget är HTTPS obligatoriskt för alla seriösa applikationer.

Den goda nyheten är att vi inte behöver göra någonting för att få HTTPS i kursprojektet. Render, Railway och Vercel ger oss automatiskt HTTPS-certifikat (via Let's Encrypt) för alla tjänster vi driftsätter. URL:en blir `https://...` från start och vi kan lita på att trafiken är krypterad.

En sak vi däremot bör göra är att tvinga HTTPS – om någon anropar `http://` ska vi omdirigera till `https://`. På de flesta plattformar sker detta automatiskt, men det är värt att verifiera. Vi kan också lägga till en "Strict-Transport-Security"-header som talar om för webbläsaren att alltid använda HTTPS i framtiden.

### CORS-policy för produktion

Vi har redan pratat om CORS, men det är värt att upprepa: i produktion ska CORS-konfigurationen vara strikt. Endast specifika ursprung som vi vet ska få göra anrop ska tillåtas, inte asterisken. När boiler room-projektet driftsätts kommer vi att behöva uppdatera CORS-listan på backenden för att inkludera frontendens produktions-URL.

```
// app.js
const allowedOrigins = [
 process.env.FRONTEND_URL, // Produktions-frontend
 'http://localhost:5173', // Lokal utveckling
].filter(Boolean); // Filtrera bort undefined

app.use(cors({
 origin: allowedOrigins,
 credentials: true,
}));
```

### Rate limiting

Rate limiting är att begränsa hur många anrop en klient får göra under en viss tidsperiod. Det skyddar mot flera saker: brute force-attacker mot login-endpoint, oavsiktlig överbelastning från buggig klientkod, och rena DoS-försök (Denial of Service).

Express har ett populärt paket som heter `express-rate-limit` som är enkelt att använda:

```
npm install express-rate-limit
```

```
// app.js
```



```

const rateLimit = require('express-rate-limit');

// Generell limiter: 100 anrop per 15 minuter per IP
const generalLimiter = rateLimit({
 windowMs: 15 * 60 * 1000,
 max: 100,
 message: { error: 'För många anrop, vänta en stund' },
});

// Striktare limiter för login: 5 försök per 15 minuter per IP
const loginLimiter = rateLimit({
 windowMs: 15 * 60 * 1000,
 max: 5,
 message: { error: 'För många inloggningsförsök' },
});

app.use('/api/', generalLimiter);
app.use('/api/auth/login', loginLimiter);

```

Notera att vi har olika begränsningar för olika endpoints. login-endpoint är extra känslig eftersom den är mål för brute force-försök, så vi sätter en mycket striktare gräns där. Generella endpoints får vara mer generösa.

## Helmet – säkerhetsheaders

Helmet är ett middleware-paket som sätter ett antal HTTP-headers som hjälper till med vanliga säkerhetsproblem. Det är en "set and forget"-lösning som ger oss bättre default-säkerhet med en enda rad kod:

```

npm install helmet

// app.js
const helmet = require('helmet');

app.use(helmet());

```

Med helmet på plats sätter Express bland annat X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security och flera andra headers som tillsammans förhindrar en rad vanliga attacker. Det är en av de billigaste säkerhetsåtgärderna vi kan vidta och bör finnas i alla våra produktionstjänster.

## Felmeddelanden i produktion

Vi diskuterade i vecka 6 att vi inte ska exponera känslig information i felmeddelanden. I produktion är det extra viktigt – stack traces och databasfel kan avslöja systemets struktur för en angripare. Vår error handler ska skilja på utveckling och produktion:

```

// middleware/errorHandler.js
function errorHandler(err, req, res, next) {
 console.error(err); // Logga alltid internt

 const isProd = process.env.NODE_ENV === 'production';

```

```
res.status(err.status || 500).json({
 message: err.message || 'Ett internt fel uppstod',
 // Inkludera stack trace endast i utveckling
 ...(isProd ? {} : { stack: err.stack }),
});
}

module.exports = errorHandler;
```

Loggning av det fullständiga felet med `console.error` är fortfarande viktigt – vi behöver kunna felsöka problemen i produktion. Men det vi skickar tillbaka till klienten ska vara minimalt.

## Driftsättning i praktiken – steg för steg

Låt oss gå igenom hela driftsättningsflödet från början till slut, så att vi har en checklista att följa när vi själva ska göra det. Vi utgår från att vi har ett färdigt API som fungerar lokalt och en MongoDB Atlas-databas redo att användas.

### Förberedelser i koden

25. Säkerställ att server.js använder process.env.PORT med fallback till 3000.
26. Säkerställ att MongoDB-anslutningen läser från process.env.MONGODB\_URI.
27. Säkerställ att JWT-secret läses från process.env.JWT\_SECRET.
28. Lägg till en .env.example i repositoriet med alla miljövariabel-namn (utan värden) som dokumentation.
29. Verifiera att .env och node\_modules finns i .gitignore.
30. Lägg till ett "start"-script i package.json som startar servern: "start": "node src/server.js".
31. Pusha alla ändringar till GitHub.

### Driftsätta backend till Render

32. Logga in på render.com med GitHub-kontot.
33. Klicka "New +" → "Web Service".
34. Välj repositoriet och konfigurera: Name, Region (välj en nära användarna, exempelvis Frankfurt för Sverige), Branch (main), Build Command (npm install), Start Command (npm start).
35. Under "Environment Variables" lägger vi till: MONGODB\_URI, JWT\_SECRET, FRONTEND\_URL, NODE\_ENV (med värdet production).
36. Klicka "Create Web Service". Render bygger och driftsätter, vilket tar 2–5 minuter.
37. Notera URL:en (något i stil med https://min-backend.onrender.com) – den behövs när vi konfigurerar frontenden.

### Verifiera att det fungerar

När Render rapporterar att tjänsten är "Live" testar vi den med Thunder Client eller Postman. Anropa en publik endpoint (exempelvis GET /api/books) och kontrollera att vi får ett svar. Om vi har en healthcheck-endpoint (exempelvis GET / som svarar med en enkel OK-status) kan vi börja där.

Om vi får fel: kolla loggarna i Render-konsolen. De vanligaste problemen vid första deploy är saknade miljövariabler, fel start-script eller att MongoDB Atlas inte tillåter anslutningen från Render. Loggarna visar nästan alltid vad som är fel.

### Driftsätta frontend till Vercel

Frontend-driftsättningen sker separat. På Vercel:

38. Logga in på vercel.com med GitHub-kontot.
39. Klicka "New Project" och välj frontend-repositoriet.
40. Vercel detekterar Vite automatiskt och föreslår rätt build-konfiguration.
41. Lägg till miljövariabel VITE\_API\_URL med värdet av Render-backendens URL (https://min-backend.onrender.com).

42. Klicka "Deploy". Efter någon minut har vi en URL för frontenden.

När frontenden är driftsatt måste vi gå tillbaka till Render-backenden och uppdatera `FRONTEND_URL`-variabeln med Vercel-frontendens URL. Annars kommer CORS att blockera anropen från frontenden.

## Iterativ deployment

När allt är på plats är efterföljande deploys triviala. Vi gör en kodändring lokalt, committar och pushar till GitHub. Render och Vercel märker pushen och börjar automatiskt en ny deploy. Inom några minuter är ändringen live i produktion.

Det är ett av de stora fördelarna med moderna plattformar: en deploy är lika enkel som en git push. Vi kan iterera snabbt och ofta, vilket är värdefullt för utbildningssyfte och för riktiga projekt. Det är också anledningen till att kontinuerlig deployment är standard i moderna utvecklingsteam – det blir så enkelt att man helt enkelt gör det hela tiden.

## Checkpoint: Förberedelse inför kursvecka 10

Vecka 10 är veckan då allt vi byggt börjar bli en riktig produkt. Det betyder också att förberedelserna är mer praktiska än tidigare – det räcker inte att förstå koncepten, vi behöver också ha verktygen och kontona på plats för att hinna med övningen att driftsätta vårt API.

Kontrollera att ni har följande klart inför workshopen:

43. Ett konto på render.com (eller railway.app) – skapa det i förväg, helst kopplat till GitHub.
44. Ett konto på vercel.com – också kopplat till GitHub.
45. MongoDB Atlas-kontot är redo med ett kluster och en databasanvändare med starkt lösenord.
46. Ert API-projekt ligger i ett GitHub-repository, antingen privat eller publikt.
47. Verifiera att API:et startar korrekt lokalt med `npm start` (inte bara med `nodemon` eller `dev-script`).

Ta också med er dessa frågor och reflektioner till workshopen: Hur ser dataflödet ut i ert boiler room-projekt – kan ni rita det som ett sekvensdiagram? Vilka miljövariabler kommer ni att behöva i produktion? Vilka säkerhetsåtgärder är viktigast för just er applikation?

Om ni stöter på problem under förberedelserna, notera vilket steg som strular och vilket felmeddelande ni får. Vi löser det tillsammans under checkpointen så att alla kommer in i workshopen med en fungerande utgångspunkt.

## Sammanfattning

Under kursvecka 10 har vi tagit klivet från ett lokalt API till en driftsatt fullstackapplikation. Det är ett kliv som omfattar både teknisk konfiguration och ett nytt sätt att tänka på systemet. Vår backend är inte längre bara en process som körs på vår egen dator – den är en tjänst som lever på internet, är åtkomlig från hela världen och måste vara robust nog att hantera det.

Vi har förstått hur frontend och backend kopplas ihop i en modern fullstackarkitektur. De är två separata applikationer med separata kodbasen, separata driftsättningar och separata ansvarsområden, kopplade endast via det API-kontrakt vi själva designat. CORS är mekanismen som tillåter anrop mellan dem, och vi har lärt oss både att konfigurera det för utveckling och produktion samt att felsöka när det inte fungerar. Proxy-konfigurationen i Vite ger oss en smidig utvecklingsupplevelse där CORS inte är ett bekymmer lokalt.

JWT-flödet sträcker sig nu hela vägen från React till Express och tillbaka. Frontenden lagrar token, lägger till den i Authorization-headern vid varje skyddat anrop, och hanterar 401-svar genom att logga ut användaren. Vi har också tittat på avvägningen mellan localStorage och HTTP-only cookies, och förstått att klientsidesvalidering aldrig ersätter serverns auth-middleware – båda lagren behövs.

Visualisering av dataflödet är ett verktyg för både dokumentation och felsökning. Sekvensdiagram visar tidsförloppet i en specifik funktion, komponentdiagram visar systemets struktur, och båda är ovärderliga när vi felsöker eller förklarar systemet för någon annan. Att gå metodiskt genom flödet steg för steg är ofta snabbaste vägen till att hitta en bugg.

Driftsättning till molnmiljö är enklare än någonsin tack vare plattformar som Render, Railway och Vercel. Vi kopplar ett GitHub-repository, konfigurerar miljövariabler, och plattformen sköter resten: bygger, driftsätter och tillhandahåller en URL med automatisk HTTPS. Skillnaden mellan utveckling och produktion ligger främst i konfigurationen – andra portar, andra databaser, strängare CORS, och miljövariabler som hanteras av plattformen istället för en .env-fil. Den som vant sig att alltid läsa konfiguration från process.env har redan gjort den största delen av jobbet.

Produktionshärdning innebär att vi lägger till skydd mot vanliga attacker och missbruk. HTTPS ger vi oss gratis från plattformen, strikt CORS-policy begränsar vilka som får anropa API:et, rate limiting skyddar mot brute force och överbelastning, helmet sätter säkerhetsheaders, och vi exponerar inte stack traces i produktionsfel. Ingen av dessa åtgärder är komplicerade – men alla måste finnas på plats för att tjänsten ska vara redo att möta verkligheten.

Nästa vecka avslutar vi den teoretiska delen av kursen med dokumentation, teamprojektets finalisering och förberedelser inför examinationen. Vi tar med oss det vi gjort den här veckan – en driftsatt applikation – och ser till att den är väl dokumenterad så att andra kan förstå, granska och bygga vidare på den.

## Resurser för fördjupning

Följande resurser rekommenderas för de som vill fördjupa sina kunskaper inom fullstack-integration och driftsättning.

### Officiell dokumentation

MDN Web Docs: CORS – grundlig genomgång av Same-Origin Policy och CORS-mekanismen:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Render Documentation – officiell guide för att driftsätta Node.js-applikationer:

<https://render.com/docs>

Vercel Documentation – officiell guide för att driftsätta frontend-applikationer:

<https://vercel.com/docs>

MongoDB Atlas Documentation – guide för att konfigurera Atlas i produktion:

<https://www.mongodb.com/docs/atlas>

Vite – proxy-konfiguration i utvecklingsservern:

<https://vite.dev/config/server-options.html#server-proxy>

### Rekommenderade guider

"The Twelve-Factor App" är en uppsättning principer för moderna molnapplikationer som har blivit standard i industrin. Avsnitten om konfiguration, miljöer och processer är direkt tillämpliga på det vi gör den här veckan:

<https://12factor.net>

"Express Production Best Practices" – officiell guide från Express om hur man förbereder en applikation för produktion. Täcker säkerhet, prestanda och driftsättning:

<https://expressjs.com/en/advanced/best-practice-security.html>

"OWASP Cheat Sheet Series" innehåller praktiska checklistor för säkerhet i produktion. Cheat sheets för REST Security och Node.js är särskilt relevanta:

<https://cheatsheetseries.owasp.org>

## Videomaterial

"CORS in 100 Seconds" och längre djupdykningar i CORS finns i flera välgjorda format på YouTube. För den som vill förstå CORS på djupet rekommenderas en längre genomgång (15-30 minuter) snarare än en snabb summering.

"Deploy Node.js App to Render" – sök på YouTube efter en aktuell genomgång (2024 eller senare) som visar hela processen visuellt. Att se någon annan göra stegen kan vara värdefullt komplement till denna text.

"Mermaid Sequence Diagrams" – kort introduktion till syntaxen vi använder för sekvensdiagram. Praktisk för boiler room-projektets dokumentation.



# **Studiematerial**

## **Dokumentation, teamprojekt och projektfinalisering**

Kursvecka 11

## Introduktion

Vi har kommit långt. Tio veckor in i kursen har vi gått från att aldrig ha rört serversidan till att ha byggt riktiga, stabila API:er som hanterar autentisering, validering, felhantering, rollbaserad åtkomst och dataskydd. Vi har kopplat ihop vårt API med en MongoDB-databas, skrivit tester och driftsatt vår applikation till en molnmiljö. Det är en imponerande resa, och vi gör klokt i att ta ett ögonblick och konstatera att vi nu har de tekniska byggstenarna på plats.

Den här veckan handlar inte om att lära oss en ny teknik. Den handlar om att göra det vi redan byggt redo att lämnas över. Det är ett perspektivskifte som är värt att markera tydligt: hittills har vi skrivit kod för oss själva och för utbildaren. Från och med nu skriver vi kod som ska kunna förstås, underhållas och vidareutvecklas av andra utvecklare som aldrig har sett vårt projekt förut. Det är en helt annan kvalitetsnivå.

Den här övergången är inte bara en akademisk övning. När vi börjar arbeta som utvecklare kommer ingen att be oss skriva ett system från grunden i ensamhet. Vi kommer att gå in i befintliga kodbasen, läsa dokumentation, ställa frågor till kollegor och själva skriva kod som andra ska kunna ta över. Förmågan att dokumentera tydligt, samarbeta i Git, granska andras kod och skriva överlämningar är minst lika viktig som förmågan att skriva en korrekt route-handler. I många fall är den till och med viktigare.

Vi lägger också grunden inför examinationen i vecka 12. Det är ett tillfälle att stanna upp, se över helheten i vårt boiler room-projekt och säkerställa att alla bitar finns på plats. Vi kommer att gå igenom vad som examineras och hur, och vi tar tid att repetera de centrala koncepten från kursens första nio veckor så att kunskapstestet inte blir en överraskning.

Slutligen är det viktigt att vi ser den här veckan som en möjlighet att lyfta projektets kvalitet ett snäpp till. Den dokumentation vi skriver nu, den kodgranskning vi utför och den disciplin vi visar i Git-workflowet är alla saker som syns i slutprodukten. Ett välorganiserat projekt med tydliga README-filer, dokumenterad arkitektur och välstrukturerad commit-historik signalerar professionalism. Det är så vi vill att vårt projekt ska upplevas av den som öppnar det för första gången.

## Dokumentation som hantverk

Dokumentation har en oförtjänt dålig rykte. Många utvecklare uppfattar den som något som måste skrivas i efterhand, något som ingen läser och något som blir inaktuellt nästan omedelbart. Den synen är förståelig men felaktig. Bra dokumentation är inte ett separat steg som kommer efter koden – det är en integrerad del av det vi levererar. När vi inte dokumenterar lämnar vi över ett halvfärdigt arbete, oavsett hur välskriven koden är.

Det är värt att stanna vid frågan: vem dokumenterar vi för? Svaret är inte 'för oss själva i framtiden' även om det är en biprodukt. Vi dokumenterar för andra utvecklare som ska arbeta med systemet, för frontend-team som ska konsumera vårt API, för säkerhetsansvariga som behöver förstå dataflödet, för nyanställda som ska förstå arkitekturen, och för oss själva när vi återvänder till projektet om sex månader och inte längre minns varför vi byggt något på ett visst sätt. Varje av dessa läsare har olika behov, och bra dokumentation tjänar dem alla utan att överbelasta någon.

### Vad utgör bra dokumentation?

Det är frestande att tro att mer dokumentation alltid är bättre. Det är fel. Dokumentation som är för omfattande blir lika oanvändbar som dokumentation som saknas helt – ingen läser den, och de bitar som faktiskt är viktiga drunknar i bruset. Bra dokumentation karaktäriseras av flera egenskaper:

- Den är aktuell. Föråldrad dokumentation är värre än ingen dokumentation alls eftersom den lurar läsaren. Vi måste därför hålla dokumentation och kod synkroniserade.
- Den är skiktad. En läsare ska kunna förstå projektets syfte på trettio sekunder, dess uppbyggnad på fem minuter och detaljerna när hen behöver dem. Allt får inte komma på en gång.
- Den är konkret. Generella formuleringar typ 'systemet hanterar användardata på ett säkert sätt' säger ingenting. Vi behöver konkreta beskrivningar: vilka endpoints, vilka data, vilka mekanismer.
- Den är ärlig. Dokumentationen ska beskriva systemet som det faktiskt är, inte som vi önskade att det var. Kända begränsningar och tekniska skulder ska nämnas explicit.
- Den är användarcentrerad. Den utgår från vad läsaren behöver veta för att kunna agera, inte från vad vi själva tycker är intressant att berätta.

En ofta förbisedd aspekt är att dokumentation och kod inte är två separata produkter utan en sammanhängande helhet. När vi väljer beskrivande variabelnamn, skriver tydliga commitmeddelanden, namnger filer logiskt och strukturerar mappar konsekvent så dokumenterar vi också. Den 'bästa' dokumentationen är ofta den som inte behöver skrivas eftersom koden i sig själv är tillräckligt tydlig för att förklara sig.

### Tre nivåer av dokumentation

För vårt boiler room-projekt arbetar vi med tre kompletterande nivåer av dokumentation. Det är nyttigt att skilja mellan dem eftersom de tjänar olika syften och har olika läsare:

- Projektnivå – README, arkitekturdiagram och installationsinstruktioner. Riktat sig till någon som möter projektet för första gången och behöver förstå vad det är, hur det körs och hur det är strukturerat.
- API-nivå – Endpoint-dokumentation, autentiseringsflöden och dataflödesdiagram. Riktat sig till frontend-utvecklare och andra som ska konsumera API:et.

- Kod-nivå – Kommentarer i koden där det faktiskt behövs, JSDoc-blocks för publika funktioner och meningsfulla namn. Riktar sig till någon som behöver förändra eller felsöka en specifik del av systemet.

Varje nivå ska innehålla det som är mest relevant för dess läsare. README ska inte innehålla detaljer om varje endpoint – det hör hemma i API-dokumentationen. Endpoint-dokumentationen ska inte upprepa hur projektet körs lokalt – det hör hemma i README. Vi separerar därför inte bara av estetiska skäl, utan för att varje typ av läsare ska hitta det den söker utan att behöva sila.

## README – projektets ansikte utåt

När någon öppnar vårt repository på GitHub är det första de ser README-filen. Det är inte en överdrift att säga att README-filen är det viktigaste enskilda dokumentet i ett projekt. Den avgör om läsaren omedelbart förstår vad projektet är, om de kan komma igång på fem minuter eller om de ger upp och tittar på något annat. Vi ska därför ta vår README på största allvar och inte se den som ett pliktskyldigt dokument som klargör pannan.

En bra README gör fem saker, i ungefär den här ordningen: den förklarar vad projektet är på en mening, motiverar varför det finns, beskriver hur man installerar och kör det, dokumenterar de viktigaste funktionerna och pekar vidare till mer detaljerad dokumentation. Det är allt. README är inte en användarmanual, inte en API-referens och inte en tutorial – det är en startpunkt.

### Struktur för README

Vi rekommenderar följande struktur för vårt boiler room-projekts README. Strukturen är inte huggen i sten, men varje sektion löser ett konkret problem för läsaren och bör därför vara med:

- Projekttitel och kort beskrivning – En till två meningar som svarar på 'vad är det här?'.
- Funktioner – En punktlista över de viktigaste sakerna systemet kan göra. Hjälper läsaren att snabbt avgöra om projektet är relevant för dem.
- Teknisk stack – Vilka tekniker används? Node.js-version, Express, MongoDB, etc. Hjälper läsaren att avgöra om de har rätt förkunskaper.
- Förutsättningar – Vad måste finnas installerat innan man kan köra projektet? Node.js-version, MongoDB-konto, eventuella API-nycklar.
- Installation – Steg-för-steg-instruktioner. Bör vara så tydliga att en utomstående utvecklare kan följa dem utan att gissa.
- Miljövariabler – En lista över alla nödvändiga miljövariabler med en kort beskrivning av varje. Inkludera ALDRIG riktiga värden – bara namn och syfte.
- Användning – Hur man kör projektet (utvecklingsläge, produktion, tester). Inklusive de viktigaste npm-skripten.
- API-översikt – En kort lista över de viktigaste endpoint-grupperna med länk till mer detaljerad API-dokumentation.
- Mappstruktur – En översikt av hur projektet är organiserat. Hjälper läsaren att snabbt förstå var saker finns.
- Tester – Hur man kör testsviten och vad den täcker.
- Författare och licens – Vem har byggt det och under vilka villkor får det användas. För boiler room-projektet listar vi gruppmedlemmarna här.

### Exempel på en README-skelett

Här är ett konkret exempel på hur de inledande sektionerna i en README kan se ut. Markdown-syntaxen är enkel, men intrycket den ger är professionellt:

```
Recipes API
```

```
Ett REST API för att hantera receptbibliotek med autentisering
och rollbaserad åtkomst. Byggt med Node.js, Express och MongoDB.
```

```
Funktioner
```

- CRUD för recept med ägarskap per användare
- JWT-baserad autentisering
- Rollbaserad åtkomst (user, admin)
- Inputvalidering med Zod
- Strukturerad loggning med pino

#### ## Teknisk stack

- Node.js 20 LTS
- Express 4.x
- MongoDB med Mongoose
- TypeScript
- Jest och Supertest för tester

#### ## Installation

1. Kлона repot: ``git clone <repo-url>``
2. Installera beroenden: ``npm install``
3. Skapa en .env-fil baserad på .env.example
4. Starta utvecklingsservern: ``npm run dev``

Det vi vill undvika är README-filer som antingen är tomma (bara titel och en mening) eller överlastade (innehåller hela kodbasen som dokumentation). Sikta på att en kompetent utvecklare ska kunna gå från att kлона repot till att ha en körande lokal miljö på max tio minuter genom att enbart följa README. Om det tar längre tid är README ofullständigt eller missvisande.

## Miljövariabler – dokumentera utan att läcka

En särskilt känslig del av README är dokumentationen av miljövariabler. Vi behöver berätta för läsaren vilka variabler som krävs och vad de gör, men vi får aldrig commit:a riktiga värden till repot. Lösningen är välkänd: vi har en .env.example-fil i versionshanteringen som listar alla nödvändiga variabler med tomma värden eller exempel, och en .env-fil som är listad i .gitignore och innehåller de faktiska värdena lokalt.

Ett exempel på en .env.example-fil för vårt projekt:

```
Servern
PORT=3000
NODE_ENV=development

Databas
MONGODB_URI=mongodb://localhost:27017/recipes_dev

Autentisering
JWT_SECRET=change_me_in_production
JWT_EXPIRES_IN=7d

Loggning
LOG_LEVEL=info
```

Notera att vi för JWT\_SECRET inte sätter värdet 'change\_me\_in\_production' av en slump. Det är ett exempelvärde som tydligt signalerar att det inte ska användas i skarp drift. Vi inkluderar också en kommentar i README som påminner om att alla hemligheter i produktionsmiljön ska genereras med kryptografiskt säkra metoder, exempelvis med node -e "console.log(require('crypto').randomBytes(64).toString('hex'))".

## API-dokumentation

När en frontend-utvecklare ska konsumera vårt API kommer de att läsa API-dokumentationen mer än någon annan del av projektet. Det är där de slår upp endpoints, kontrollerar förväntade request-format, hittar exempelresponser och förstår autentiseringsflöden. Vår API-dokumentation är därför ett av projektets viktigaste leveranser, och den förtjänar samma omsorg som vår kod.

I vecka 9 introducerade vi tanken på API-dokumentation och nämnde Swagger/OpenAPI som standard. Den här veckan tar vi det vidare och pratar om vad en god API-dokumentation faktiskt ska innehålla, oavsett om vi väljer att skriva den för hand i Markdown eller generera den från en OpenAPI-specifikation.

### Vad varje endpoint behöver dokumenteras med

För varje publik endpoint i vårt API behöver vi dokumentera ett antal saker. Om någon av dessa saknas blir konsumenten tvungen att gissa eller läsa vår källkod – båda är dåliga alternativ. Minimumet vi vill ha med är:

- HTTP-metod och sökväg – exempelvis 'POST /api/recipes'. Detta står först.
- Sammanfattning – en mening som beskriver vad endpointen gör och för vem den är tänkt.
- Autentisering – krävs det en JWT-token? Krävs en specifik roll? Detta måste vara glasklart.
- Path-parametrar – om sökvägen innehåller variabler (t.ex. /:id), beskriv vad de är och vilken typ.
- Query-parametrar – för GET-endpoints är det vanligt med parametrar för filtrering, sortering eller paginering.
- Request body – för POST/PUT/PATCH, beskriv vilka fält som förväntas, vilka är obligatoriska och vilka regler de följer.
- Lyckat svar – statuskod (200, 201, 204) och ett exempel på svar-body.
- Möjliga fel – vilka statuskoder kan returneras under olika omständigheter (400, 401, 403, 404, 500) och vad orsakar dem.

### Exempel: dokumentera en endpoint

För att göra det här konkret tittar vi på hur en enskild endpoint kan dokumenteras i Markdown. Tänk dig en POST-endpoint för att skapa ett nytt recept:

```
POST /api/recipes
```

Skapar ett nytt recept. Den autentiserade användaren blir ägare.

**\*\*Autentisering\*\*:** Krävs (JWT-token i Authorization-header)

**\*\*Roll\*\*:** user eller admin

**\*\*Request body\*\*** (JSON):

```
```json
{
  "title": "Pasta carbonara",
  "servings": 4,
  "ingredients": ["spaghetti", "ägg", "pancetta"],
}
```



```

    "instructions": "Koka pastan ..."
  }
  ...

  **Validering**:
  - title: sträng, 3-100 tecken, obligatorisk
  - servings: heltal, 1-50, obligatorisk
  - ingredients: array av strängar, minst 1, obligatorisk
  - instructions: sträng, max 5000 tecken, obligatorisk

  **Lyckat svar**: 201 Created

  ```json
 {
 "id": "6543abc...",
 "title": "Pasta carbonara",
 "createdAt": "2025-11-12T10:30:00Z"
 }
 ...

 Möjliga fel:
 - 400 Bad Request - ogiltig request body
 - 401 Unauthorized - saknad eller ogiltig token
 - 403 Forbidden - användaren saknar behörighet

```

Ett exempel som detta ger en frontend-utvecklare allt som behövs för att implementera anropet utan att behöva fråga oss eller läsa vår kod. Notera att vi inte bara listar fält utan också anger valideringsregler – det gör att frontend kan implementera samma regler i sina formulär och ge omedelbar återkoppling till användaren innan anropet ens skickas.

## OpenAPI och Swagger

För större projekt blir handskriven Markdown-dokumentation snabbt svår att underhålla. OpenAPI (tidigare känt som Swagger) är en specifikation för att beskriva REST API:er i ett strukturerat format, oftast YAML eller JSON. Genom att skriva en OpenAPI-fil får vi flera fördelar samtidigt: dokumentationen kan genereras automatiskt som en interaktiv webbsida via verktyg som Swagger UI, klienter kan generera kod i olika språk, och dokumentationen blir maskinläsbar.

För vårt boiler room-projekt är ett rimligt mål att antingen skriva en OpenAPI-spec eller skriva en grundlig Markdown-dokumentation. Båda är acceptabla. Det viktiga är att den är komplett – alla publika endpoints ska finnas med, autentiseringen ska vara förklarad, och en frontend-utvecklare ska kunna komma igång utan att läsa källkoden.

Ett pragmatiskt mellanting är att använda paketet `swagger-jsdoc` som tillåter oss att skriva OpenAPI-kommentarer direkt över våra route-handlers, och sedan generera dokumentationen automatiskt. Det gör det lättare att hålla dokumentationen synkroniserad med koden eftersom den bor i samma fil.

## Dokumentera autentiseringsflödet

En specialfall som förtjänar egen uppmärksamhet är autentiseringsflödet. Det är en av de delar av API:et som frontend måste implementera först, och det är också den del där fel får

störst konsekvenser. Vi vill därför ha ett separat avsnitt i dokumentationen som tydligt beskriver hur autentisering fungerar:

- Hur registrerar man en ny användare? Vilken endpoint, vilka fält och vad händer i bakgrunden (ex. lösenord hashas med bcrypt).
- Hur loggar man in? Vilken endpoint, vilken response (token + ev. användarinfo), vad är token-livslängden.
- Hur skickas token i efterföljande anrop? Authorization-header med Bearer-prefix är standarden vi använder.
- Vad händer när token går ut? Returnerar vi 401, och hur ska klienten hantera det?
- Hur fungerar rollbaserad åtkomst? Vilka roller finns, hur tilldelas de och vilka endpoints kräver vilka roller?

Det är frestande att tänka att frontend-utvecklaren 'redan vet hur JWT fungerar', men vårt jobb är att dokumentera vårt API:s specifika implementation. Token-livslängden, refresh-strategin och felhanteringen är våra val, och de behöver kommuniceras explicit.

## Dokumentera systemarkitekturen

Utöver dokumentation per endpoint behöver vi också beskriva hur systemet hänger ihop som helhet. Det är arkitekturdokumentation, och dess syfte är att ge en läsare en mental modell av hur de olika delarna interagerar utan att de behöver dyka in i kodbasen. För ett fullstacksystem som vårt boiler room-projekt innebär det att vi beskriver hur frontend, backend och databas samverkar, vilka externa tjänster som är inblandade och hur data flödar mellan dem.

En vanlig invändning är att arkitekturdiagram blir inaktuella så fort de är ritade. Det är delvis sant, men slutsatsen ska inte vara att vi slutar rita diagram – det ska vara att vi väljer rätt detaljnivå. Diagram på hög abstraktionsnivå (vilka tjänster finns, hur kommunicerar de) ändras sällan. Diagram på låg detaljnivå (varje funktion och variabel) ändras ständigt. Vi siktar på den höga nivån.

### Mermaid – diagram som kod

Vi introducerade Mermaid kort i vecka 9. Det är ett textbaserat sätt att beskriva diagram som GitHub, GitLab och de flesta moderna dokumentationsverktyg renderar automatiskt. Det betyder att vi kan ha våra arkitekturdiagram direkt i vår Markdown-dokumentation, versionshanterade tillsammans med koden. När arkitekturen ändras uppdaterar vi diagrammet i samma commit som koden.

Här är ett exempel på ett enkelt arkitekturdiagram för vårt boiler room-projekt skrivet i Mermaid-syntax. Det visar de tre huvudkomponenterna och hur de kommunicerar:

```
```mermaid
graph LR
  Client[React-frontend] -->|HTTPS / JSON| API[Express API]
  API -->|Mongoose| DB[(MongoDB Atlas)]
  API -->|pino| Logs[Logg-aggregator]
```
```

Det här diagrammet är lågdetaljerat med flit. Det säger inget om enskilda endpoints, ingenting om databasmodeller och ingenting om vilka middleware som körs. Det berättar däremot något viktigt: tre boxar och tre pilar. Det är vårt system. Den som läser kan på ett ögonkast förstå strukturen och därefter dyka in i detaljerna.

### Komponentdiagram för backend

För backend i sig är det ofta värdefullt att ha ett mer detaljerat diagram som visar hur lagren i applikationen hänger ihop. Det är en bild som är särskilt användbar för en ny utvecklare som ska göra sin första ändring i kodbasen och behöver veta var saker hör hemma.

```
```mermaid
graph TB
  Routes[Routes] --> Controllers[Controllers]
  Controllers --> Services[Services]
  Services --> Models[Mongoose Models]
  Models --> DB[(MongoDB)]

  Auth[Auth Middleware] --> Routes
  Validation[Zod Validation Middleware] --> Routes
  ErrorHandler[Error Handler] --> Routes
```
```

...

Det vi vinner på ett sådant diagram är att vi visualiserar separation of concerns – det vi pratat om sedan vecka 2. Routes hanterar HTTP-detaljer och delegerar till controllers. Controllers tar hand om koordinering och delegerar till services. Services innehåller affärslogik och pratar med models. Models pratar med databasen. Middleware-paret (auth och validation) körs före routes. När någon ny tittar på vår kod är detta kartan de använder för att navigera.

## Sekvensdiagram för API-anrop

Sekvensdiagram visar hur olika delar av systemet samverkar i tid – vad händer steg för steg när ett anrop kommer in? De är särskilt användbara för att dokumentera komplexa flöden som autentisering, eftersom de gör det otvetydigt vad som ska hända i vilken ordning.

Här är ett exempel på ett sekvensdiagram för inloggning:

```

```mermaid
sequenceDiagram
    participant C as Client
    participant A as API
    participant DB as MongoDB

    C->>A: POST /api/auth/login (email, password)
    A->>A: Validera input med Zod
    A->>DB: Hitta användare med email
    DB-->>A: Användardokument med hashat lösenord
    A->>A: Jämför lösenord med bcrypt.compare
    A->>A: Skapa JWT-token
    A-->>C: 200 OK med token

    Note over C: Spara token i state

    C->>A: GET /api/recipes (Bearer token)
    A->>A: Verifiera token i auth-middleware
    A->>DB: Hämta recept för användaren
    DB-->>A: Lista över recept
    A-->>C: 200 OK med recept
  ```
 ...

```

Diagrammet visar det vi behöver veta i en blick: vilka anrop sker, vilka parametrar skickas med, vilka steg sker internt och vilka svar returneras. Det är dokumentation som backend-, frontend- och säkerhetsansvariga alla kan luta sig mot. Om någon undrar 'när verifieras token?' eller 'när hashas lösenord?' så är svaret rakt fram.

## Andra verktyg: draw.io och Excalidraw

Mermaid är utmärkt för diagram som är tillräckligt enkla att uttrycka i text, men för mer komplexa illustrationer kan det bli klumpigt. För större bilder, exempelvis ett komplett systemdiagram med flera mikrotjänster, externa API:er och datakällor, är verktyg som draw.io (numera diagrams.net) eller Excalidraw bättre val. De producerar bilder som vi kan exportera som PNG eller SVG och inkludera i vår dokumentation.

Excalidraw har en handritad estetik som ofta gör arkitekturdiagram lätta att förstå – de signalerar 'detta är en grov skiss av strukturen', vilket är ärlig kommunikation jämfört med ett ultraprecist tekniskt diagram som ger ett falskt intryck av att alla detaljer redan är genomtänkta. För kursens projekt är båda valen helt acceptabla. Det viktiga är att diagrammen finns med i dokumentationen och att de är läsbara.

## Överlämningsdokumentation för teamprojekt

Boiler room-projektet är ett teamprojekt och ska, i den verkliga världen, ha kunnat lämnas över till en annan grupp utvecklare som tar vid där vi slutar. Det ställer krav som går utöver standard-API-dokumentation. En överlämning handlar inte bara om vad systemet är, utan också om vad som hänt under utvecklingens gång, vad som inte är klart och vad som är värt att veta för att inte snubbla på okända ledningar.

Den klassiska liknelsen är att vi bygger ett hus och därefter lämnar nycklarna till någon annan. Om vi bara lämnar nycklarna utan att säga något missar mottagaren att källaren ibland tar in vatten vid ihållande regn, att fönstret i sovrummet inte går att öppna och att de gröna ledningarna i pannrummet faktiskt är från en gammal installation. Bra överlämning är att gå igenom huset tillsammans med mottagaren och peka ut allt som inte är uppenbart.

### Vad ska finnas med i en överlämning?

En överlämningsdokumentation byggs ovanpå README, API-dokumentation och arkitekturdiagram. Den lägger till information som är specifik för just överlämningen – sådant som inte är permanent dokumentation av systemet utan en ögonblicksbild av läget när vi släpper taget. Den brukar innehålla:

- Status – vad är färdigt, vad är delvis klart och vad är oimplementerat? Var ärlig.
- Kända problem – buggar, prestandaproblem eller designval som ännu inte är åtgärdade men som vi vet om.
- Tekniska skulder – ställen där vi tagit en genväg som behöver lösas senare. Beskriv vad som är tillfälligt och vad som är meningen att det ska bli.
- Roadmap – vad är logiskt nästa steg? Vilka funktioner var planerade men inte hann med?
- Beslutslogg (ADR) – för viktiga arkitekturella beslut, varför valde vi MongoDB framför PostgreSQL? Varför valde vi JWT framför sessions? Sådant är värdefullt att förstå för den som kommer in senare.
- Tredjepartstjänster – vilka externa tjänster är systemet beroende av (MongoDB Atlas, Render, etc.)? Vem äger kontona och hur kommer mottagaren åt dem?
- Kontakt – vem i teamet kan svara på frågor om vilka delar? Det är värdefullt under den första tiden efter överlämning.

### Architecture Decision Records (ADR)

Ett begrepp som är värt att känna till är Architecture Decision Records, ADR. En ADR är ett kort dokument (gärna max en sida) som dokumenterar ett enda viktigt arkitekturellt beslut. Strukturen är standardiserad och består typiskt av: kontext (vad är problemet vi löser?), beslut (vad valde vi?), konsekvenser (vad innebär valet?) och alternativ (vad övervägde vi också?).

För vårt boiler room-projekt behöver vi inte skriva ADR för varje litet beslut, men det är värt att skriva några för de stora valen. Exempel på beslut som förtjänar en ADR i vårt projekt: val av databas, val av autentiseringsmetod, val av valideringsbibliotek, val av loggningsbibliotek. Att skriva ned varför vi gjort dessa val, även kortfattat, är guld värt för framtida utvecklare.

Ett ADR-exempel kan se ut ungefär så här:

```
ADR-001: Använd MongoDB som primär databas
```

Status: Antaget

Datum: 2025-11-01

## ## Kontext

Projektet behöver en databas som hanterar recept, användare och kommentarer. Datamängden är blygsam och scheman förväntas utvecklas under projektets gång.

## ## Beslut

Vi använder MongoDB Atlas via Mongoose.

## ## Konsekvenser

- Snabb iteration med flexibla scheman
- Tar inte nytta av relationsfunktioner som JOIN
- Kräver disciplinerad användning av populate

## ## Alternativ

PostgreSQL övervägdes. Avfärdades pga kortare startsträcka för teamet med dokumentdatabas.

En ADR är inte ämnad att uppdateras när läget ändras. Om vi senare byter från MongoDB till PostgreSQL skriver vi en ny ADR (ADR-007 t.ex.) som markerar ADR-001 som 'ersatt'. Det innebär att ADR-mappen blir en historisk logg över hur projektet har resonerat över tid – något som är mycket värdefullt för en ny utvecklare som vill förstå inte bara hur systemet är, utan hur det blivit som det är.

## GDPR-hänsyn i dokumentationen

I vecka 9 introducerade vi GDPR och dess krav på dataminimering, säker loggning och rättigheterna för registrerade. Den här veckan tar vi det vidare från ett dokumentationsperspektiv. GDPR är inte bara en teknisk angelägenhet – det är också ett dokumentationskrav. Den som driver en tjänst som hanterar personuppgifter måste kunna redogöra för vilken data som lagras, varför den lagras och hur den skyddas. Den dokumentationen är något vi som utvecklare bidrar till.

När vi dokumenterar GDPR-aspekter i vårt projekt är vårt primära mål att ge en läsare – kanske en projektledare, en jurist eller en dataskyddsombud – möjlighet att besvara frågor som: 'Lagrar vi personuppgifter i denna applikation? Vilka? Varför? Hur länge? Vem har åtkomst?'. Om svaret kräver att vederbörande läser källkoden har vi misslyckats.

### Dataregister – vad lagrar vi?

En central del av GDPR-dokumentationen är ett dataregister – en lista över alla personuppgifter vi behandlar. För vårt boiler room-projekt är registret ofta ganska kort, men det ska ändå finnas. För varje typ av personuppgift dokumenterar vi:

- Vilken data – exempelvis e-postadress, namn, profilbild.
- Var lagras den – vilken collection i MongoDB, vilket fält.
- Varför lagras den – det rättsliga grundbegreppet (samtycke, avtal, berättigat intresse) och det funktionella syftet.
- Hur länge lagras den – är det en livslång lagring eller har vi en utgångsperiod?
- Vem har åtkomst – endast användaren själv? Administratörer? Externa tjänster?
- Hur skyddas den – exempelvis lösenord haschas med bcrypt, transport är HTTPS, tillgång kräver autentisering.

Ett konkret exempel: vårt fält `users.email` lagrar e-postadressen som samlas in vid registrering, används som unik identifierare och för kommunikation kring kontot, lagras tills användaren raderar sitt konto, är åtkomlig endast av användaren själv och systemadministratörer, och är inte lagrad i hashat eller krypterat format eftersom vi behöver kunna söka på den.

Dokumentationen ovan tar några rader att skriva, men den besvarar omedelbart sex frågor som annars skulle kräva utredning. Det är en investering som lönar sig dubbelt – juridiskt och för utvecklarteamet självt.

### Dataminimering i dokumentationen

Det räcker inte med att vi tillämpar dataminimering i koden – vi behöver också kunna förklara varför vi tillämpar den. Det betyder att för varje fält vi har i våra datamodeller ska vi kunna besvara frågan 'varför har vi det här fältet?'. Om svaret är 'jag vet inte, det fanns med från början' eller 'det kan vara bra att ha' så ska fältet bort. Den disciplinen tjänar både slutanvändarens integritet och vårt eget framtida arbete.

När vi dokumenterar varje datafält uppmuntrar det oss också att ifrågasätta dess existens. Om vi inte kan motivera varför vi lagrar telefonnumret är det dags att ta bort fältet. Den här reflekterande processen är en av GDPR:s mer underskattade fördelar – den tvingar fram en disciplin som i längden ger oss enklare och mer fokuserade datamodeller.



## Loggning och GDPR

Vi pratade i vecka 9 om att inte logga personuppgifter. Det är minst lika viktigt att vi dokumenterar vår loggningspolicy. Att skriva en kort sektion i README eller i en separat LOGGING.md som beskriver vad som loggas och vad som aldrig loggas är ett sätt att göra detta till en explicit del av kontraktet med användaren och med teamet.

En loggningspolicy kan se ut ungefär så här:

```
Loggning
```

```
Vi loggar följande information för varje request:
```

- HTTP-metod och sökväg
- Statuskod
- Svarstid
- Användar-ID (men inte e-post eller namn)
- IP-adress (anonymiserad efter 30 dagar)

```
Vi loggar ALDRIG:
```

- Lösenord (i klartext eller hashat)
- E-postadresser eller namn
- Innehållet i request body
- Authorization-headers
- Personnummer eller andra känsliga uppgifter

Det här dokumentet är inte bara för utvecklare – det är också en explicit redogörelse som vi kan visa om någon undrar. Om en användare ber oss om alla loggade uppgifter om dem (vilket de har rätt till enligt GDPR) så vet vi exakt vad vi behöver leverera och vad vi inte behöver leta efter eftersom det aldrig loggades.

## Git-workflow i team

Hittills i utbildningen har vi använt Git i grupp under boiler room-arbetet, men ofta utan en formaliserad arbetsprocess. När projektet växer i komplexitet och vi närmar oss överlämning behöver vi en tydligare struktur. Ett genomtänkt Git-workflow är inte byråkrati – det är ett sätt att förhindra att ändringar krockar, att tappade commits inte uppstår och att kodgranskning blir en naturlig del av flödet.

Det finns flera etablerade workflow-modeller. För vårt projekt rekommenderar vi en variant av GitHub Flow: en stabil main-branch där produktionsklar kod alltid finns, feature-branches för varje ny funktion eller bugggrättning, och pull requests för att integrera ändringar tillbaka till main. Det är en enkel modell som passar bra för team i vår storlek och för projekt i vår fas.

### Branchstrategi

Vi rekommenderar följande regler för branches i vårt projekt. Att vara konsekvent kring det här är en av de billigaste investeringarna vi kan göra för att hålla projektet hanterbart:

- main – ska alltid vara stabil. Endast pull requests som passerat kodgranskning får mergas in. Direktcommits till main är förbjudna.
- feature/\* – för nya funktioner. Exempel: feature/recipe-comments, feature/admin-dashboard.
- fix/\* – för bugggrättningar. Exempel: fix/login-redirect, fix/missing-validation.
- refactor/\* – för kodförbättringar utan ny funktionalitet. Exempel: refactor/extract-auth-service.
- docs/\* – för dokumentationsändringar. Exempel: docs/api-readme, docs/architecture-diagram.

Notera att vi använder beskrivande namn, inte branch-1 eller robins-branch. Den som senare läser commit-historiken eller en pull request ska kunna se i en blick vad arbetet handlat om. Det är en liten gest mot framtiden som kostar nästan ingenting men betyder mycket.

### Commit-meddelanden som dokumentation

Commit-meddelanden är en form av dokumentation som många underskattar. När vi senare felsöker ett problem och kör git blame eller git log för att förstå när något infördes så är det de meddelandena vi läser. Om de är intetsägande ('fix', 'update', 'wip') tappar vi värdefull kontext. Vi rekommenderar därför formatet conventional commits, som är en lättviktskonvention för commit-meddelanden:

- feat: lägg till endpoint för att kommentera recept
- fix: hantera saknad token utan att krascha
- refactor: bryt ut user-validation till egen modul
- docs: uppdatera README med nya miljövariabler
- test: lägg till tester för auth-middleware
- chore: uppgradera mongoose till senaste versionen

Prefixen (feat, fix, refactor, docs, test, chore) gör det enkelt att skanna historiken och förstå vilken typ av ändring varje commit utgör. För större ändringar kan vi också skriva en längre beskrivning under första raden, men vi använder då ett tomrum mellan rubriken och beskrivningen:

```
feat: implementera rollbaserad åtkomst för administratörer
```

Lägger till middleware som kontrollerar att användaren har rollen 'admin' innan den får tillgång till administrativa endpoints. Implementationen följer mönstret från ADR-003.

Ett bra commit-meddelande svarar på två frågor: 'vad ändrades?' (i rubriken) och 'varför?' (i beskrivningen, om det inte är uppenbart). Vad-frågan kan ofta utläsas av diffen, men varför-frågan finns sällan i koden. Det är där commit-meddelandet adderar värde.

## Pull requests

En pull request (PR) är inte bara ett tekniskt steg för att merge kod – det är ett tillfälle att kommunicera med teamet om vad ändringen gör och varför. En PR med titeln 'feat: lägg till kommentarer' och en tom beskrivning lämnar all tolkning till granskaren. En PR med en tydlig beskrivning gör granskningen avsevärt enklare och snabbare.

Vi rekommenderar följande mall för PR-beskrivningar:

```
Vad gör den här PR:en?
Lägger till möjligheten för inloggade användare att
kommentera recept. Inkluderar nya endpoints, schema och tester.

Varför?
Krav från sprint 3 i boiler room-projektet.

Hur kan det testas?
1. Logga in som user
2. POST /api/recipes/:id/comments med en kommentar
3. Verifiera att den dyker upp i GET /api/recipes/:id

Checklista
- [x] Tester finns och passerar
- [x] README uppdaterad
- [x] Inga konsolloggar kvar
- [x] Inputvalidering med Zod
```

En sådan beskrivning gör det enkelt för en granskare att veta vad de letar efter, hur de ska testa ändringen själva och om författaren har tänkt på de uppenbara sakerna. Det räddar tid för alla inblandade och gör koden bättre.

## Merge eller rebase?

En klassisk debatt i Git-världen är om man ska använda merge-commits eller rebase när man integrerar en feature-branch i main. Det finns inget objektivt rätt svar – båda har för- och nackdelar. För vårt projekt rekommenderar vi att teamet tar ett medvetet beslut och håller sig till det:

- Merge commits skapar en explicit punkt i historiken där en feature integrerats. Historiken blir mer komplex men bevarar exakt vad som hände när. Bra för att spåra när och varför något ändrades.
- Rebase ger en linjär historia som är lättare att läsa men förlorar information om när feature-branches integrerades. Bra för team som värdesätter läsbarhet i historiken.

- Squash merge slår ihop alla commits från feature-branchen till en enda commit på main. Bra om feature-branchens enskilda commits är 'work in progress' och inte värda att bevara individuellt.

För vårt boiler room-projekt rekommenderar vi squash merge som standard. Det ger en ren historia på main där varje commit motsvarar en feature eller fix, och det tar bort 'fixa stavfel'-commits som vi alla skriver under utvecklingens gång. Vad teamet faktiskt väljer spelar mindre roll än att alla håller sig till samma val.

## Ansvarsfördelning i ett team

Hittills har vi pratat om mekanismerna i Git, men minst lika viktigt är det sociala kontraktet inom teamet. Vem ansvarar för vilka delar? Vem måste granska en PR innan den mergas? När gör vi tekniska beslut som hela teamet, och när tar enskilda utvecklare egna beslut? Det här är frågor som varje team behöver svara på explicit, även om svaren är pragmatiska och inte huggna i sten.

För vårt boiler room-projekt rekommenderar vi följande grundregler. De är bara förslag – varje grupp får anpassa efter sin egen situation, men något i stil med detta är värt att etablera:

- Ingen direktpush till main. Allt går via PR.
- Minst en annan teammedlem ska godkänna PR:en före merge.
- Den som öppnar PR:en mergar den själv efter godkännande – det säkerställer att de är medvetna om när deras kod hamnar i main.
- Större arkitekturella beslut (val av bibliotek, ändringar i datamodeller) diskuteras med hela teamet före implementation.
- Daglig synkning under boiler room om vad var och en arbetar med. Det förhindrar att två personer omedvetet ändrar samma fil samtidigt.

Det är också nyttigt att ha en informell ansvarsfördelning där varje del av kodbasen har en eller två 'ägare' som är mer insatta i den än andra. Det betyder inte att ingen annan får röra koden – det betyder att om man har frågor om en specifik modul finns det en uppenbar person att vända sig till. Det förkortar avstånd och sprider ändå kunskapen i teamet eftersom alla också gör PR-granskningar tvärs över ägargränserna.

## Kodgranskning – code review

Kodgranskning, eller code review, är processen där en eller flera utvecklare läser och kommenterar någons kodändring innan den mergas in i huvudbranchen. Det är en av de mest värdefulla kvalitetspraxis som finns inom mjukvaruutveckling, både för att fånga buggar tidigt och för att sprida kunskap inom teamet. Vi introducerades för konceptet i kurs 3, vecka 9. Den här veckan tar vi det vidare i ett backend-sammanhang där felaktig kod kan ha allvarigare säkerhetskONSEKVENSER än i frontend.

Det är viktigt att förstå att kodgranskning inte är en personlig kritik. När någon ger feedback på vår kod handlar det om koden, inte om oss som utvecklare. Den som tar kommentarer personligt blir en sämre granskare och en sämre granskad – båda lider. Den som ger kommentarer hånfullt eller nedlåtande skapar en kultur där andra hellre struntar i att granska. En framgångsrik granskningskultur kräver att alla parter visar respekt och tar feedback i vetskap om att det är menat att förbättra produkten.

### Vad letar vi efter när vi granskar?

När vi granskar någon annans kod är det lätt att fastna på små stilistiska detaljer (lediga tabbar, semikolon, namnkonventioner) och missa de viktiga frågorna. Stilistiska detaljer ska helst hanteras automatiskt med Prettier och ESLint. När vi granskar manuellt fokuserar vi på saker som verktygen inte kan fånga:

- Korrekthet – gör koden vad den ska? Är edge cases hanterade? Vad händer vid tom array, null, oväntade datatyper?
- Säkerhet – finns det inputvalidering? Hashas lösenord? Exponerar felmeddelanden information vi inte vill ge ut? Är tokens skyddade?
- Felhantering – hanteras fel konsekvent? Skickas de vidare till central error handler? Riskerar någon kod att kasta ohanterade undantag?
- Tester – finns det tester? Täcker de både lyckade fall och felfall? Testar de det som faktiskt är intressant att testa?
- Läsbarhet – kan en utomstående läsa koden och förstå syftet? Är variabelnamn talande? Är funktioner lagom stora?
- Konsistens – följer ändringen samma mönster som resten av kodbasen? Eller introducerar den ett nytt sätt att lösa något som redan har en etablerad lösning?
- Prestanda – finns det uppenbara prestandaproblem? N+1 queries? Onödiga datatransporter? För kursens skala behöver vi inte mikrooptimera, men uppenbara problem ska påpekas.

### En checklista för kodgranskning av backend-kod

För att göra granskningen mer systematisk är det användbart att ha en checklista som vi går igenom när vi granskar en PR. Den nedan är specifik för det vi byggt under kursens gång och är ett bra utgångsläge för boiler room-projektet:

1. Validering – Validerar nya endpoints sin input med Zod? Är schemana strikta nog?
2. Autentisering – Skyddas endpoints som ska skyddas? Är auth-middleware korrekt placerad?
3. Auktorisering – Kontrolleras roller där det behövs? Får en user-roll inte access till admin-routes?
4. Felhantering – Skickas fel vidare till next(error)? Returneras rätt statuskoder?

5. Loggning – Loggas inga personuppgifter? Är loggnivåerna rätt valda (info, warn, error)?
6. Hemligheter – Finns inga API-nycklar, JWT-secrets eller lösenord i koden? Allt via miljövariabler?
7. Statuskoder – Används rätt HTTP-statuskoder (201 för create, 204 för delete utan body, 404 för not found)?
8. Tester – Finns tester för nya funktioner? Tar de hand om både glada vägen och felscenarier?
9. Dokumentation – Är README och API-dokumentationen uppdaterad i samma PR?
10. Konsolloggar – Finns inga slumpmässiga console.log kvar från utvecklingen?

En checklista som denna kan bockas av i PR-beskrivningen av författaren själv innan granskningen påbörjas. Det signalerar till granskaren att vissa saker redan är genomgångna och tillåter granskaren att fokusera på den specifika kodlogiken.

## Hur ger vi bra feedback?

Hur vi formulerar feedback har stor effekt på hur den tas emot. Det finns några enkla principer som gör skillnad i en kodgranskning:

- Fokusera på koden, inte på personen. 'Den här funktionen kan inte hantera tom array' är bättre än 'du har glömt att hantera tom array'.
- Var konkret. 'Det här ser konstigt ut' är inte användbart. 'Den här loopen kommer att kasta om items är undefined – kan vi guarda mot det?' är användbart.
- Förslag, inte diktat. 'Vad tänker du om att bryta ut den här i en egen funktion?' är ofta bättre än 'bryt ut den här'. Inbjud till dialog.
- Skilj mellan måste, bör och kan. Vissa kommentarer är blockerande (säkerhet, korrekthet). Andra är förslag (stil, läsbarhet). Markera vilken kategori varje kommentar tillhör.
- Beröm också. Om något är välgjort, säg det. Det är lätt att bara påpeka problem och ge intrycket av att hela PR:en var dålig.

En användbar konvention som många team använder är att prefixa kommentarer med en kategori: 'nit:' för småsaker som inte är blockerande, 'q:' för frågor man undrar över, 'suggestion:' för förslag, och inget prefix för saker som måste åtgärdas. Det gör att författaren snabbt kan se vad som är avgörande och vad som är diskussion.

## Hur tar vi emot feedback?

Att vara den som granskas är ofta svårare än att vara den som granskar. Det är lätt att bli defensiv när någon påpekar problem i kod vi just spenderat timmar på. Några förhållningssätt som hjälper:

- Anta gott uppsåt. Granskaren försöker hjälpa, inte sänka.
- Tacka för feedback, även när vi inte håller med. Det signalerar att samtalet är välkommet.
- Om du inte håller med, förklara varför istället för att bara avvisa kommentaren. Ofta är det informationsbrist på ena eller andra sidan.
- Om en kommentar leder till en ändring, gör ändringen i en ny commit istället för att skriva om historik. Det gör att granskaren kan se exakt vad som ändrats sedan deras kommentar.

- Ta bra feedback med dig till nästa PR. Om någon påpekat samma sak två gånger är det ett mönster vi vill internalisera.

Vi vill skapa en kultur där kodgranskning ses som något positivt, inte ett hinder. Bra granskningskulturer ger oss alla bättre kod och, kanske ännu viktigare, gör oss själva till bättre utvecklare. Att läsa andras kod regelbundet är ett av de snabbaste sätten att lära sig nya mönster och undvika misstag.

## Slutförberedelser inför examinationen

Examinationsveckan i vecka 12 består av tre moment: ett skriftligt kunskapstest som täcker kursmål 1-6 (kunskaper), en skriftlig reflektion som också täcker kursmål 1-6, och en projektinlämning i grupp som täcker kursmål 7-12 (färdigheter). Alla tre genomförs under vecka 12, men förberedelserna görs nu. Det är vecka 11 som är vår sista möjlighet att lägga den extra timmen där den behövs som mest.

Vi tar tid att gå igenom vad varje moment förväntar sig av oss. Det är inte meningen att examinationen ska komma som en överraskning – tvärtom ska vi gå in i den med en klar bild av vad som testas och hur. Det minskar stressen och tillåter oss att fokusera på att visa vad vi kan, inte att gissa vad utbildaren kommer att fråga.

### Skriftligt kunskapstest – kursmål 1-6

Kunskapstestet täcker de teoretiska kursmålen och är det moment där vi visar att vi inte bara kan kopiera kod, utan också förstår varför saker fungerar som de gör. Frågorna kommer att vara av typen som testar förståelse – inte memorering. Det är därför inte meningsfullt att lära sig syntax utantill. Det som testas är begrepp och resonemang.

Vi förväntas kunna besvara frågor som:

- Förklara vad stateless innebär för REST API-design och vilka konsekvenser det har.
- Beskriv skillnaden mellan dokumentdatabaser och relationsdatabaser och när vi väljer det ena framför det andra.
- Förklara hur TypeScript bidrar till säkrare backend-kod och vilka delar av Express som typas.
- Vad innebär OWASP Top 3, och vilka mekanismer använder vi för att skydda mot dem?
- Beskriv principen 'don't trust the client' och dess praktiska konsekvenser.
- Förklara skillnaden mellan autentisering och auktorisering, och hur RBAC implementeras.
- Hur påverkar GDPR vår API-design och datahantering, särskilt vad gäller dataminimering och loggning?

Notera att frågorna ovan är exempel på typen av frågor – inte de exakta frågorna. Det viktiga är att vi kan resonera kring koncepten, inte att vi kan recitera definitioner. Om vi har varit aktiva under workshops och boiler room-arbetet och läst studiematerialen så är vi väl förberedda. Den här veckan är till för att fylla luckor och repetera, inte för att lära sig allt från scratch.

### Skriftlig reflektion

Den skriftliga reflektionen kompletterar kunskapstestet genom att låta oss visa vår förståelse i ett mer fritt format. Den är individuell och handlar om vår egen läranderesor under kursen: vad vi lärt oss, vad som varit utmanande, hur vi tänker på backend-utveckling nu jämfört med innan kursen, och hur vi planerar att fortsätta lära oss.

Reflektionen ska inte vara en sammanfattning av kursinnehållet – det är vad kunskapstestet är till för. Den ska vara ett ärligt och genomtänkt dokument där vi visar att vi har bearbetat materialet personligt. Konkreta exempel från boiler room-projektet eller workshops är värdefulla. Generella formuleringar typ 'jag har lärt mig mycket' är inte. En reflektion på två-tre sidor är typiskt rimligt – det handlar om kvaliteten i tänkandet, inte mängden text.



## Projektlämnning i grupp – kursmål 7-12

Projektlämnningen är där våra färdigheter examineras. Det är boiler room-projektet vi byggt under kursens gång, och bedömningen baseras på den färdiga produkten samt den dokumentation som ingår. Det är därför veckan 11 är så viktig – den är vår sista chans att finslipa kvaliteten innan inlämning.

Inlämningen förväntas innehålla:

- En körande applikation – driftsatt i molnmiljö (Render eller motsvarande) som visar att den faktiskt fungerar end-to-end.
- Källkod i ett offentligt eller delat Git-repository med en commit-historik som visar gruppens samarbete över tid.
- En komplett README som ger en utomstående utvecklare allt de behöver för att förstå och köra projektet.
- API-dokumentation som täcker alla endpoints med autentiseringskrav, request/response-exempel och felscenarier.
- Arkitekturdiagram som visar hur frontend, backend och databas samverkar samt sekvensdiagram för det centrala flödet i applikationen.
- En testsvit med tester för CRUD-operationer, autentisering och åtminstone några av de centrala flödena.
- En överlämningsdokumentation med status, kända problem och eventuella ADR-dokument för viktiga arkitekturella beslut.
- GDPR-dokumentation som beskriver vilka personuppgifter som hanteras, var och varför.

Listan kan kännas omfattande, men de flesta delarna har vi byggt löpande under kursens gång. Veckan 11 handlar om att samla ihop, strukturera och fylla i de luckor som finns. Det är inte meningen att vi ska skriva all dokumentation från noll på en vecka – det är meningen att vi ska konsolidera det vi redan har.

## Vad bedöms i projektet?

Bedömningen av projektet utgår från kursmål 7-12 (färdigheterna). Det innebär att vi bedöms inte bara på 'fungerar det?' utan på hur väl vi visar de specifika färdigheter som kursen är till för att utveckla. Konkret betyder det:

- Att backend är strukturerad i Node.js/Express med tydlig separation av routes, controllers och models, och att TypeScript används där det är relevant.
- Att CRUD-operationer är implementerade och testade mot en databas, och att kopplingen mellan kod och datalager är säker och stabil.
- Att inputvalidering finns på alla relevanta endpoints och att felhantering är konsekvent och säker.
- Att grundläggande skydd mot OWASP Top 3 är implementerade: lösenord hashas, autentisering används för skyddade routes, RBAC fungerar och hemligheter hanteras via miljövariabler.
- Att projektet är driftsatt och dokumenterat med hänsyn till GDPR och dataminimering.
- Att gruppen visat ett fungerande samarbete med tydlig ansvarsfördelning, vilket bland annat syns i Git-historiken och PR-arbetet.

Det sista är värt att betona. Bedömningen är inte enbart av slutprodukten – den är också av processen. Ett projekt som har en sund Git-historik med commits från alla gruppmedlemmar,

dokumenterade pull requests och tydlig progression över tid signalerar något helt annat än ett projekt där en person commit:at allt sista veckan.

## Repetition – kursens centrala koncept

För att inte komma in i kunskapstestet med kursmaterialet i dimma tar vi nu ett varv genom de viktigaste koncepten från kursens första nio veckor. Det här ska inte ersätta att vi går tillbaka och läser studiematerialet – det är en checklista för att identifiera vad som behöver mer uppmärksamhet och vad som redan sitter.

### Vecka 1-3: Node.js, Express och TypeScript

Vi började kursen med att förstå vad Node.js är: en JavaScript-runtime byggd på V8 med eventloop och non-blocking I/O. Vi lärde oss att eventloopen tillåter Node.js att hantera många simultana anslutningar utan att skapa en tråd per anslutning. Express byggde vi ovanpå Node.js – det gav oss bekväm routing, request- och response-objekt samt middleware-konceptet.

Middleware är funktioner som körs i en kedja för varje inkommande request. De kan modifiera req och res, eller kalla next() för att gå vidare i kedjan. Vi använder middleware för loggning (morgan), parsing av request body (express.json), CORS, validering och autentisering. Att förstå middleware-kedjan på djupet är centralt för allt vi gör i Express.

REST-principerna gav oss riktlinjer för API-design: resursorienterade URL:er med substantiv i plural, HTTP-metoder som anger åtgärden (GET, POST, PUT, PATCH, DELETE), korrekta statuskoder och stateless kommunikation. TypeScript adderade vi för typad kod – Request, Response, NextFunction från Express, samt egna interfaces för datamodeller och API-kontrakt.

### Vecka 4-5: MongoDB, Mongoose och CRUD

MongoDB är en dokumentdatabas där data lagras som JSON-liknande BSON-dokument. Det skiljer sig från relationsdatabaser där data lagras i tabeller med rader och kolumner. Dokumentdatabaser är flexibla – schemana kan utvecklas över tid – men kräver disciplin eftersom databasen i sig inte erbjuder samma typ av strukturella garantier som SQL.

Mongoose är en ODM som lägger ett strukturlager ovanpå MongoDB. Vi definierar schemas som beskriver hur våra dokument ska se ut, inklusive typer, valideringsregler och default-värden. Mongoose-modeller ger oss CRUD-metoder: create, find, findById, findByIdAndUpdate, findByIdAndDelete och fler.

Relationer i MongoDB hanteras antingen genom embedding (lagra relaterad data inuti samma dokument) eller referencing (lagra ett ID som pekar på ett annat dokument). Embedding är bra för data som alltid läses ihop. Referencing är bra för data som lever självständigt eller där relationen är många-till-många. Populate i Mongoose är hur vi 'följer' en referens när vi läser data.

Testning kom in i vecka 5. Jest och Supertest är våra verktyg. Vi skriver tester mot en testdatabas (ofta MongoDB Memory Server) och verifierar att våra endpoints beter sig som de ska – både i lyckade fall och i felfall.

### Vecka 6: Validering och felhantering

Validering är en av de viktigaste säkerhetsmekanismerna i ett API. Principen är 'don't trust the client': all data som kommer in måste valideras innan den används. Vi använder Zod – ett schemabaserat valideringsbibliotek med TypeScript-integration – för att deklarativt beskriva vilken data vi förväntar oss. Med safeParse får vi antingen den validerade datan eller en lista med alla valideringsfel.

Felhantering centraliserar vi med en error handler middleware i Express. Genom att anropa `next(error)` i våra route handlers skickar vi alla fel till en gemensam plats där de översätts till HTTP-svar. Vi har egna felklasser (`ValidationError`, `NotFoundError`, `UnauthorizedError`) som gör koden mer uttrycksfull och låter oss skilja förväntade applikationsfel från oväntade programmeringsfel.

## Vecka 7-8: Autentisering, OWASP och RBAC

OWASP Top 3 består av Broken Access Control, Cryptographic Failures och Injection. Det är de tre vanligaste säkerhetshoten mot webbapplikationer enligt OWASP. Vi skyddar oss mot Broken Access Control med autentisering och auktorisering. Mot Cryptographic Failures med `bcrypt` för lösenord och HTTPS för transport. Mot Injection med parametriserade databasanrop via `Mongoose`.

Autentisering använder vi JSON Web Tokens (JWT). Vid inloggning verifierar vi lösenordet (`bcrypt.compare` mot den hashade versionen) och utfärdar en token som innehåller användarens ID. Klienten skickar tokenen i `Authorization`-headern på efterföljande anrop. Vår `auth`-middleware verifierar tokenen och lägger till användarinfo på `req`-objektet.

Auktorisering är skilt från autentisering. Det handlar om vad en autentiserad användare får göra. RBAC (Role-Based Access Control) är vår implementation: varje användare har en roll (`user`, `admin`), och vissa endpoints kräver specifika roller. RBAC-middleware kontrollerar `req.user.role` och avvisar med 403 Forbidden om rollen inte räcker.

Miljövariabler hanterar vi med `dotenv` för utveckling och med molnpartnerns inställningar för produktion. Hemligheter (`JWT_SECRET`, `MONGODB_URI`, externa API-nycklar) hör aldrig hemma i kod eller i Git – de bor alltid i miljön.

## Vecka 9-10: GDPR, loggning och driftsättning

GDPR ställer krav på att vi behandlar personuppgifter ansvarsfullt. Dataminimering – samla bara in det som behövs. Säker loggning – aldrig persondata i loggar. Rätten att bli glömd – möjlighet att radera användardata. Vi dokumenterar vilken data vi behandlar och varför, vilket är både ett juridiskt krav och en god utvecklingspraxis.

Strukturerad loggning gör vi med `pino` eller `winston`. Loggar är JSON-objekt vilket gör dem maskinläsbara och sökbara. Vi har olika loggnivåer: `error`, `warn`, `info`, `debug`. I produktion brukar vi sätta nivån till `info` eller högre för att inte drunkna i loggar.

Driftsättningen tog oss till en molnmiljö (Render eller motsvarande) med MongoDB Atlas som databas. Förutsättningarna för produktion är HTTPS, korrekt CORS-policy, miljövariabler för hemligheter och rate limiting för att skydda mot missbruk. När vi är driftsatta är systemet inte längre vårt eget – det är åtkomligt för omvärlden, vilket innebär ett annat säkerhetsperspektiv.

## Checkpoint: Förberedelse inför kursvecka 11

Inför checkpointen tisdag ska vi ha läst igenom detta studiematerial och ha en bild av vart vårt boiler room-projekt står. Det är den här veckan vi skiftar fokus från att bygga nya funktioner till att finslipa, dokumentera och förbereda inlämning. Det är ett bra tillfälle att stanna upp och inventera.

Kontrollera att gruppen har följande klart inför boiler room måndag:

11. Projektet är driftsatt och fungerar i produktion (eller ni vet exakt vad som saknas för att det ska göra det).
12. Ni har en lista över vad som är klart, vad som är delvis klart och vad som inte är klart.
13. Ni har identifierat vilka delar av dokumentationen som behöver mest arbete (README? API-dokumentation? Arkitekturdiagram?).
14. Ni har funderat på ansvarsfördelning för veckan: vem skriver vad, vem granskar vem.

Ta med er följande tankar till checkpointen: Var står ni i förhållande till bedömningskriterierna? Finns det något i listan över förväntade delar i projektinlämningen som ni vet att ni inte hinner med? Vad är planen för att hantera det? Och, kanske viktigast: hur fördelar ni arbetsbördan inför examinationen så att ingen i gruppen känner att de bär hela ansvaret ensam?

Det är vanligt att känna stress inför examination. Det är också vanligt att inte allt blir helt klart – det är okej. Vi siktar på att leverera ett projekt som visar vad vi lärt oss, inte ett perfekt produkt. Att vara ärliga om vad som är klart och vad som inte är klart är inte en svaghet utan ett tecken på professionalism.

## Sammanfattning

Under kursvecka 11 har vi flyttat fokus från att bygga ny funktionalitet till att göra det vi redan byggt redo att lämnas över. Det är ett perspektivskifte från 'vad gör koden?' till 'kan andra förstå och underhålla den?'. Den övergången är central i utvecklarens yrkesliv – sällan bygger vi system i isolation, oftast deltar vi i pågående arbete tillsammans med andra.

Vi har etablerat dokumentation som ett hantverk i tre nivåer: projektnivå (README, arkitekturdiagram), API-nivå (endpoint-dokumentation, autentiseringsflöden) och kodnivå (kommentarer, JSDoc, namn). Varje nivå tjänar sin egen läsare och har sin egen detaljnivå. Bra dokumentation är aktuell, skiktad, konkret, ärlig och användarcentrerad.

README är projektets ansikte utåt. Den beskriver vad projektet är, hur man installerar det, vilka miljövariabler som krävs och hur man kör det. En kompetent utvecklare ska kunna gå från klonat repo till körande lokal miljö på max tio minuter genom att enbart följa README. API-dokumentationen kompletterar med detaljer om varje endpoint: autentisering, request body, response, möjliga fel.

Arkitekturdiagram dokumenterar hur systemet hänger ihop som helhet. Vi använder Mermaid för enklare diagram direkt i Markdown, och draw.io eller Excalidraw för mer komplexa illustrationer. Sekvensdiagram visar hur olika delar samverkar i tid och är särskilt värdefulla för att dokumentera autentiseringsflöden och andra flerstegs-flöden.

Överlämningsdokumentationen tillför sådant som inte är permanent dokumentation av systemet utan en ögonblicksbild av läget vid överlämning: status, kända problem, tekniska skulder, roadmap och beslutsloggar (ADR). En ADR dokumenterar ett enskilt arkitekturellt beslut – vad valde vi, varför och vilka alternativ övervägde vi.

GDPR-dokumentationen handlar om att kunna besvara frågor om vilka personuppgifter vi behandlar, var, varför och hur länge. Ett dataregister listar alla personuppgifter med deras syfte, lagringsplats, retentionstid och åtkomstkrav. Loggningspolicy gör explicit vad vi loggar och, kanske viktigare, vad vi aldrig loggar.

Git-workflowet i team kräver disciplin. Vi använder feature-branches med beskrivande namn, conventional commit-meddelanden, pull requests med tydliga beskrivningar och kodgranskning före merge. Ansvarsfördelningen är social lika mycket som teknisk – varje team behöver komma överens om sina egna regler kring direktpush, godkännanden och beslutsmandat.

Kodgranskning är ett av de mest värdefulla kvalitetspraxis som finns. Vi granskar för korrekthet, säkerhet, felhantering, tester, läsbarhet och konsistens. Vi formulerar feedback respektfullt, fokuserar på kod istället för person och skiljer mellan måste, bör och kan. Vi tar emot feedback med öppet sinne och antar gott uppsåt.

Inför examinationen i vecka 12 förbereder vi oss på tre fronter: kunskapstestet (kursmål 1-6), den skriftliga reflektionen (kursmål 1-6) och projektinlämningen i grupp (kursmål 7-12). Den här veckan är vår sista chans att finslipa kvaliteten och fylla luckor. Vi går in i examinationen med en klar bild av vad som testas, så att vi kan fokusera på att visa vad vi kan.

Med det här studiematerialet avrundar vi kursens reguljära innehåll. I vecka 12 träffas vi för examinationen, och därefter går vi vidare i utbildningen med en gedigen grund i backend-utveckling. Tack för att ni varit med på resan så här långt – nu samlar vi ihop och visar vad vi kan.

## Resurser för fördjupning

Nedan följer resurser för de som vill fördjupa sina kunskaper om dokumentation, teamarbete och kodgranskning.

### Dokumentation och diagram

Mermaid - officiell dokumentation och live editor för diagram-som-kod direkt i Markdown:

<https://mermaid.js.org>

Excalidraw - enkelt och webbaserat verktyg för att rita arkitekturdiagram med handritad estetik:

<https://excalidraw.com>

OpenAPI Specification - den fullständiga specifikationen för OpenAPI/Swagger:

<https://swagger.io/specification/>

"Make a README" - en konkret och välskriven guide om hur man skriver en bra README. Innehåller också en mall vi kan utgå från:

<https://www.makeareadme.com>

### Architecture Decision Records

"Documenting Architecture Decisions" av Michael Nygard - den ursprungliga artikeln som introducerade ADR-konceptet. Kort, tydlig och fortfarande relevant trots att den är från 2011:

<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

"adr-tools" - ett kommandoradsverktyg för att skapa och hantera ADR-dokument med en enkel mall. Bra startpunkt för team som vill prova ADR utan att överarbeta processen:

<https://github.com/npryce/adr-tools>

### Git och samarbete

"Conventional Commits" - specifikationen för commit-meddelanden som vi rekommenderar i projektet:

<https://www.conventionalcommits.org>

"GitHub Flow" - en kort guide till en enkel branch-strategi som passar bra för mindre team och vårt boiler room-projekt:

<https://docs.github.com/en/get-started/using-github/github-flow>

"Pro Git" av Scott Chacon och Ben Straub - boken är gratis online och är referensverket för Git. Kapitlet om branching och workflows är extra relevant just nu:

<https://git-scm.com/book/en/v2>

## Kodgranskning

"Google's Engineering Practices Documentation" - Googles publika guide till kodgranskning. Den beskriver både hur man granskar och hur man tar emot granskning, och är en av de bästa resurserna som finns på området:

<https://google.github.io/eng-practices/review/>

"The Art of Readable Code" av Dustin Boswell och Trevor Foucher - en bok om hur man skriver kod som är lätt att läsa och granska. Många av principerna är direkt applicerbara på det vi byggt under kursen.

## Videomaterial

"Writing READMEs" och "How to write a Git commit message" finns i flera välgjorda format på YouTube. Sök på respektive term och välj nyare videor (2023 eller senare) för att få exempel som är relevanta för moderna projekt.

"Code Review Best Practices" från större tekniska konferenser (GOTO, NDC, JSConf) är ofta välarbetade och pedagogiska. De ger ett bredare perspektiv än enskilda artiklar och visar hur etablerade utvecklare resonerar i praktiken.



